Epidemiología: Nivel Avanzado

Instituto Nacional de Epidemiología Dr. Juan H. Jara

¡Les damos la bienvenida al curso!

Es un gusto recibirles en esta nueva edición del curso, diseñado para profundizar en el análisis de datos provenientes de distintos diseños de estudio mediante el uso de modelos estadísticos.

A lo largo del curso, abordaremos herramientas analíticas avanzadas que nos permitirán modelar adecuadamente la relación entre exposiciones y desenlaces en estudios transversales, de casos y controles, de cohortes y experimentales. Nos enfocaremos en la aplicación práctica de estos modelos, en la interpretación de resultados y en los supuestos que los sustentan, promoviendo una mirada crítica sobre su uso en la investigación epidemiológica.

Esperamos que este espacio sea una experiencia formativa enriquecedora y un nuevo impulso en su desarrollo como investigadoras e investigadores en salud.

El equipo docente

Christian Ballejo

Andrea Silva

Tamara Ricardo

Ramiro Dana Smith

Julia Maxwell

Declaración de uso de IA generativa

Se utilizaron herramientas de inteligencia artificial generativa (ChatGPT, versión GPT-5.3) para la revisión gramatical y ortográfica del manuscrito. El contenido final fue evaluado, validado y editado manualmente por los/as autores/as, quienes asumen la responsabilidad por su integridad.

Parte I.

Unidad 1

1. Introducción a R



Figura 1.1.: Artwork por @allison_horst

⚠ Estos materiales, que forman parte del **Módulo 1 del Curso de Epidemiología Avanzada**, no pretenden cubrir exhaustivamente todos los temas relevantes de un curso de lenguaje R. Su objetivo es introducir únicamente aquellos aspectos más importantes y necesarios para su aplicación en los módulos siguientes.

1.1. ¿Qué es R?

El sitio oficial **r-project.org** define a R como *"un entorno de software libre para gráficos y computación estadística. Se compila y se ejecuta en una amplia variedad de plataformas UNIX, Windows y MacOS."*

Profundizando en su descripción, podemos decir que R es un *lenguaje de programación interpretado, orientado a objetos, multiplataforma y de código abierto (open source) aplicado al manejo y análisis de datos estadísticos.*

A continuación, detallamos cada una de sus características:

R es un lenguaje de programación estadístico

Si bien posee un entorno y se puede utilizar como calculadora avanzada o para simulaciones, R es fundamentalmente un lenguaje de programación. Presenta una estructura y reglas de sintaxis propias, así como gran variedad de funciones desarrolladas con fines estadísticos.

R es un lenguaje orientado a objetos

R implementa conceptos de la programación orientada a objetos, lo cual le permite ofrecer simpleza y flexibilidad en el manejo de datos. En R, todo con lo que trabajamos —variables, funciones, datos, resultados— son objetos que pueden ser modificados o combinados con otros objetos.

R es un lenguaje interpretado

R no requiere compilación previa. Los scripts se ejecutan directamente mediante el intérprete del lenguaje, que devuelve resultados de forma inmediata.

R es un lenguaje multiplataforma

R puede instalarse y utilizarse en sistemas operativos Linux, Windows y MacOS. En todos ellos funciona de la misma manera, lo que garantiza que los scripts pueden correr en cualquier plataforma sin necesidad de modificaciones.

R es software libre y de código abierto

R se distribuye gratuitamente bajo licencia GNU - GPL (*General Public License*), lo que otorga a los usuarios la libertad de usar, estudiar, compartir y modificar el software. Esto ha favorecido el crecimiento de una comunidad global activa y colaborativa.

1.2. Breve historia del lenguaje

R tiene su origen en el lenguaje **S**, desarrollado en los años 70 en los laboratorios Bell de AT&T (actualmente Lucent Technologies). Posteriormente, S dio lugar a una versión comercial llamada **S-Plus**, distribuida por Insightful Corporation.

En 1995, los profesores de estadística **Ross Ihaka** y **Robert Gentleman**, de la Universidad de Auckland (Nueva Zelanda) iniciaron el **"Proyecto R"**, con la intención de desarrollar un programa estadístico inspirado en el lenguaje S pero de dominio público.

Aunque R es considerado un dialecto de S, existen diferencias importantes en el diseño de ambos lenguajes.

El software está desarrollado principalmente en lenguaje C++, con algunas rutinas en Fortran. El nombre "R" hace referencia a las iniciales de sus creadores: Ross y Robert. Actualmente, R es mantenido por un grupo internacional de desarrolladores voluntarios conocido como el **Core Development Team**.

1.3. Scripts de R

Un script es un archivo de texto plano que contiene una secuencia de instrucciones para ser ejecutadas por el intérprete de R.

El término *script* puede traducirse como guión, archivo de órdenes, archivo de procesamiento por lotes o archivo de sintaxis.

Puede crearse con cualquier editor de texto o con entornos especializados, y permite ser leído, modificado, guardado y ejecutado de forma completa o línea por línea.

Una de sus principales ventajas es su **reutilización**: los scripts pueden adaptarse fácilmente a distintos análisis o contextos, lo que facilita la replicabilidad del trabajo.

1.4. Características generales del lenguaje

R posee una **sintaxis textual precisa**. Como en otros lenguajes de programación, la escritura debe ser exacta: distingue entre mayúsculas y minúsculas (*case sensitive*), y cada línea escrita en la consola comienza con el símbolo > (*prompt*[1]).

1.5. Documentación del código

La documentación es una tarea fundamental en cualquier lenguaje de programación, ya que nos permite entender el propósito del script, facilita su mantenimiento y posibilita su reutilización tanto por quien lo creó como por otras personas.

En R, la documentación de los scripts se realiza a través de **comentarios**, indicados con el símbolo #. Todo lo que sigue a ese símbolo es ignorado por el intérprete cuando se ejecute el código:

```
# esto es una línea de comentario y el intérprete no la ejecuta
```

Así que a la hora de documentar es preferible abusar de estos comentarios que no utilizarlos.

1.6. Funciones

Las órdenes elementales en R se denominan comandos o **funciones**. Algunas se conocen como "integradas" ya que están incluidas en el núcleo del lenguaje (R `base`) y otras provienen de **paquetes** adicionales.

La mayoría de las ofrecidas en los paquetes o librerías están elaboradas con R, dado que todos los usuarios podemos crear nuevas funciones con el mismo lenguaje.

Toda función tiene un nombre y puede recibir argumentos (obligatorios u opcionales), que se colocan entre **paréntesis y separados por comas**. Incluso algunas funciones que no tienen asociado argumentos necesitan, en su sintaxis, a los paréntesis `()`.

Siempre una función devuelve un resultado, un valor o realiza una acción:

[1] Símbolo que aparece en la pantalla de la computadora indicando que el sistema está esperando información del usuario o que el sistema está listo para recibir instrucciones del usuario.

```
nombre_de_la_función(arg_1, arg_2, arg_n)
```

Como el interprete de R no permite errores en la sintaxis de las expresiones, debemos atender a los siguientes puntos a la hora de escribirlas:

La sintaxis habitual de una función es la siguiente:

```
funcion(arg1, arg2, arg3, ...)
```

Los títulos de los argumentos pueden escribirse y mediante un igual agregar el valor correspondiente:

```
funcion(arg1 = 32, arg2 = 5, arg3 = 65, ...)
```

Se puede omitir el título del argumento y escribir directamente el valor, pero en este caso, hay que respetar el orden definido por la función:

```
funcion(32, 5, 65, ...)
```

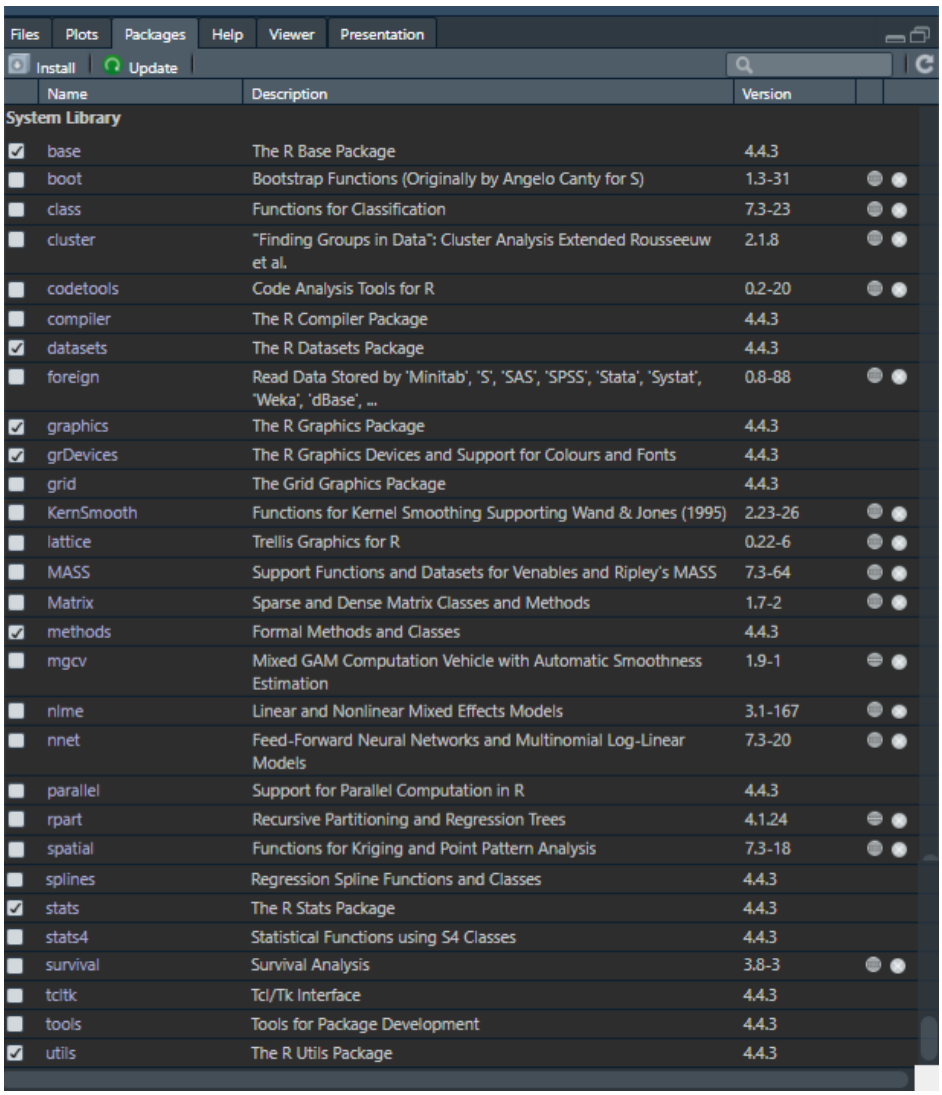
Los valores numéricos, lógicos, especiales y objetos van escritos en forma directa y cuando escribimos caracteres (texto) van necesariamente encerrados entre comillas:

```
funcion(arg1 = 3, arg2 = NA, arg3 = TRUE, arg4 = "less", arg5 = x, ...)
```

1.7. Paquetes o librerías

Los paquetes, también conocidos como *librerías*, son conjuntos de funciones, datos y documentación organizados organizadas en torno a una temática específica, que permiten ampliar las capacidades del sistema base de R.

Al instalar R, se incorpora un núcleo básico que queda activo automáticamente en cada sesión (`base`, `datasets`, `graphics`, `grDevices`, `methods`, `stats` y `utils`). Junto a este núcleo, se incluye un conjunto de paquetes recomendados que forman parte de la distribución oficial y pueden activarse manualmente cuando se los necesita. No obstante, la verdadera potencia de R reside en la posibilidad de incorporar nuevos paquetes desarrollados por la comunidad, lo que permite extender continuamente sus funcionalidades.



En la actualidad, existen más de 17.000 paquetes disponibles para una amplia variedad de aplicaciones, número que crece mes a mes gracias a la naturaleza *open source* del lenguaje. Cualquier persona puede desarrollar y compartir sus propios paquetes, aunque no todos llegan a publicarse en el repositorio oficial CRAN (Comprehensive R Archive Network), que actúa como principal fuente de distribución.

Los paquetes pueden descargarse directamente desde CRAN o instalarse a partir de archivos comprimidos locales (como `.zip` o `.tar.gz`). Una vez instalados, se los puede activar en cualquier análisis.

1.7.1. Dependencias

En muchos casos, al utilizar un paquete, este necesita de otros para funcionar correctamente. Esta relación se conoce como **dependencia**, y es una característica central en el ecosistema de R.

La mayoría de las funciones incluidas en los paquetes están desarrolladas en el propio lenguaje R y, durante su construcción, es habitual que recurran a funciones ya existentes en otros paquetes. Por ejemplo, una función nueva puede hacer uso de una función auxiliar que no pertenece al sistema base, sino a otro paquete externo.

Cuando intentamos ejecutar una función que depende de otras no disponibles en nuestra instalación, R no podrá encontrar esas funciones y nos devolverá un mensaje de error indicando que no reconoce el nombre

solicitado. Este tipo de errores suele alertar sobre funciones “desconocidas” o “no encontradas”, lo que indica que falta instalar o activar alguno de los paquetes requeridos.

Para evitar estos inconvenientes, R intenta resolver automáticamente las dependencias cuando instalamos un paquete desde el repositorio oficial CRAN. Si el paquete necesita otros para funcionar, el sistema detecta esta relación y se encarga de instalar también aquellos paquetes auxiliares. De todos modos, si alguno no se instala correctamente o no se activa en la sesión, será necesario hacerlo de forma manual.

1.8. Errores y advertencias

El lenguaje R es muy preciso en su sintaxis. Cometer errores al escribir funciones u objetos es común durante el aprendizaje, y el intérprete de R devuelve mensajes de error cuando detecta algo incorrecto.

Uno de los aspectos clave a tener en cuenta es que R **distingue entre mayúsculas y minúsculas** (*case sensitive*), por lo que no es lo mismo escribir `a` que `A`.

Existen tres grupos de mensajes de error:

- Errores de sintaxis
- Error de objeto no encontrado
- Otros errores

1.8.1. Errores de sintaxis

Ocurre cuando R no puede interpretar una línea de código porque algo está mal escrito. Los errores de sintaxis más frecuentes incluyen:

- Paréntesis, comas, comillas, corchetes o llaves mal colocados o ausentes.
- Argumentos mal separados o incorrectamente definidos.

Por ejemplo la función `rep()` repite valores una cantidad de veces. Tiene dos argumentos, `x` donde se coloca el valor a repetir y `times` donde se define la cantidad de veces:

```
rep(x = 3, times = 4) # repetimos 4 veces 3 con rep()
```

```
[1] 3 3 3 3
```

Si nos olvidamos de cerrar el paréntesis:

```
rep(x = 3, times = 4
```

```
Error in parse(text = input): <text>:2:0: unexpected end of input
1: rep(x = 3, times = 4
   ^
```

Si omitimos la coma entre argumentos:

```
rep(x = 3 times = 4)
```

```
Error in parse(text = input): <text>:1:11: unexpected symbol
1: rep(x = 3 times
              ^
```

Si usamos un nombre de argumento no válido:

```
rep(y = 3, times = 4)
```

```
Error in `rep()`:
! attempt to replicate an object of type 'symbol'
```

Si escribimos mal el nombre de la función:

```
rop(x = 3, times = 4)
```

```
Error in `rop()`:
! could not find function "rop"
```

Este último error se asemeja a un error de objeto no encontrado, aunque tiene origen en un problema de sintaxis.

Los mensajes de error en general y sobre todo al principio pueden parecer extraños y difíciles de entender, pero con un poco de práctica podemos inferir donde está el problema.

1.8.2. Error de objeto no encontrado

Este tipo de error se produce cuando R no reconoce un objeto utilizado en el código. Las causas más frecuentes incluyen:

- El nombre del objeto está mal escrito (por ejemplo, errores de sintaxis o uso incorrecto de mayúsculas/minúsculas).
- El objeto pertenece a un paquete o archivo que no fue cargado.
- Faltan comillas al declarar caracteres.
- Otras causas similares.

Volvamos al ejemplo anterior, ahora repitiendo un valor tipo `character`:

```
rep(x = "A", times = 4) # repetimos 4 veces "A" con rep()
```

```
[1] "A" "A" "A" "A"
```

Si olvidamos las comillas:

```r
rep(x = A, times = 4) # repetimos 4 veces A con rep()
```

```
Error:
! object 'A' not found
```

1.8.3. Otros errores

Son aquellos que se generan por causas distintas a errores de sintaxis o de objeto no encontrado. Por ejemplo:

```r
"x" + 10
```

```
Error in `"x" + 10`:
! non-numeric argument to binary operator
```

El código anterior genera un mensaje de error porque intenta sumar objetos de tipos incompatibles entre sí.

Veamos otro ejemplo:

```r
t.test(1)
```

```
Error in `t.test.default()`:
! not enough 'x' observations
```

En este caso, el error se debe a que no hay suficientes observaciones para realizar una prueba t de Student.

Otro caso frecuente ocurre cuando se intenta acceder a una posición inexistente en un vector:

```r
x <- c(1, 2, 3)
x[[5]]
```

```
Error in `x[[5]]`:
! subscript out of bounds
```

Este mensaje de error indica que el subíndice está fuera de los límites del objeto (`subscript out of bounds`).

1.8.4. Advertencias

Las advertencia no son tan serias como un error, o al menos no lo parece, ya que permiten el código se ejecute igual. Pero puede ocurrir que ignorar una advertencia llegue a ser algo muy serio, si esto implica que la salida de la función es equivocada.

Ignorar advertencias puede llevar a conclusiones erróneas, por lo que es recomendable prestarles atención y entender su causa.

Por ejemplo:

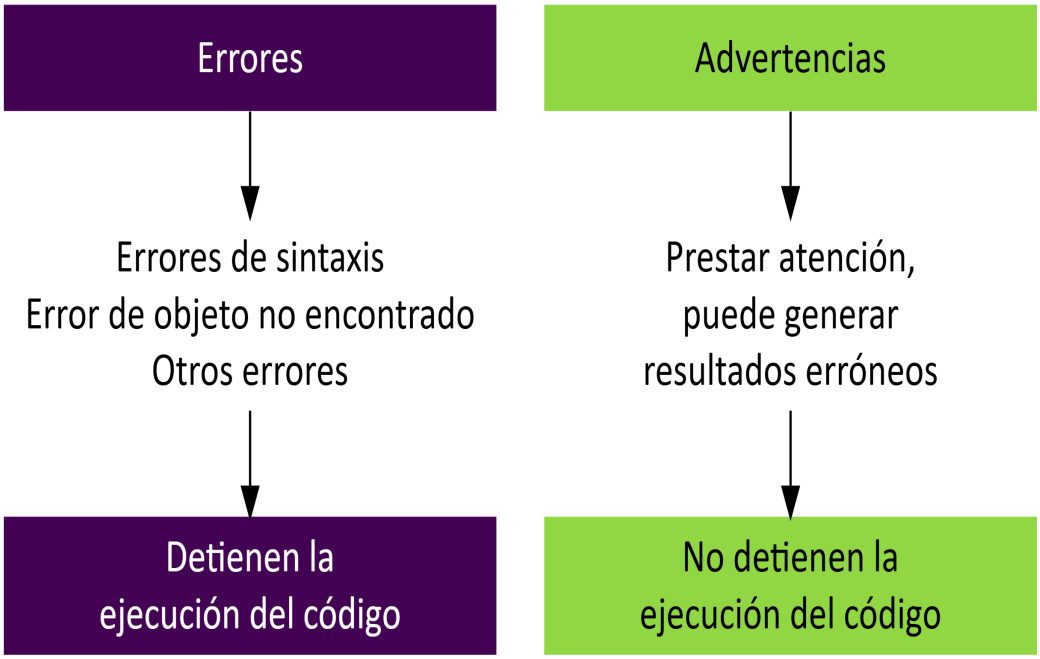
```
log(-1)
```

```
Warning in log(-1): NaNs produced
```

```
[1] NaN
```

la función genera una advertencia porque el logaritmo de un número negativo no está definido en los reales, y R devuelve un valor `NaN` (*Not a Number*).

1.8.5. Resumiendo...



1.9. Creación de objetos

Todas las declaraciones donde se crean objetos usan el símbolo de asignación <−:

```
nombre_objeto <- valor
```

Veámoslo en un ejemplo:

```
a <- 1
```

En este caso asignamos el valor 1 al objeto `a`. El objeto `a` es un vector de una posición (un solo valor). Si llamamos al objeto, el intérprete devuelve el valor asignado previamente:

```
a
```

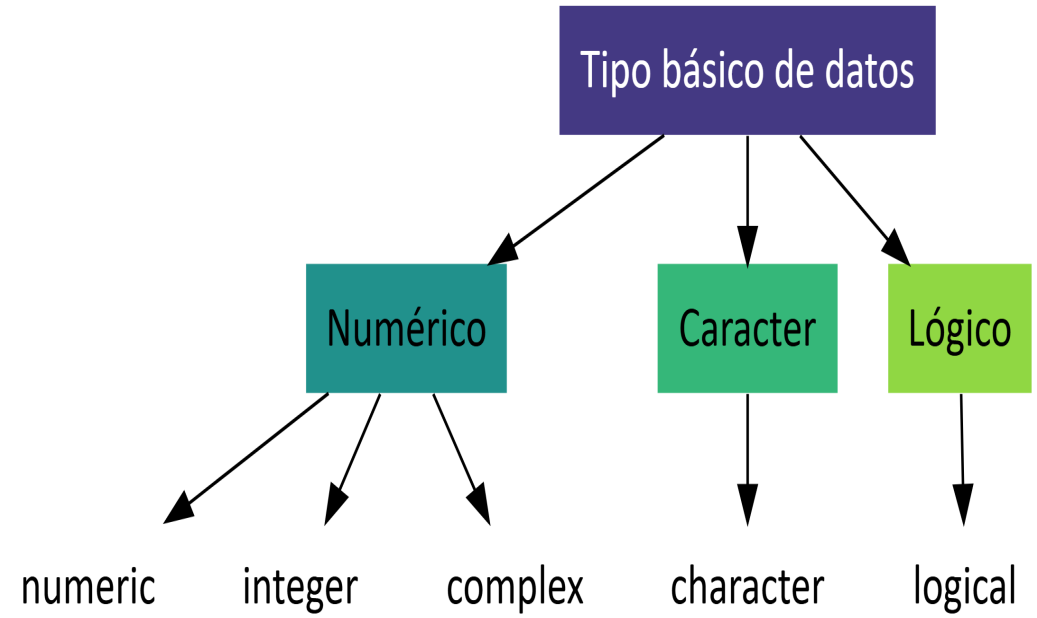
```
[1] 1
```

Observemos que, además de devolvernos el valor, aparece delante un número entre corchetes `[1]`. Este número es la ubicación o **índice** del comienzo del objeto. En este caso, como el vector tiene una sola posición, indica que el primer valor mostrado empieza en la posición 1.

1.10. Estructuras de datos

Los objetos contenedores de datos más simples pertenecen a cinco clases atómicas, que son:

- `integer` (números enteros)
- `numeric` (números reales)
- `complex` (números complejos)
- `character` (cadenas de caracteres)
- `logical` (valores lógicos: `TRUE` o `FALSE`)



Sin embargo, cada una de estas clases de datos no se encuentran de manera aislada, sino encapsuladas dentro de la clase de objeto más básica de R, a la que se denomina **vector**.

En RStudio, el sistema de colores ayuda a distinguir los tipos de valores:

```
# Número
1111
```

```
[1] 1111
```

```
# Caracter
"esto es una cadena de texto"
```

```
[1] "esto es una cadena de texto"
```

```
# Lógico
TRUE
```

```
[1] TRUE
```

```
# Objeto
a <- 1
```

1.10.1. Vectores

Un vector es un conjunto de valores (números o símbolos), del mismo tipo ordenados de la forma (elemento 1, elemento 2, … , elemento n), donde n es la longitud o tamaño del vector.

Los atributos principales son:

- **Tipo**: puede ser `integer`, `numeric`, `character`, `complex` o `logical`.
- **Longitud**: cantidad de elementos que contiene el objeto,

El vector más simple contiene un solo dato, por ejemplo un vector de longitud 1 y tipo `numeric`:

```
vec1 <- 1
vec1
```

```
[1] 1
```

Otro vector más grande por ejemplo podría ser `(1, 5, 2)`. En este caso también es del tipo `numeric` pero tiene una longitud de 3 elementos:

```
vec2 <- c(1, 5, 2)
vec2
```

```
[1] 1 5 2
```

Para concatenar elementos usamos la función `c()`, dentro de la cual van los valores separados por comas. El orden de los elementos importa, en nuestro ejemplo la primera posición la ocupa el 1, la segunda el 5 y la tercera el 2. Si tuvieramos otro vector `(5, 1, 2)`, no sería lo mismo porque los valores están ordenados de forma diferente.

Para conocer la longitud del vector usamos:

```
length(vec2)
```

```
[1] 3
```

Nos informa que `vec2` tiene 3 elementos.

Para conocer el tipo de dato ejecutamos:

```
class(vec2)
```

```
[1] "numeric"
```

Podemos ver que los datos almacenados en este segundo ejemplo cumplen con la definición en lo que respecta al tipo de dato, ya que cada elemento es del mismo tipo (`numeric`).

Veamos un ejemplo de asignación de otro tipo de dato atómico, como es el `character`:

```
vec3 <- "Hola"
vec3
```

```
[1] "Hola"
```

Siempre que escribamos contenido de tipo `character` debemos hacerlo entre comillas. En este caso generamos el vector `vec3` con el contenido `"Hola"`, que, a pesar de ser una palabra compuesta de varios caracteres, dentro del vector `vec3` esta ocupa una sola posición:

```
length(vec3)
```

```
[1] 1
```

Respecto al tipo de dato si usamos la función `class()` tendremos:

```
class(vec3)
```

```
[1] "character"
```

1.10.2. Factores

Un factor es un objeto especialmente diseñado para contener datos categóricos y se asocia particularmente con las *variables cualitativas*.

En su estructura interna está compuesto por dos vectores:

- Un vector de índices enteros.
- Un vector de categorías (niveles) de tipo `character`, a los que hace referencia el primer vector.

Existen de dos tipos: factores nominales y factores ordinales. En el caso del segundo se establece un orden en los niveles.

Normalmente, obtenemos un tipo `factor` de convertir un vector u otro tipo de objeto con caracteres, pero para mostrar un ejemplo lo realizamos con la función `factor()`:

```
factor1 <- factor(
  x = c("a", "b", "a", "c", "b", "a"),
  levels = c("a", "b", "c")
)
factor1
```

```
[1] a b a c b a
Levels: a b c
```

Creamos el objeto `factor1` con 6 elementos caracteres y tres niveles sin orden.

Además de la practicidad de trabajar con factores, muchas funciones de R requieren que las variables categóricas estén en formato factor para funcionar correctamente.

1.10.3. *Dataframe*

Un *dataframe* es un objeto diseñado para contener conjuntos de datos y representa una estructura bidimensional similar a una tabla, con filas y columnas. Cada columna puede almacenar elementos de distintos tipos (por ejemplo, numéricos, de texto o lógicos), siempre que todos tengan la misma longitud.

Las columnas suelen tener nombres únicos, lo que permite referenciarlas fácilmente como si fueran variables individuales dentro del conjunto de datos.

Este es el tipo de objeto que se utiliza habitualmente para almacenar información importada desde archivos externos (por ejemplo, archivos de texto separados por comas o planillas de Excel), y con el que más frecuentemente trabajamos en los análisis.

Desde el punto de vista estructural, un *dataframe* está compuesto por una serie de vectores de igual longitud dispuestos verticalmente, uno al lado del otro, conformando las columnas de la tabla.

Veamos un ejemplo:

```
# Historia clínica
HC <- c("F324", "G21", "G34", "F231")

# Edad
edad <- c(34, 32, 34, 54)

# Sexo
sexo <- c("M", "V", "V", "M")

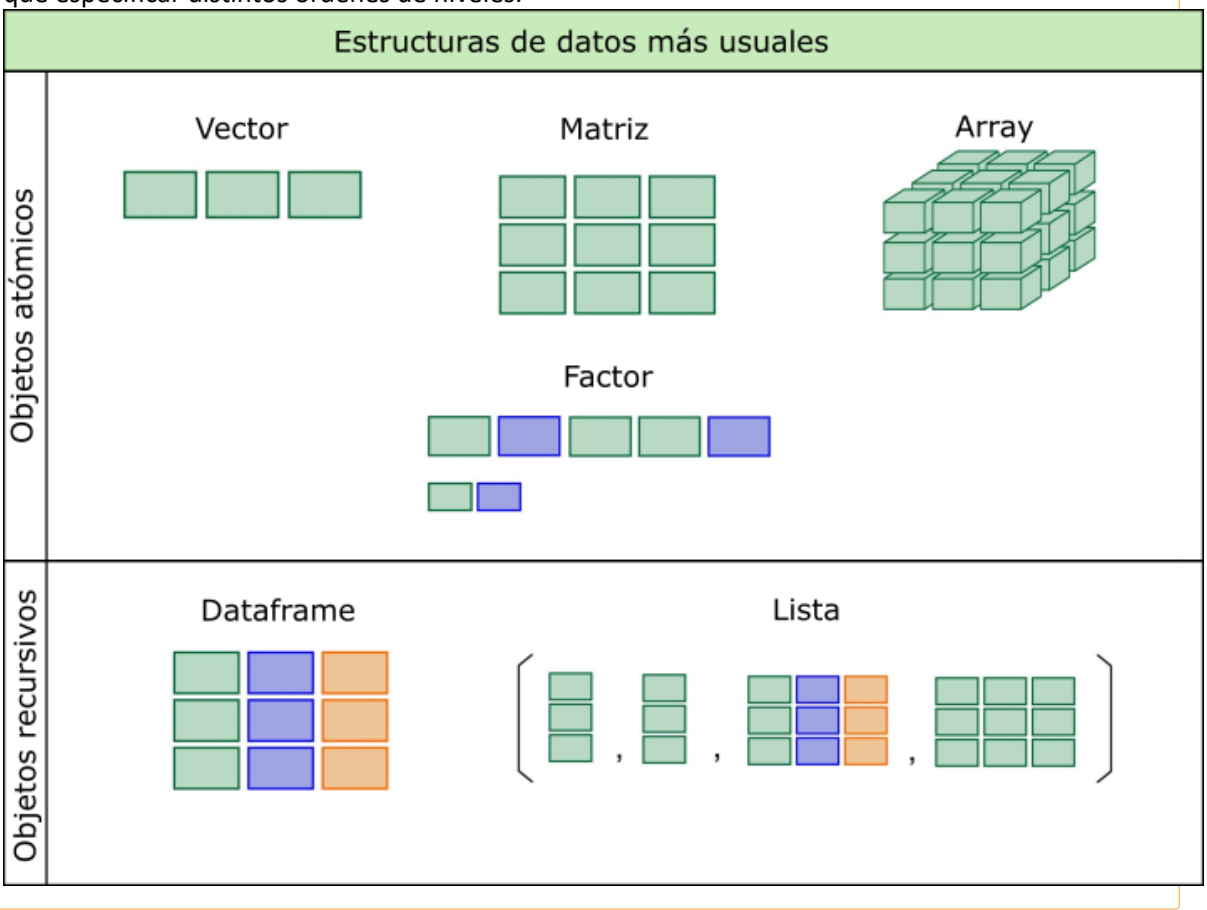
# dataframe
df1 <- data.frame(HC, edad, sexo)

df1
```

```
    HC edad sexo
1 F324   34    M
2  G21   32    V
3  G34   34    V
4 F231   54    M
```

Creamos tres vectores con datos de supuestos individuos, su historia clinica, la edad y el sexo. Luego mediante la función `data.frame()` "unimos" esos vectores en forma vertical para formar un *dataframe* de 3 variables y 4 observaciones.

⚠ Existen otras estructuras de datos que aparecen en la siguiente figura. Las más habituales en nuestro trabajo son los vectores y los *dataframes*. Los factores serán necesarios cuando tengamos que especificar distintos órdenes de niveles.



1.11. Operadores

Además de funciones, el lenguaje R cuenta con operadores de uso relativamente intuitivo, que permiten realizar operaciones de diferentes tipos con los objetos que contienen datos.

1.11.1. Operadores aritméticos

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División
^	Potencia
%%	Módulo

Operador	Descripción
%/%	División de enteros

Los operadores aritméticos se utilizan como si el lenguaje fuese una calculadora:

```
# Suma
2 + 5
```

```
[1] 7
```

```
# Resta
3 - 2
```

```
[1] 1
```

```
# Multiplicación
9 * 3
```

```
[1] 27
```

```
# División
10 / 2
```

```
[1] 5
```

```
# Potencia
5^2
```

```
[1] 25
```

También se pueden hacer operaciones con los objetos que almacenan valores numéricos:

```
a <- 3

b <- 6

(a + b) * b
```

```
[1] 54
```

Y funciona con objetos con más de un elemento, aplicando aritmética vectorial, donde las operaciones se realizan elemento a elemento:

```r
a <- c(1, 2, 3)

a * 3
```

```
[1] 3 6 9
```

O bien, con operaciones entre los objetos, donde se realiza entre los elementos de la misma posición:

```r
a <- c(1, 2, 3)

a * a
```

```
[1] 1 4 9
```

1.11.2. Operadores relacionales

Operador	Descripción
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que
==	Igual que
!=	No igual que

Habitualmente estos operadores se utilizan asiduamente en expresiones para indicar relaciones entre valores.

Podemos ver su funcionamiento en el ejemplo siguiente:

```r
a <- c(3, 8, 2)

a == c(3, 4, 5)
```

```
[1]  TRUE FALSE FALSE
```

El lenguaje evalúa las comparaciones que hace el operador relacional igual (en este caso) y en aquellos valores que coinciden devuelve `TRUE` y en los que no hay coincidencia devuelve `FALSE`.

Lo mismo sucede con los otros operadores relacionales:

```r
a <- c(4, 8, 10)

a > 8
```

```
[1] FALSE FALSE  TRUE
```

```
a < 8
```

```
[1]  TRUE FALSE FALSE
```

```
a != 8
```

```
[1]  TRUE FALSE  TRUE
```

1.11.3. Operadores lógicos

Operador	Descripción
!	NOT
&	AND booleano
&&	AND booleano para vectores de longitud 1
|	OR booleano
||	OR booleano para vectores de longitud 1

Cuando queremos conectar algunas de las expresiones relacionales hacemos uso de estos operadores lógicos típicos (AND, OR, NOT).

Para ejemplificar podemos hacer:

```
a <- c(1:8)

a
```

```
[1] 1 2 3 4 5 6 7 8
```

```
(a > 3) & (a < 7)
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE  TRUE FALSE FALSE
```

En el caso anterior usamos el operador & como conector AND de dos expresiones relacionales donde el lenguaje devuelve `TRUE` en el rango mayor a 3 y menor a 7 (valores 4,5 y 6).

1.12. Valores especiales

Existen algunos valores especiales para datos con expresiones reservadas en R, entre ellos encontramos los valores `NA`, `NaN`, `Inf` y `NULL`.

Operador	Significado	Descripción
NA	Not available	Es la forma de expresar a los valores perdidos o faltantes (missing values)

Operador	Significado	Descripción
NaN	Not a number	Utilizado para resultados de operaciones que devuelven error numérico
Inf	Infinity	Valor infinito (positivo)
-Inf	Infinity	Valor infinito (negativo)
NULL	Null	Valor nulo

El más relevante de estos valores especiales es el `NA` que sirve para indicar que no hay valor en esa posición o elemento de un objeto.

1.13. Secuencias regulares

Además de concatenar elementos con la función `c()`, existen tres formas comunes de generar secuencias regulares.

La primera es mediante un **operador secuencial** (`:`), que genera una secuencia de enteros entre dos valores, ya sea en forma ascendente o descendente:

```
1:10
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
10:1
```

```
 [1] 10  9  8  7  6  5  4  3  2  1
```

La segunda forma es mediante la función `seq()`, cuyos argumentos principales son `from`, `to` y `by`. Esta función permite mayor flexibilidad:

```
seq(from = 1, to = 20, by = 2)
```

```
 [1]  1  3  5  7  9 11 13 15 17 19
```

El ejemplo anterior genera una secuencia de números que comienza en 1 y llega hasta 20, avanzando de dos en dos.

Algunos otros ejemplos de la misma función:

```
seq(from = 0.1, to = 0.9, by = 0.1)
```

```
[1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

```
seq(from = -5, to = 5, by = 1)
```

```
 [1] -5 -4 -3 -2 -1  0  1  2  3  4  5
```

```
seq(from = 300, to = 0, by = -50)
```

```
[1] 300 250 200 150 100  50   0
```

La tercera opción es la función `rep()`, que permite duplicar valores. Su forma más básica es `rep(x, times = n)`, donde se repite el valor `x` la cantidad de veces indicada por `n`.

Algunos ejemplos:

```
rep(x = 2, times = 5)
```

```
[1] 2 2 2 2 2
```

```
# combinada con el operador :
rep(1:4, 5)
```

```
 [1] 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4
```

```
# combinada con la función c()
rep(c(4.5, 6.8, 7.2), 2)
```

```
[1] 4.5 6.8 7.2 4.5 6.8 7.2
```

1.14. Índices

R implementa una forma eficiente y flexible de acceder selectivamente a elementos de un objeto basado en una indexación interna.

Para acceder a un índice se utiliza la notación de corchetes. Por ejemplo, si tenemos un vector `x` con varios elementos y queremos acceder al segundo, usamos `x[2]`.

Esta forma de indexar los elementos de los distintos objetos nos permite realizar muchas operaciones desde el simple llamado, hasta seleccionar, eliminar o modificar valores.

Veamos algunos ejemplos:

```
# generamos un vector x con 5 letras
x <- c("a", "b", "c", "d", "e")

# llamamos a la primer posición y nos devuelve su valor
x[1]
```

```
[1] "a"
```

```
# llamamos a la tercer posición y nos devuelve su valor
x[3]
```

```
[1] "c"
```

```
# llamamos a las posiciones 1 y 3 juntas mediante c()
x[c(1, 3)]
```

```
[1] "a" "c"
```

```
# llamamos a las posiciones menos la 1 y la 4
x[-c(1, 4)]
```

```
[1] "b" "c" "e"
```

```
# creamos otro vector y con los valores 1,2 y 5
y <- c(1, 2, 5)

# utilizamos el vector y como índice
x[y]
```

```
[1] "a" "b" "e"
```

```
# asignamos el valor "h" a la posición 2 del vector x
x[2] <- "h"

x
```

```
[1] "a" "h" "c" "d" "e"
```

```
# creamos el vector z eliminando la posición 5 de x
z <- x[-5]

z
```

```
[1] "a" "h" "c" "d"
```

Estos ejemplos muestran las múltiples posibilidades que ofrece el uso de índices para manipular objetos en R, lo que hace al lenguaje muy poderoso y versátil.

Cuando se aplican índices a estructuras bidimensionales, como matrices o *dataframes*, la notación general es:

```
nombre[índice de fila, índice de columna]
```

1.15. Gestión de factores

Al presentar las distintas estructuras de datos mencionamos que los factores suelen generarse a partir de vectores u otros objetos de tipo carácter.

Para entender cómo funcionan, partamos de un vector con datos categóricos:

```
respuesta <- c("Si", "No", "No", "Si", "Si", "Si", "No")
```

Creamos un vector llamado `respuesta` con 7 elementos de tipo carácter, en el que se repiten las categorías `"Si"` y `"No"`.

Podemos confirmar que se trata de un vector y que su contenido es de tipo carácter:

```
# preguntamos si sexo es un vector
is.vector(respuesta)
```

```
[1] TRUE
```

```
# visualizamos el tipo de dato de sexo
class(respuesta)
```

```
[1] "character"
```

Para crear un factor a partir de este vector debemos utilizar la función `factor()`:

```
respuesta <- factor(respuesta)

respuesta
```

```
[1] Si No No Si Si Si No
Levels: No Si
```

En la salida observamos los siete elementos y, debajo, una línea con los niveles del factor, indicados por `Levels`. Estos niveles son identificados automáticamente.

Podemos verificar cómo cambió el tipo de objeto:

```
# preguntamos si respuesta es un vector
is.vector(respuesta)
```

```
[1] FALSE
```

```
# preguntamos si respuesta es un factor
is.factor(respuesta)
```

```
[1] TRUE
```

```
# visualizamos el tipo de dato de respuesta
class(respuesta)
```

```
[1] "factor"
```

Aunque visualmente vemos palabras, internamente R trata al factor como un tipo especial basado en números. ¿Por qué sucede esto? Veámoslo con más detalle:

```
str(respuesta)
```

```
 Factor w/ 2 levels "No","Si": 2 1 1 2 2 2 1
```

La función `str()` devuelve la estructura interna del objeto. En este caso, muestra que `respuesta` tiene dos niveles y que cada elemento del vector es representado por un número (1 o 2), donde `1` corresponde a `"No"` y 2 a `"Si"`.

Esto significa que la estructura de los factores está compuesta por dos vectores: uno numérico que funciona como índice de enteros, que sustituye al vector de caracteres original, y el otro es un vector de caracteres, que contiene los niveles o categorías, a los que hace referencia el primer vector.

Para ver solo los niveles o categorías del factor podemos usar:

```
levels(respuesta)
```

```
[1] "No" "Si"
```

En los factores nominales donde no importa el orden, la función `factor()` implementa el orden alfabético para determinar a qué índice numérico pertenece cada categoría. Es claro en el ejemplo que a `No` le asigna el `1` y a `Si` el 2.

Pero si nos encontramos frente a una variable cualitativa ordinal vamos a necesitar indicarle a la función cual es el orden de las categorías.

Vamos al siguiente ejemplo:

```
salud <- c(4, 3, 1, 3, 2, 2, 3, 3, 1)
salud
```

```
[1] 4 3 1 3 2 2 3 3 1
```

Tenemos en el vector salud algunos códigos numéricos que representan nivel de salud de personas registradas por una encuesta donde `1` significa mala salud, 2 regular, 3 buena y 4 muy buena.

Procedemos a crear el factor `nivsalud` a partir de este vector:

```
nivsalud <- factor(
  salud,
  label = c("Mala", "Regular", "Buena", "Muy buena"),
  levels = 1:4
)

nivsalud
```

```
[1] Muy buena Buena     Mala      Buena     Regular   Regular   Buena
[8] Buena     Mala
Levels: Mala Regular Buena Muy buena
```

Aquí utilizamos dos argumentos adicionales:

- `labels`: define las etiquetas que queremos mostrar para cada categoría.
- `levels`: indica el orden de los niveles originales (en este caso, 1 a 4).

Aquí es necesario definir estos argumentos porque, a diferencia del factor `respuesta`, el vector `salud` contiene números en lugar de las palabras correspondientes a las categorías.

Hasta aquí hemos creado un factor pero si miramos sus niveles no encontraremos señales que sigan un orden específico:

```
levels(nivsalud)
```

```
[1] "Mala"      "Regular"   "Buena"     "Muy buena"
```

Sin embargo, aún no hemos definido un **orden** explícito entre las categorías. Para hacerlo, agregamos el argumento `ordered = TRUE`:

```
nivsalud <- factor(
  salud,
  label = c("Mala", "Regular", "Buena", "Muy buena"),
  levels = 1:4,
  ordered = TRUE
)

nivsalud
```

```
[1] Muy buena Buena     Mala      Buena     Regular   Regular   Buena
[8] Buena     Mala
Levels: Mala < Regular < Buena < Muy buena
```

Al incorporar este argumento, estamos diciendo que los niveles tienen orden natural. Esto puede observarse en la salida donde se muestra que `"Mala"` < `"Regular"` < `"Buena"` < `"Muy buena"`.

1.16. Gestión de *dataframes*

En R, la estructura utilizada para almacenar datos provenientes de fuentes externas (como archivos .csv, planillas de cálculo, bases SQL, etc.) es el *dataframe*.

A modo de ejemplo, vamos a construir un *dataframe* llamado `datos` con 4 variables y 5 observaciones. Las variables serán:

- `id`: identificador numérico entero y correlativo.
- `edad`: edad en años.
- `sexo`: codificado como `"V"` para varón y `"M"` para mujer.
- `trabaja`: variable lógica, donde `TRUE` (`T`) representa que la persona trabaja y `FALSE` (`F`) que no lo hace.

```
# construimos el vector id
id <- 1:5

# construimos el vector edad
edad <- c(23, 43, 12, 65, 37)

# construimos el vector sexo
sexo <- c("V", "M", "M", "V", "M")

# construimos el vector trabaja
trabaja <- c(T, T, F, F, T)

# construimos el dataframe datos
datos <- data.frame(id, edad, sexo, trabaja)
```

Aunque para el ejemplo construimos un *dataframe* manualmente, lo habitual en la práctica es leer archivos externos que se importan directamente como *dataframes*.

Algunas de las funciones generales que podemos aplicar son:

```
# pedimos el número de columnas (variables)
ncol(datos)
```

```
[1] 4
```

```
# pedimos el número de filas (registros u observaciones)
nrow(datos)
```

```
[1] 5
```

```
# pedimos las dimensiones del dataframe (observaciones,variables)
dim(datos)
```

```
[1] 5 4
```

También podemos visualizar como se compone el objeto `datos` aplicando `str()` que devuelve la estructura interna de cualquier objeto en R.

```
str(datos)
```

```
'data.frame':   5 obs. of  4 variables:
 $ id     : int  1 2 3 4 5
 $ edad   : num  23 43 12 65 37
 $ sexo   : chr  "V" "M" "M" "V" ...
 $ trabaja: logi  TRUE TRUE FALSE FALSE TRUE
```

Esta salida nos informa que:

- `datos` es un *dataframe* con 5 observaciones y 4 variables.
- `id` es un entero (`int`).
- `edad` es numérica (`num`).
- `sexo` es de tipo carácter (`chr`).
- `trabaja` es lógica (`logi`).

Además, `str()` muestra los primeros valores de cada variable, que en este caso son todos, ya que el *dataframe* tiene solo 5 registros.

Cuando necesitemos llamar al contenido de alguna columna o variable del *dataframe*, utilizamos la siguiente notación:

```
nombre_del_dataframe$nombre_de_la_variable
```

Por ejemplo, si queremos mostrar el contenido de la variable `sexo` del objeto `datos` hacemos:

```
datos$sexo
```

```
[1] "V" "M" "M" "V" "M"
```

Esto devuelve todos los valores de la variable `sexo`.

También podemos acceder a los elementos del *dataframe* usando **indexación por filas y columnas**:

```
# pedimos la tercer variable (sexo)
datos[, 3]
```

```
[1] "V" "M" "M" "V" "M"
```

Observemos que las dos salidas son idénticas, dado que muestran todas las observaciones de la variable `sexo`, aunque la solicitud sea de manera diferente.

Algunos otros ejemplos de uso de índices:

```
# observación 1 de la variable 2
datos[1, 2]
```

```
[1] 23
```

```
# observación 4 de todas las variables
datos[4, ]
```

```
  id edad sexo trabaja
4  4   65    V   FALSE
```

```r
# observación 1,2 y 3 de la variable 3
datos[1:3, 3]
```

```
[1] "V" "M" "M"
```

```r
# observación 5 de las variables 1 y 4
datos[5, c(1, 4)]
```

```
  id trabaja
5  5    TRUE
```

Por último, vamos podemos mostrar y gestionar los nombres de las variables con la función `names()`:

```r
names(datos)
```

```
[1] "id"      "edad"    "sexo"    "trabaja"
```

1.17. Fórmulas

En R, las fórmulas se utilizan principalmente para describir modelos estadísticos, y las vamos a emplear a lo largo de toda la cursada.

Las fórmulas se escriben utilizando operadores como `~`, `+`, `*`, entre otros. Se usan para especificar modelos como regresiones, ANOVA, pruebas de hipótesis y, en algunos casos, también para generar ciertos tipos de gráficos.

Formula genérica

Aquí la formula está dada por el operador `~`, a la izquierda está la variable de respuesta o dependiente, a la derecha la o las variables explicativas o independientes. El esquema general es el siguiente:

```r
variable_dependiente ~ variable_independiente
```

El símbolo ~ (llamado virgulilla) puede interpretarse como *"es modelada por"* o *"en función de"*.

Por ejemplo, en una regresión lineal se utiliza la función base `lm()` y el argumento principal de esta función es una formula:

```r
regresion_lineal <- lm(variable_respuesta ~ variable_explicativa, data = datos)
```

Además de la fórmula, la función `lm()` incluye el argumento `data =`, que es fundamental: allí se le indica a R el *dataframe* en el que debe buscar las variables involucradas en el modelo.

La siguiente tabla muestra los usos de los los operadores más comunes dentro de una formula:

Operador	Ejemplo	Descripción
+	+x	Incluye la variable x
-	-x	Excluye la variable x
:	x : z	Incluye la interacción de la variable x con z
*	x * z	Incluye ambas variables y la interacción entre ellas

2. Introducción a RStudio

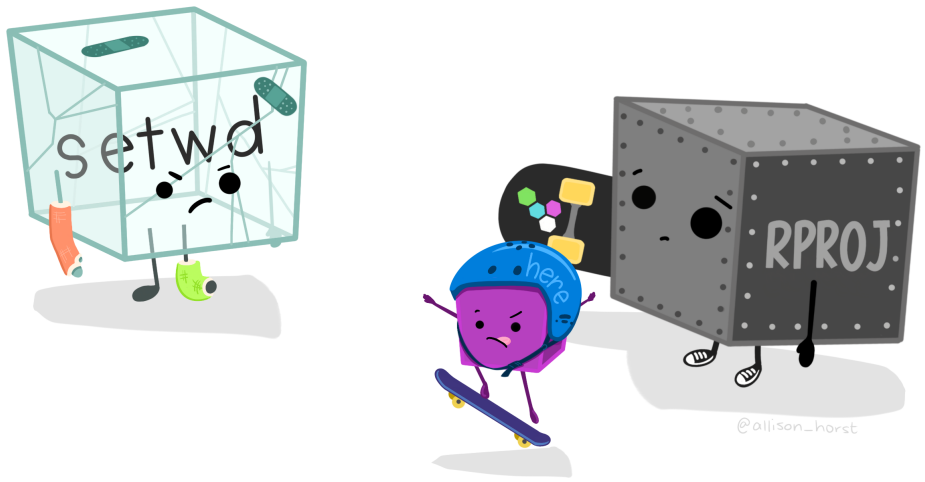


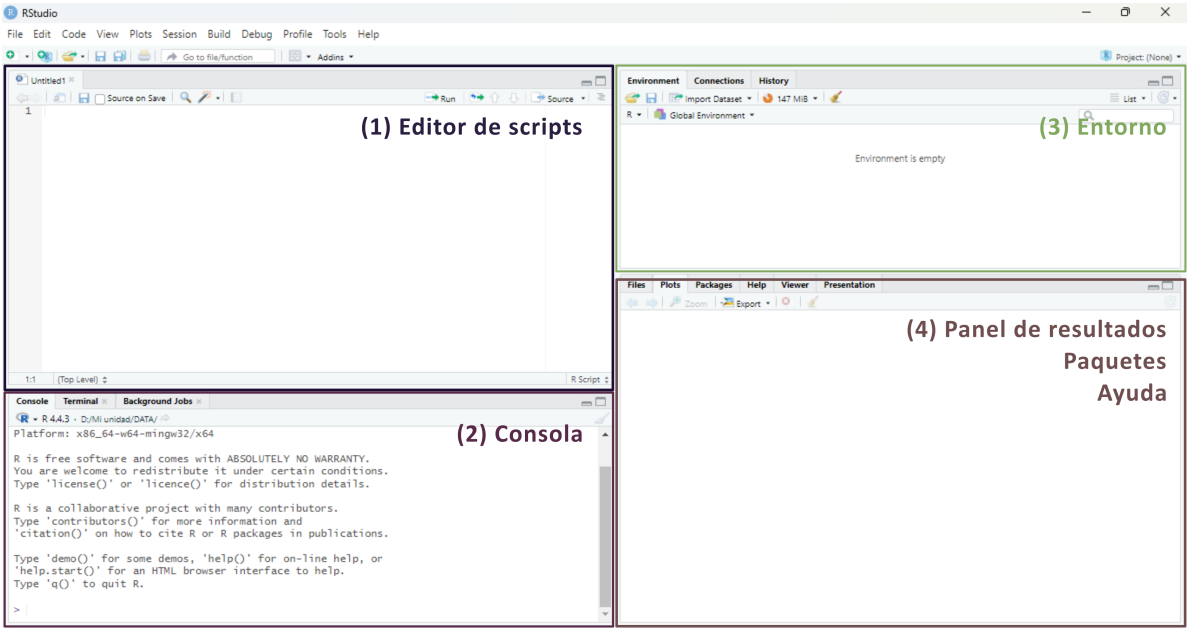
Figura 2.1.: Artwork por @allison_horst

2.1. Primeros pasos y generalidades

RStudio Desktop (2025, Posit Software) es un **entorno de desarrollo integrado** (*IDE*, por sus siglas en inglés) diseñado específicamente para trabajar con el lenguaje R.

Es una herramienta **multiplataforma y de código abierto** que facilita la programación, el análisis de datos y la elaboración de informes científicos. Ofrece una integración fluida con otros componentes del ecosistema de R, como R Markdown, Quarto, control de versiones (por ejemplo, Git) y la gestión de proyectos.

RStudio presenta una interfaz unificada compuesta por distintos paneles, lo que permite organizar el trabajo de forma clara y eficiente:



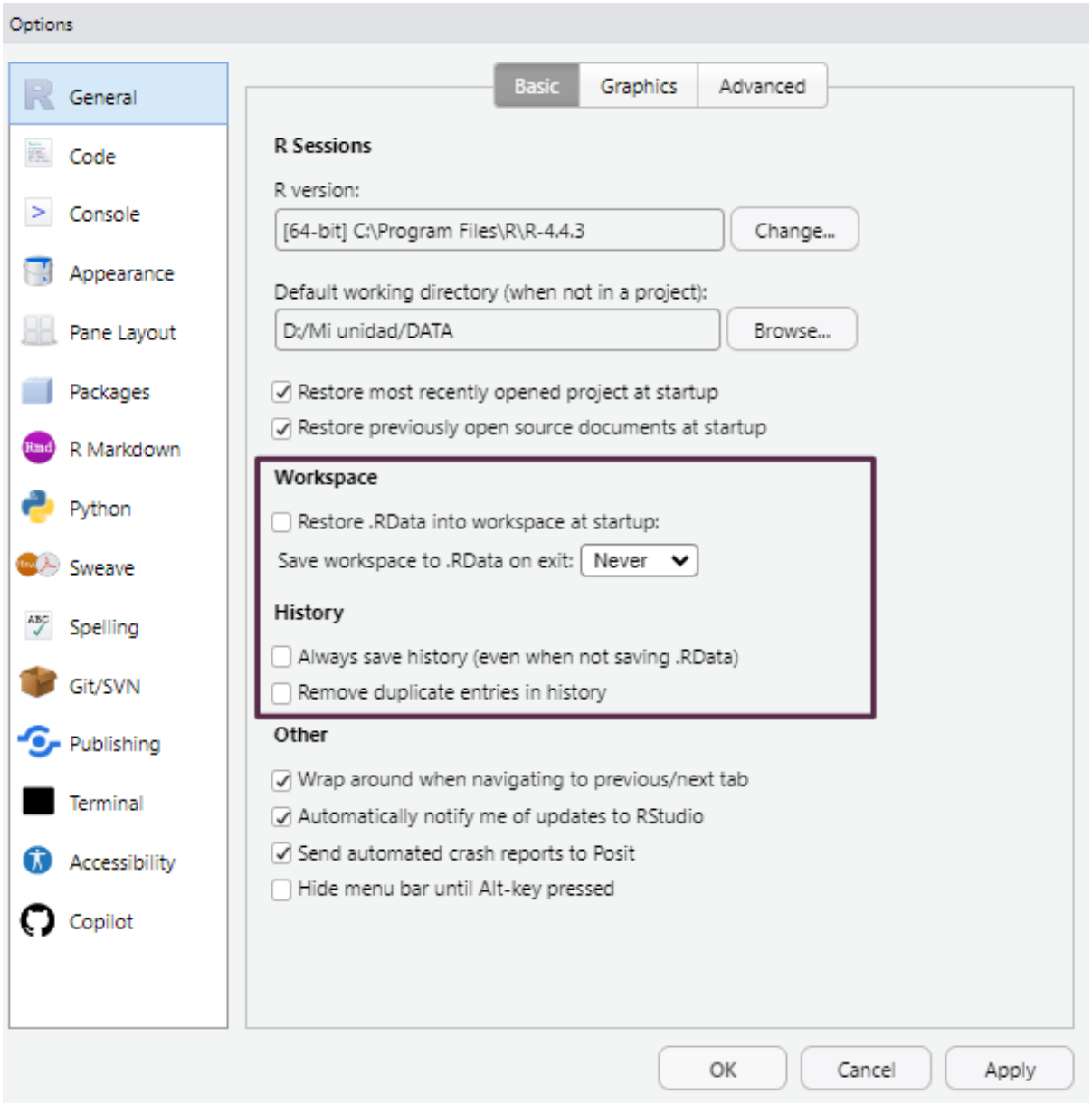
1. **Editor de scripts** (*Source*): permite crear y editar scripts de R, así como documentos R Markdown o Quarto.

2. **Consola de R** (*Console*): muestra la salida del código ejecutado y ejecuta código de R de forma inmediata.

3. **Entorno** (*Environment*): muestra los objetos creados durante la sesión, como vectores, *dataframes* o funciones.

4. **Panel de resultados** (*Output*): presenta los gráficos, tablas, visualizaciones en HTML y también incluye un explorador de archivos, visor de paquetes instalados y panel de ayuda.

Para garantizar la reproducibilidad de los resultados, es recomendable evitar el guardado automático del historial de objetos entre sesiones, ya que puede generar confusión. Para desactivar esta opción, ir a `Tools > Global Options` y desmarcar las casillas de las opciones **Workspace** y **History** como se muestra en la siguiente imagen:



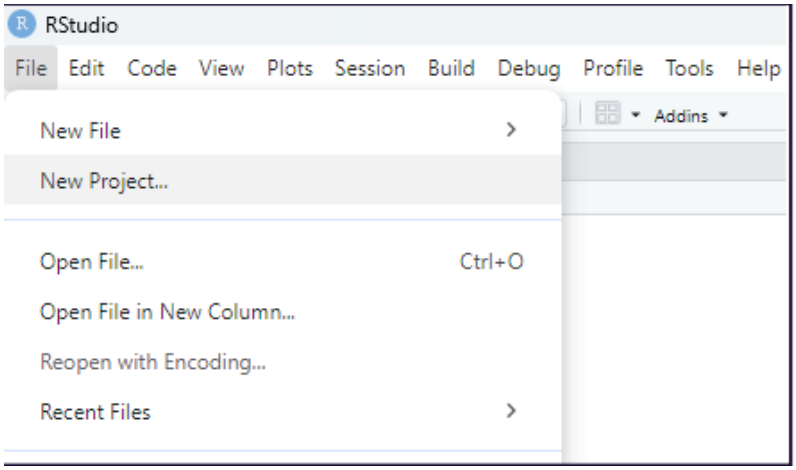
2.2. Proyectos de RStudio

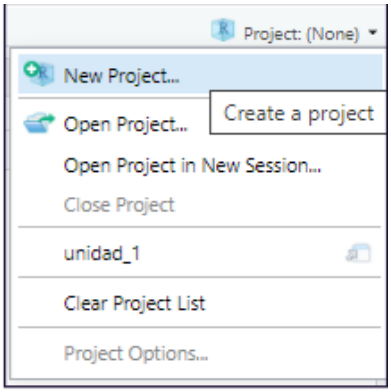
Los proyectos de RStudio permiten organizar de forma estructurada todo el material asociado a un análisis: scripts, informes, bases de datos, imágenes, etc. Cada proyecto se vincula a una carpeta específica del sistema de archivos, y RStudio la utiliza como directorio de trabajo por defecto. Esta carpeta puede estar ubicada en cualquier parte del sistema de almacenamiento que deseemos (disco rígido, pendrive, disco externo, etc).

Trabajar con proyectos facilita la importación de datos y evita errores relacionados con rutas relativas o absolutas.

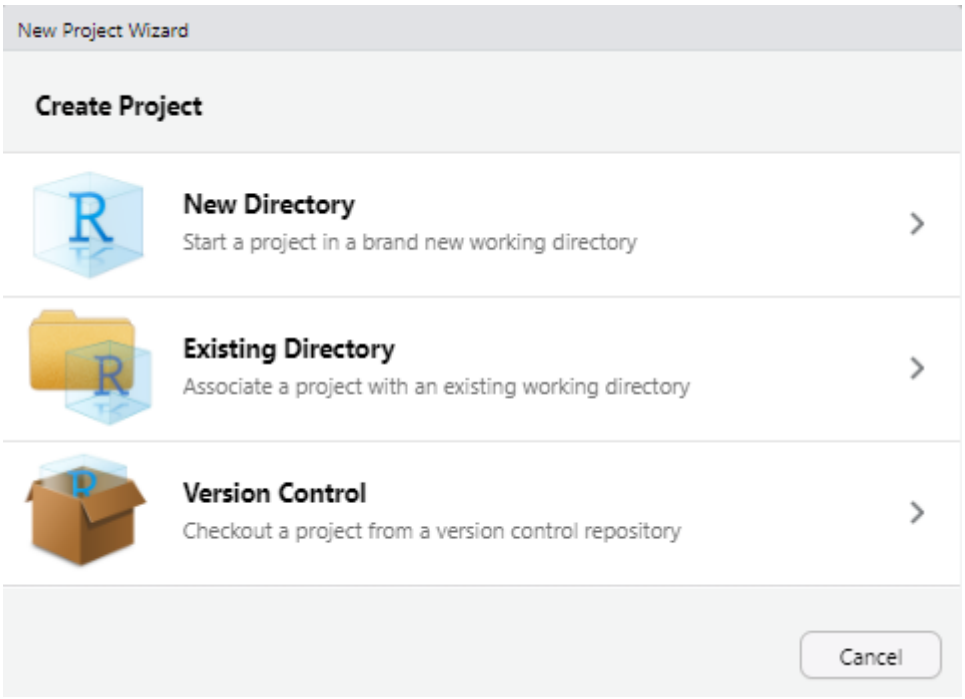
2.2.1. Crear un proyecto

Para crear un nuevo proyecto, se puede utilizar el menú `File > New Project...`. También accedemos a generar un proyecto nuevo a partir de pulsar el acceso directo `New Project...` ubicado en la esquina superior derecha de la interfaz:



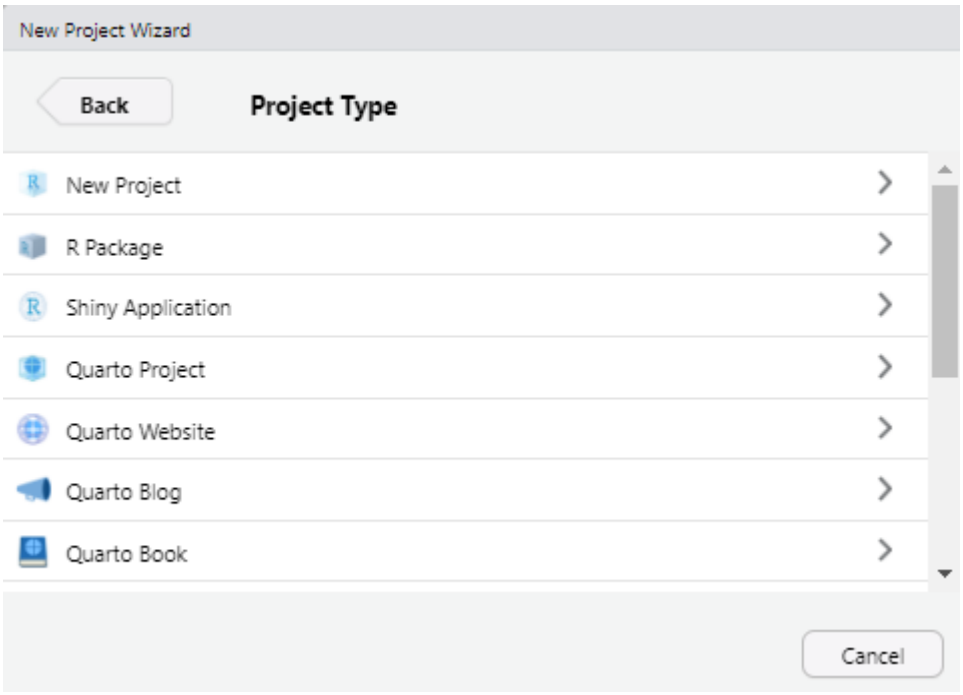


En cualquiera de los dos casos aparecerá un cuadro de diálogo que presenta tres opciones para crear el **nuevo proyecto** de RStudio:



- **New Directory**: crea una nueva carpeta para el proyecto a la que deberemos asignarle un nombre, es la opción más habitual. Todos los archivos de configuración aparecerán asociados a esta nueva carpeta.
- **Existing Directory**: vincula el proyecto a una *carpeta ya existente* que contenga archivos previos con los que deseamos trabajar.
- **Version Control**: permite clonar un repositorio (Git o SVN). Esta opción no se utilizará durante el curso.

Una vez que creamos un nuevo proyecto con la opción **New Directory**, aparecerá una pantalla con una lista de tipos de proyectos que se pueden crear en RStudio:

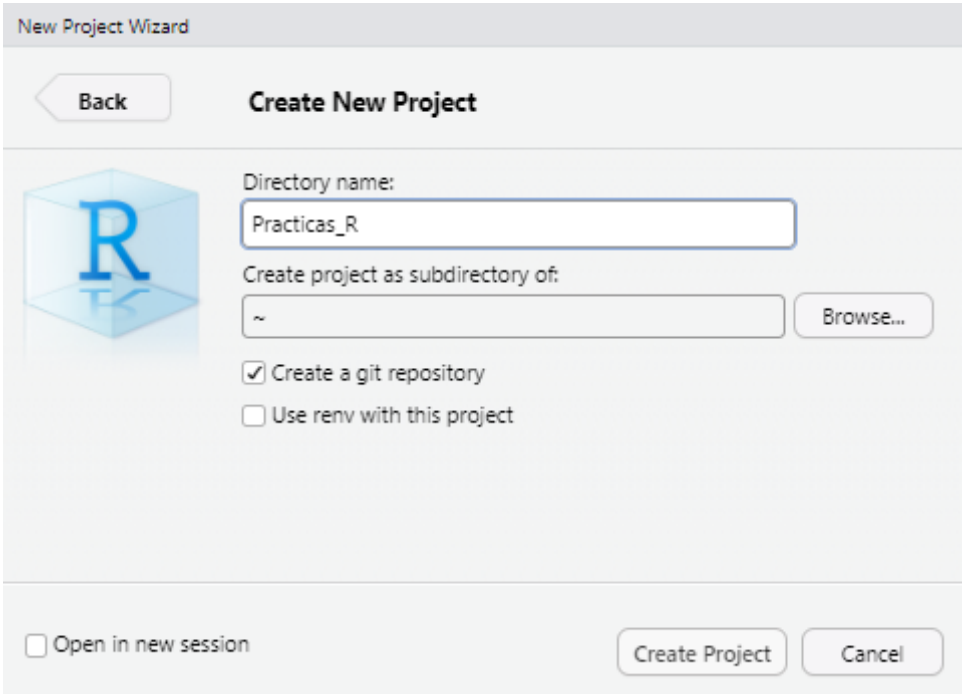


Durante este curso utilizaremos siempre la **primera opción** (**New Project**). Las demás opciones están pensadas para usos más específicos, como el desarrollo de paquetes de R, sitios web o documentos con Quarto o R Markdown.

Al seleccionar **New Project**, se abrirá una nueva ventana con los siguientes campos:

- **Directory name:** aquí debemos escribir el nombre del nuevo directorio (carpeta) que también será el nombre del proyecto. Por ejemplo, podemos llamarlo `Practicas_R`.

- **Create project as subdirectory of:** este campo permite definir en qué ubicación del sistema de archivos se guardará el proyecto. Podemos hacer clic en el botón **Browse…** para abrir el explorador de archivos y seleccionar la carpeta contenedora. En nuestro ejemplo, lo ubicaremos dentro de **Mis Documentos**.



Una vez completados estos campos, hacemos clic en **Create Project**.

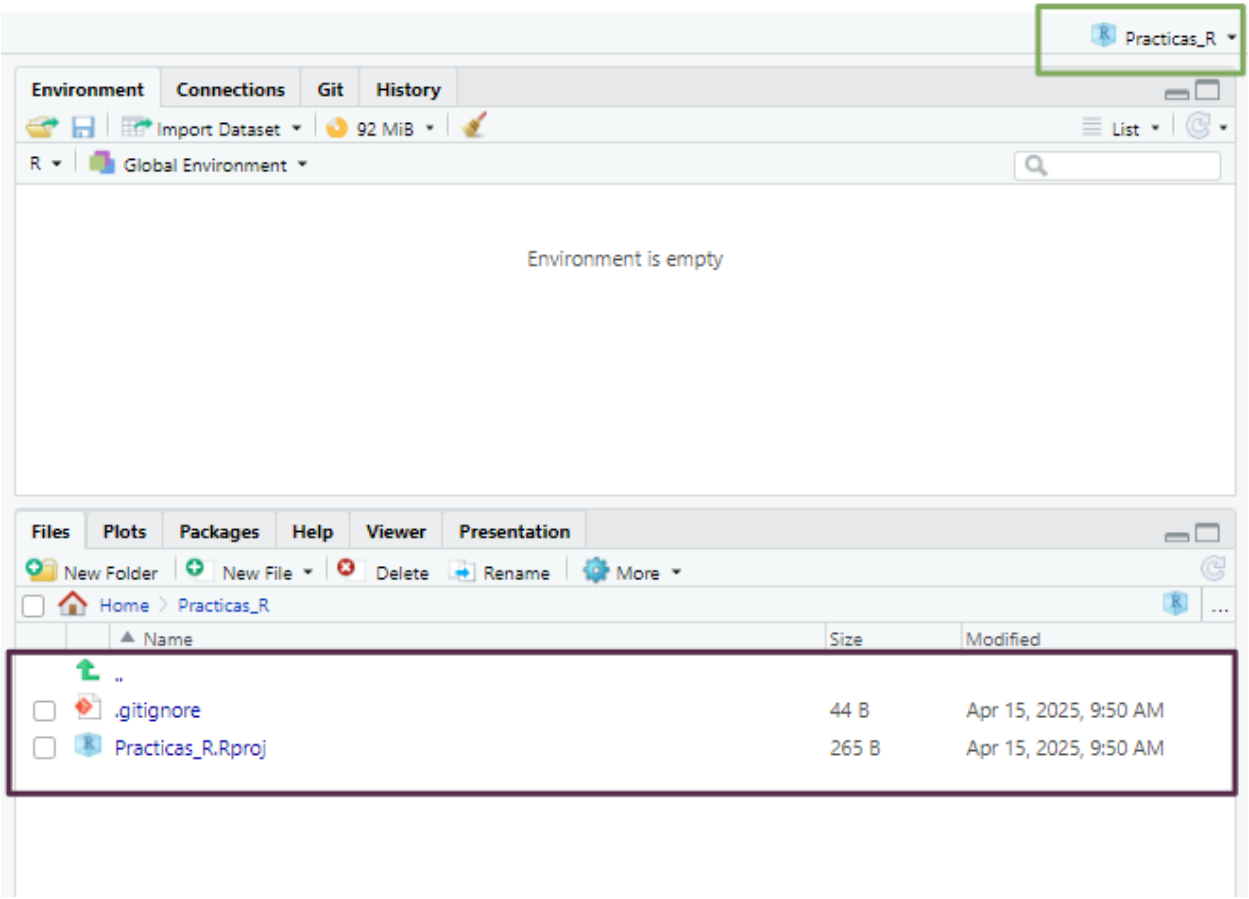
Los proyectos en RStudio tienen entornos independientes, lo que significa que si cerramos un proyecto o cambiamos a otro, la configuración de cada uno se mantendrá inalterable sin interferir con los demás.

Esto incluye los scripts abiertos, el directorio de trabajo, las pestañas que dejamos activas y otros elementos del entorno que puedan ser necesarios para continuar con un análisis. Este sistema permite mantener organizados los distintos trabajos que llevamos adelante.

Echemos un vistazo a lo que RStudio realizó:

- En primer lugar el panel **Files** (pantalla inferior derecha) apunta a la nueva carpeta **Practicas_R** y dentro de ella vemos un nuevo archivo el nombre del proyecto y la extensión `.Rproj`. Este archivo contiene todas las configuraciones para su proyecto.

- El otro cambio se observa en la parte superior derecha, que muestra el nombre del proyecto. Si hacemos click en él, se desplegará el menú de proyectos. Desde aquí se puede abrir y cerrar proyectos, navegar rápidamente a proyectos que se han abierto recientemente y configurar las opciones de RStudio para cada uno de ellos.



2.2.2. Abrir un proyecto existente

Cuando el proyecto ya existe, sea porque lo creamos nosotros o porque alguien nos pasó una carpeta con un proyecto de RStudio creado vamos a visualizar dentro de esa carpeta un archivo con extensión `.Rproj`.

La forma más veloz para abrir el proyecto es ejecutar este archivo (debería abrir una sesión de RStudio con el proyecto activo). La otra forma es desde el menú superior derecho de RStudio en la opción **Open project…** y luego buscando en nuestro directorio el mismo archivo `.Rproj`.

Nota

El menú de proyectos del área superior derecha va guardando como elementos recientes los proyectos que se van abriendo y también es una forma rápida de acceder a ellos pulsando sobre estos atajos.

2.3. Scripts en RStudio

Como vimos **anteriormente**, un script es un archivo de código que contiene una secuencia de instrucciones escritas en R. Estos scripts pueden ser reutilizados, modificados y compartidos, lo que los convierte en una herramienta fundamental para garantizar la reproducibilidad del trabajo.

2.3.1. Crear un nuevo script

Tenemos varias formas de crear un script nuevo:

- Desde el menú superior: `File > New File > R Script`
- Con el atajo de teclado: `Ctrl + Shift + N`
- Desde la barra de herramientas: presionando el ícono 

2.3.2. Ejecutar scripts

La forma habitual de ejecutar el contenido de un script es línea por línea, usando alguna de las siguientes opciones:

- Presionando el botón 

del editor de código de RStudio
- Mediante el atajo de teclado `Ctrl + Enter`

Para ejecutar una línea, simplemente ubicamos el cursor en cualquier parte de ella y presionamos el comando correspondiente. Luego de ejecutarse, el cursor avanzará automáticamente a la siguiente línea de código.

Mientras ejecutamos cada línea debemos ir observando la salida en la consola (panel inferior izquierdo) y también los cambios que se dan en el panel *Environment* (panel superior derecho) donde aparecerán los objetos que vayamos creando y manipulando.

2.3.3. Edición de scripts

Modificar o agregar líneas al script puede hacerse directamente en el editor. Cada vez que realizamos un cambio, es necesario **volver a ejecutar la línea** o bloque modificado para que los cambios se reflejen en el entorno de trabajo.

Podemos probar y modificar tantas veces como sea necesario. Sin embargo, debemos tener presente que cada manipulación en los objetos se mantiene hasta que se vuelvan a cambiar y a veces, cuando los objetos están vinculados con otras líneas de código posteriores tenemos que tener cuidado que se mantenga la consistencia del script.

Por ejemplo: si definimos un vector numérico para realizar cálculos matemáticos, pero luego lo sobrescribimos con un valor de tipo caracter, los cálculos posteriores producirán un error y RStudio nos informará de esto en la consola.

Por eso, es clave observar el contenido de los objetos en el panel **Environment**, lo que nos ayuda a evitar errores y operaciones incoherentes.

2.3.4. Guardado de scripts

Cualquier agregado o modificación que hayamos realizado al script y nos interese mantener nos obligará a guardar el archivo de código editado.

Existen distintas formas de guardar un script:

- Desde el menú superior: `File > Save`
- Con el atajo de teclado: `Ctrl + S`
- Presionando el ícono del disquete azul

Si en cambio quisiera guardarlo como otro archivo para mantener el script original, podemos guardarlo con diferente nombre o en otra ubicación mediante `File > Save As...`

2.3.5. Abrir scripts existentes

Los scripts que construyamos o nos compartan siempre tendrán extensión `.R` y, por lo general, se encontrarán dentro de un proyecto.

Para abrir estos archivos `.R` podemos:

- Desde el menú superior: `File > Open file...`
- Con el atajo de teclado: `Ctrl + O`
- Haciendo click sobre el archivo desde el panel *Files*
- Presionando el botón de la carpeta amarilla

Esto abrirá el script en una nueva pestaña dentro del editor de código.

2.3.6. ¿Cómo trabajaremos en este curso?

En general, utilizaremos scripts dentro de **proyectos de RStudio**. La secuencia recomendada será:

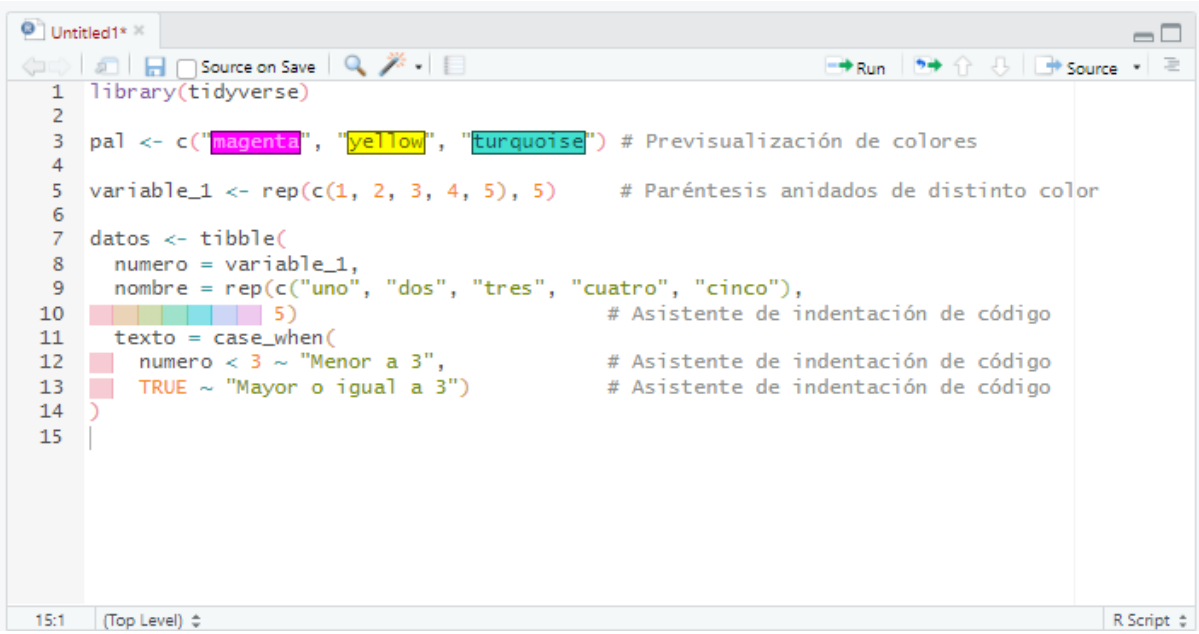
1. **Descargar** desde el Aula Virtual un archivo comprimido conteniendo la carpeta, el proyecto, los scripts y archivos de datos.
2. **Descomprimir** el archivo en la ubicación que deseamos (recomendamos crear una carpeta destinada al curso).
3. Abrir la carpeta y ejecutar el archivo de proyecto `.Rproj`.
4. Una vez abierto RStudio con el proyecto activo, ubicamos los scripts desde el panel **Files**.
5. **Ejecutar** cada línea del script, leyendo la documentación del código y observando la salida en la consola y los cambios en el entorno

2.4. Herramientas de RStudio

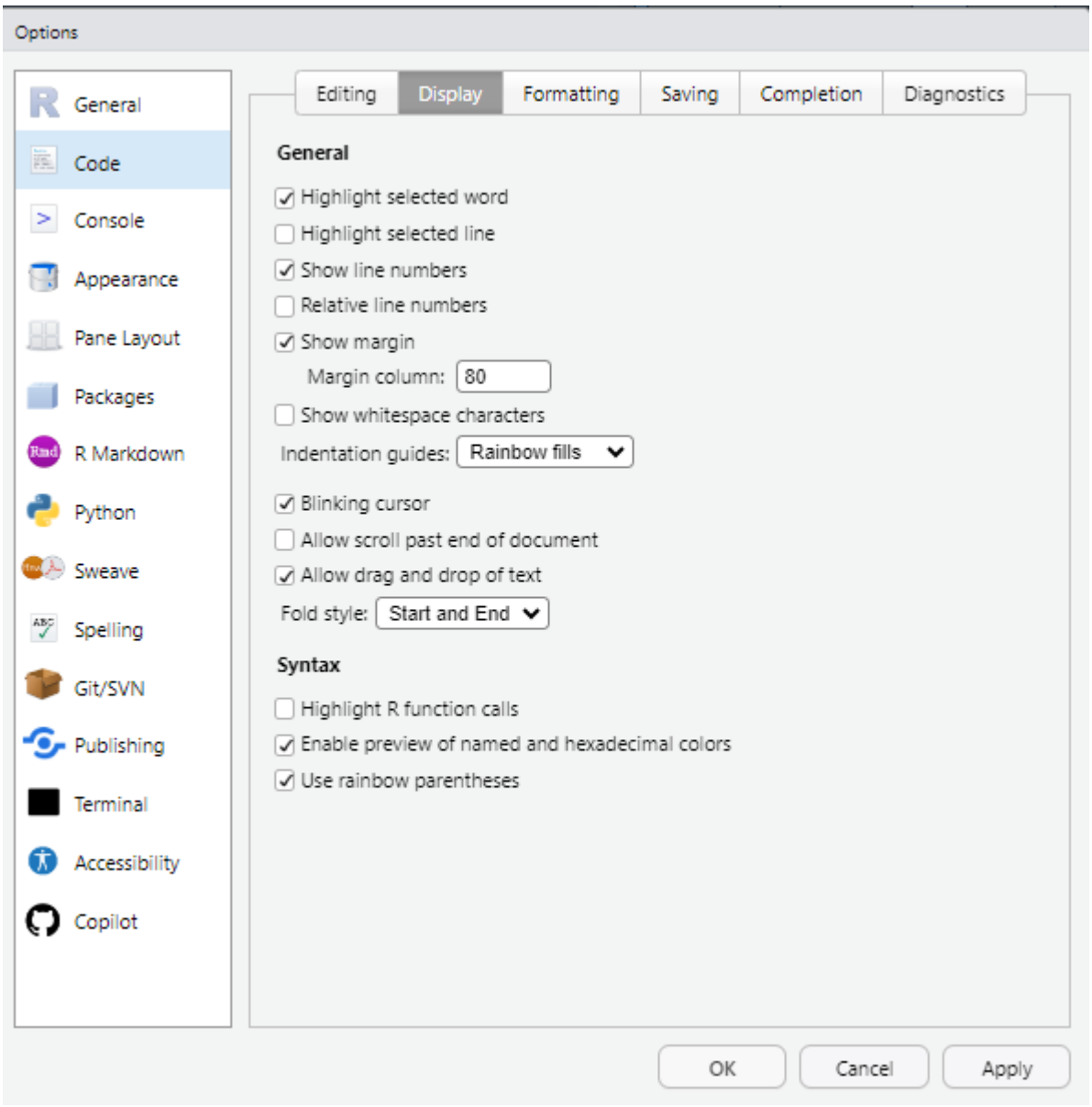
Algunas de las herramientas fundamentales de RStudio son el asistente de código, la ayuda en línea y el historial de comandos.

2.4.1. Asistente de código

Al escribir en el editor o la consola, la tecla `Tab` activa el autocompletado de funciones, nombres de objetos y argumentos, agilizando la escritura y reduciendo errores de sintaxis. En versiones recientes, el asistente también permite la previsualización de colores en los gráficos, resaltar los paréntesis de cierre en funciones anidadas con distintos colores y gestionar automáticamente la indentación del código.

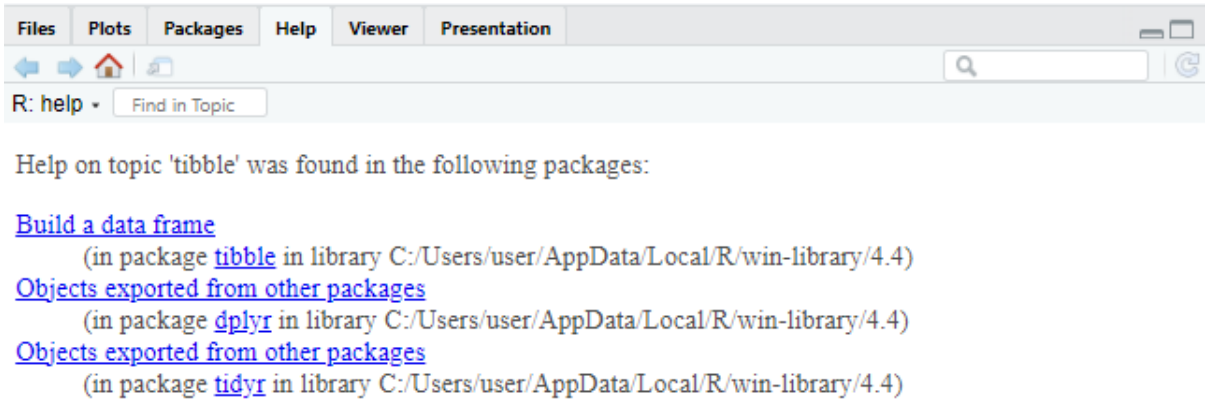


Muchas de estas opciones se pueden configurar desde el menú `Code` y desde `Tools > Global Options > Code`(pestañas **Editing** y **Display**).



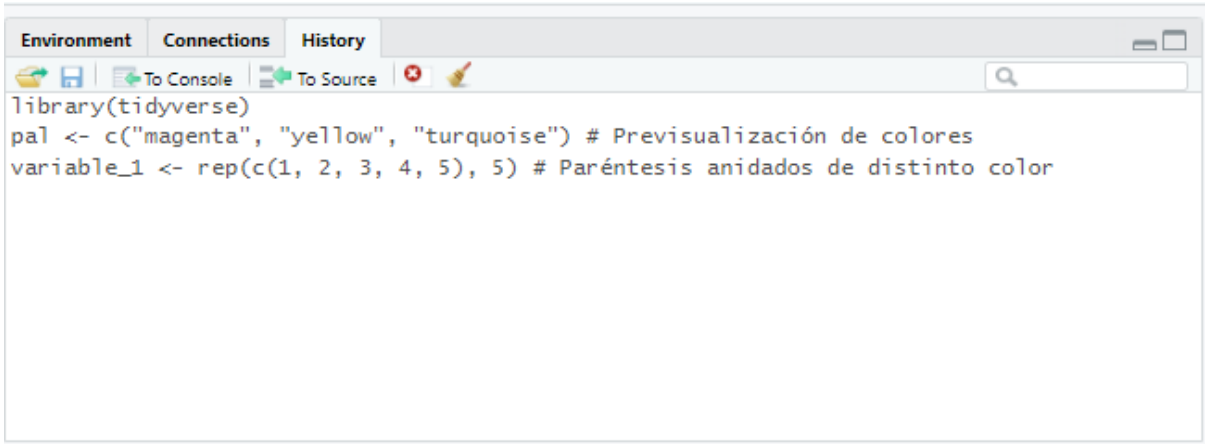
2.4.2. Ayuda en línea

Al posicionar el cursor sobre el nombre de una función en el editor y presionar `F1`, se accede directamente a la documentación correspondiente en el panel **Help** (habitualmente ubicado en la esquina inferior derecha).



2.4.3. Historial de comandos

En la consola, al usar las teclas de flecha arriba/abajo, se puede navegar por los comandos ejecutados durante la sesión actual. Además, el panel **History** (parte superior derecha) almacena los comandos de todas las sesiones previas, permitiendo reutilizarlos con un clic en **To Console** (`Enter`) o **To Source** (`Shift + Enter`), según se desee insertarlos en la consola o en el script activo.



2.5. Atajos de teclado (Windows)

Menú	Descripción
Archivo (File)	
Ctrl + Shift + N	Crea un nuevo script
Ctrl + O	Abre un script guardado
Ctrl + S	Guarda el script activo
Ctrl + W	Cierra el script activo
Ctrl + Q	Sale del programa RStudio
Edición (Edit)	
Ctrl + F	Abre la ventana de búsqueda (para buscar palabras dentro de un script)
Ctrl + L	Limpia la consola
Código (Code)	

Menú	Descripción
Ctrl + Enter	Ejecuta la línea de código donde está situado el cursor
Ctrl + Alt + R	Ejecuta todo el código del script activo
Ctrl + Shift + N	Inserta nueva sección de código
Ctrl + Shift + R	Inserta nueva sección de comentarios de texto
Sesión (Session)	
Ctrl + Shift + H	Abre la ventana para establecer directorio de trabajo
Ctrl + Shift + F10	Reinicia la sesión de R
Herramientas (Tools)	
Alt + Shift + K	Abre la lista de ayuda de atajos de teclado

2.6. Paquetes

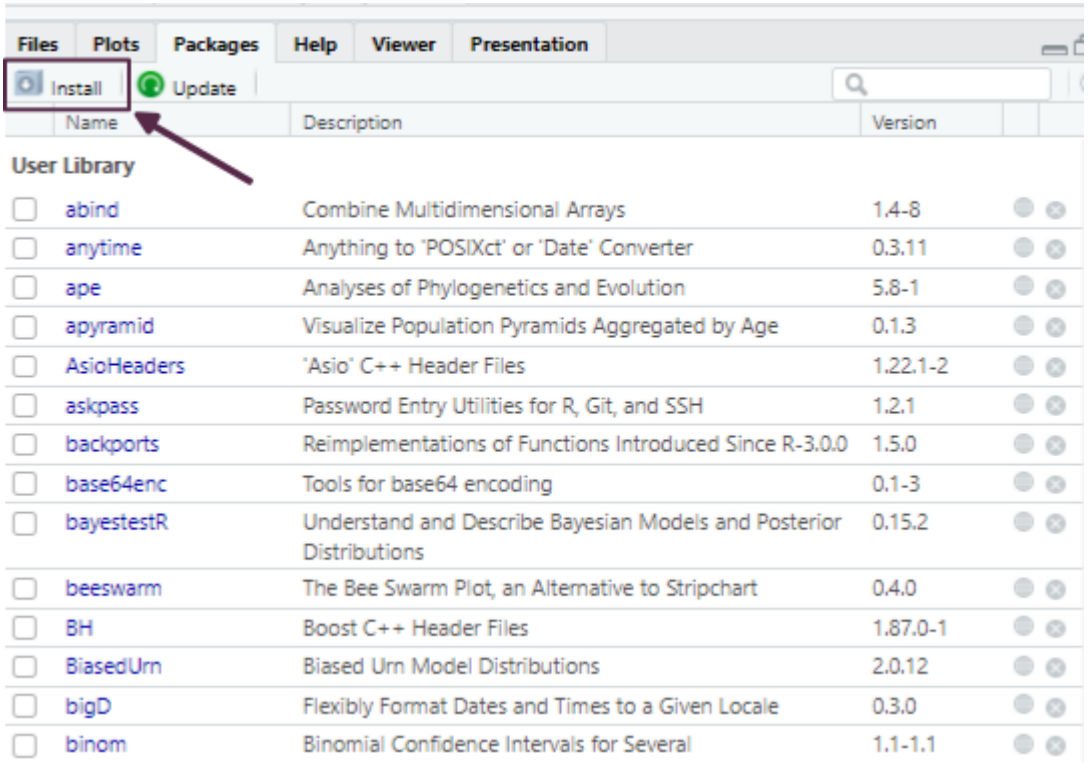
Existen dos formas principales de descargar paquetes: directamente desde RStudio o desde el **sitio web de CRAN**, descargándolos como archivos comprimidos. Si el equipo cuenta con conexión a Internet, lo más práctico es realizar la descarga directamente desde RStudio. En cambio, si no se dispone de acceso permanente a la red, es posible descargar los paquetes desde otro equipo y luego transferirlos como archivos `.zip` o `.tar.gz` al equipo donde se encuentra instalado R.

Cuando accedemos al sitio web de CRAN y buscamos un paquete específico, encontraremos información útil como una breve descripción del paquete, el número de versión, la fecha de publicación, el nombre del autor, documentación asociada y enlaces de descarga específicos para cada sistema operativo.

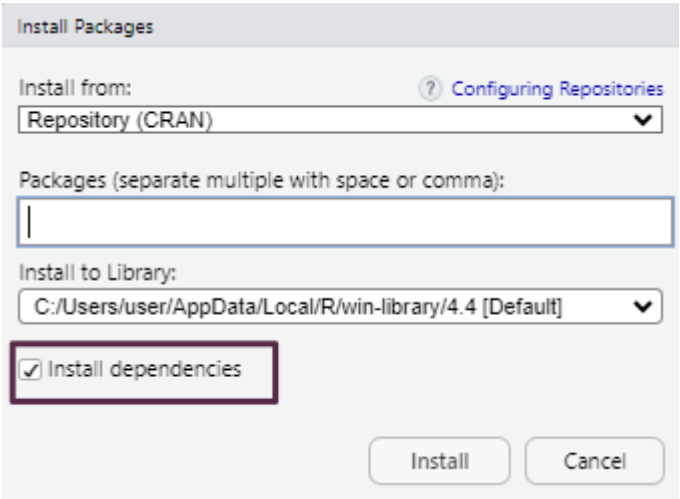
Dado que la mayoría de las computadoras hoy en día cuentan con acceso a Internet, en este curso nos enfocaremos en la instalación y activación de paquetes directamente desde RStudio.

Dentro del entorno de RStudio, la gestión de paquetes se realiza desde la pestaña *Packages*, ubicada en el panel inferior derecho. Esta interfaz facilita tareas como la instalación, actualización y activación de paquetes, y cada acción que realizamos desde la interfaz gráfica se traduce internamente en la ejecución de funciones del lenguaje R que pueden observarse en la consola.

Para instalar un paquete nuevo, simplemente pulsamos el botón *Install* dentro de la pestaña *Packages*, lo cual abrirá una ventana emergente.



Allí podemos escribir el nombre del paquete que deseamos instalar. Para asegurarnos de que también se descarguen e instalen automáticamente los paquetes de los que depende, es importante tildar la opción *Install dependencies*.



Como alternativa, también podemos realizar la instalación desde el editor de scripts mediante el siguiente comando:

```
install.packages("nombre_del_paquete", dependencies = TRUE)
```

> Para facilitar la instalación de los paquetes requeridos durante el curso, recomendamos descargar el archivo **“Paquetes y datos para el curso”**, descomprimirlo y ejecutar el script que se encuentra en su interior.

2.7. Lectura de archivos de datos

R permite importar tablas de datos desde diversos formatos, tanto utilizando funciones de **R base** como funciones provistas por paquetes específicos.

El formato más común es el **texto plano** (ASCII), donde los valores están organizados en columnas separadas por caracteres delimitadores. Los separadores más habituales incluyen:

- Coma (`,`)
- Punto y coma (`;`)
- Tabulación (`\t`)
- Barra vertical (`|`)

Estos archivos suelen tener una **cabecera** (*header*) en la primera fila con los nombres de las variables, y cada columna debe contener datos del mismo tipo (números, texto, lógicos, etc.).

Para importar correctamente un archivo es importante conocer su estructura:

- Si incluye o no cabecera.
- Qué carácter se usa como separador.
- El tipo de codificación (UTF-8, Latin1, etc.).

Dado que son archivos de texto, pueden visualizarse con editores simples como el Bloc de Notas o desde RStudio, lo que facilita su inspección previa.

Para cargar los datos desde un archivo de texto plano usamos el código:

```
datos <- read.xxx("mis_datos.xxx")
```

(Se debe reemplazar `read.xxx()` por la función correspondiente: `read.table()`, `read.csv()`, `read_delim()`, `read_excel()`, etc., según la extensión del archivo).

R también permite cargar bases de datos incluidas en paquetes instalados mediante:

```
data(nombre_datos)

datos <- nombre_datos
```

2.8. Buenas prácticas

Adoptar buenas prácticas desde el inicio mejora la reproducibilidad, facilita el trabajo colaborativo y reduce errores. Algunas recomendaciones clave son:

- Trabajar **siempre** dentro de un **proyecto de RStudio** (`.Rproj`). Esto permite organizar los archivos, mantener rutas relativas consistentes y acceder a funcionalidades específicas como control de versiones o panel de archivos integrados.
- Incluir al comienzo de cada script las líneas de **activación de paquetes** necesarios, utilizando la función `library()`.

- **Cargar los datos** una vez activados los paquetes, para garantizar que todas las funciones requeridas estén disponibles.
- Documentar el código mediante **comentarios** iniciados con #. Esto permite entender qué hace cada bloque de código, facilitando futuras modificaciones o revisiones.
- **Usar espacios e indentación adecuada** para mejorar la legibilidad. Esto es especialmente importante en estructuras anidadas (como condicionales, bucles o funciones).

3. Introducción a `tidyverse`



3.1. Introducción

Tidyverse es el nombre que recibe el conjunto de paquetes desarrollados y/o promovidos por **Hadley Wickham** (jefe científico en Posit/RStudio) y su equipo, orientado al trabajo de ciencia de datos con R (Wickham et al. 2019). Estos paquetes están diseñados para integrarse de manera coherente, compartiendo una misma filosofía de diseño conocida como ***The tidy tools manifesto***.

Los cuatro principios básicos sobre los que se construye *tidyverse* son:

- Reutilización de estructuras de datos
- Resolución de problemas complejos combinando varias piezas sencillas
- Uso de programación funcional
- Diseño orientado a las personas

Los paquetes incluidos cubren todas las etapas del análisis de datos dentro de R: importación y ordenamiento de los datos (*tidy data*), transformación, visualización, modelado y la posterior comunicación de resultados.

La palabra *tidy* se traduce como "ordenado", y hace referencia a una estructura específica que deben cumplir los datos:

- Cada ***variable*** es una *columna* de la tabla de datos.
- Cada ***observación*** es una *fila* de la tabla de datos.
- Cada ***tabla*** responde a una *unidad de observación o* análisis.



Además de los paquetes principales, al instalar `tidyverse` se incluyen otros que permiten trabajar con fechas, cadenas de caracteres o factores, también siguiendo los mismos principios.

Uno de los objetivos de los desarrolladores fue dotar a la sintaxis de estos paquetes de una **gramática** clara: funciones cuyos nombres y argumentos permiten construir "*frases*" que sean semánticamente comprensibles. Un ejemplo de esto se ve en el paquete `dplyr`, donde la mayoría de las funciones son verbos en inglés como `filter()`, `mutate()`, `summarise()`, lo que facilita su lectura y comprensión.

El paquete `tidyverse` puede instalarse desde el repositorio oficial CRAN mediante el menú *Packages* de RStudio, o ejecutando el siguiente código:

```
install.packages("tidyverse")
```

Una vez instalado, se activa mediante:

```
library(tidyverse)
```

Al activarlo, se muestra un mensaje con la versión instalada, la lista de paquetes que se cargan automáticamente y posibles conflictos de nombres entre funciones. Esto es habitual cuando se utilizan múltiples paquetes, ya que algunas funciones pueden llamarse igual. Por ejemplo, la función `filter()` existe tanto en el paquete `stats` como en `dplyr`. Al cargar `tidyverse`, R avisa de esta superposición:

```
dplyr::filter() masks stats::filter()
```

Cuando necesitamos asegurarnos de que estamos usando la función de un paquete específico, se recomienda usar la notación `::`, por ejemplo:

```
# Función filter() del paquete stats
stats::filter()

# Función filter() del paquete tidyverse
dplyr::filter()
```

Una estrategia útil cuando trabajamos con varios paquetes es cargar `tidyverse` al final de la lista de paquetes, para que sus funciones sobrescriban las de otros paquetes si fuese necesario:

```
library(stats)
library(tidyverse)
```

Los paquetes que se instalan con la versión actual de `tidyverse` pueden consultarse ejecutando:

```
tidyverse_packages()
```

```
 [1] "broom"         "conflicted"    "cli"           "dbplyr"
 [5] "dplyr"         "dtplyr"        "forcats"       "ggplot2"
 [9] "googledrive"   "googlesheets4" "haven"         "hms"
[13] "httr"          "jsonlite"      "lubridate"     "magrittr"
[17] "modelr"        "pillar"        "purrr"         "ragg"
[21] "readr"         "readxl"        "reprex"        "rlang"
[25] "rstudioapi"    "rvest"         "stringr"       "tibble"
[29] "tidyr"         "xml2"          "tidyverse"
```

Además, existen muchos otros paquetes que siguen la misma filosofía pero no están incluidos por defecto. En esos casos, deben instalarse y activarse individualmente.

> ⚠ Para profundizar el uso de `tidyverse`, se recomienda consultar:
> Sitio oficial: https://www.tidyverse.org/
> Libro *R para Ciencia de Datos*: https://es.r4ds.hadley.nz/
> Nueva versión (*r4ds 2e*): https://r4ds.hadley.nz/
> EpiRhandbook en español: https://www.epirhandbook.com/es/index.es.html



3.2. *Dataframes* con `tibble`

Uno de los paquetes que forman parte del núcleo básico de *tidyverse* es `tibble` (Müller y Wickham 2026), que introduce una versión moderna del objeto `data.frame`. Todas las funciones que generan tablas de datos en *tidyverse* devuelven objetos *tibble* (`tbl_df`), los cuales son más eficientes y amigables para el flujo de trabajo.

Las principales ventajas de trabajar con `tibble` son:

- Impresión en consola más legible y controlada (muestran un número limitado de filas y columnas).
- No cambian automáticamente el tipo de datos.
- Permiten nombres de columnas con espacios o caracteres especiales si se encierran entre comillas invertidas ` (aunque **no se recomienda**).

Para crear un objeto *tibble* manualmente usamos el siguiente código:

```
datos <- tibble(
  nombre = c("Ana", "Luis", "María"),
  edad = c(34, 28, 45),
  altura = c(1.65, 1.80, 1.70)
)
```

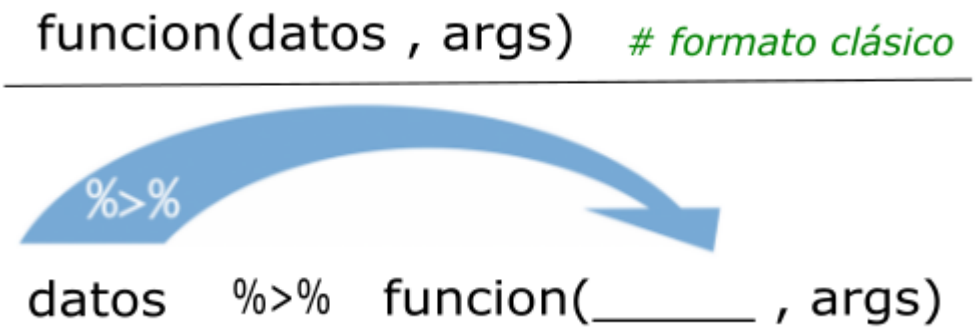


3.3. Tuberías con magrittr

Una de las incorporaciones más útiles y transversales del ecosistema *tidyverse* es el uso de "*tuberías*" o *pipe operators*. Una tubería conecta un bloque de código con otro, permitiendo encadenar operaciones de manera legible. El operador %>%, proveniente del paquete `magrittr` (Bache y Wickham 2026), transforma llamadas de funciones anidadas (con múltiples paréntesis) en una secuencia de pasos más simple de leer y escribir.

A partir de la versión 4.1.0 de R, también se incorporó una **tubería nativa** (|>), con un comportamiento muy similar. Ambas opciones son válidas y su uso es prácticamente equivalente.

Este enfoque refleja el principio de que *cada función representa un paso en una secuencia lógica de transformación de datos*. La forma de trabajar se puede ver en el siguiente esquema general:



A continuación, mostramos un ejemplo comparativo de cómo cambia la sintaxis usando el dataset incorporado en R `mtcars`, que contiene datos sobre autos:

```
head(sqrt(mtcars))
```

```
                      mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs am     gear     carb
Mazda RX4         4.057093  0  1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0  1 2.000000 2.000000
Datsun 710        4.313931  1  1 2.000000 1.000000
Hornet 4 Drive    4.409082  1  0 1.732051 1.000000
Hornet Sportabout 4.125530  0  0 1.732051 1.414214
Valiant           4.496665  1  0 1.732051 1.000000
```

En la línea de código anterior estamos pidiendo mostrar la cabecera (6 primeras observaciones de la tabla de datos) de la raíz cuadrada de los valores de la tabla `mtcars`, en formato del lenguaje clásico (anidado).

Ahora activamos `magrittr` y ejecutamos la línea anterior en formato tubería:

```
# Activa paquete
library(magrittr)

# Formato tubería
mtcars %>%
  sqrt() %>%
  head()
```

```
                      mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs am     gear     carb
Mazda RX4         4.057093  0  1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0  1 2.000000 2.000000
Datsun 710        4.313931  1  1 2.000000 1.000000
Hornet 4 Drive    4.409082  1  0 1.732051 1.000000
Hornet Sportabout 4.125530  0  0 1.732051 1.414214
Valiant           4.496665  1  0 1.732051 1.000000
```

Podemos hacer lo mismo con la tubería nativa de R sin activar ningún paquete (revisar que esté activada desde `Tools > Global Options`):

```
# Tubería nativa
mtcars |>
  sqrt() |>
  head()
```

```
                      mpg      cyl     disp        hp     drat       wt
Mazda RX4         4.582576 2.449490 12.64911 10.488088 1.974842 1.618641
Mazda RX4 Wag     4.582576 2.449490 12.64911 10.488088 1.974842 1.695582
Datsun 710        4.774935 2.000000 10.39230  9.643651 1.962142 1.523155
Hornet 4 Drive    4.626013 2.449490 16.06238 10.488088 1.754993 1.793042
Hornet Sportabout 4.324350 2.828427 18.97367 13.228757 1.774824 1.854724
Valiant           4.254409 2.449490 15.00000 10.246951 1.661325 1.860108
                      qsec vs am     gear     carb
Mazda RX4         4.057093  0  1 2.000000 2.000000
Mazda RX4 Wag     4.125530  0  1 2.000000 2.000000
Datsun 710        4.313931  1  1 2.000000 1.000000
Hornet 4 Drive    4.409082  1  0 1.732051 1.000000
Hornet Sportabout 4.125530  0  0 1.732051 1.414214
Valiant           4.496665  1  0 1.732051 1.000000
```

Las tuberías le dan mucha mas claridad al código separándolo en partes, como si fuesen oraciones de un párrafo.



3.4. Archivos de texto plano con `readr`

El paquete `readr` (Wickham, Hester, et al. 2026) contiene funciones similares a las de la familia `read.table()` de R base, pero desarrollados bajo el ecosistema `tidyverse`.

Los archivos de texto plano (ASCII u otras codificaciones) son universalmente utilizados por la mayoría de los gestores de bases de datos y planillas de cálculo. Generalmente se encuentran con extensiones `.txt` o `.csv` (por *comma-separated values*) y son el tipo de archivo de datos más habitual en R.

Estos datos planos tienen dos características principales:

- La cabecera (en inglés *header*).
- El carácter o símbolo separador que indica la separación de columnas: pueden estar separadas por comas, punto y coma, tabulación, etc.

La presencia o no de una cabecera se maneja con los argumentos `col_names` y `skip`:

- `col_names = TRUE` indica que la primera fila contiene los nombres de las columnas (cabecera).
- `col_names = FALSE` indica que no hay cabecera y las columnas se nombran automáticamente (`X1`, `X2`, etc).
- `skip = 0` (valor por defecto) lee los datos desde la primera fila, pero si hay encabezados complejos (por ejemplo, títulos y subtítulos), se puede indicar cuántas filas deben omitirse. Ejemplo: `skip = 5` omite las primeras 5 filas del archivo.

Otro aspecto a considerar es el carácter **separador** utilizado para indicar la separación entre columnas. Los separadores más comunes son:

- coma (`,`)
- punto y coma (`;`)
- tabulación (`TAB`)
- espacio (`" "`)
- barra vertical o *pipe* (`|`)

Funciones de lectura

Algunas de las funciones del paquete asumen un separador particular. Por caso `read_csv()` lee separados por coma y `read_tsv()` separado por tabulaciones, pero la función `read_delim()` permite que definamos el separador a través del argumento `delim`.

En forma detallada el paquete `readr` soporta siete formatos de archivo a partir de siete funciones:

- `read_csv()`: archivos separados por comas (CSV).
- `read_tsv()`: archivos separados por tabulaciones.
- `read_delim()`: archivos separados con delimitadores generales.
- `read_fwf()`: archivos con columnas de ancho fijo.
- `read_table()`: archivos formato tabla con columnas separadas por espacios.
- `read_log()`: archivos log web.

En comparación con las funciones R `base`, las funciones de `readr`:

- Usan un esquema de nombres consistente de parámetros.
- Son más rápidas.
- Analizan eficientemente los formatos de datos comunes (especialmente fechas y horas).
- Muestra una barra de progreso para archivos grandes.
- Vienen incluidas dentro de `tidyverse` pero también pueden usarse de forma independiente:

```
library(readr)
```

A modo de ejemplo, leeremos un archivo sin cabecera separado por comas bajo el nombre `datos`:

```
datos <- read_csv("datos/ejemplo-datos.csv", col_names = F)
datos
```

```
# A tibble: 4 x 5
     X1 X2       X3       X4    X5
  <dbl> <chr>    <chr>    <chr> <date>
1     9 Leone    Fernando M     1958-12-24
2    26 Garcia   Esteban  M     1954-01-21
3    35 Salamone Nicolas  M     1993-06-27
4    48 Gonzalez Viviana  F     1965-06-21
```

Leemos el mismo archivo con cabecera y separado por punto y comas, bajo el nombre `info`:

```
info <- read_csv2("datos/ejemplo-datos-header.csv", col_names = T)
info
```

```
# A tibble: 4 x 5
   Iden Apellido Nombre   Sexo  FNac
  <dbl> <chr>    <chr>    <chr> <date>
1     9 Leone    Fernando M     1958-12-24
2    26 Garcia   Laura    M     1954-01-21
3    35 Salamone Nicolas  M     1993-06-27
4    48 Gonzalez Viviana  F     1965-06-21
```

En estos ejemplos:

- `read_csv()` espera comas como separador
- `read_csv2()` espera punto y coma como separador

Al leer un archivo, `readr` intenta adivinar automáticamente el tipo de dato de cada columna (*parse*). Si no hay cabecera, los nombres de columna serán `X1`, `X2`, etc.

Podemos inspeccionar la estructura de un *dataframe* con `glimpse()`:

```
glimpse(info)
```

```
Rows: 4
Columns: 5
$ Iden     <dbl> 9, 26, 35, 48
$ Apellido <chr> "Leone", "Garcia", "Salamone", "Gonzalez"
$ Nombre   <chr> "Fernando", "Laura", "Nicolas", "Viviana"
$ Sexo     <chr> "M", "M", "M", "F"
$ FNac     <date> 1958-12-24, 1954-01-21, 1993-06-27, 1965-06-21
```

Los tipos de datos posibles son:

- `character` (`<chr>`)
- `integer`, `double` o `numeric` (`<int>`, `<dbl>`)

- logical (<lgl>)
- date, datetime, etc.

Por ejemplo, columnas con enteros pueden aparecer como <dbl> si se interpretan como double, y las fechas como <date>.

Agregamos unos argumentos más y ejemplificamos la sintaxis con read_delim() para leer un archivo con cabecera compleja (la tabla comienza en la fila 9) separado por caracteres | (*pipes*).

```
read_delim(
  "ejemplo-datos-header-skip.txt",
  col_names = T,
  skip = 8,
  delim = "|"
)
```

> **Importante:** No olvides asignar la lectura a un nombre para guardar el *dataframe* dentro del entorno de trabajo (por ejemplo: datos <-).

Funciones de escritura

El paquete también incluye funciones para escribir archivos de texto plano, con formatos espejo de las funciones de lectura más comunes:

- write_csv(): escribe archivos separados por comas
- write_csv2(): escribe archivos separados por punto y comas
- write_tsv(): escribe archivos separados por tabulaciones
- write_delim(): escribe archivos separados con delimitadores definidos por el usuario

Los argumentos son generales y para el caso del último más extensos, dado que hay que definir cual es el separador que deseamos en el archivo. Podemos consultarlos con el siguiente código:

```
args(write_delim)
```

```
function (x, file, delim = " ", na = "NA", append = FALSE, col_names = !append,
    quote = c("needed", "all", "none"), escape = c("double",
        "backslash", "none"), eol = "\n", num_threads = readr_threads(),
    progress = show_progress())
NULL
```

Por ejemplo para exportar un conjunto de datos en texto plano al que denominaremos "ejemplo.csv" con separador **punto y coma** y **cabecera incluida** podemos hacer:

```
write_delim(x = datos, file = "ejemplo.csv", delim = ";")
```

o más sencillo, usando la función específica write_csv2():

```
write_csv2(datos, "ejemplo.csv") # define cabecera y separador ;
```



3.5. Lectura de hojas de cálculo con readxl

Uno de los formatos más comunes para almacenar datos son las hojas de cálculo, en particular las creadas con Microsoft Excel. El paquete `readxl` (Wickham y Bryan 2026), parte del ecosistema `tidyverse`, permite leer este tipo de archivos.

`readxl` es compatible con hojas de cálculo de Excel 97-2003, con extensión `.xls`, y con versiones más recientes, con extensión `.xlsx`.

Una primera función útil es `excel_sheets()`, que permite conocer y listar los nombres de las hojas contenidas en un archivo Excel (también llamado libro o *workbook*).

Por ejemplo, supongamos que tenemos un archivo denominado "`datos.xlsx`" y queremos saber por cuantas hojas está compuesto y que nombre tienen:

```
library(readxl) # hay que activarlo independientemente de tidyverse

excel_sheets("datos/datos.xlsx")
```

```
[1] "diabetes"   "vigilancia" "mortalidad"
```

Esto devuelve, por ejemplo, tres hojas: "`diabetes`", "`vigilancia`" y "`mortalidad`".

Para leer una de estas hojas utilizamos la función `read_excel()`, cuyos argumentos principales son:

```
args(read_excel)
```

```
function (path, sheet = NULL, range = NULL, col_names = TRUE,
    col_types = NULL, na = "", trim_ws = TRUE, skip = 0, n_max = Inf,
    guess_max = min(1000, n_max), progress = readxl_progress(),
    .name_repair = "unique")
NULL
```

Entre los más relevantes encontramos:

- `path`: nombre del archivo y su ubicación (entre comillas)
- `sheet`: nombre de la hoja o su número de orden

- col_names: si es TRUE, toma la primera fila como nombres de las columnas
- skip: permite saltear un número determinado de filas antes de comenzar la lectura

Al ejecutar read_excel(), internamente se utiliza la función excel_format() para detectar si el archivo es .xls o .xlsx, y luego se aplica la función específica para cada caso: read_xls() o read_xlsx(). Estas funciones también pueden usarse directamente si se desea.

Supongamos ahora que queremos leer la hoja llamada "diabetes":

```
diabetes <- read_excel(
  path = "datos/datos.xlsx",
  sheet = "diabetes",
  col_names = T
)

# mostramos las 6 primeras observaciones
head(diabetes)
```

```
# A tibble: 6 x 8
    A1C  hba1 GLUCB   SOG Tol_Glucosa    DM    SM  HOMA
  <dbl> <dbl> <dbl> <dbl> <chr>       <dbl> <dbl> <dbl>
1  6.17   7.9   101   122 IFG             0     1  4.04
2  5.58   7.2   103   100 IFG             0     0  5.03
3  5.38   7.1   103    90 IFG             0     1  2.92
4  5.38   6.6   109    96 IFG             0     1  4.79
5  5.19   6.3   107    69 IFG             0     1  3.06
6  4.89   6      NA   117 IFG             0     0  5.77
```

Observemos que en los argumentos escribimos el nombre del archivo que se encuentra en nuestro proyecto y por lo tanto en nuestra carpeta activa, el nombre de la hoja y nos aseguramos que la primer fila representa a la cabecera de la tabla (sus nombres de variables).

Como readxl forma parte del ecosistema tidyverse el formato de salida es un *tibble*. En este caso de 23 observaciones por 8 variables.

Ahora leamos la segunda hoja de nombre "vigilancia":

```
vigilancia <- read_excel(path = "datos/datos.xlsx", sheet = 2, col_names = F)

# mostramos las 6 primeras observaciones
head(vigilancia)
```

```
# A tibble: 6 x 9
   ...1 ...2        ...3      ...4  ...5  ...6  ...7 ...8  ...9
  <dbl> <chr>      <dbl>     <dbl> <dbl> <dbl> <dbl> <chr> <chr>
1   875 09/28/2015  2015 544080000     1    31     1 F     VIGILANCIA EN SALUD ~
2   875 42317       2015 544080000     1    35     1 F     VIGILANCIA EN SALUD ~
3   875 42317       2015 544080000     1    47     1 F     VIGILANCIA EN SALUD ~
4   307 09/26/2015  2015 544005273     1    23     1 M     VIGILANCIA INTEGRADA~
5   307 09/24/2015  2015 544005273     1    19     1 M     VIGILANCIA INTEGRADA~
6   875 09/28/2015  2015 544080000     1    63     1 F     VIGILANCIA EN SALUD ~
```

En este caso, en lugar del nombre de la hoja usamos un 2 que es su ubicación y especificamos `col_names = FALSE` porque el conjunto de datos no tiene cabecera. `readxl` asignará nombres genéricos como `...1`, `...2`, etc.

Finalmente leamos la última hoja disponible del archivo:

```
mortalidad <- read_excel(
  path = "datos/datos.xlsx",
  sheet = "mortalidad",
  col_names = T,
  skip = 1
)

# mostramos las 6 primeras observaciones
head(mortalidad)
```

```
# A tibble: 5 x 10
  grupo_edad grupo.I.1.1 grupo.II.1.1 grupo.III.1.1 grupo.I.2.1 grupo.II.2.1
  <chr>            <dbl>        <dbl>          <dbl>       <dbl>        <dbl>
1 30-44               41          202            222         539         1438
2 45-59               99         1071            181         759         6210
3 60-69              114         1782            119         985         9238
4 70-79              221         2336            119        1571        12369
5 80+                362         2492             81        2523        14642
# i 4 more variables: grupo.III.2.1 <dbl>, grupo.I.3.1 <dbl>,
#   grupo.II.3.1 <dbl>, grupo.III.3.1 <dbl>
```

Lo novedoso de esta lectura es el argumento `skip = 1` que debimos incorporar dado que, en este caso, la hoja de Excel comienza con una línea de título que no pertenece al conjunto de datos. También que el argumento `sheet` permite el nombre de la hoja elegida entre comillas.

Además de los argumentos generales de `read_xl()`, podemos mencionar estos otros:

- `n_max`: número máximo de filas a leer.
- `range`: rango de celdas a importar (como en Excel, por ejemplo `"B3:D87"`).
- `col_types`: define el tipo de datos de cada columna. Valores posibles: `"numeric"`, `"logical"`, `"text"`, `"date"`, `"skip"` (no leer la columna), `"guess"` (modo predeterminado: la función decide automáticamente el tipo).
- `na`: carácter o vector de caracteres que se deben interpretar como valores perdidos (`NA`). Por defecto, las celdas vacías se interpretan así.

> ⚠ En los siguientes capítulos se abordarán dos componentes fundamentales del ecosistema `tidyverse`: `dplyr`, para la manipulación de datos y la construcción de tablas de resumen, y `ggplot2`, para la elaboración de gráficos estadísticos.

3.6. Referencias

4. Gestión de datos con `dplyr`



4.1. Introducción

El paquete `dplyr` (Wickham, François, et al. 2026) fue desarrollado por *Hadley Wickham* como una versión optimizada del paquete `plyr` (Wickham 2011).

Su principal contribución es ofrecer una *gramática* para la manipulación de datos, basada en funciones que actúan como verbos, lo que facilita la lectura y comprensión del código.

Las funciones clave del paquete permiten realizar las siguientes acciones (verbos):

- `select()`: selecciona un conjunto de columnas (variables)
- `rename()`: renombra variables en un conjunto de datos
- `filter()`: selecciona un conjunto de filas (observaciones) según una o varias condiciones lógicas
- `arrange()`: reordena las filas de un conjunto de datos
- `mutate()`: añade nuevas variables/columnas o transforma variables existentes
- `summarise()`/ `summarize()`: genera resúmenes estadísticos de diferentes variables en el conjunto de datos
- `group_by()`: agrupa las observaciones en función de una o más variables, lo que permite realizar operaciones por grupo
- `count()`: contabiliza valores que se repiten, generando una tabla de frecuencias

Además, al ser parte del ecosistema *tidyverse*, `dplyr` integra al operador *pipe* (%> %) formando una única secuencia de procesamiento o *pipeline*.

4.1.1. Argumentos comunes

Todas las funciones, básicamente, tienen en común una serie de argumentos.

1. El primer argumento es el nombre del conjunto de datos (objeto donde esta nuestra tabla de datos).

2. Los otros argumentos describen que hacer con el conjunto de datos especificado en el primer argumento, podemos referirnos a las columnas en el objeto directamente sin utilizar el operador $, es decir sólo con el nombre de la columna/variable.

3. El valor de retorno es un nuevo conjunto de datos.

4. Los conjuntos de datos deben estar bien organizados/estructurados, es decir debe existir una observación por columna y, cada columna representar una variable, medida o característica de esa observación. Es decir, debe cumplir con *tidy data*.

4.1.2. Activación del paquete

`dplyr` está incluído en el núcleo base de `tidyverse`, por lo que se encuentra disponible si tenemos activado a este último.

También se puede activar en forma independiente:

```
library(dplyr)
```

4.1.3. Conjunto de datos para ejemplo

Para visualizar y comprender el funcionamiento de estos "verbos" de manipulación, resulta muy útil contar con ejemplos concretos. Por eso, en esta unidad trabajaremos con un conjunto de datos que nos permitirá practicar el uso de las funciones del paquete.

> Recuerden que pueden **descargar los datos** utilizados en los ejemplos del curso y descomprimirlos en la carpeta donde tengan guardado su proyecto de RStudio.

Uno de los archivos incluidos, "`noti-vih.csv`", contiene registros de notificaciones de VIH por jurisdicción en Argentina correspondientes a los años 2015 y 2016.

```
# asignamos la lectura a datos
datos <- read_csv("datos/noti-vih.csv")

# mostramos las 6 primeras observaciones
head(datos)
```

```
# A tibble: 6 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 CABA          2015   901  3054237
4 CABA          2016   427  3050000
5 Catamarca     2015    69   396552
6 Catamarca     2016    51   401575
```

4.2. Función `select()`

La función `select()` permite elegir columnas específicas de un conjunto de datos, devolviendo una versión "recortada por columnas" del mismo.

A continuación, exploramos algunas formas útiles de seleccionar variables:

Seleccionar todas las variables excepto `pob`:

```
datos |>
  select(-pob)
```

```
# A tibble: 48 x 3
   jurisdiccion   año casos
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  1513
 2 Buenos Aires  2016   957
 3 CABA          2015   901
 4 CABA          2016   427
 5 Catamarca     2015    69
 6 Catamarca     2016    51
 7 Chaco         2015    15
 8 Chaco         2016     9
 9 Chubut        2015   110
10 Chubut        2016    89
# i 38 more rows
```

Otra forma para el mismo resultado anterior (mediante el operador rango `:`):

```
datos |>
  select(jurisdiccion:casos)
```

```
# A tibble: 48 x 3
   jurisdiccion   año casos
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  1513
 2 Buenos Aires  2016   957
 3 CABA          2015   901
 4 CABA          2016   427
 5 Catamarca     2015    69
 6 Catamarca     2016    51
 7 Chaco         2015    15
 8 Chaco         2016     9
 9 Chubut        2015   110
10 Chubut        2016    89
# i 38 more rows
```

Seleccionar solamente las variables `jurisdiccion` y `casos`:

```
datos |>
  select(jurisdiccion, casos)
```

```
# A tibble: 48 x 2
   jurisdiccion casos
   <chr>        <dbl>
 1 Buenos Aires  1513
 2 Buenos Aires   957
 3 CABA           901
 4 CABA           427
 5 Catamarca       69
 6 Catamarca       51
 7 Chaco           15
 8 Chaco            9
 9 Chubut         110
10 Chubut          89
# i 38 more rows
```

Lo mismo que el ejemplo anterior, pero usando la posición de las columnas:

```
datos |>
  select(1, 3)
```

```
# A tibble: 48 x 2
   jurisdiccion casos
   <chr>        <dbl>
 1 Buenos Aires  1513
 2 Buenos Aires   957
 3 CABA           901
 4 CABA           427
 5 Catamarca       69
 6 Catamarca       51
 7 Chaco           15
 8 Chaco            9
 9 Chubut         110
10 Chubut          89
# i 38 more rows
```

Mover la variable año al inicio y mantener todas las demás:

```
datos |>
  select("año", everything())
```

```
# A tibble: 48 x 4
     año jurisdiccion casos      pob
   <dbl> <chr>        <dbl>    <dbl>
 1  2015 Buenos Aires  1513 16626374
 2  2016 Buenos Aires   957 16789474
```

```
 3  2015 CABA           901  3054237
 4  2016 CABA           427  3050000
 5  2015 Catamarca       69   396552
 6  2016 Catamarca       51   401575
 7  2015 Chaco           15  1153846
 8  2016 Chaco            9  1125000
 9  2015 Chubut         110   567010
10  2016 Chubut          89   577922
# i 38 more rows
```

Cambiar el nombre de una de la variable año y mover al inicio:

```
datos |>
  select(anio = año, everything())
```

```
# A tibble: 48 x 4
    anio jurisdiccion casos      pob
   <dbl> <chr>        <dbl>    <dbl>
 1  2015 Buenos Aires  1513 16626374
 2  2016 Buenos Aires   957 16789474
 3  2015 CABA           901  3054237
 4  2016 CABA           427  3050000
 5  2015 Catamarca       69   396552
 6  2016 Catamarca       51   401575
 7  2015 Chaco           15  1153846
 8  2016 Chaco            9  1125000
 9  2015 Chubut         110   567010
10  2016 Chubut          89   577922
# i 38 more rows
```

Otros posibles argumentos son:

- `starts_with()`: selecciona todas las columnas que comiencen con el patrón indicado.
- `ends_with()`: selecciona todas las columnas que terminen con el patrón indicado.
- `contains()`: selecciona las columnas que posean el patrón indicado.
- `matches()`: similar a `contains()`, pero permite poner una expresión regular.
- `all_of()`: selecciona las variables pasadas en un vector (todos los nombres deben estar presentes o devuelve un error).
- `any_of()`: idem anterior excepto que no se genera ningún error para los nombres que no existen.
- `num_range()`: selecciona variables con nombre combinados con caracteres y números (ejemplo: `num_range("x", 1:3)` selecciona las variables `x1`, `x2` y `x3`.
- `where()`: aplica una función a todas las variables y selecciona aquellas para las cuales la función regresa TRUE (por ejemplo: `is.numeric()` para seleccionar todas las variables numéricas).

4.3. Función rename()

La función rename() puede considerarse una extensión de select(). Si bien select() también permite renombrar variables, no resulta muy útil para este fin, ya que descarta todas las variables que no se mencionan explícitamente.

En cambio, rename() permite cambiar el nombre de una o más variables sin eliminar las demás. Solo se modifican los nombres indicados, y el resto del conjunto de datos permanece sin cambios.

Ejemplo: renombrar la variable pob como población:

```
datos |>
  rename("población" = pob)
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos población
   <chr>        <dbl> <dbl>     <dbl>
 1 Buenos Aires  2015  1513  16626374
 2 Buenos Aires  2016   957  16789474
 3 CABA          2015   901   3054237
 4 CABA          2016   427   3050000
 5 Catamarca     2015    69    396552
 6 Catamarca     2016    51    401575
 7 Chaco         2015    15   1153846
 8 Chaco         2016     9   1125000
 9 Chubut        2015   110    567010
10 Chubut        2016    89    577922
# i 38 more rows
```

Se puede renombrar una serie de columnas usando rename_with(), por ejemplo para agregar el prefijo "n_" a las variables de conteo:

```
datos |>
  rename_with(
    .cols = c(casos, pob), # Columnas a renombrar
    .fn = ~ paste0("n_", .x) # Función o vector para renombrar columnas
  )
```

```
# A tibble: 48 x 4
   jurisdiccion   año n_casos    n_pob
   <chr>        <dbl>   <dbl>    <dbl>
 1 Buenos Aires  2015    1513 16626374
 2 Buenos Aires  2016     957 16789474
 3 CABA          2015     901  3054237
 4 CABA          2016     427  3050000
 5 Catamarca     2015      69   396552
 6 Catamarca     2016      51   401575
 7 Chaco         2015      15  1153846
 8 Chaco         2016       9  1125000
 9 Chubut        2015     110   567010
```

```
10 Chubut        2016      89    577922
# i 38 more rows
```

4.4. Función `filter()`

La función `filter()` permite seleccionar filas de un conjunto de datos, produciendo un subconjunto de observaciones.

Veamos un ejemplo sencillo con nuestros datos:

```
datos |>
  filter(jurisdiccion == "Tucuman")
```

```
# A tibble: 2 x 4
  jurisdiccion   año casos     pob
  <chr>        <dbl> <dbl>   <dbl>
1 Tucuman       2015   258 1592593
2 Tucuman       2016   246 1618421
```

Utiliza los mismos operadores de comparación propios del lenguaje R:

Operador	Descripción
<	Menor que
>	Menor que
<=	Menor o igual que
>=	Mayor o igual que
==	Igual que
!=	No igual que
%in %	Es parte de
is.na()	Es un valor ausente
!is.na()	No es un valor ausente

Lo mismo con los operadores lógicos que se utilizan como conectores entre las expresiones:

Operador	Descripción
&	AND booleano
|	OR booleano
xor()	OR exclusivo
!	NOT
any()	cualquier TRUE
all()	todos TRUE

Cuando usamos múltiples argumentos separados por coma dentro de `filter()`, estas se combinan con un operador **AND** implícito, es decir, cada expresión debe ser verdadera para que la fila sea incluida en la salida.

Por ejemplo, filtramos las observaciones que cumplan que `casos` sea mayor a 100 **y** que `pob` sea menor a 1.000.000:

```
datos |>
  filter(casos > 100, pob < 1000000)
```

```
# A tibble: 7 x 4
  jurisdiccion   año casos    pob
  <chr>        <dbl> <dbl>  <dbl>
1 Chubut        2015   110 567010
2 Jujuy         2015   160 727273
3 Jujuy         2016   133 734807
4 Neuquen       2015   109 619318
5 Neuquen       2016   101 627329
6 Rio Negro     2015   112 700000
7 Rio Negro     2016   105 709459
```

Para combinar condiciones dentro de una misma variable usamos el operador OR (|) o, de forma más práctica, %in%:

```
# Con OR
datos |>
  filter(jurisdiccion == "Buenos Aires" | jurisdiccion == "La Pampa")
```

```
# A tibble: 4 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 La Pampa      2015    57   343373
4 La Pampa      2016    67   345361
```

```
# Con %in%
datos |>
  filter(jurisdiccion %in% c("Buenos Aires", "La Pampa"))
```

```
# A tibble: 4 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2015  1513 16626374
2 Buenos Aires  2016   957 16789474
3 La Pampa      2015    57   343373
4 La Pampa      2016    67   345361
```

En el siguiente ejemplo, filtramos observaciones del año 2016 con más de 200 casos. El uso de & es equivalente al uso de coma:

```
datos |>
  filter(año == "2016" & casos > 200)
```

```
# A tibble: 6 x 4
  jurisdiccion   año casos      pob
  <chr>        <dbl> <dbl>    <dbl>
1 Buenos Aires  2016   957 16789474
2 CABA          2016   427  3050000
3 Cordoba       2016   368  3607843
4 Mendoza       2016   254  1909774
5 Salta         2016   230  1352941
6 Tucuman       2016   246  1618421
```

Por último, podemos usar `xor()` para seleccionar observaciones que cumplan **solo una** de las condiciones, pero no ambas. Por ejemplo, el siguiente filtro selecciona registros donde el año sea 2016 ó los casos sean mayores a 200, pero no ambos al mismo tiempo (es decir que no se den ambos en `TRUE`):

```
datos |>
  filter(xor(año == "2016", casos > 200))
```

```
# A tibble: 25 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 CABA          2015   901  3054237
 3 Catamarca     2016    51   401575
 4 Chaco         2016     9  1125000
 5 Chubut        2016    89   577922
 6 Cordoba       2015   468  3572519
 7 Corrientes    2016    99  1076087
 8 Entre Rios    2016   109  1329268
 9 Formosa       2016    60   582524
10 Jujuy         2016   133   734807
# i 15 more rows
```

En las últimas versiones de `dplyr` se añadió la función `filter_out()`, que equivale a usar el operador `!` para negar una condición. Por ejemplo, si quisiéramos excluir todos los datos de la provincia de Mendoza:

```
datos |>
  filter_out(jurisdiccion == "Mendoza")
```

```
# A tibble: 46 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 Buenos Aires  2016   957 16789474
 3 CABA          2015   901  3054237
 4 CABA          2016   427  3050000
 5 Catamarca     2015    69   396552
 6 Catamarca     2016    51   401575
 7 Chaco         2015    15  1153846
 8 Chaco         2016     9  1125000
```

```
 9 Chubut        2015   110   567010
10 Chubut        2016    89   577922
# i 36 more rows
```

4.5. Función `arrange()`

La función `arrange()` se utiliza para ordenar las filas de un conjunto de datos de acuerdo a una o varias columnas/variables. Por defecto, el ordenamiento es ascendente alfanumérico.

Ordenamos la tabla por la variable `pob` (forma ascendente predeterminada):

```
datos |>
  arrange(pob)
```

```
# A tibble: 48 x 4
   jurisdiccion       año casos    pob
   <chr>            <dbl> <dbl>  <dbl>
 1 Tierra del Fuego  2015    36 152542
 2 Tierra del Fuego  2016    34 156682
 3 Santa Cruz        2015    65 320197
 4 Santa Cruz        2016    59 329609
 5 La Pampa          2015    57 343373
 6 La Pampa          2016    67 345361
 7 La Rioja          2015    41 369369
 8 La Rioja          2016     6 375000
 9 Catamarca         2015    69 396552
10 Catamarca         2016    51 401575
# i 38 more rows
```

Para ordenar en forma descendente podemos utilizar `desc()` dentro de los argumentos de `arrange()`:

```
datos |>
  arrange(desc(pob))
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2016   957 16789474
 2 Buenos Aires  2015  1513 16626374
 3 Cordoba       2016   368  3607843
 4 Cordoba       2015   468  3572519
 5 Santa Fe      2016   170  3400000
 6 Santa Fe      2015   301  3382022
 7 CABA          2015   901  3054237
 8 CABA          2016   427  3050000
 9 Mendoza       2016   254  1909774
10 Mendoza       2015   316  1880952
# i 38 more rows
```

Que equivale a:

```
datos |>
  arrange(-pob)
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2016   957 16789474
 2 Buenos Aires  2015  1513 16626374
 3 Cordoba       2016   368  3607843
 4 Cordoba       2015   468  3572519
 5 Santa Fe      2016   170  3400000
 6 Santa Fe      2015   301  3382022
 7 CABA          2015   901  3054237
 8 CABA          2016   427  3050000
 9 Mendoza       2016   254  1909774
10 Mendoza       2015   316  1880952
# i 38 more rows
```

Podemos combinar ordenamientos. Por ejemplo, en forma alfabética ascendente para `jusrisdiccion` y luego numérica descendente para `casos`:

```
datos |>
  arrange(jurisdiccion, desc(casos))
```

```
# A tibble: 48 x 4
   jurisdiccion   año casos      pob
   <chr>        <dbl> <dbl>    <dbl>
 1 Buenos Aires  2015  1513 16626374
 2 Buenos Aires  2016   957 16789474
 3 CABA          2015   901  3054237
 4 CABA          2016   427  3050000
 5 Catamarca     2015    69   396552
 6 Catamarca     2016    51   401575
 7 Chaco         2015    15  1153846
 8 Chaco         2016     9  1125000
 9 Chubut        2015   110   567010
10 Chubut        2016    89   577922
# i 38 more rows
```

4.6. Función `mutate()`

Esta función nos permite transformar variables dentro de un conjunto de datos. A menudo tendremos la necesidad de modificar variables existentes o crear nuevas variables a partir de las ya disponibles. La función `mutate()` nos ofrece una forma clara y eficiente de realizar este tipo de operaciones.

Por ejemplo, podríamos querer calcular tasas crudas para cada jurisdicción y año, en función del número de casos y de la población total:

```
datos |>
  mutate(tasa = casos / pob * 100000)
```

```
# A tibble: 48 x 5
   jurisdiccion   año casos      pob  tasa
   <chr>        <dbl> <dbl>    <dbl> <dbl>
 1 Buenos Aires  2015  1513 16626374  9.10
 2 Buenos Aires  2016   957 16789474  5.70
 3 CABA          2015   901  3054237 29.5
 4 CABA          2016   427  3050000 14
 5 Catamarca     2015    69   396552 17.4
 6 Catamarca     2016    51   401575 12.7
 7 Chaco         2015    15  1153846  1.30
 8 Chaco         2016     9  1125000  0.8
 9 Chubut        2015   110   567010 19.4
10 Chubut        2016    89   577922 15.4
# i 38 more rows
```

En este caso, `mutate()` calcula la tasa cruda por 100.000 habitantes e incorpora una nueva variable (`tasa`) con los resultados correspondientes a cada observación.

También se pueden construir múltiples variables en la misma expresión, solamente separadas por comas:

```
datos |>
  mutate(
    tasaxcien_mil = casos / pob * 100000,
    tasaxdiez_mil = casos / pob * 10000
  )
```

```
# A tibble: 48 x 6
   jurisdiccion   año casos      pob tasaxcien_mil tasaxdiez_mil
   <chr>        <dbl> <dbl>    <dbl>         <dbl>         <dbl>
 1 Buenos Aires  2015  1513 16626374          9.10         0.910
 2 Buenos Aires  2016   957 16789474          5.70         0.570
 3 CABA          2015   901  3054237         29.5          2.95
 4 CABA          2016   427  3050000         14            1.4
 5 Catamarca     2015    69   396552         17.4          1.74
 6 Catamarca     2016    51   401575         12.7          1.27
 7 Chaco         2015    15  1153846          1.30         0.130
 8 Chaco         2016     9  1125000          0.8          0.08
 9 Chubut        2015   110   567010         19.4          1.94
10 Chubut        2016    89   577922         15.4          1.54
# i 38 more rows
```

Si deseamos que estas nuevas variables se incorporen de forma permanente al conjunto de datos (y no solo se muestren en la consola), debemos utilizar el operador de asignación <−:

```
datos <- datos |>
  mutate(
    tasaxcien_mil = casos / pob * 100000,
    tasaxdiez_mil = casos / pob * 10000
  )
```

Un aspecto fundamental es que las funciones utilizadas dentro de `mutate()` deben estar *vectorizadas*: deben aceptar un vector de entrada y devolver otro vector del mismo tamaño como salida.

Existen muchas funciones que se pueden utilizar dentro de `mutate()`. A continuación se presentan algunas útiles:

- **Operadores aritméticos**: +, −, *, /, ^.
- **Aritmética modular**: %/ % (división entera) y %% (resto), donde `x == y * (x %/% y) + (x %% y)`.
- **Funciones matemáticas**: `log()`, `log2()`, `log10()`, `exp()`, `sqrt()`, `abs()`, entre otras.
- **Valores acumulados**: `cumsum()`, `cumprod()`, `cummin()`, `cummax()` y `cummean()`.
- **Clasificación o *ranking***: funciones como `min_rank()` permiten asignar rangos (1º, 2º, etc.). Por defecto, los valores más pequeños reciben rangos más bajos. Si se desea invertir el orden, puede utilizarse `desc(x)`.

En versiones recientes de `dplyr`, `mutate()` incorpora argumentos que facilitan el trabajo con datos:

- `.by`: permite aplicar transformaciones por grupos de manera **temporal**, sin necesidad de usar `group_by()`
- `.before` / `.after`: permiten ubicar la nueva variable antes o después de otra columna del conjunto de datos

Finalmente, si se utiliza en `mutate()` el mismo nombre de una variable que ya existe en la tabla, dicha variable será sobrescrita (por ejemplo, al cambiarle el tipo de `character` a `factor`). Si se desea crear una variable nueva, se debe utilizar un nombre que no esté previamente en el conjunto de datos.

4.7. Funciones `group_by()` y `summarise()`

Estas dos funciones suelen utilizan en conjunto para calcular **resúmenes estadísticos por grupos**. En el caso de `summarise()` / `summarize()`, podemos reducir un conjunto de datos a uno o más valores resumen.

Por ejemplo:

```
datos |>
  summarise(
    promedio_casos = mean(casos),
    casos_totales = sum(casos)
  )
```

```
# A tibble: 1 x 2
  promedio_casos casos_totales
           <dbl>         <dbl>
1           192.          9211
```

Así, se obtiene una tabla con una sola fila que contiene el promedio y el total de casos en todo el conjunto de datos.

La función `group_by()` permite definir grupos dentro del conjunto de datos según una o más variables. A partir de ese momento, las operaciones siguientes se aplican **de forma independiente dentro de cada grupo**.

Por ejemplo, si queremos calcular los mismos indicadores por año:

```
datos |>
  group_by(año) |>
  summarise(
    promedio_casos = mean(casos),
    casos_totales = sum(casos)
  )
```

```
# A tibble: 2 x 3
    año promedio_casos casos_totales
  <dbl>          <dbl>         <dbl>
1  2015           224.          5369
2  2016           160.          3842
```

El resultado es una tabla con una fila por cada año, mostrando los valores correspondientes a cada grupo.

También es posible agrupar por más de una variable. Por ejemplo, calcular una tasa por jurisdicción y año:

```
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000
  )
```

```
# A tibble: 48 x 3
# Groups:   jurisdiccion [24]
   jurisdiccion   año  tasa
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5
 4 CABA          2016 14
 5 Catamarca     2015 17.4
 6 Catamarca     2016 12.7
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8
 9 Chubut        2015 19.4
10 Chubut        2016 15.4
# i 38 more rows
```

Después de usar `group_by()`, los datos permanecen agrupados. Si se desea continuar trabajando sin esa estructura, se puede utilizar la función `ungroup()` o el argumento `.groups = "drop"` dentro de `summarise()`:

```
# Descartar grupos con ungroup()
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000
  ) |>
  ungroup()
```

```
# A tibble: 48 x 3
   jurisdiccion   año  tasa
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5
 4 CABA          2016 14
 5 Catamarca     2015 17.4
 6 Catamarca     2016 12.7
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8
 9 Chubut        2015 19.4
10 Chubut        2016 15.4
# i 38 more rows
```

```
# Descartar grupos dentro de summarise()
datos |>
  group_by(jurisdiccion, año) |>
  summarise(
    tasa = casos / pob * 100000,
    .groups = "drop"
  )
```

```
# A tibble: 48 x 3
   jurisdiccion   año  tasa
   <chr>        <dbl> <dbl>
 1 Buenos Aires  2015  9.10
 2 Buenos Aires  2016  5.70
 3 CABA          2015 29.5
 4 CABA          2016 14
 5 Catamarca     2015 17.4
 6 Catamarca     2016 12.7
 7 Chaco         2015  1.30
 8 Chaco         2016  0.8
 9 Chubut        2015 19.4
10 Chubut        2016 15.4
# i 38 more rows
```

4.7.1. Funciones de resumen más frecuentes

Algunas de las funciones del R base que se pueden utilizar dentro de los argumentos de esta función son:

- min(): mínimo
- max(): máximo
- mean(): media
- median(): mediana
- var(): varianza
- sd(): desvío
- sum(): sumatoria

Otras funciones que se pueden incorporar las provee el mismo paquete dplyr, por ejemplo:

- first(): primer valor en el vector.
- last(): último valor en el vector.
- n(): número de valores en el vector.
- n_distinct(): números de valores distintos en el vector.

4.7.2. Combinaciones

En los ejemplos anteriores vimos cómo se van integrando algunas de las funciones mediante el uso del operador de tubería %>% o |>. La idea detrás de esta "*gramática de los datos*" que propone el paquete dplyr es poder encadenar acciones de forma legible y lógica, construyendo oraciones más complejas paso a paso.

Veamos un ejemplo que integra muchas de las funciones vistas hasta ahora:

> Obtener una nueva tabla con las tasas crudas de casos notificados de VIH, por año y jurisdicción, **mayores a 20 por 100.000 habitantes**, ordenadas de **mayor a menor**.

```
datos |> # siempre partimos de los datos
  group_by(año, jurisdiccion) |> # agrupamos
  summarise(tasa = casos / pob * 100000) |> # resumimos
  filter(tasa > 20) |> # filtramos
  arrange(desc(tasa)) # ordenamos
```

```
# A tibble: 5 x 3
# Groups:   año [2]
    año jurisdiccion      tasa
  <dbl> <chr>            <dbl>
1  2015 CABA              29.5
2  2015 Tierra del Fuego  23.6
3  2015 Jujuy             22.0
4  2016 Tierra del Fuego  21.7
5  2015 Santa Cruz        20.3
```

Una buena práctica para construir este tipo de código es escribir cada paso de la operación **en una línea separada**. Esto no solo mejora la legibilidad, sino que facilita la identificación de errores o la modificación de pasos específicos.

Este ejemplo muestra claramente el poder y la claridad que se logran al combinar funciones como group_by(), summarise(), filter() y arrange() en una misma operación fluida y coherente.

4.8. Función `count()`

Esta función permite contar rápidamente los valores únicos de una o más variables en un conjunto de datos. Es especialmente útil para crear tablas de frecuencias absolutas, que posteriormente pueden ser usadas para calcular frecuencias relativas.

Tiene un par de argumentos opcionales:

- `name`: Define el nombre de la columna que contendrá el conteo. Por defecto, esta columna se llama `n`.
- `sort`: Ordena la tabla de frecuencias de mayor a menor (por defecto, no realiza ninguna ordenación).
- `wt`: Permite incorporar una variable que funcione como ponderación (o factor de expansión) para el cálculo de la frecuencia.

Un ejemplo básico de su uso es contar las observaciones por cada valor único de la variable `jurisdiccion` en el conjunto de datos:

```
datos |>
  count(jurisdiccion)
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <int>
 1 Buenos Aires     2
 2 CABA             2
 3 Catamarca        2
 4 Chaco            2
 5 Chubut           2
 6 Cordoba          2
 7 Corrientes       2
 8 Entre Rios       2
 9 Formosa          2
10 Jujuy            2
# i 14 more rows
```

Usando el argumento `wt` obtenemos el conteo de casos por jurisdicción:

```
datos |>
  count(jurisdiccion, wt = casos)
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <dbl>
 1 Buenos Aires  2470
 2 CABA          1328
 3 Catamarca      120
 4 Chaco           24
 5 Chubut         199
 6 Cordoba        836
 7 Corrientes     213
```

```
 8 Entre Rios     253
 9 Formosa        124
10 Jujuy          293
# i 14 more rows
```

Lo cual es equivalente a:

```
datos |>
  group_by(jurisdiccion) |>
  summarise(
    n = sum(casos, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 24 x 2
   jurisdiccion     n
   <chr>        <dbl>
 1 Buenos Aires  2470
 2 CABA          1328
 3 Catamarca      120
 4 Chaco           24
 5 Chubut         199
 6 Cordoba        836
 7 Corrientes     213
 8 Entre Rios     253
 9 Formosa        124
10 Jujuy          293
# i 14 more rows
```

Referencias

5. Gráficos estadísticos con `ggplot2`

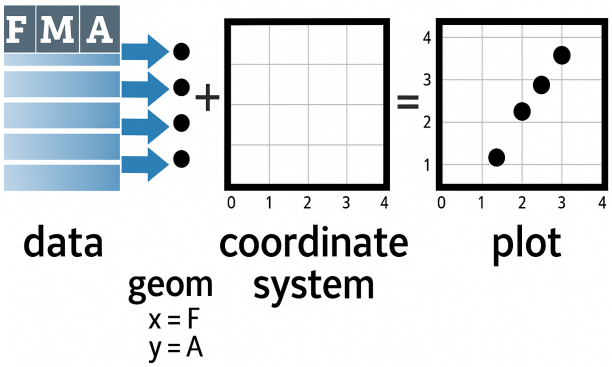


5.1. Introducción

El paquete `ggplot2` (Wickham 2016) es parte del ecosistema `tidyverse` y se autodefine como una librería para "*crear elegantes visualizaciones de datos utilizando una gramática de gráficos*". Proporciona una forma intuitiva de construir gráficos basada en **The Grammar of Graphics** a través de un sistema basado en tres componentes básicos:

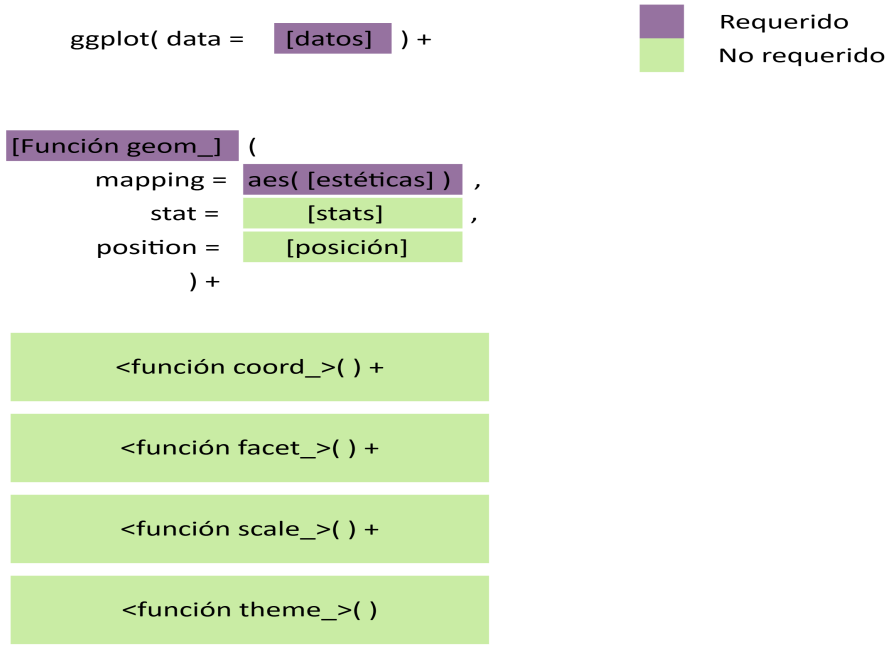
- datos
- coordenadas
- objetos geométricos

La estructura para construir un gráfico es la siguiente:



5.2. Anatomía de gráficos con ggplot2

La construcción de gráficos en ggplot2 se organiza a partir de una gramática que puede entenderse a través de sus componentes fundamentales:



- `data`: el conjunto de datos que vamos a graficar, debe contener toda la información necesaria.
- `aes()`: el mapeo estético (*aesthetic mapping*) es donde se declaran las variables que se representarán en el gráfico (por ejemplo, los ejes X e Y o cómo se asignarán los colores).
- `geoms`: representaciones gráficas de los datos (puntos, líneas, barras, cajas, etc.). Son las "geometrías" que dibujan el gráfico.
- `stats`: transformaciones estadísticas sobre los datos (medias, regresiones, suavizados, etc.), que permiten resumir la información.
- `scales`: controlan cómo se representan los datos (ejes, colores, leyendas).
- `coordinate systems`: sistema de coordenadas utilizado para representar el gráfico en un plano bidimensional.
- `facets`:permiten dividir los datos en subconjuntos y generar gráficos en paneles (viñetas).
- `theme`: controla la apariencia general de los elementos no asociados a los datos (fondos, tipografías, bordes, etc.).

5.2.1. Ejemplo con datos

Antes de mostrar como usar estos componentes, cargaremos la base de datos "`facultad.csv`", que contiene datos ficticios sobre ingresantes a una facultad (por ejemplo, sexo, edad, talla y peso). Este conjunto se utilizará en los ejemplos:

```
# Cargar datos
facultad <- read_csv("datos/facultad.csv")

# Mostramos las 6 primeras observaciones
head(facultad)
```

```
# A tibble: 6 x 18
     hc sexo   edad ant_diabetes ant_tbc ant_cancer ant_obesidad ant_ecv ant_ht
  <dbl> <chr> <dbl> <chr>        <chr>   <chr>      <chr>        <chr>   <chr>
1 26880 M        17 NO           NO      NO         SI           NO      SI
2 26775 M        18 SI           NO      NO         NO           NO      NO
3 26877 M        18 SI           NO      SI         NO           NO      SI
4 26776 M        18 NO           NO      NO         SI           SI      NO
5 26718 M        18 NO           NO      NO         NO           NO      SI
6 26738 M        18 NO           NO      NO         NO           NO      SI
# i 9 more variables: ant_col <chr>, fuma <chr>, edadini <dbl>, cantidad <dbl>,
#   col <dbl>, peso <dbl>, talla <dbl>, sist <dbl>, diast <dbl>
```

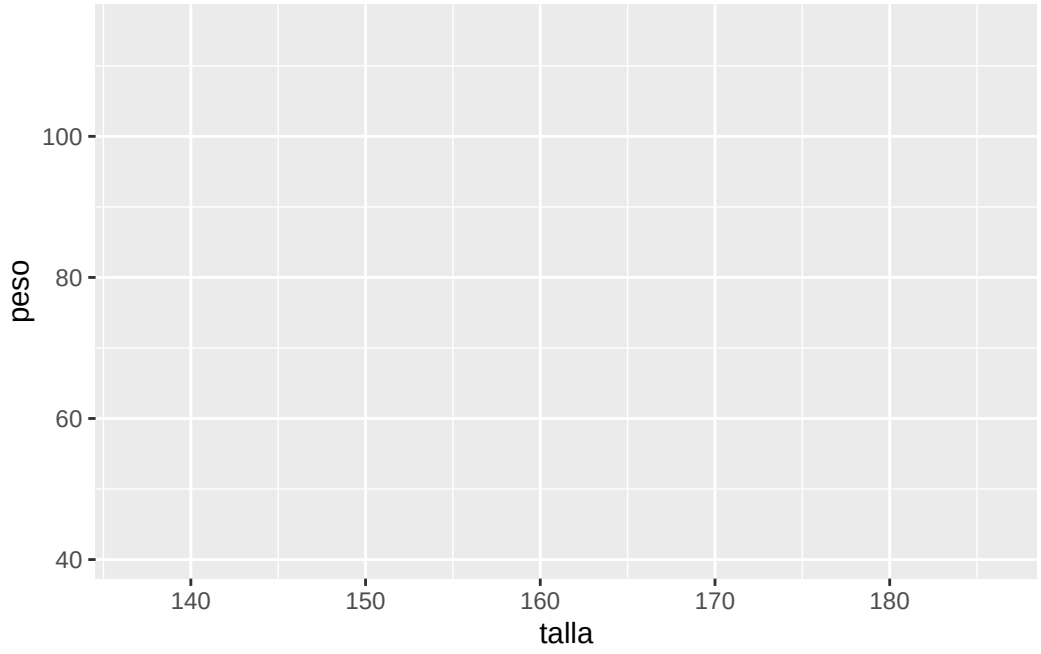
5.2.2. Mapeo estético

La función básica para construir un gráfico en ggplot2 es `ggplot()`, que crea una estructura inicial (vacía) a la que le irán agregando capas. Anteriormente dijimos que la función `aes()` define el contenido estético del gráfico, es decir, indica cómo se representan visualmente las variables (posición, color, tamaño, forma, etc.).

Es importante notar que: - Si aes() se incluye dentro de ggplot(), el mapeo se aplica a todas las capas. - Si se incluye dentro de una geometría, se aplica solo a esa capa. - aes() genera automáticamente leyendas cuando corresponde.

Veamos cómo funciona y cuáles son sus implicancias:

```
facultad |>
  # solo la capa estética aes()
  ggplot(aes(x = talla, y = peso))
```



Este código genera un gráfico vacío que contiene únicamente los ejes definidos (`peso` y `talla`), pero aún no muestra los datos.

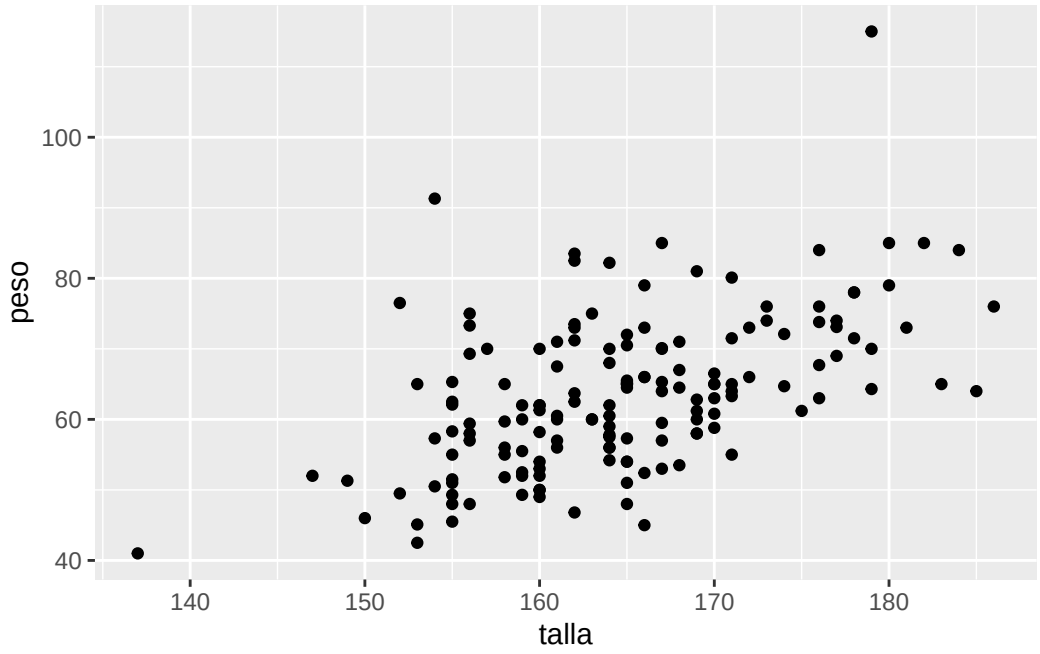
5.2.3. Capas geométricas

Para visualizar los datos, es necesario agregar capas geométricas mediante funciones `geom_`. Estas representan los objetos gráficos que se dibujan en el gráfico.

Las capas se agregan utilizando el operador +:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso)) +

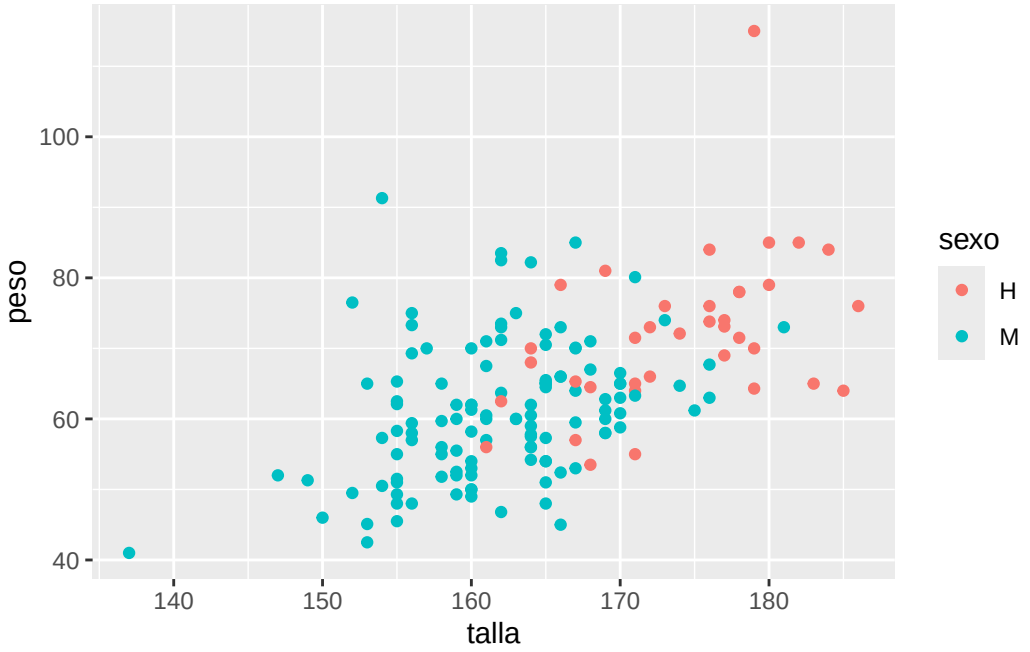
  # agregamos la capa geométrica de puntos
  geom_point()
```



Podemos diferenciar los puntos según `sexo` incorporando esa variable al mapeo estético:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point()
```

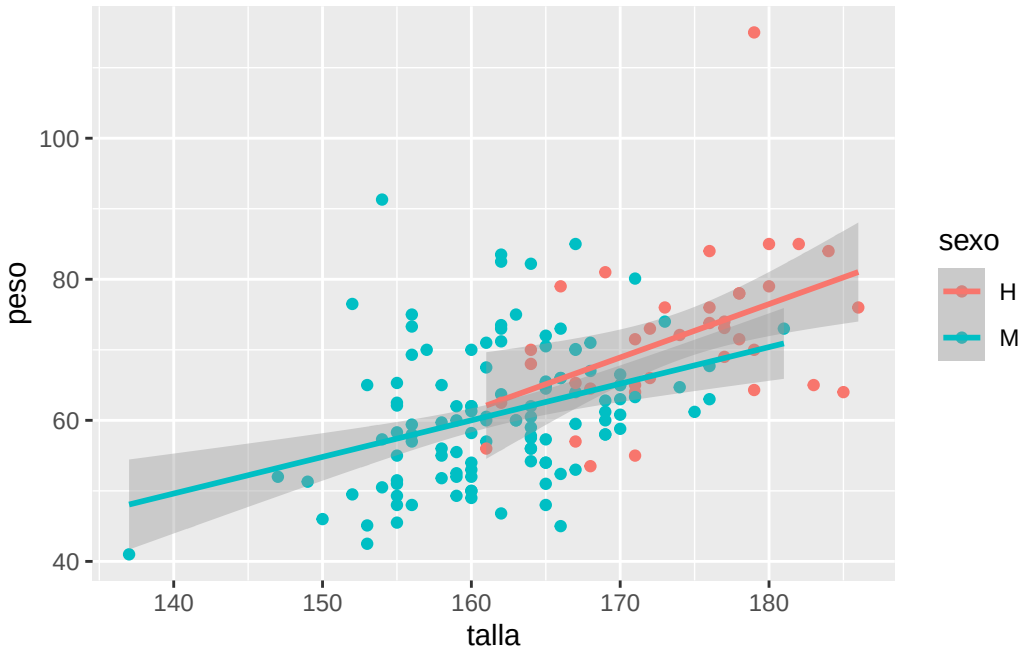


También es posible superponer capas. Por ejemplo, agregar rectas de regresión:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point() +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```



La función `geom_smooth()` permite aplicar distintos métodos de ajuste. En este caso, `"lm"` indica una regresión lineal, incluyendo su intervalo de confianza.

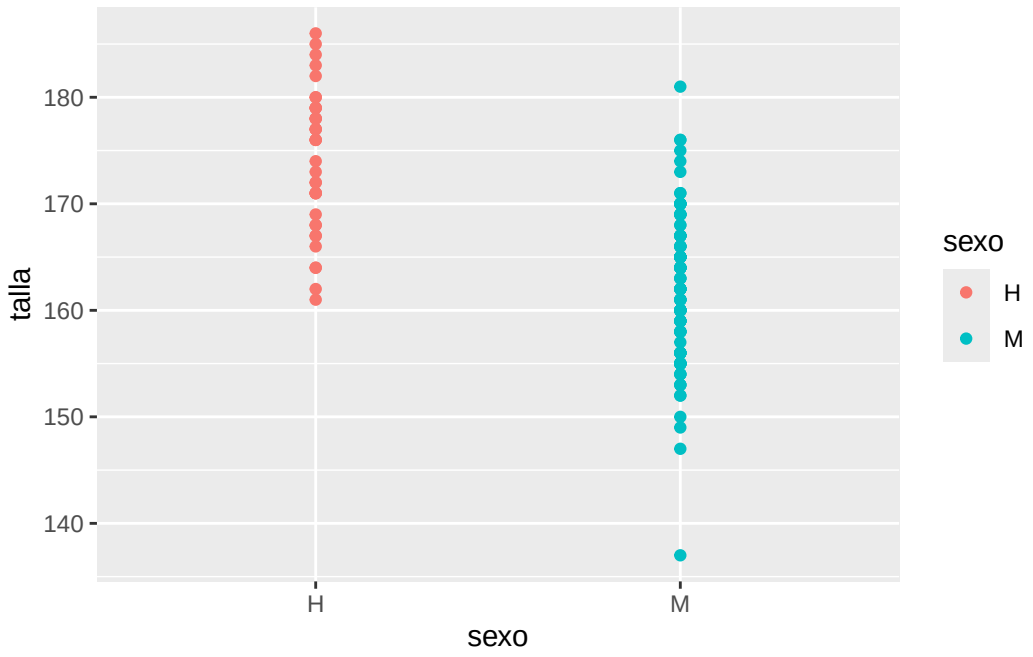
Algunas de las capas geométricas más utilizadas son:

- Histogramas: `geom_histogram()`
- Gráfico de caja y bigotes (*boxplot*): `geom_boxplot()`
- Gráfico de barras: `geom_bar()`
- Gráfico de columnas: `geom_col()`
- Gráfico de puntos (*scatter plot*): `geom_point()` o `geom_jitter()`
- Gráfico de líneas: `geom_line()` o `geom_path()`
- Gráfico de violín: `geom_violin()`
- Tendencias: `geom_smooth()`
- Curvas de densidad: `geom_density()`

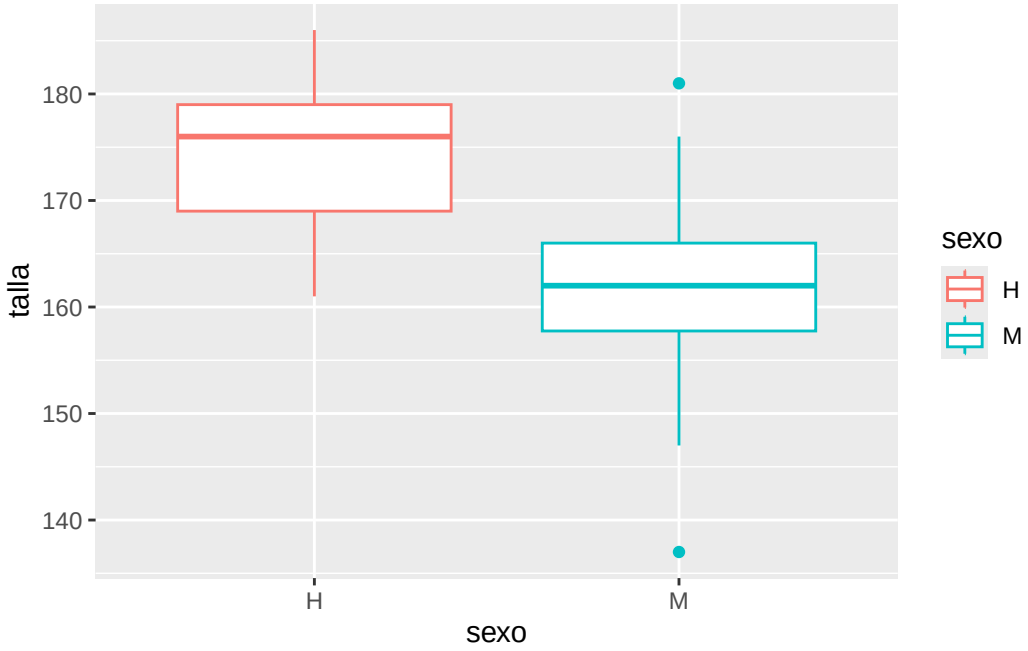
Todas estas funciones pueden aplicarse sobre los mismos datos, y su uso depende del objetivo del análisis. Las capas se grafican secuencialmente, de modo que la última geometría en añadirse aparece adelante.

A continuación, algunos ejemplos para visualizar la relación entre `sexo` y `talla` utilizando distintas capas geométricas:

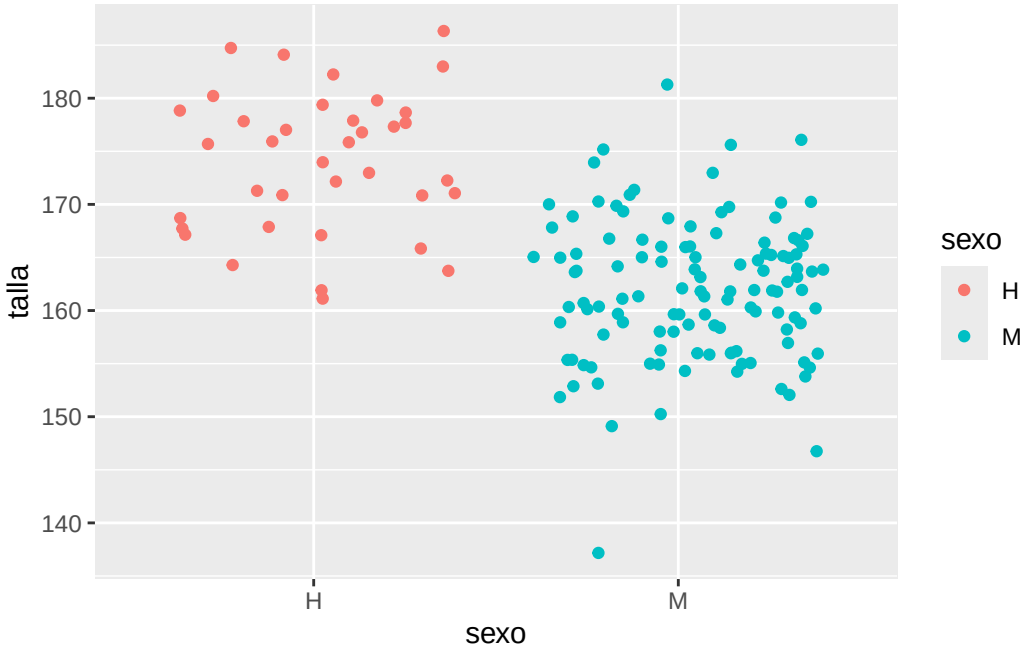
```
# Gráfico de puntos
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica de puntos
  geom_point()
```



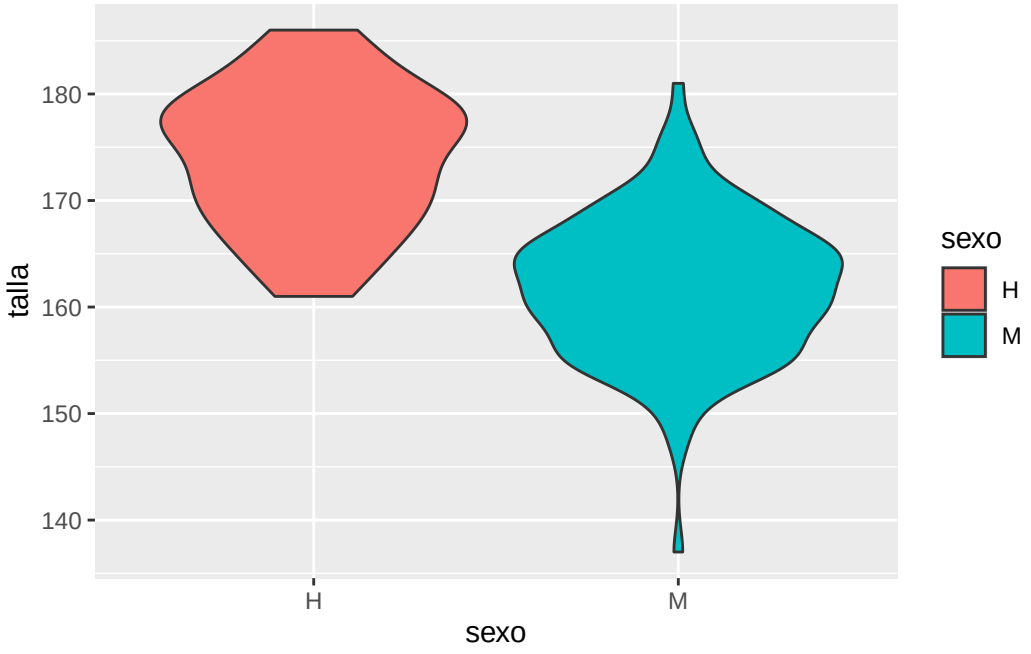
```
# Boxplot
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica de boxplot
  geom_boxplot()
```




```
# Entramado de puntos
facultad |>
  ggplot(aes(x = sexo, y = talla, color = sexo)) +
  # capa geométrica jitter (entramado de puntos)
  geom_jitter()
```




```
# Gráfico de violín
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +
  # capa geométrica de violin
  geom_violin()
```



En el último caso se utiliza `fill` en lugar de `color`. Mientras `color` controla el contorno, `fill` define el relleno de las geometrías.

5.2.4. Estéticas estáticas y dinámicas

En ggplot2, las estéticas son propiedades visuales asociadas a los datos en las capas geométricas. No incluyen elementos como títulos o etiquetas.

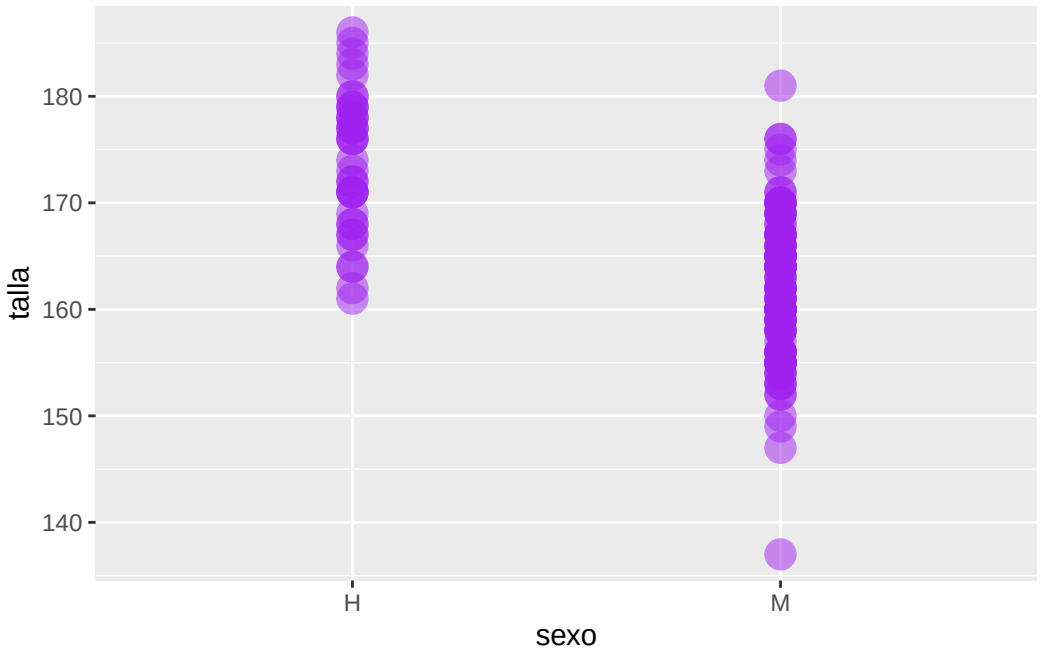
Algunas estéticas frecuentes:

- `color` / `colour`
- `fill`
- `alpha`
- `size`
- `linewidth` / `lwd`
- `linetype`/ `lty`

Valores estáticos

Si queremos que todas las observaciones tengan una misma estética, colocamos los argumentos por fuera de `aes()`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla)) +
  # capa geométrica de puntos
  geom_point(
    color = "purple", # Color de los puntos
    size = 5, # Tamaño de los puntos
    alpha = .5 # Transparencia de los puntos
  )
```



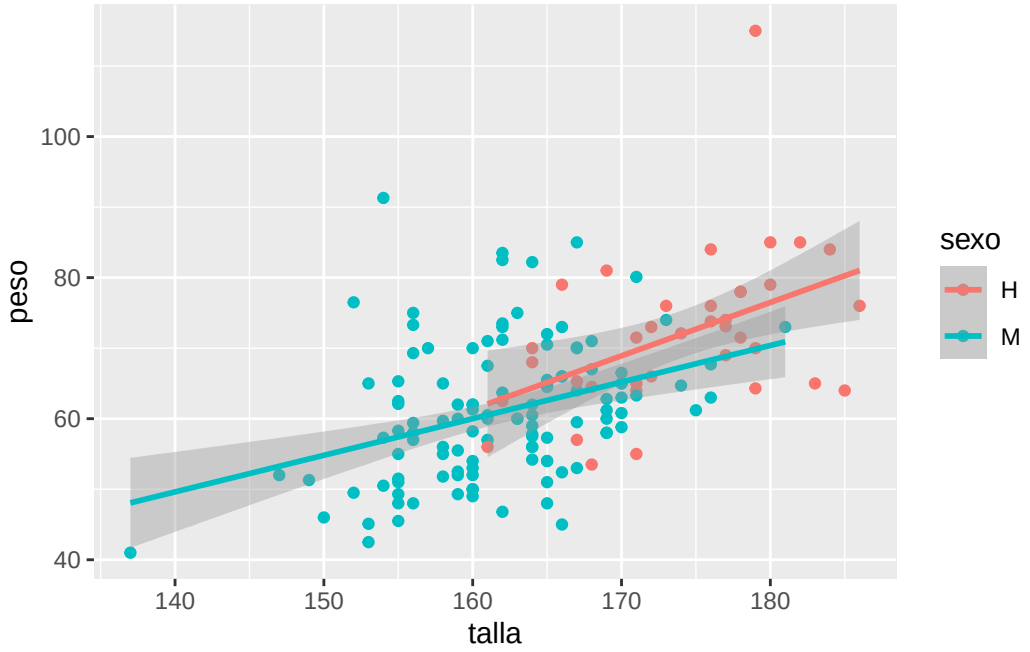
Valores dinámicos

Se definen dentro de `aes()` y aplican a todas las capas:

```
facultad |>
  # Mapeo estético
  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # agregamos la capa geométrica de puntos
  geom_point() +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```

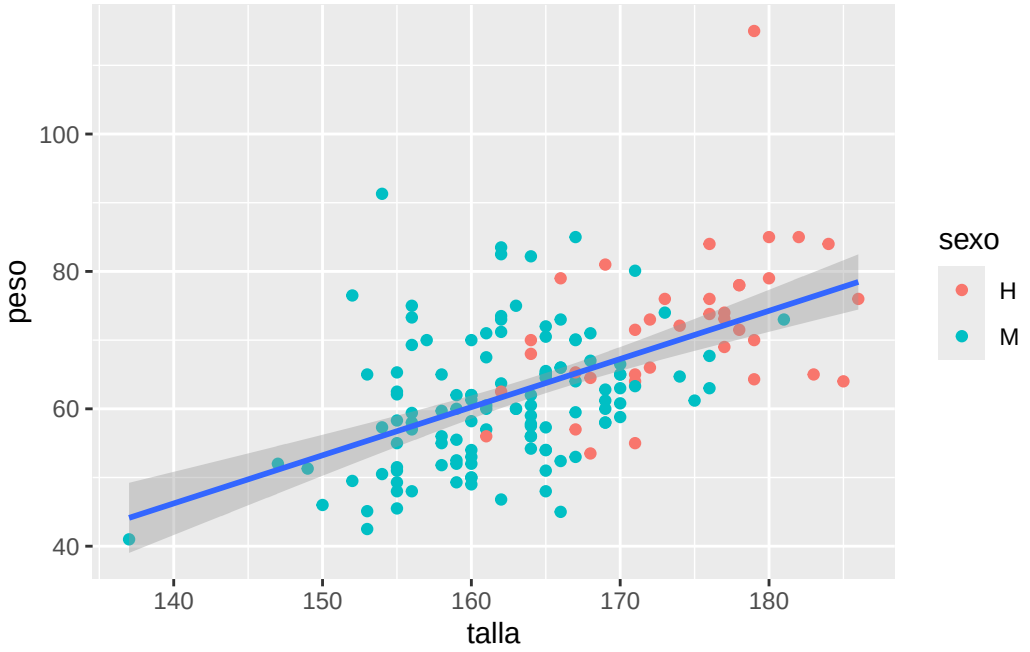


En este otro ejemplo, el color por sexo solo se aplica a los puntos de `geom_point()`, mientras que la regresión se calcula sobre el conjunto completo:

```
facultad |>
  # mapeo estético
  ggplot(aes(x = talla, y = peso)) +

  # agregamos la capa geométrica de puntos coloreada por sexo
  geom_point(aes(color = sexo)) +

  # agregamos una segunda capa geométrica para la recta de regresión
  geom_smooth(method = "lm")
```



Este comportamiento permite una gran flexibilidad al construir gráficos, definiendo qué capas responden a cada mapeo estético.

5.3. Personalización de escalas

El sistema de escalas en `ggplot2` permite ajustar distintos aspectos visuales de un gráfico, como colores de contorno y relleno, ejes, tamaños y tipos de línea, entre otros.

Todas las funciones asociadas a escalas comienzan con el prefijo `scale_`, seguido del atributo que se desea modificar (por ejemplo, `scale_fill_` para el color de relleno).

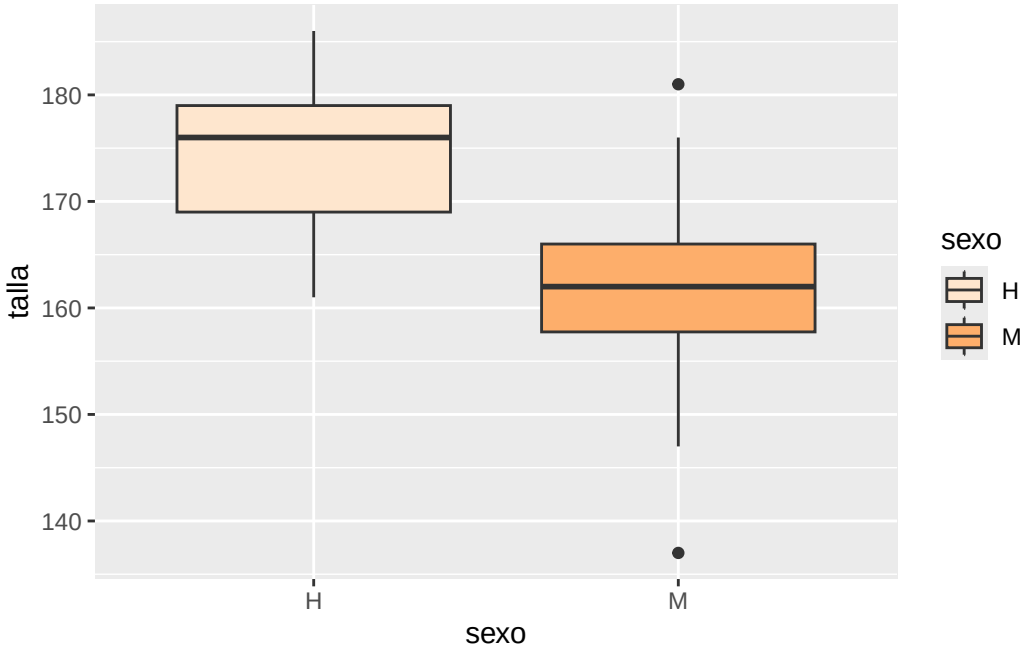
5.3.1. Escalas de colores

A continuación, se presentan algunos ejemplos utilizando el conjunto de datos `facultad`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta de naranjas
  scale_fill_brewer(palette = "Oranges")
```



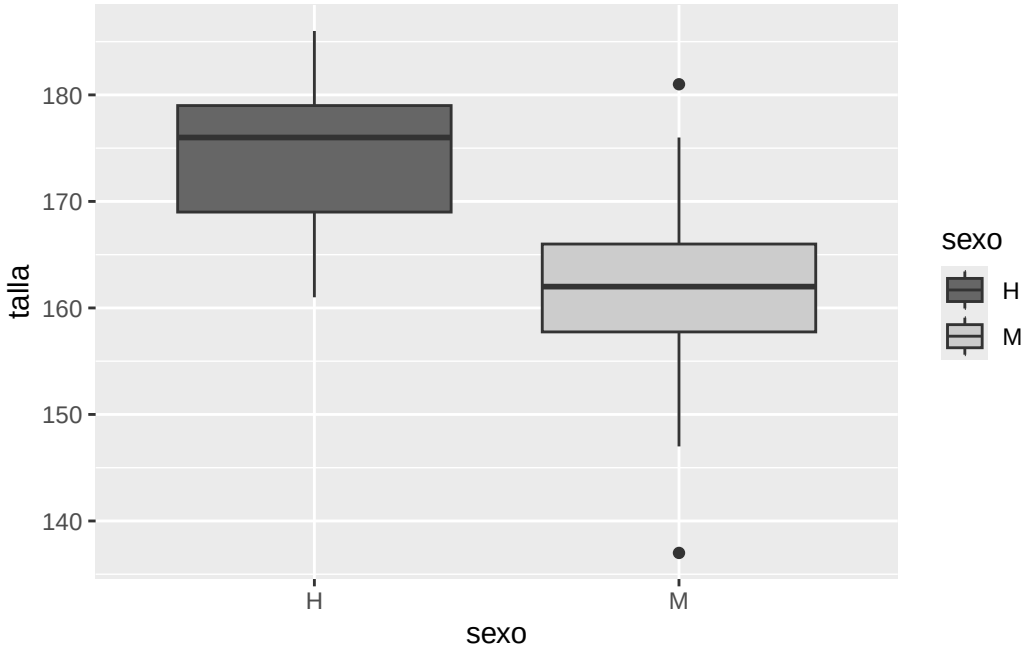
En este ejemplo, se aplica la función `scale_fill_brewer()`, que utiliza una paleta de colores (en este caso, `"Oranges"`) y se vincula con el argumento `fill` definido en `aes()`, determinando el color de relleno del *boxplot*.

Otra alternativa es utilizar una escala de grises mediante `scale_fill_grey()`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala de grises
  scale_fill_grey(start = 0.4, end = 0.8)
```



Se recomienda trabajar con paletas de colores accesibles para personas con daltonismo y otras dificultades en la percepción visual, como las incluidas en las dependencias `RColorBrewer` y `viridisLite`, así como en paquetes específicos con paletas *colorblind-friendly*, como `scico` (Pedersen y Crameri 2023) y `cols4all` (Tennekes y Puts 2023).

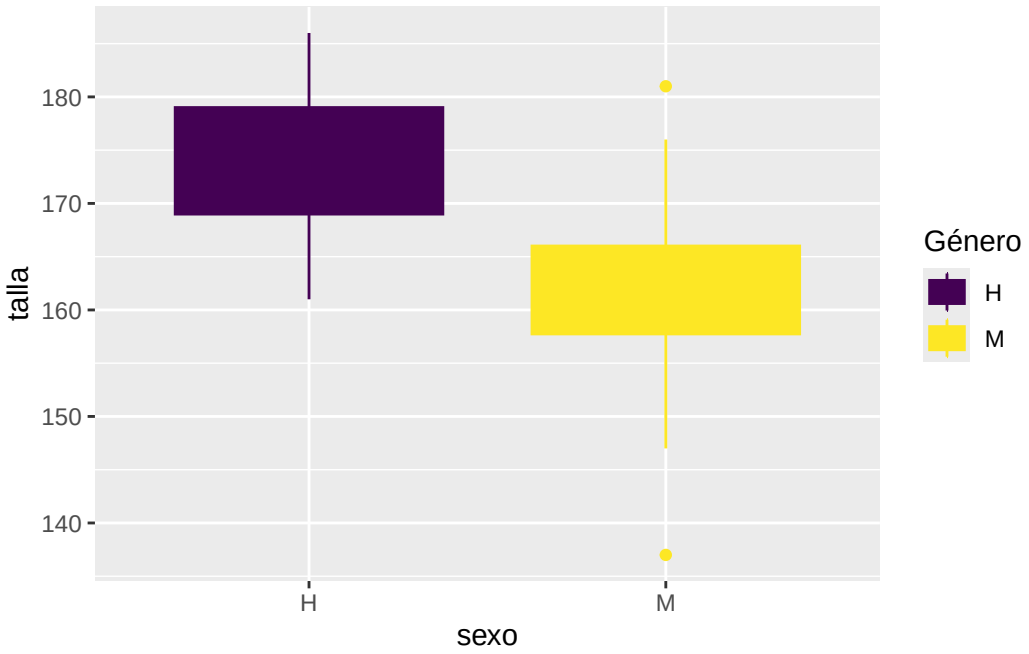
En versiones recientes de `ggplot2`, varias funciones de escala permiten utilizar el argumento `aesthetics`, que facilita aplicar una misma escala a múltiples estéticas simultáneamente. Esto resulta especialmente útil cuando queremos usar una misma paleta de colores tanto para el contorno (`color`) como para el relleno (`fill`).

Retomando el ejemplo anterior:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo, color = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala viridis
  scale_fill_viridis_d(
    name = "Género",
    aesthetics = c("fill", "color")
  )
```



En este caso, la paleta `"viridis"` se aplica simultáneamente al relleno y al contorno de las cajas, evitando inconsistencias visuales entre ambas estéticas.

Sin `aesthetics` los colores de relleno y de línea deberían especificarse como:

```
# paleta en escala viridis
scale_fill_viridis_d(name = "Género") +
  scale_color_viridis_d()
```

Lo que duplica código y puede desincronizar los cambios en una paleta.

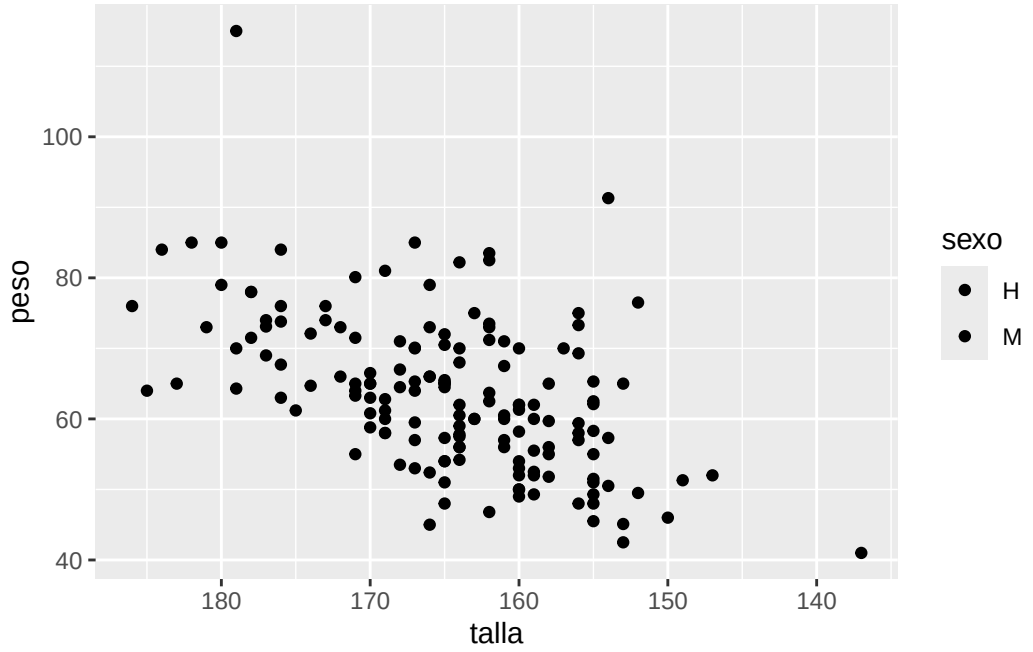
5.3.2. Escalas de ejes

También es posible modificar las escalas de los ejes. Por ejemplo, invertir el eje X con `scale_x_reverse()`:

```
facultad |>
  ggplot(aes(x = talla, y = peso, fill = sexo)) +

  # capa geométrica de puntos
  geom_point() +

  # invierte el eje x
  scale_x_reverse()
```



La inclusión de `scale_x_reverse()` provoca que la variable `talla` se muestre en orden descendente (de mayor a menor).

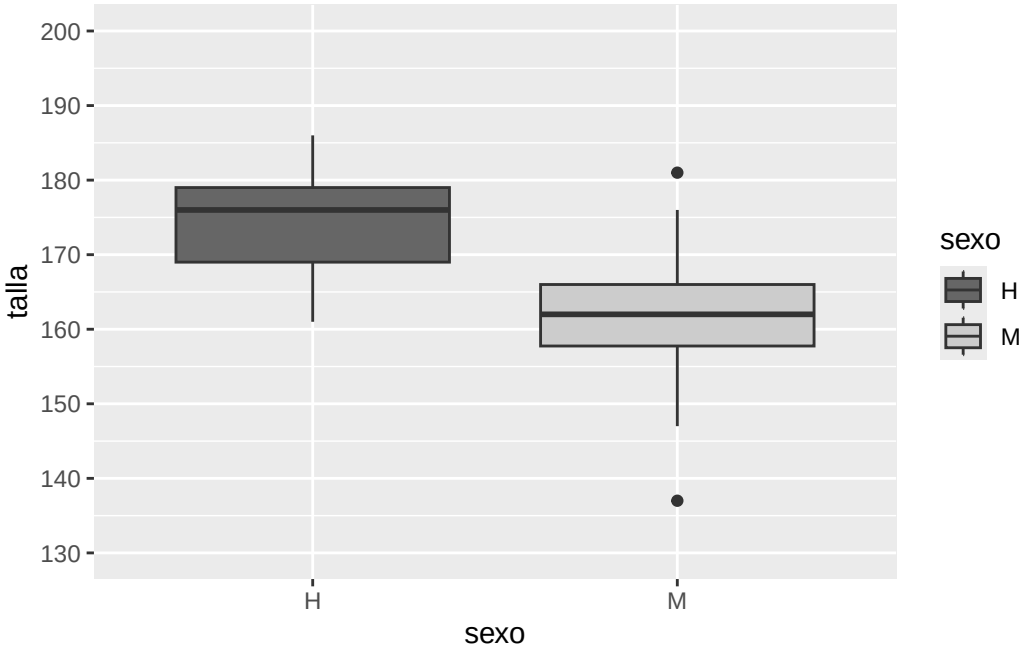
Por último, se pueden personalizar los cortes del eje Y mediante `scale_y_continuous()`. En el siguiente ejemplo, se define que el eje Y vaya de 130 a 200 cm, con marcas cada 10 unidades:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta en escala de grises
  scale_fill_grey(start = 0.4, end = 0.8) +

  # puntos de corte del eje Y
  scale_y_continuous(
    limits = c(130, 200), # límites
    breaks = seq(130, 200, 10)
  ) # nro de etiquetas
```



5.4. Transformaciones estadísticas

En ggplot2, algunos gráficos no requieren transformaciones estadísticas explícitas, como los gráficos de dispersión. Sin embargo, otros —como histogramas, *boxplots* o curvas de tendencia— incorporan **transformaciones estadísticas implícitas** que pueden modificarse o personalizarse.

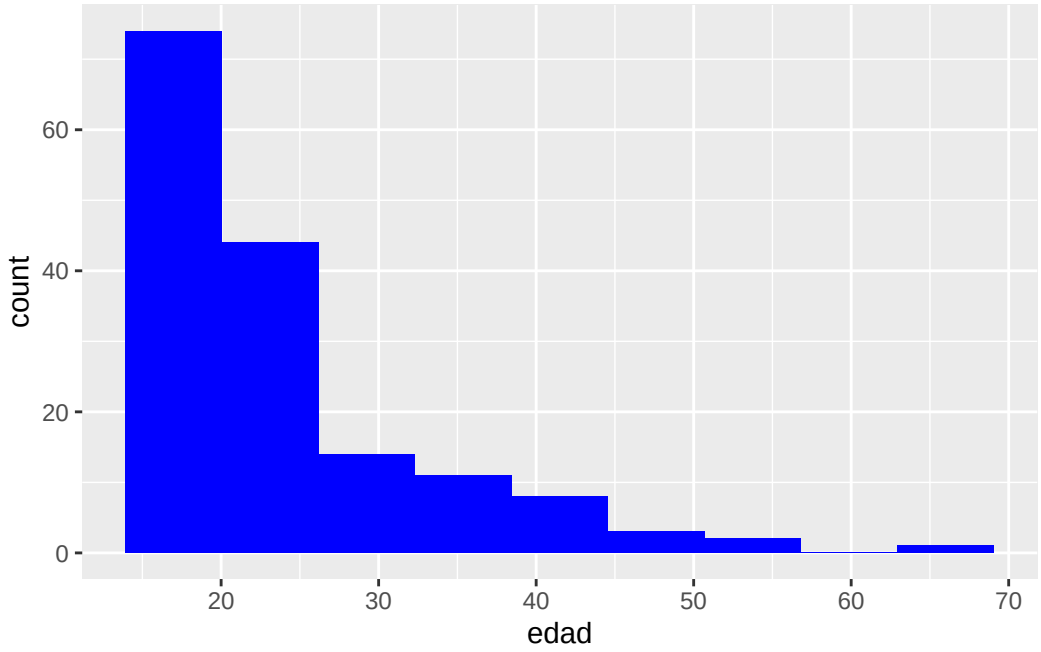
Estas transformaciones pueden:

- estar integradas dentro de las funciones geométricas (`geom_`)
- agregarse como capas independientes mediante funciones `stat_`

Por ejemplo, en un histograma la transformación estadística consiste en agrupar los datos en intervalos (clases) y contar las observaciones en cada uno. Esta transformación está incluida en `geom_histogram()`, pero puede parametrizarse:

```
facultad |>
  ggplot(aes(edad)) +

  # capa geométrica histograma
  geom_histogram(bins = nclass.Sturges(facultad$edad), fill = "Blue")
```



En este caso, se utiliza la regla de Sturges —a través de `nclass.Sturges()`— para determinar automáticamente la cantidad de clases de la variable `edad`.

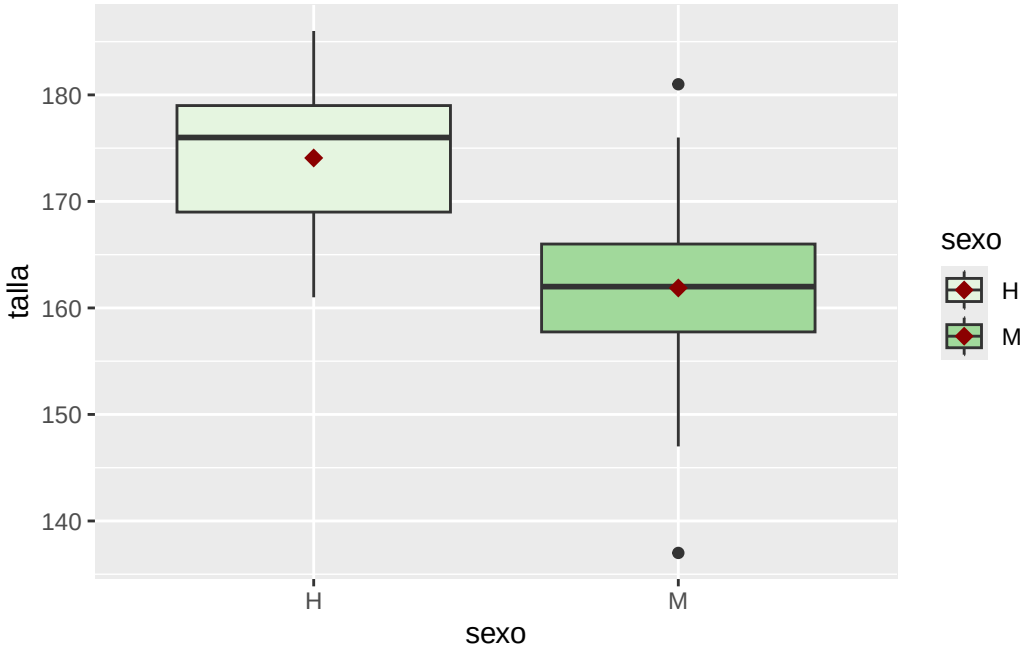
También es posible agregar transformaciones estadísticas como capas adicionales. Por ejemplo, si queremos mostrar la media de `talla` para cada grupo de `sexo` sobre un *boxplot*, podemos utilizar `stat_summary()`:

```
facultad |>
  ggplot(aes(x = sexo, y = talla, fill = sexo)) +

  # capa geométrica boxplot
  geom_boxplot() +

  # paleta de tonos verdes
  scale_fill_brewer(palette = "Greens") +

  # añade capa summary
  stat_summary(
    fun = mean,
    color = "darkred",
    geom = "point",
    shape = 18,
    size = 3
  )
```



La función `stat_summary()` permite calcular estadísticas (como `mean`, `median`, `sd`, entre otras) y representarlas mediante una geometría. En este caso, la media de `talla` se muestra como un punto rojo oscuro (`color = "darkred"`), con forma romboidal (`shape = 18`) y tamaño destacado (`size = 3`).

5.5. Facetado

El **facetado** permite dividir un gráfico en múltiples paneles (o viñetas) según los niveles de una o más variables categóricas. Esto resulta especialmente útil para comparar patrones entre grupos sin sobrecargar un único gráfico.

En ggplot2 existen dos funciones principales para implementar esta técnica:

- `facet_wrap()`: divide los datos según una única variable categórica, generando paneles dispuestos automáticamente en filas y columnas.
- `facet_grid()`: permite crear una matriz de paneles a partir de dos variables categóricas, una definida en las filas y otra en las columnas.

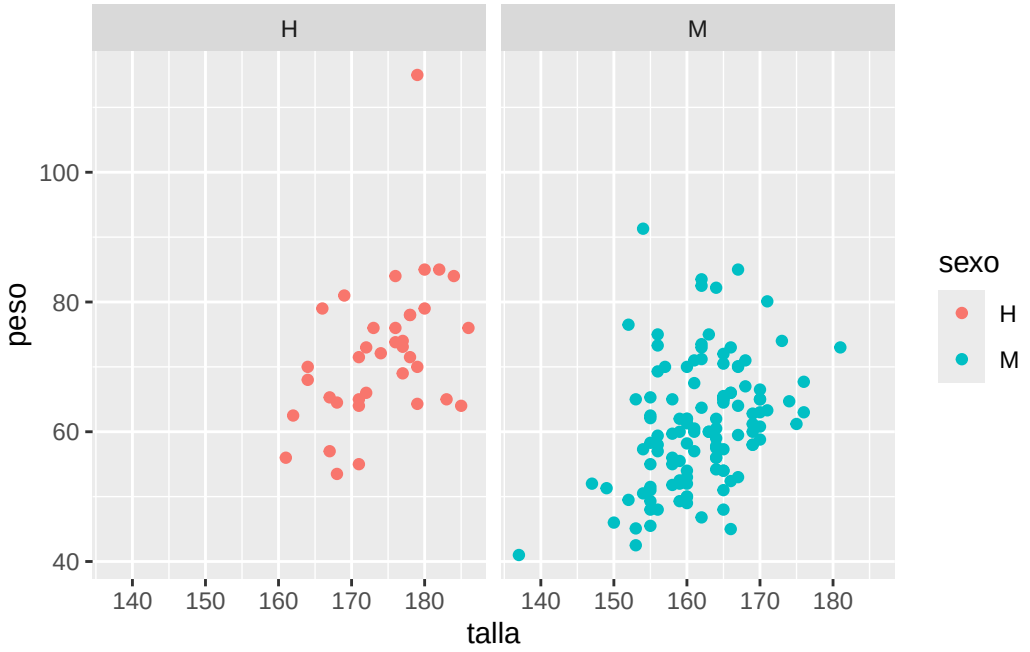
Retomemos el gráfico de dispersión entre `talla` y `peso`, y generemos paneles separados para cada nivel de la variable `sexo` usando `facet_wrap()`:

```
facultad |>

  ggplot(aes(x = talla, y = peso, color = sexo)) +

  # capa geométrica de puntos
  geom_point() +

  # separa en paneles por sexo
  facet_wrap(~sexo)
```



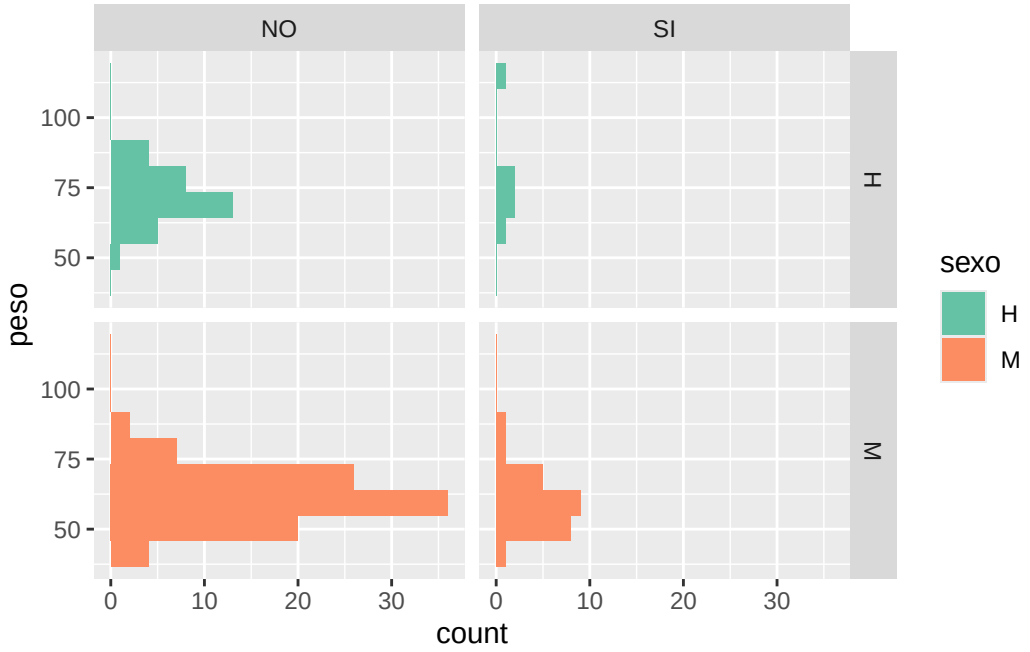
Ahora utilizamos `facet_grid()` para crear una matriz de histogramas a partir del cruce de las variables `sexo` y `fuma`. En cada panel se representa la distribución de `peso`:

```
facultad |>
  ggplot(aes(y = peso, fill = sexo)) +

  # capa geométrica histograma
  geom_histogram(bins = nclass.Sturges(facultad$peso)) +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # separa en paneles por sexo y fuma
  facet_grid(sexo ~ fuma)
```



Como se observa en estos ejemplos, el facetado se integra naturalmente con otros componentes de la gramática de gráficos: mapeo estético (`aes()`), capas geométricas (`geom_`) y escalas (`scale_`).

El número de combinaciones posibles es muy amplio, dada la flexibilidad de `ggplot2`. Sin embargo, el objetivo de este material es comprender los principios fundamentales sobre los que se construye esta *gramática de los gráficos*.

5.6. Sistema de coordenadas

En algunos casos, puede ser útil modificar el sistema de coordenadas predeterminado de un gráfico. Mientras que las funciones `scale_` controlan **cómo se representan los datos**, las funciones de coordenadas (`coord_`) modifican **cómo se visualiza el espacio del gráfico**.

En `ggplot2` una de las transformaciones más comunes es invertir la orientación de los ejes mediante `coord_flip()`, lo que resulta útil, por ejemplo, para presentar gráficos de barras en forma horizontal:

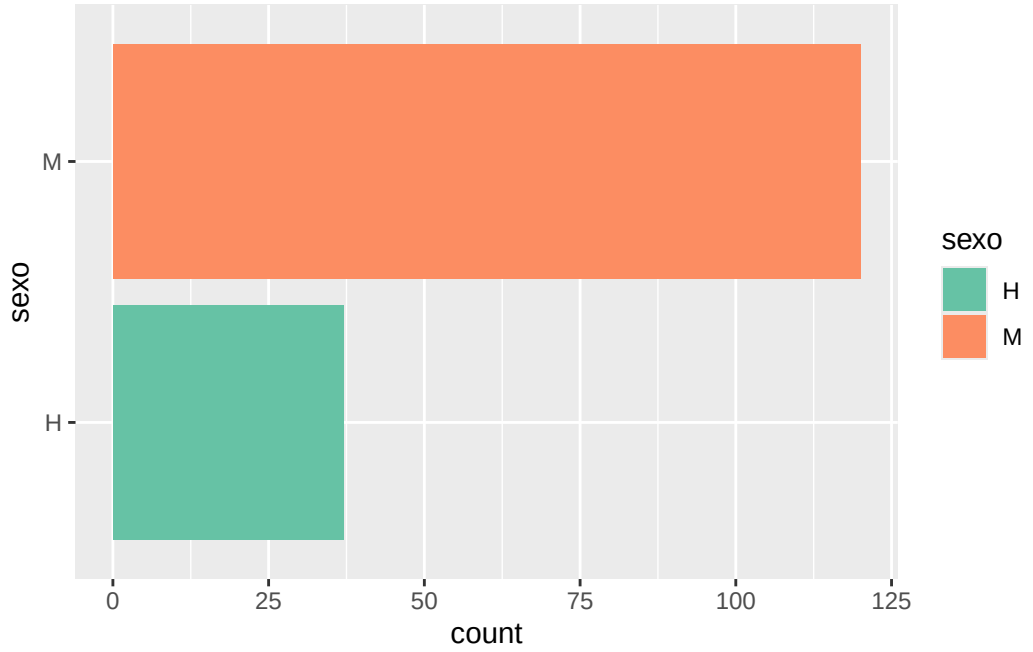
```
facultad |>

  ggplot(aes(sexo, fill = sexo)) +

  # capa geométrica barras
  geom_bar() +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # invierte disposición de ejes
  coord_flip()
```



5.7. Temas

ggplot2 incluye un conjunto de temas gráficos predefinidos que permiten modificar el aspecto general del gráfico. El tema por defecto es `theme_gray()`, pero puede cambiarse agregando una capa `theme_*()` dentro del gráfico.

Repetimos el gráfico anterior, esta vez utilizando el tema blanco y negro (`theme_bw()`):

```
facultad |>

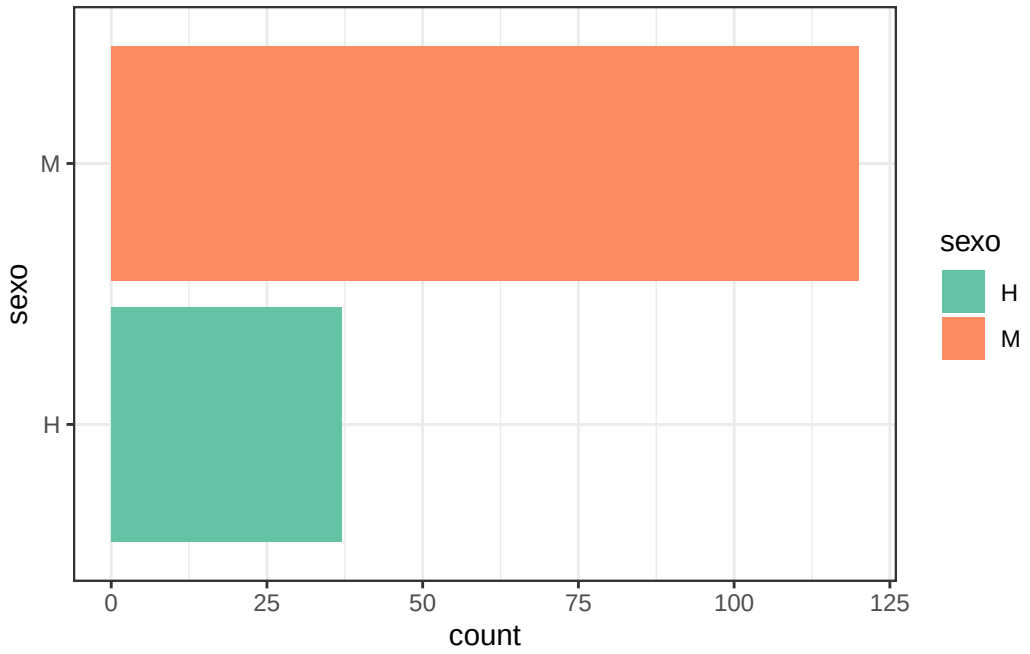
  ggplot(aes(sexo, fill = sexo)) +

  # capa geométrica barras
  geom_bar() +

  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # invierte disposición de ejes
  coord_flip() +

  # tema en blanco y negro
  theme_bw()
```



También podemos aplicar un tema de fondo oscuro con `theme_dark()`:

```
facultad |>

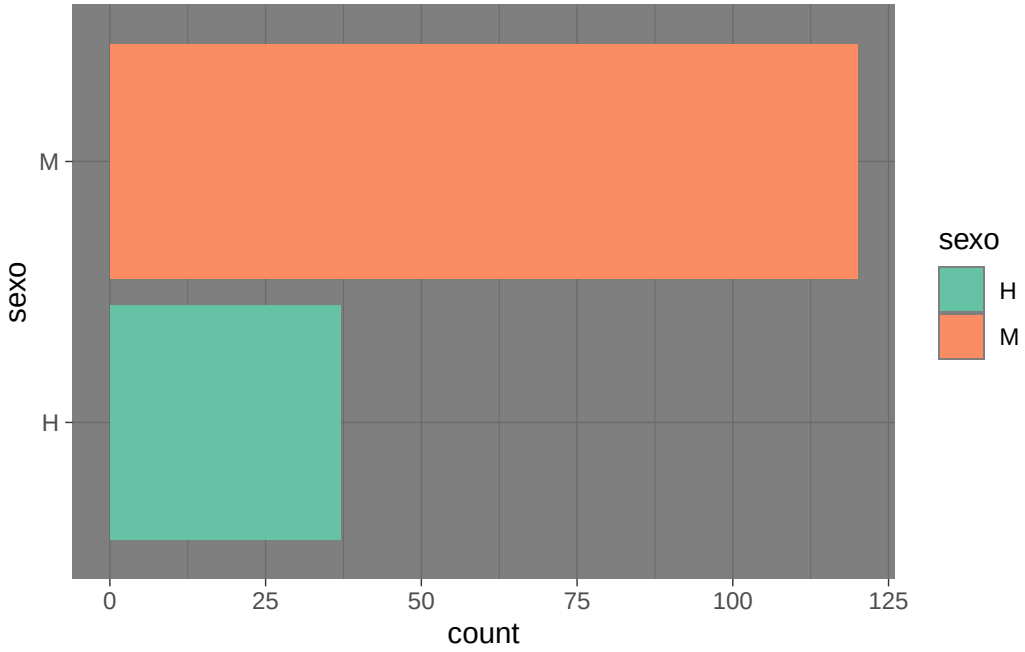
  ggplot(aes(sexo, fill = sexo)) +

  # capa geométrica barras
  geom_bar() +

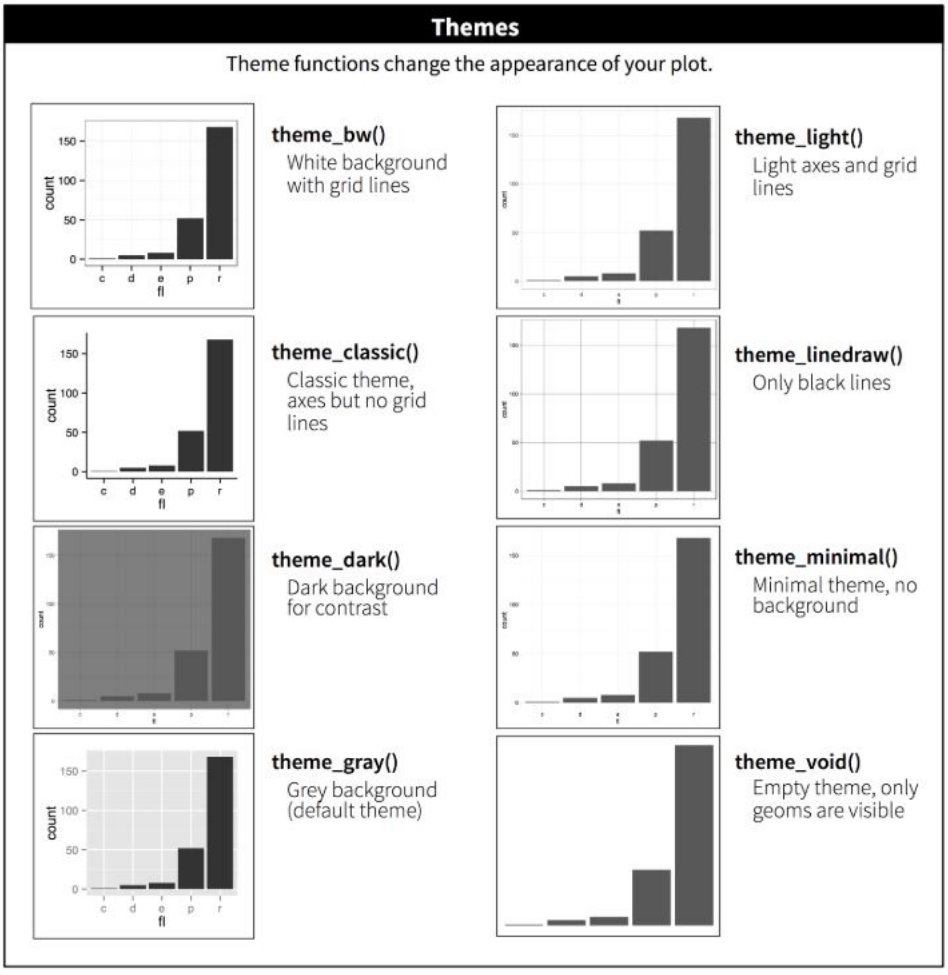
  # paleta de colores
  scale_fill_brewer(palette = "Set2") +

  # invierte disposición de ejes
  coord_flip() +

  # tema con fondo oscuro
  theme_dark()
```



A continuación se muestra un cuadro con los principales temas disponibles en `ggplot2` y sus características visuales:

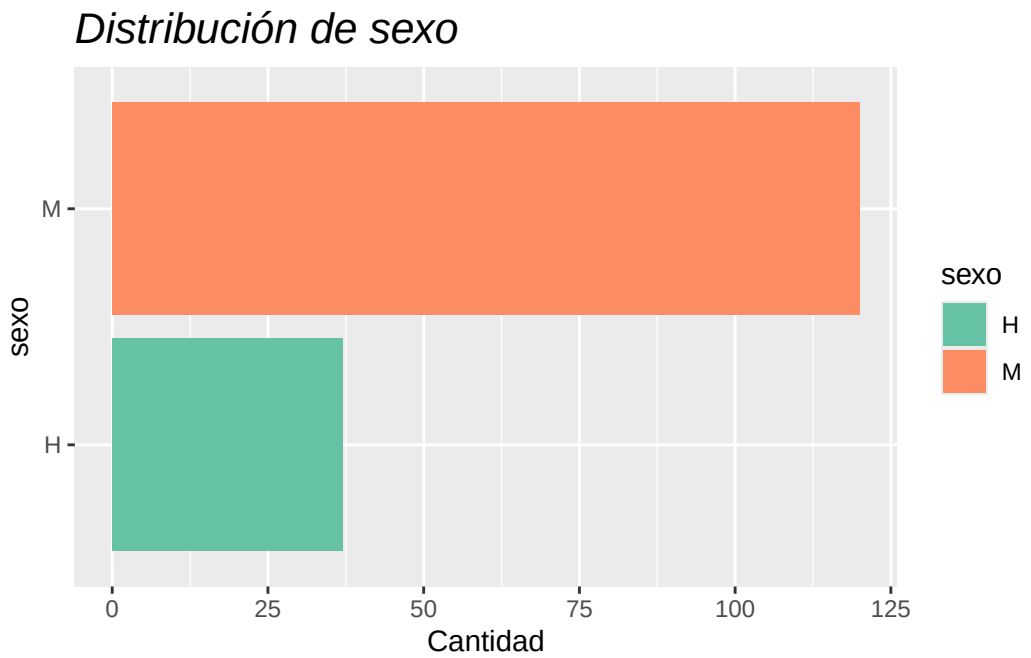


Además del aspecto general del gráfico, es posible agregar títulos, subtítulos y etiquetas de ejes con la función `labs()`. Por ejemplo:

```
labs(
  x = "Etiqueta X",
  y = "Etiqueta Y",
  title = "Título del gráfico",
  subtitle = "Subtítulo del gráfico"
)
```

También se pueden ajustar detalles del texto, como tipo de fuente o tamaño, mediante la función `theme()`:

```
facultad |>
  ggplot(aes(sexo, fill = sexo)) +
  # capa geométrica barras
  geom_bar() +
  # paleta de colores
  scale_fill_brewer(palette = "Set2") +
  # invierte disposición de ejes
  coord_flip() +
  # cambia etiquetas de los ejes
  labs(y = "Cantidad", title = "Distribución de sexo") +
  # modifica el estilo de fuente del título
  theme(plot.title = element_text(face = "italic", size = 16))
```



5.8. Paquete esquisse



El paqueteesquisse (Meyer y Perrier 2025) es una extensión de ggplot2 que permite crear gráficos de manera interactiva, mediante una interfaz gráfica intuitiva basada en el sistema de arrastrar y soltar (*drag & drop*).

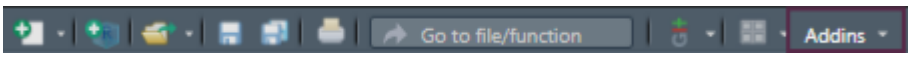
Con esquisse es posible:

- Explorar visualmente los datos según su tipo.
- Asignar variables a diferentes estéticas del gráfico (ejes, color, tamaño, etc.).
- Exportar los resultados en distintos formatos.
- Recuperar el código R que genera el gráfico, facilitando su reproducción y modificación.

El paquete se instala mediante el menú **Packages** de RStudio o ejecutando:

```
install.packages("esquisse")
```

Luego se puede acceder a la aplicación por medio del acceso Addins:



o ejecutando en consola:

```
esquisser()
```

También se puede agregar el nombre de la tabla de datos dentro de los paréntesis

```
esquisser(datos)
```

Para más detalles, se puede consultar la viñeta del paquete disponible en CRAN o en su repositorio de GitHub.

5.9. Extensiones de ggplot2

Una de las grandes fortalezas de ggplot2 es su ecosistema de extensiones. Hasta el momento, existen más de 158 **paquetes** desarrollados para ampliar sus funcionalidades, muchos de ellos diseñados para facilitar tareas específicas o agregar nuevas capas, geometrías, temas, escalas o herramientas interactivas.

Algunas de las extensiones que utilizaremos durante el curso son:

- patchwork (Pedersen 2025): permite combinar múltiples gráficos de ggplot2 en una misma figura de forma sencilla y controlada.
- GGally (Schloerke et al. 2025): extiende ggplot2 con funciones para crear matrices de gráficos (como ggpairs()), útiles para análisis exploratorio multivariado.
- ggpubr (Kassambara 2026a): facilita la creación de gráficos listos para publicar, con funciones simplificadas para agregar títulos, etiquetas, comparaciones estadísticas y anotaciones.
- see (Lüdecke et al. 2021): ofrece temas, paletas de colores y escalas compatibles con personas con daltonismo, además de geometrías personalizadas.
- survminer(Kassambara et al. 2026): diseñado para la visualización de análisis de supervivencia (como curvas de Kaplan-Meier) utilizando ggplot2.
- ggfortify(Horikoshi y Tang 2018): permite graficar objetos estadísticos como modelos lineales, PCA o series temporales sin necesidad de escribir código ggplot2 desde cero.

Estas extensiones permiten ampliar la flexibilidad y expresividad gráfica de ggplot2 sin perder la estructura gramatical que lo caracteriza.

Referencias

6. Análisis exploratorio de datos



6.1. Introducción

El análisis exploratorio de datos (conocido como **EDA**, su sigla en inglés) es un enfoque de análisis fundamental para resumir y visualizar las características importantes de un conjunto de datos.

John Tukey, estadístico estadounidense, fue uno de los principales impulsores de este enfoque. En 1977 publicó el libro *Exploratory Data Analysis*, donde, entre otras contribuciones, introdujo el gráfico *boxplot* (diagrama de caja y bigotes).

En términos sencillos, antes de avanzar hacia el análisis formal o la construcción de modelos estadísticos, resulta esencial explorar, conocer y describir las variables presentes en nuestra tabla de datos.

Entre los principales objetivos perseguidos por EDA se encuentran:

- Conocer la estructura de la tabla de datos y sus tipos de variable.
- Detectar observaciones incompletas (valores *missing* o `NA`).
- Explorar la distribución de las variables de interés a partir de:
    - Estadísticos descriptivos
    - Representaciones gráficas
- Detectar valores atípicos (*outliers*).

Aclaración

En este documento utilizaremos funciones del lenguaje R basadas en la filosofía *tidyverse*, junto con otros paquetes diseñados para tareas específicas. Esto no implica que no se puedan emplear funciones del R base; sin embargo, el ecosistema `tidyverse` facilita la comprensión y legibilidad del código.

Presentaremos estas diferentes funciones de distintos paquetes que pueden servir en cada etapa de un EDA. Los paquetes con los que trabajaremos son:

- `tidyverse`
- `skimr`
- `dlookr`
- `janitor`

> ⚠ **Nota:** Algunos paquetes, como `dlookr`, pueden generar falsos positivos en la detección del antivirus durante el proceso de instalación. Sugerimos desactivar momentáneamente el antivirus para evitar inconvenientes.

Una vez instalados, podemos activar los paquetes con el siguiente código:

```
# Carga de paquetes
library(skimr)
library(janitor)
library(dlookr)
library(tidyverse)
```

Se recomienda cargar `tidyverse` al final de la lista para evitar conflictos con funciones que puedan solaparse entre paquetes.

> Es importante destacar que no existe un único camino y/o función para realizar un análisis exploratorio. Esta selección de herramientas puede adaptarse según las preferencias y necesidades de cada usuario. Por lo tanto, quienes ya tengan familiaridad con otras funciones o paquetes pueden continuar utilizándolos sin inconvenientes.

Para ilustrar los pasos del análisis exploratorio, utilizaremos un archivo con datos ficticios llamado "`datos2.txt`", que contiene variables de distintos tipos.

6.2. Conocer la estructura de la tabla de datos y sus tipos de variable

El primer paso en la exploración de un conjunto de datos es conocer su estructura y tamaño:

- El **tamaño** se refiere a la cantidad de observaciones (filas) y de variables (columnas).
- La **estructura** incluye cómo están organizadas las variables, qué tipo de datos contiene cada una y qué categorías o valores pueden tomar.

Comenzaremos por cargar los datos de ejemplo con la función `read_csv2()` de `tidyverse`:

```
datos <- read_csv2("datos/datos2.txt")
```

Una vez cargados los datos, la función `glimpse()` permite obtener una visión general de la tabla:

```
glimpse(datos)
```

```
Rows: 74
Columns: 7
$ id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,~
$ sexo    <chr> "M", "M", "M", "M", "M", "M", "M", "M", NA, "F", "F", "M", "F"~
$ edad    <dbl> 76, 68, 50, 49, 51, 68, 70, 64, 60, 57, 83, 76, 27, 34, 17, 45~
$ peso    <dbl> 71.0, 71.0, 79.0, 71.0, 87.0, 75.0, 80.0, 83.0, 69.0, 73.0, 60~
$ talla   <dbl> 167.0, 164.0, 164.0, 164.0, 167.5, 170.0, 166.0, 160.0, 160.0,~
$ trabaja <lgl> FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, NA, TRUE, TRUE, TRUE, ~
$ fecha   <chr> "20/10/2020", "20/10/2020", "20/10/2020", "05/11/2020", "05/11~
```

Esta función nos informa que la tabla contiene, por ejemplo, 74 observaciones y 7 variables, mostrando el tipo de dato de cada una y los primeros valores que aparecen.

Entre los tipos de datos más comunes que podemos encontrar se incluyen:

- `int` (*integer*): números enteros.
- `dbl` (*double*): números reales.
- `lgl` (*logical*): valores lógicos (`TRUE`, `FALSE`).
- `chr` (*character*): texto o cadenas de caracteres.
- `Date`: fechas.
- `fct` (*factor*): variables categóricas con niveles.
- `dttm` (*date-time*): fechas y horas.

Esta primera revisión de la estructura suele complementarse con el **diccionario de datos**, un recurso fundamental que describe el significado, tipo, unidad y codificación de cada variable. Este diccionario puede acompañar tanto a bases de datos generadas en investigaciones propias (fuentes primarias) como a datos provenientes de fuentes secundarias.

Es importante tener en cuenta que el tipo de dato en R no siempre coincide con la naturaleza estadística de la variable. Por ejemplo:

- Una variable codificada como `dbl` puede representar una medida cuantitativa continua, como la edad o el peso.
- Pero también puede representar una variable cualitativa codificada con números. Por ejemplo, si una variable que registra respuestas “Sí” y “No” fue codificada como `1` y `0`, su tipo de dato será numérico (`dbl` o `int`), aunque conceptualmente sea una variable categórica.

Por esta razón, además de inspeccionar el tipo de datos en R, es importante revisar el significado y el uso previsto de cada variable dentro del contexto del análisis.

6.3. Detectar observaciones incompletas

Los valores perdidos o faltantes (conocidos como *missing* en inglés), representados en R por el valor especial `NA`, constituyen un desafío importante en el análisis de datos. Su presencia puede afectar la calidad del análisis y condicionar las decisiones estadísticas posteriores.

Existen numerosos enfoques para el tratamiento de valores faltantes, incluyendo técnicas de imputación y modelado específico. Sin embargo, en este curso nos enfocaremos exclusivamente en cómo **detectar**, **contabilizar** y, en algunos casos, **excluir** valores faltantes utilizando funciones del lenguaje R.

Una forma sencilla de detectar valores faltantes es mediante la función `count()` del paquete `dplyr`. Al aplicarla a una variable, la salida incluye una fila adicional que informa cuántos valores `NA` hay:

```
datos |>
  count(trabaja)
```

```
# A tibble: 3 x 2
  trabaja     n
  <lgl>   <int>
1 FALSE      26
2 TRUE       39
3 NA          9
```

Una alternativa más completa es la función `find_na()` del paquete `dlookr` (Ryu 2025):

```
find_na(datos, rate = T)
```

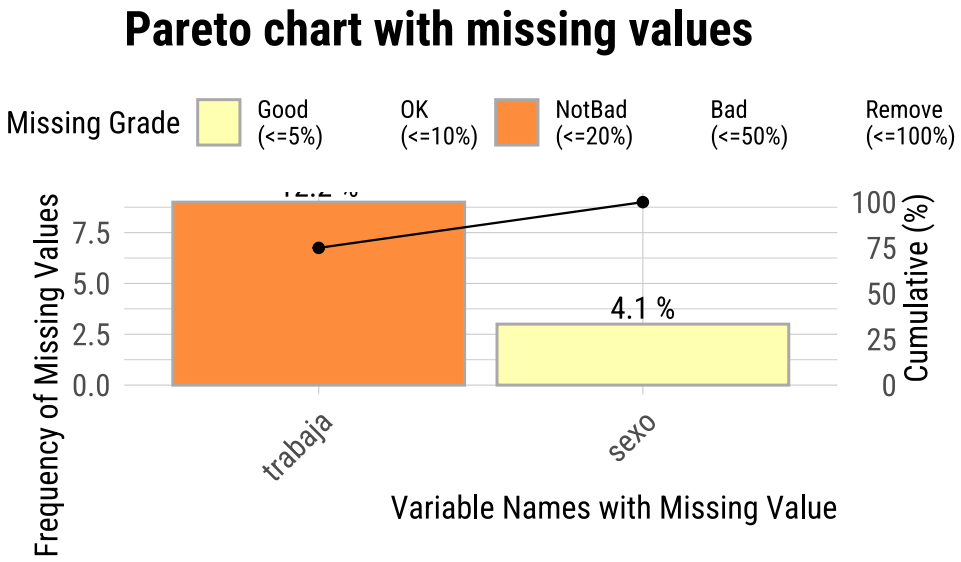
```
     id    sexo    edad    peso   talla trabaja   fecha
  0.000   4.054   0.000   0.000   0.000  12.162   0.000
```

Esta función se puede aplicar al conjunto de datos completo y devuelve, para cada variable, la cantidad y el porcentaje de valores `NA`. Por ejemplo, podríamos observar que la variable `sexo` tiene alrededor de un 4 % de valores faltantes, y la variable `trabaja`, algo más del 12 %.

Estos porcentajes pueden ayudarnos a decidir si una variable debe incluirse en un análisis o si es conveniente excluir ciertas observaciones con datos incompletos, siempre que los `NA` sean el resultado de una ausencia real de información.

El mismo paquete trae una función gráfica llamada `plot_na_pareto()`, que genera un gráfico de barras ordenados por frecuencia de valores faltantes:

```
plot_na_pareto(datos, only_na = T)
```



Finalmente, para un diagnóstico más integral de la calidad de las variables, puede utilizarse la función `diagnose()`:

```
diagnose(datos)
```

```
# A tibble: 7 x 6
  variables types     missing_count missing_percent unique_count unique_rate
  <chr>     <chr>             <int>           <dbl>        <int>       <dbl>
1 id        numeric               0            0             74      1
2 sexo      character             3            4.05           3      0.0405
3 edad      numeric               0            0             45      0.608
4 peso      numeric               0            0             56      0.757
5 talla     numeric               0            0             38      0.514
6 trabaja   logical               9           12.2            3      0.0405
7 fecha     character             0            0             11      0.149
```

Esta función ofrece un resumen detallado que incluye el tipo de variable, la cantidad de valores faltantes, la proporción de valores únicos, entre otros indicadores de utilidad para la exploración inicial.

6.4. Conocer la distribución de las variables de interés

6.4.1. Resumir variables cuantitativas

La instalación básica de R tiene incorporadas múltiples funciones estadísticas que permiten calcular medidas resumen para variables cuantitativas. Estas funciones pueden integrarse a la función `summarise()` de `tidyverse`.

Medidas de tendencia central

Las medidas de tendencia central forman parte del grupo de medidas de posición o localización, pero su objetivo principal es resumir la información en torno a un valor que representa el "centro" de la distribución. Es decir, un valor respecto al cual tienden a agruparse los demás valores.

Podemos obtener la media y la mediana de nuestros datos con el siguiente código:

```
datos |>
  summarise(
    # Media
    media = mean(edad),
    # Mediana
    mediana = median(edad)
  )
```

```
# A tibble: 1 x 2
  media mediana
  <dbl>   <dbl>
1  48.1    52.5
```

En cambio, R base no incluye una función específica para calcular la **moda**. Para obtenerla, debemos escribir una función propia o utilizar algún paquete adicional que la implemente (por ejemplo, `modeest::mlv()`).

Medidas de posición

Las medidas de posición dividen los datos en grupos con igual número de observaciones. Entre las más utilizadas se encuentran los **cuartiles** y **percentiles**.

La función `quantile()` del paquete base `stats` permite calcular cuartiles u otros percentiles. Por ejemplo, para calcular los cuartiles Q1 y Q3, indicamos en el argumento `probs` los valores 0.25 y 0.75:

```
datos |>
  summarise(
    # Primer cuartil
    cuartil1 = quantile(edad, probs = 0.25),
    # Tercer cuartil
    cuartil3 = quantile(edad, probs = 0.75)
  )
```

```
# A tibble: 1 x 2
  cuartil1 cuartil3
     <dbl>    <dbl>
1       28       64
```

Para obtener el mínimo y máximo de estos valores numéricos usamos el siguiente código:

```
datos |>
  summarise(
    # Mínimo
    minimo = min(edad),
    # Máximo
    maximo = max(edad)
  )
```

```
# A tibble: 1 x 2
  minimo maximo
   <dbl>  <dbl>
1     13     86
```

Medidas de dispersión

Las medidas de dispersión nos permiten conocer cuán dispersos o variables son los valores dentro del conjunto de datos.

Entre las más clásicas se encuentran la **varianza** y el **desvío estándar**, que se calculan fácilmente con las funciones `var()` y `sd()`:

```
datos |>
  summarise(
    # Varianza
    varianza = var(edad),
    # Desvío estándar
    desvio = sd(edad)
  )
```

```
# A tibble: 1 x 2
  varianza desvio
     <dbl>  <dbl>
1     405.   20.1
```

También puede ser útil calcular el **rango**, que se obtiene como la diferencia entre el valor máximo y el mínimo, y el **rango intercuartílico (RIC)**, mediante `IQR()`:

```
datos |>
  summarise(
    # Rango
    rango = max(edad) - min(edad),
    # Rango intercuartílico
    ric = IQR(edad)
  )
```

```
# A tibble: 1 x 2
  rango   ric
  <dbl> <dbl>
1    73    36
```

El paquete `dlookr` ofrece la función `describe()` para generar un resumen completo de las variables numéricas:

```
describe(datos, -id)
```

```
# A tibble: 3 x 26
  described_variables     n    na  mean    sd se_mean   IQR skewness kurtosis
  <chr>               <int> <int> <dbl> <dbl>   <dbl> <dbl>    <dbl>    <dbl>
1 edad                   74     0  48.1 20.1    2.34  36      -0.211  -1.11
2 peso                   74     0  72.5 13.2    1.54  17.7     0.145   0.215
3 talla                  74     0 165.   8.12   0.944  9.75    0.256   0.0363
# i 17 more variables: p00 <dbl>, p01 <dbl>, p05 <dbl>, p10 <dbl>, p20 <dbl>,
#   p25 <dbl>, p30 <dbl>, p40 <dbl>, p50 <dbl>, p60 <dbl>, p70 <dbl>,
#   p75 <dbl>, p80 <dbl>, p90 <dbl>, p95 <dbl>, p99 <dbl>, p100 <dbl>
```

Esta función puede aplicarse directamente sobre todo el conjunto de datos. Si bien selecciona automáticamente las variables numéricas, en este caso estamos excluyendo explícitamente la variable `id`, ya que un identificador no tiene interés estadístico.

El resumen que devuelve incluye:

- `na`: cantidad de observaciones con datos y con `NA`.
- `mean`: media aritmética.
- `sd`: desvío estándar de la media.
- `se_mean`: error estándar de la media.
- `IQR`: rango intercuartílico.
- Medidas de forma como la simetría (`skewness`) y la curtosis (`kurtosis`).
- Percentiles, incluyendo la mediana (`P50`) y los cuartiles (`P25` y `P75`).

6.4.2. Resumir variables cualitativas

Las variables cualitativas o categóricas pueden encontrarse en R bajo los tipos de dato `character` o `factor`. En ocasiones será necesario convertirlas a `factor`, ya que este tipo permite aplicar ciertos procedimientos específicos para variables categóricas.

Frecuencias

Podemos resumir individualmente variables cualitativas mediante las frecuencias absolutas y relativas de sus categorías. La función `count()` de `dplyr` nos muestra el conteo absoluto:

```
datos |>
  count(sexo)
```

```
# A tibble: 3 x 2
  sexo      n
  <chr> <int>
1 F        27
2 M        44
3 <NA>      3
```

En la salida se incluirán, además de las categorías presentes, las observaciones con valores faltantes (`NA`).

La inclusión o no de los valores faltantes dependerá del propósito del análisis. Para excluirlos, podemos utilizar `drop_na()`:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na()
```

```
# A tibble: 2 x 2
  sexo      n
  <chr> <int>
1 F        27
2 M        44
```

Para obtener frecuencias relativas en porcentaje:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na() |>
  # Transforma a porcentajes
  mutate(porc = 100 * n / sum(n))
```

```
# A tibble: 2 x 3
  sexo      n  porc
  <chr> <int> <dbl>
1 F        27  38.0
2 M        44  62.0
```

Redondeamos el valor del porcentaje con `round()`:

```
datos |>
  count(sexo) |>
  # Saltea los valores NA
  drop_na() |>
  # Transforma a porcentajes y redondea decimales
  mutate(
    porc = 100 * n / sum(n),
    porc = round(porc, digits = 2)
  )
```

```
# A tibble: 2 x 3
  sexo      n  porc
  <chr> <int> <dbl>
1 F        27  38.0
2 M        44  62.0
```

El paquete `janitor` (Firke 2024) ofrece una alternativa más completa mediante la función `tabyl()`:

```
datos |>
  tabyl(sexo)
```

```
 sexo  n    percent valid_percent
    F 27 0.36486486     0.3802817
    M 44 0.59459459     0.6197183
 <NA>  3 0.04054054            NA
```

Esta función muestra tanto frecuencias absolutas como relativas, incluyendo y excluyendo los valores `NA` (porcentaje sobre el total de valores válidos).

Podemos mejorar la presentación combinando otras funciones del paquete:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, show_na = F) |>
  # Añadimos totales por fila
  adorn_totals(where = "row") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
  sexo  n percent
     F 27  38.03%
     M 44  61.97%
 Total 71 100.00%
```

Tablas de contingencia

La forma más adecuada de describir la relación entre dos variables cualitativas es a través de una **tabla de contingencia**, en la cual:

- Las filas representan las categorías de una variable.
- Las columnas representan las categorías de otra variable.
- Las celdas muestran el número de observaciones correspondientes a cada combinación de categorías.

La función `tabyl()` también permite crear este tipo de tablas. A continuación, un ejemplo entre `sexo` y `trabaja` (aunque `trabaja` sea lógica, puede tratarse como categórica):

```
datos |>
  tabyl(sexo, trabaja)
```

```
 sexo FALSE TRUE NA_
    F     8   15   4
    M    17   22   5
 <NA>     1    2   0
```

Recordemos que el orden dentro de los paréntesis de la función es igual al de los índices, el primer argumento es la variable que aparecerá en las filas y el segundo la variable de las columnas. Por ese motivo, en la tabla de contingencia absoluta tenemos `sexo` en las filas y `trabaja` en las columnas.

Se puede mejorar la tabla excluyendo los valores `NA` y agregando totales por fila:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por fila
  adorn_totals(where = "row")
```

```
  sexo FALSE TRUE
     F     8   15
     M    17   22
 Total    25   37
```

Para calcular frecuencias relativas porcentuales por columna usamos el siguiente código:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales
  adorn_totals(where = "row") |>
  # Añadimos porcentajes por columna
  adorn_percentages(denominator = "col") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
  sexo   FALSE    TRUE
     F  32.00%  40.54%
     M  68.00%  59.46%
 Total 100.00% 100.00%
```

Calculamos frecuencias relativas porcentuales por fila:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por columna
  adorn_totals(where = "col") |>
  # Añadimos porcentajes por fila
  adorn_percentages(denominator = "row") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
 sexo  FALSE   TRUE   Total
    F 34.78% 65.22% 100.00%
    M 43.59% 56.41% 100.00%
```

Cambiando el argumento `denominator` por `"all"` se calculan frecuencias relativas al total:

```
datos |>
  # Excluímos valores NA
  tabyl(sexo, trabaja, show_na = F) |>
  # Añadimos totales por columna
  adorn_totals(where = "col") |>
  # Añadimos porcentajes al total
  adorn_percentages(denominator = "all") |>
  # Redondea porcentajes a 2 decimales
  adorn_pct_formatting(digits = 2)
```

```
 sexo  FALSE   TRUE  Total
    F 12.90% 24.19% 37.10%
    M 27.42% 35.48% 62.90%
```

6.4.3. Explorar variables mediante gráficos

Uno de los aportes más importantes de John Tukey al análisis de datos es la incorporación de los gráficos como herramienta exploratoria. A través de representaciones visuales podemos detectar rápidamente patrones, anomalías, valores extremos, asimetrías o relaciones entre variables.

En R, los gráficos más útiles para explorar la distribución **univariada** de las variables son:

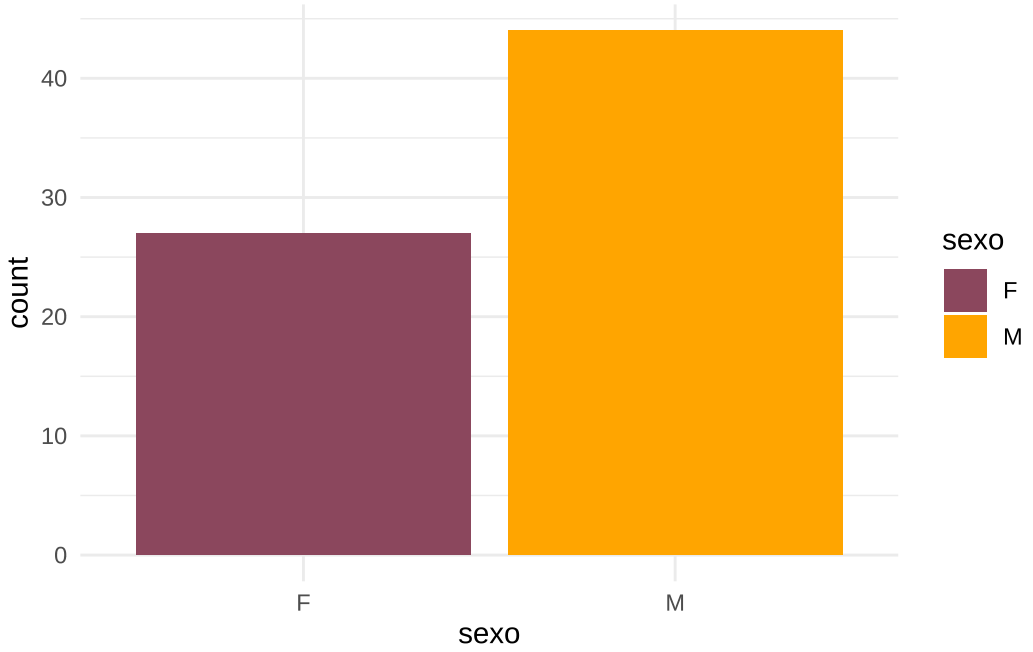
- Para variables **cualitativas**: gráficos de barras
- Para variables **cuantitativas**: histogramas, gráficos de densidad, boxplots y violin plots
- Cuando queremos explorar la relación entre **dos o más variables**, los tipos de gráficos más comunes incluyen:
    - Diagramas de dispersión (puntos)
    - Gráficos de líneas
    - Gráficos de mosaico para variables categóricas cruzadas

El lenguaje R soporta una serie de sistemas gráficos asociados a paquetes como `graphics`, `lattice`, `ggplot2`, etc. que sirven de base incluso para otros paquetes con funciones más específicas. Actualmente el estándar gráfico en R es `ggplot2`.

Barras (univariado)

El gráfico de barras permite visualizar la frecuencia de las categorías de una variable cualitativa:

```
datos |>
  # Omitimos los NA de sexo
  drop_na(sexo) |>
  # Generamos histograma
  ggplot(aes(x = sexo, fill = sexo)) +
  geom_bar() +
  scale_fill_manual(values = c("palevioletred4", "orange")) +
  theme_minimal()
```



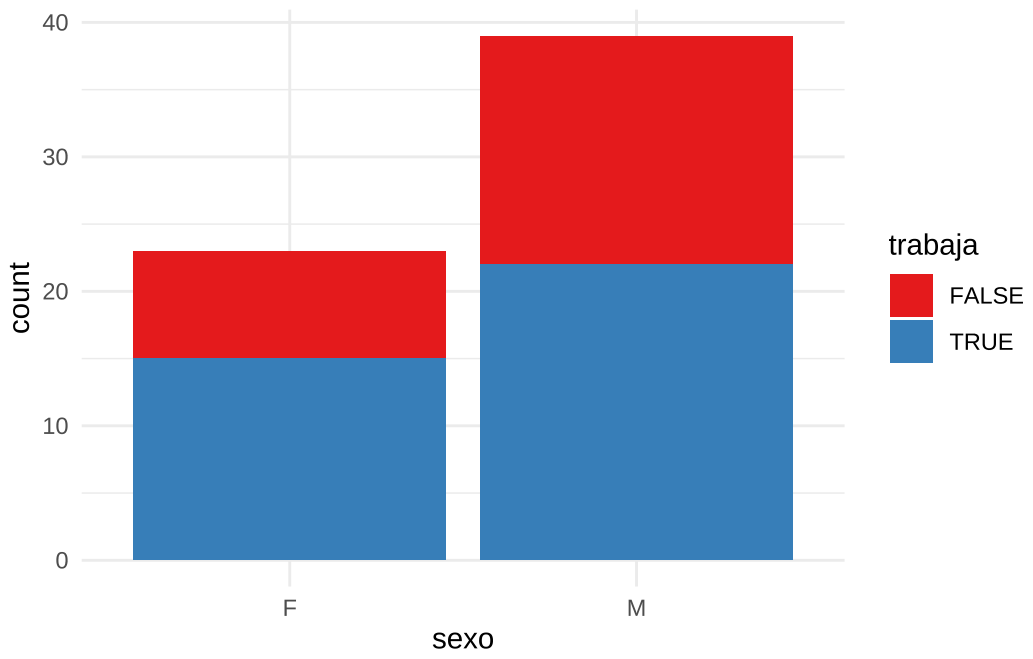
Barras (bivariado)

Cuando cruzamos dos variables categóricas, podemos representar la relación entre ambas modificando el argumento `position` de `geom_bar()`.

El argumento `position = "stack"` nos muestra los valores absolutos acumulados:

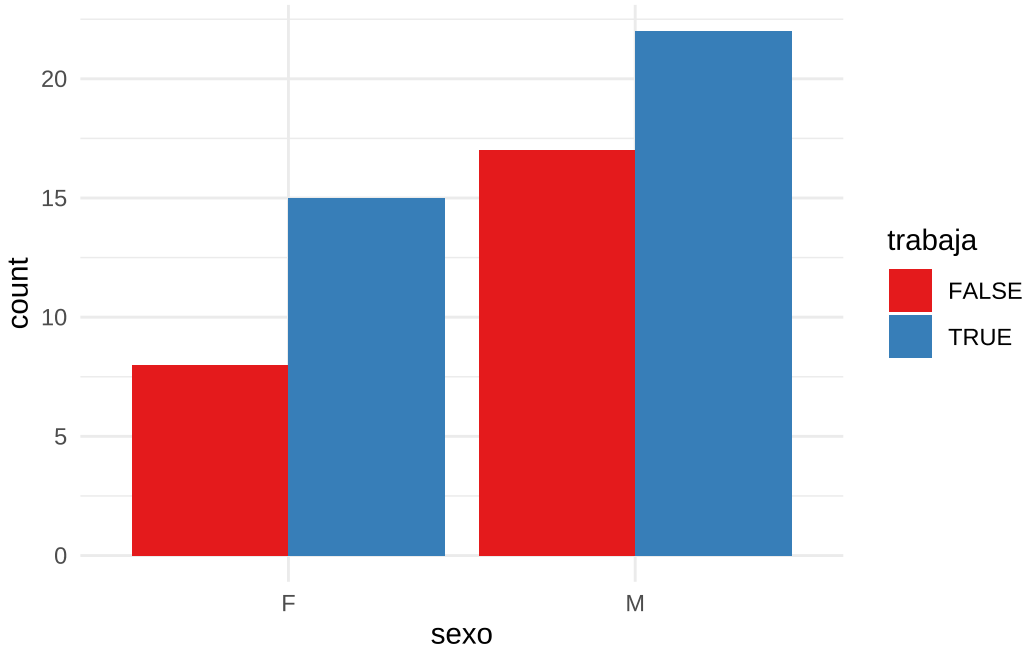
```
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
```

```
  geom_bar(position = "stack") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



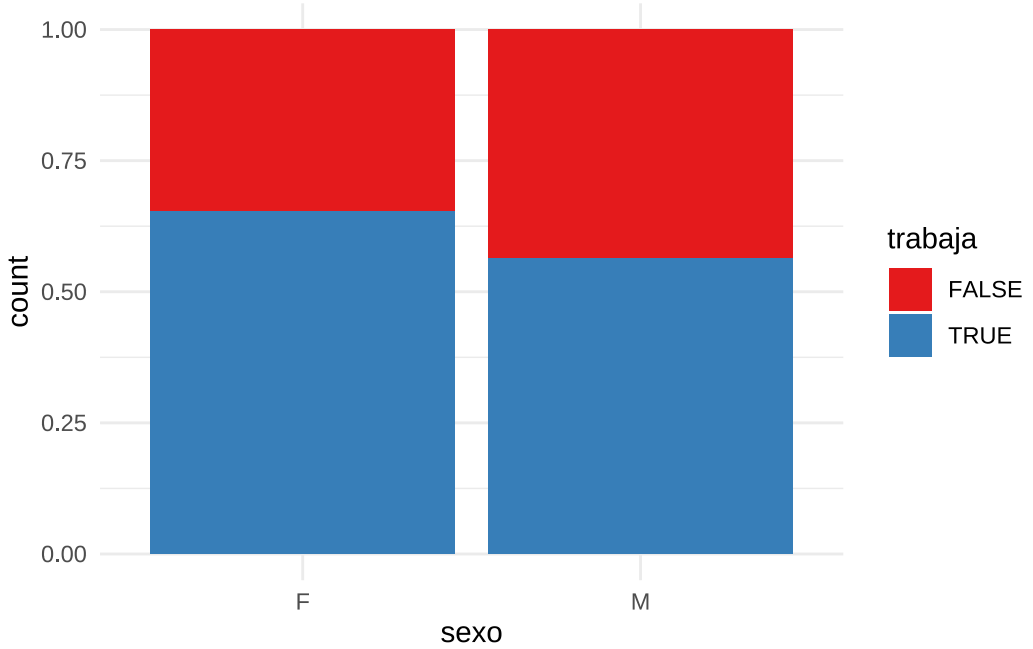
Por otro lado, el argumento `position = "dodge"` muestra las barras lado a lado, permitiendo comparar proporciones entre grupos:

```
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
  geom_bar(position = "dodge") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



Finalmente, `position = "fill"` convierte las alturas en proporciones sobre el total por grupo:

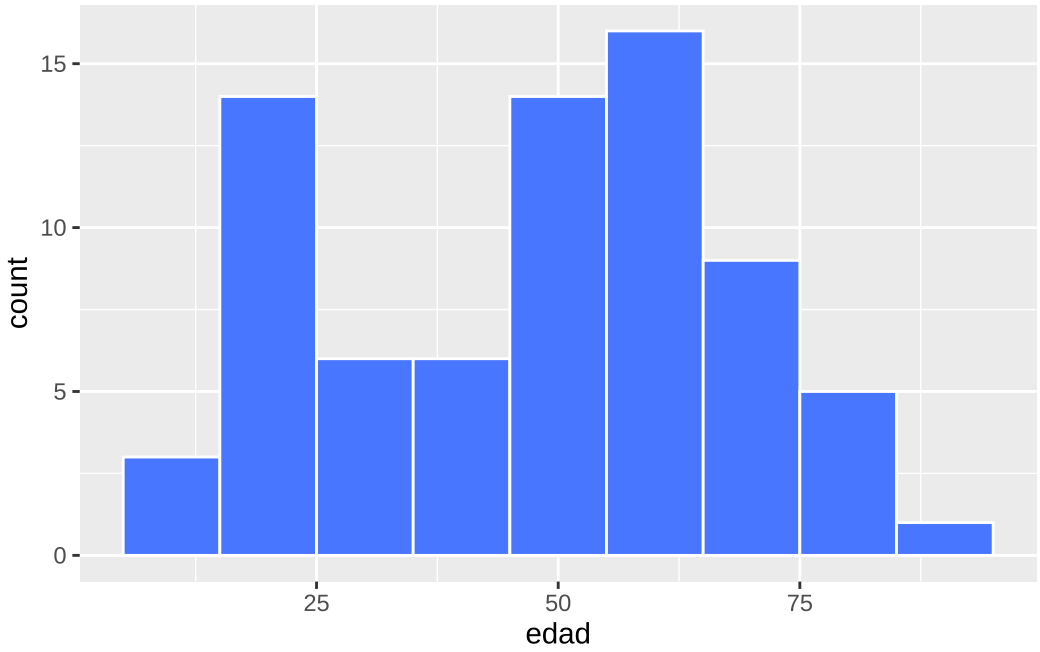
```
datos |>
  # Omitimos los NA de sexo y trabaja
  drop_na(sexo, trabaja) |>
  # Generamos gráfico de barras
  ggplot(aes(x = sexo, fill = trabaja)) +
  geom_bar(position = "fill") +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal()
```



Histograma

Representa la frecuencia de valores en intervalos definidos. Útil para observar la forma general de la distribución:

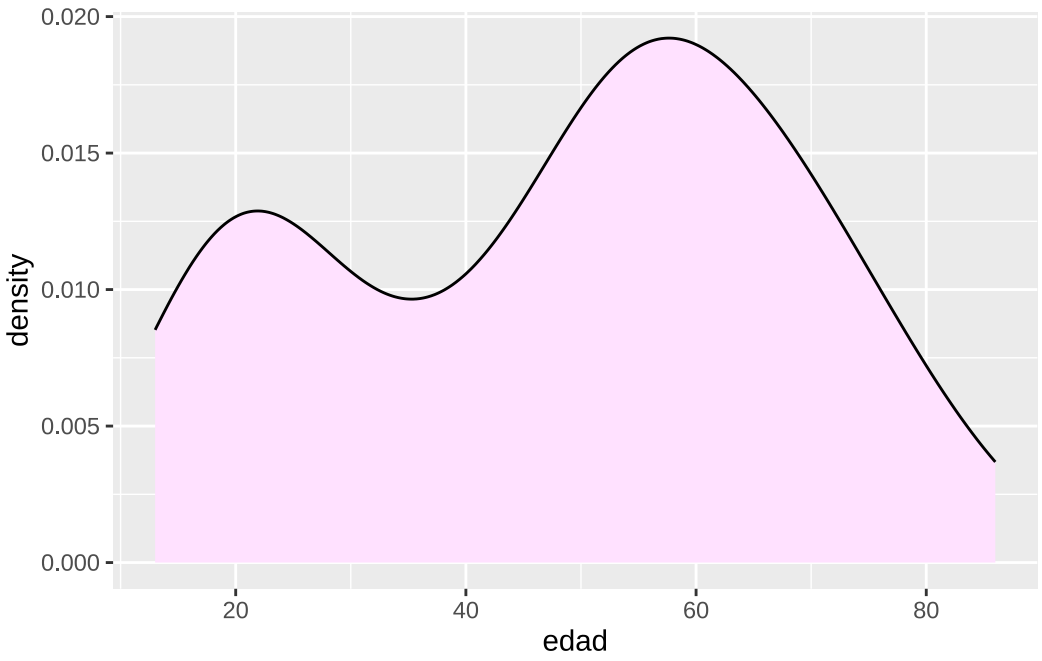
```
datos |>
  # Genera histograma
  ggplot(aes(x = edad)) +
  geom_histogram(binwidth = 10, fill = "royalblue1", color = "white")
```



Densidad

Es una estimación suave de la distribución de frecuencias:

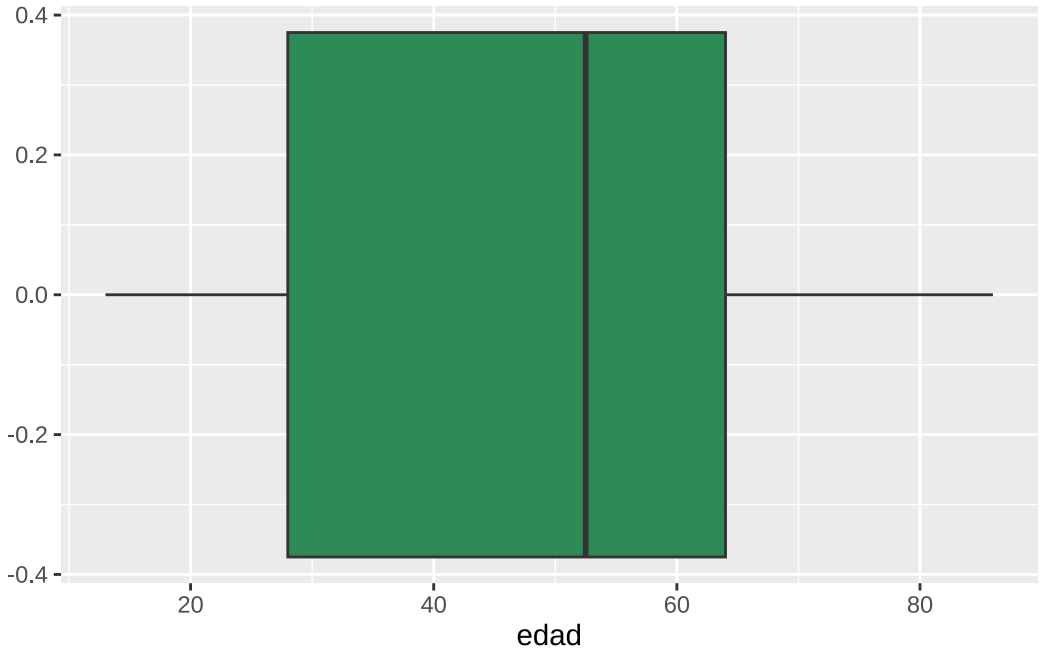
```
datos |>
  ggplot(aes(x = edad)) +
  geom_density(fill = "thistle1")
```



Boxplot

Muestra el rango intercuartílico, la mediana y los valores atípicos. Ideal para detectar asimetrías y *outliers*:

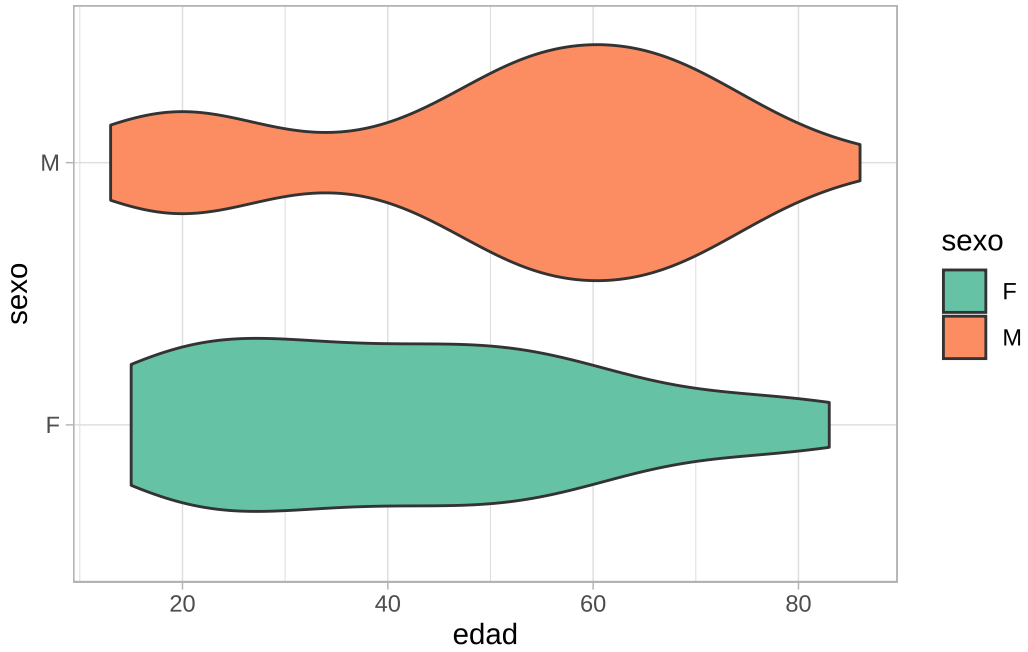
```
datos |>
  # Genera boxplot
  ggplot(aes(x = edad)) +
  geom_boxplot(fill = "seagreen4")
```



Violinplot

Combina el *boxplot* con una curva de densidad reflejada. Permite visualizar tanto la forma de la distribución como los cuantiles:

```
datos |>
  # Omitimos los NA de sexo
  drop_na(sexo) |>
  # Genera violinplot
  ggplot(aes(x = edad, y = sexo, fill = sexo)) +
  geom_violin() +
  scale_fill_brewer(palette = "Set2") +
  theme_light()
```

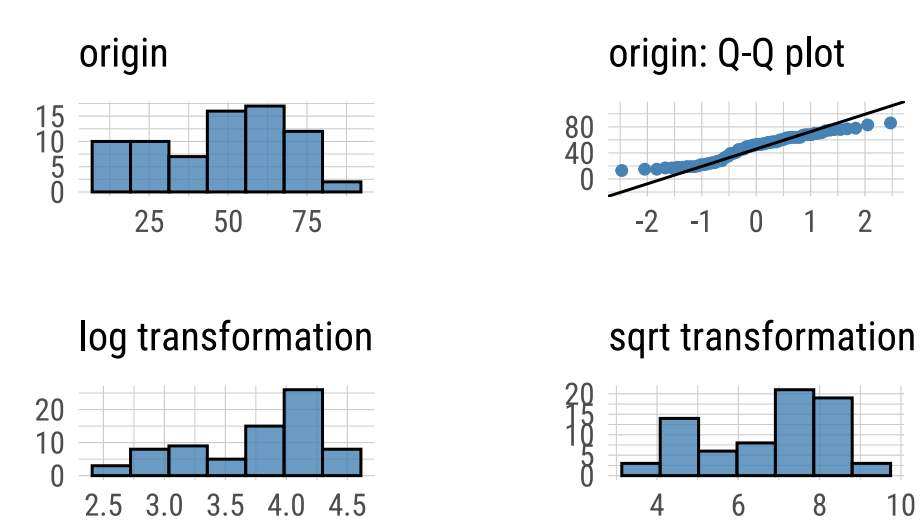


Q-Q Plot

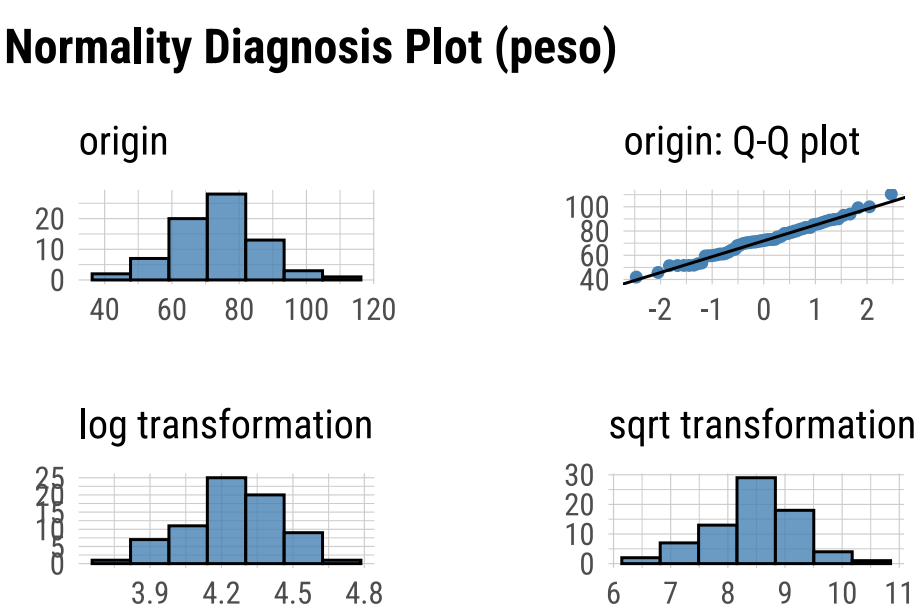
Los gráficos Q-Q (cuantil-cuantil) permiten evaluar visualmente si una variable sigue una distribución teórica, como la normal. Suelen usarse como método gráfico para analizar "normalidad", es decir cuanto se asemeja la distribución de la variable a la distribución normal o gaussiana.

La función `plot_normality()` de `dlookr` muestra un diagnóstico gráfico de normalidad de una variable usando histogramas y Q-Q plot. Además muestra otros histogramas con conversiones de datos (logarítmico y raíz cuadrada por defecto, pero también "Box-Cox" y otras):

```
# Sobre la variable edad
datos |>
  plot_normality(edad)
```




```
# Sobre la variable peso
datos |>
  plot_normality(peso)
```



Podemos decir que la variable `peso` se ajusta mejor a una distribución normal, ya que los puntos del Q-Q plot se alinean más cercanamente a la diagonal teórica.

> **Nota**: Este análisis gráfico de normalidad suele complementarse con pruebas estadísticas específicas, que abordaremos en la próxima unidad.

6.5. Detección de valores atípicos

Un valor atípico (*outlier*) es una observación que se encuentra numéricamente alejada del resto de los datos. Su presencia puede tener diferentes causas, y su tratamiento dependerá del contexto:

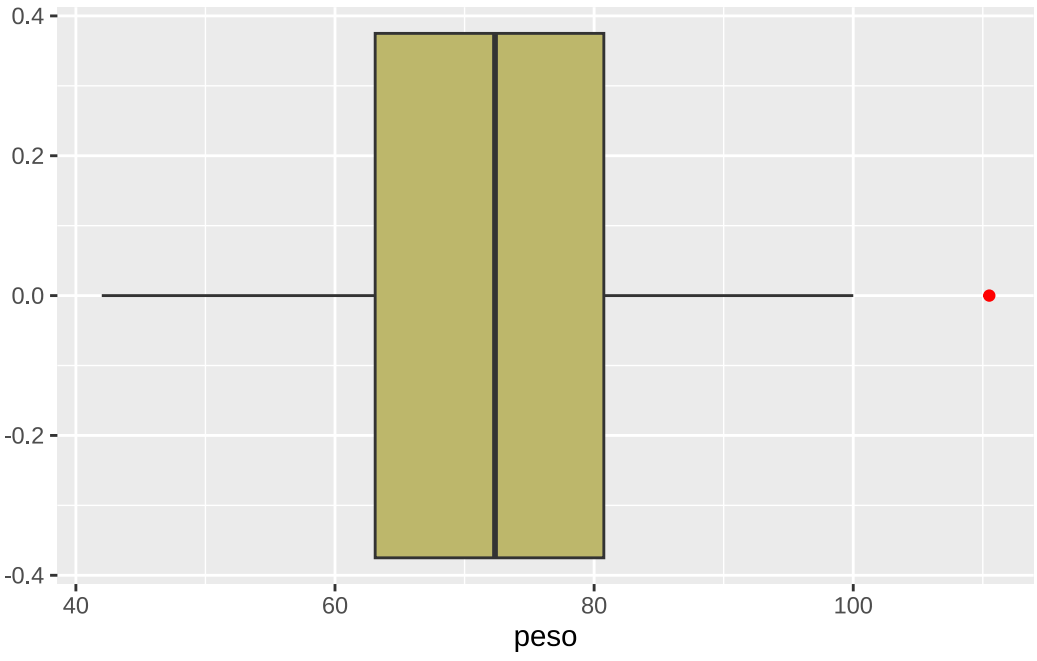
- **Errores de carga o procedimiento**: deben corregirse si se detectan.
- **Valores extremos plausibles**: pueden ser válidos, pero conviene evaluarlos en detalle.
- **Eventos extraordinarios o causas desconocidas**: si no se pueden justificar, suelen excluirse del análisis.

Estos valores pueden afectar sensiblemente ciertos estadísticos como la media, distorsionando su interpretación.

Una forma gráfica común de detectar valores atípicos es mediante los *boxplots*. Los puntos situados fuera de los "bigotes" representan posibles *outliers*.

A continuación, se presenta un ejemplo con la variable `peso`, donde se observa un valor extremo en el límite superior de la distribución (punto rojo):

```
datos |>
  ggplot(aes(x = peso)) +
  geom_boxplot(fill = "darkkhaki", outlier.color = "red")
```



Este valor coincide con el máximo observado:

```
max(datos$peso)
```

```
[1] 110.5
```

El paquete `dlookr` incluye la función `diagnose_outlier()` para la detección automatizada de valores atípicos en todas las variables numéricas de un conjunto de datos:

```
diagnose_outlier(datos)
```

```
# A tibble: 4 x 6
  variables outliers_cnt outliers_ratio outliers_mean with_mean without_mean
  <chr>            <int>          <dbl>         <dbl>     <dbl>        <dbl>
1 id                   0           0             NaN       37.5         37.5
2 edad                 0           0             NaN       48.1         48.1
3 peso                 1           1.35          110.      72.5         72.0
4 talla                0           0             NaN      165.         165.
```

Esta función devuelve una tabla que incluye, para cada variable: cantidad y proporción de *outliers* detectados, media de la variable incluyendo los *outliers*, media de la variable excluyendo los *outliers*. En función de estos dos estadísticos se puede comparar el efecto de los valores atípicos en la media.

El paquete `skimr` (Waring et al. 2026) permite obtener un resumen estadístico compacto y amigable de un conjunto de datos mediante la función `skim()`:

```
skim(datos)
```

Tabla 6.1.: Data summary

Name	datos
Number of rows	74
Number of columns	7
_____	
Column type frequency:	
character	2
logical	1
numeric	4
_____	
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
sexo	3	0.96	1	1	0	2	0
fecha	0	1.00	10	10	0	11	0

Variable type: logical

skim_variable	n_missing	complete_rate	mean	count
trabaja	9	0.88	0.6	TRU: 39, FAL: 26

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
id	0	1	37.50	21.51	1	19.25	37.50	55.75	74.0	▇▇▇▇▇
edad	0	1	48.07	20.12	13	28.00	52.50	64.00	86.0	▆▆▇▇▅
peso	0	1	72.49	13.23	42	63.10	72.35	80.75	110.5	▃▇▇▂▁
talla	0	1	164.85	8.12	148	160.00	164.00	169.75	185.0	▃▇▇▃▂

Además, puede integrarse fácilmente con la gramática `tidyverse`. Por ejemplo, podemos explorar estadísticas descriptivas de variables numéricas agrupadas por `sexo`:

```
datos |>
  # Excluye NAs de sexo
  drop_na(sexo) |>
  # Agrupa por sexo
  group_by(sexo) |>
  # Solo variables numéricas - id
  select(where(is.numeric), -id) |>
  # Explora outliers
  skim()
```

Tabla 6.2.: Data summary

Name	select(…)
Number of rows	71
Number of columns	4
_____	
Column type frequency:	
numeric	3
_____	
Group variables	sexo

Variable type: numeric

skim_variable	sexo	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
edad	F	0	1	41.89	19.64	15.0	26.00	39.0	54.50	83.0	▆▇▇▅▂
edad	M	0	1	51.59	19.56	13.0	40.75	55.5	64.75	86.0	▃▅▆▇▅
peso	F	0	1	63.88	12.02	42.0	53.30	61.0	72.00	85.6	▅▇▆▃▂
peso	M	0	1	77.97	11.49	51.8	71.00	77.2	83.75	110.5	▂▆▇▂▁
talla	F	0	1	158.57	6.08	148.0	155.00	159.5	162.25	172.0	▃▅▇▆▂
talla	M	0	1	168.85	6.58	158.0	164.00	167.0	173.00	185.0	▂▇▇▃▂

En este ejemplo, mostramos resultados de variables numéricas menos de `id` agrupados por `sexo` (sin considerar valores `NA` en las categorías de sexo).

Referencias

7. Hoja de Estilo del lenguaje R

7.1. Convenciones de estilo R

R es bastante indulgente con la forma en que escribimos código, a diferencia de otros lenguajes como Python, donde un espacio mal puesto puede arruinar el script. Sin embargo, adoptar una guía de estilo mejora la legibilidad, facilita la colaboración y reduce errores.

Las siguientes líneas de código producen el mismo resultado, pero no todas son igual de claras:

```
# Opción 1
mpg |>
  filter(cty > 10, class == "compact")

# Opción 2
mpg |> filter(cty > 10, class == "compact")

# Opción 3
mpg |>
  filter(cty > 10,
         class == "compact")

# Opción 4
mpg |> filter(cty>10, class=="compact")

# Opción 5
filter(mpg,cty>10,class=="compact")

# Opción 6
mpg |>
filter(cty > 10,
                          class == "compact")

# Opción 7
filter ( mpg,cty>10,     class=="compact" )
```

Las tres primeras versiones son más legibles. El resto, aunque válidas, son difíciles de seguir, especialmente en trabajos colaborativos o materiales docentes.

7.2. Guía de estilo de `tidyverse`

Para ayudar a mejorar la legibilidad y facilitar el compartir código con otros, el equipo de Tidyverse publicó una guía concisa con ejemplos claros de buenas y malas formas de escribir código, nombres de variables,

sangría, líneas largas, y más:

 https://style.tidyverse.org/

Además, RStudio incluye herramientas para aplicar estas convenciones automáticamente. Por ejemplo, seleccionando el código y presionando `Ctrl + i` (Windows) se puede reidentar el texto. No siempre es perfecto, pero es realmente útil para lograr la sangría correcta sin tener que presionar manualmente espacio muchas veces.

7.2.1. Espaciado

Colocar espacios después de las comas:

 Correcto

```
filter(mpg, cty > 10)
```

 Incorrecto

```
filter(mpg , cty > 10)

filter(mpg ,cty > 10)

filter(mpg,cty > 10)
```

Colocar espacios después de comas y alrededor de operadores (+, −, >, =, etc.) mejora la legibilidad. También se deben evitar espacios innecesarios dentro de paréntesis:

 Correcto

```
filter(mpg, cty > 10)
```

 Incorrecto

```
filter(mpg, cty>10)

filter(mpg, cty> 10)

filter(mpg, cty >10)
```

No colocar espacios alrededor de paréntesis que sean parte de funciones:

 Correcto

```
filter(mpg, cty > 10)
```

 Incorrecto

```
filter (mpg, cty > 10)

filter ( mpg, cty > 10)

filter( mpg, cty > 10 )
```

7.2.2. Líneas largas

En general, es una buena práctica no tener líneas de código muy largas. Se recomienda limitar las líneas a **80 caracteres**. Para visualizarlo, en RStudio vamos a `Tools > Global Options > Code > Displayy` seleccionamos la casilla **Show margin.**

Se sugiere agregar saltos de línea dentro de las líneas de código más largas, los mismos deben colocarse luego de las comas y los argumentos se deben alinear dentro de la función:

Correcto

```
filter(mpg, cty > 10, class == "compact")


filter(mpg, cty > 10,
       class == "compact")


filter(mpg,
       cty > 10,
       class == "compact")

filter(mpg,
       cty > 10,
       class %in% c("compact", "pickup", "midsize", "subcompact",
                    "suv", "2seater", "minivan"))
```

Incorrecto

```
filter(mpg, cty > 10, class %in% c("compact", "pickup", "midsize", "subcompact", "suv", "2seat
```

7.2.3. Tuberías y capas ggplot2

Colocar cada paso de la tubería (%>% ó |>) en una línea separada, con sangría de dos espacios debajo del operador:

Correcto

```
mpg |>
  filter(cty > 10) |>
  group_by(class) |>
  summarize(avg_hwy = mean(hwy))
```

Incorrecto

```
# Mal
mpg |> filter(cty > 10) |> group_by(class) |>
  summarize(avg_hwy = mean(hwy))

# Muy mal
mpg |> filter(cty > 10) |> group_by(class) |> summarize(avg_hwy = mean(hwy))

# Tan mal que no funciona
mpg |>
  filter(cty > 10)
  |> group_by(class)
  |> summarize(avg_hwy = mean(hwy))
```

Lo mismo aplica para las capas de gráficos de `ggplot2`, usando el conector + al final de la línea y debajo sangría de dos espacios:

Correcto

```
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() +
  geom_smooth() +
  theme_bw()
```

Incorrecto

```
# Mal
ggplot(mpg, aes(x = cty, y = hwy, color = class)) +
  geom_point() + geom_smooth() +
  theme_bw()

# Muy mal
ggplot(mpg, aes(x = cty, y = hwy, color = class)) + geom_point() + geom_smooth() + theme_bw(

# Tan mal que no funciona
ggplot(mpg, aes(x = cty, y = hwy, color = class))
  + geom_point()
  + geom_smooth()
  + theme_bw()
```

7.2.4. Comentarios

Los comentarios deben comenzar con # seguido de un espacio:

Correcto

```
# Bien
```

Incorrecto

```
#Mal

      #Mal
```

Si el comentario es corto, se puede incluir en la misma línea, separado por al menos dos espacios para mejorar la legibilidad:

```
mpg |>
  filter(cty > 10) |>  # filtro filas donde cty es 10 o más
  group_by(class) |>  # estratifica por class
  summarize(avg_hwy = mean(hwy))  # resume la media de hwy por cada grupo
```

Se puede agregar espacios adicionales para alinear los comentarios en línea, si lo deseamos:

```
mpg |>
  filter(cty > 10) |>             # filtro filas donde cty es 10 o más
  group_by(class) |>              # estratifica por class
  summarize(avg_hwy = mean(hwy))  # resume la media de hwy por cada grupo
```

Si el comentario es muy largo, podemos dividirlo en varias líneas. RStudio incluye una herramienta útil para comentarios largos: `Code > Reflow Comment` los ajusta automáticamente al ancho deseado.

7.2.5. Nombres de columnas

Se recomienda utilizar nombres de columnas en un formato consistente y compatible con el enfoque del ecosistema `tidyverse`. Esto facilita la manipulación de datos y reduce errores en el código.

En particular, se sugiere evitar:

- Nombres excesivamente largos
- Uso de mayúsculas
- Signos de puntuación, acentos y caracteres especiales
- Espacios en blanco

En su lugar, es recomendable utilizar:

- Nombres breves pero descriptivos
- Letras minúsculas

palabras separadas por guiones bajos (_) o puntos (.)

Correcto

```
names(mpg)
```

```
 [1] "manufacturer" "model"        "displ"        "year"         "cyl"
 [6] "trans"        "drv"          "cty"          "hwy"          "fl"
[11] "class"
```

Incorrecto

En bases de datos existentes:

```
names(iris)
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

Al crear un nuevo dataset:

```
datos <- tibble(
  Jurisdicción = c(rep("Buenos Aires", 10), rep("CABA", 10)),
  CasosTotales = sample(100, 20),
  POBLACIÓN = c(rep(16626374, 10), rep(3054237, 10)),
)

names(datos)
```

```
[1] "Jurisdicción" "CasosTotales" "POBLACIÓN"
```

Al crear una nueva variable o una tabla resumen:

```
# Tan mal que da error
datos  |>
summarise(tasa-incidencia(100k) = CasosTotales/POBLACIÓN * 100000)
```

```
Error in parse(text = input): <text>:3:30: unexpected symbol
2: datos  |>
3: summarise(tasa-incidencia(100k
                               ^
```

En caso de que nuestros datos tengan nombres de columnas no compatibles con `tidyverse`, se recomienda usar la función `clean_names()` del paquete `janitor` para estandarizarlas:

```
library(janitor)

iris |>
  clean_names() |>
  head(10)
```

```
   sepal_length sepal_width petal_length petal_width species
1           5.1         3.5          1.4         0.2  setosa
2           4.9         3.0          1.4         0.2  setosa
3           4.7         3.2          1.3         0.2  setosa
4           4.6         3.1          1.5         0.2  setosa
5           5.0         3.6          1.4         0.2  setosa
6           5.4         3.9          1.7         0.4  setosa
7           4.6         3.4          1.4         0.3  setosa
8           5.0         3.4          1.5         0.2  setosa
9           4.4         2.9          1.4         0.2  setosa
10          4.9         3.1          1.5         0.1  setosa
```

```
datos <- datos |>
  clean_names()

names(datos)
```

```
[1] "jurisdiccion"  "casos_totales" "poblacion"
```

Importante

Las versiones recientes de RStudio y Positron (el nuevo entorno de desarrollo de Posit) incorporan soporte para **Air**, un formateador de código de alto rendimiento desarrollado en Rust. Esta herramienta aplica de manera automática los estándares de estilo recomendados por el ecosistema de `tidyverse`, lo que optimiza la legibilidad y simplifica el mantenimiento de los scripts.
Para habilitar esta función en RStudio hay que entrar a:
`Tools > Global Options > Code > Formatter`
Y habilitar las opciones correspondientes.

Parte II.

Unidad 2

8. Estudios epidemiológicos

Un estudio epidemiológico es un proceso que se desarrolla en varias fases: desde la definición del **problema de investigación**, la selección de la estrategia o **diseño del estudio**, hasta la planificación de las **actividades**. En esencia, el diseño constituye la estrategia metodológica elegida para alcanzar los objetivos planteados.

8.1. Clasificación de estudios epidemiológicos

Existen distintos criterios para clasificar los estudios epidemiológicos. La siguiente tabla (Tabla 8.1) resume esta clasificación, que probablemente ya hayan abordado en cursos previos:

Tabla 8.1.: Clasificación de estudios epidemiológicos.

Poblacionales	Individuales	Observacionales	Experimentales
Análisis de situación	Serie de casos	Estudios de casos y controles	Ensayos clínicos
Estudios Ecológicos	Reporte de casos	Estudios de cohortes	Ensayos comunitarios y/o de campo
	Estudios transversales		

Toda investigación debe partir de una revisión actualizada del conocimiento sobre el tema a abordar. Esto implica delimitar el problema de investigación de forma precisa, definiendo objetivos, propósito, justificación e hipótesis. Para ello, es fundamental conocer las características y ventajas de cada tipo de estudio epidemiológico, evaluar su adecuación a los objetivos planteados y considerar los recursos disponibles en términos de tiempo, personal y presupuesto.

8.1.1. Clasificación GRADE

Existen propuestas que jerarquizan la calidad de la evidencia según el tipo de diseño, bajo el supuesto de que algunos están más expuestos a sesgos que otros. Esta jerarquización permite orientar la toma de decisiones clínicas y de salud pública.

El análisis riguroso de la evidencia facilita establecer grados de recomendación para intervenciones diagnósticas, terapéuticas o preventivas. Uno de los sistemas más utilizados para clasificar la calidad de la evidencia y la fuerza de las recomendaciones es **GRADE** (*Grading of Recommendations Assessment, Development and Evaluation*), que permite sintetizar la calidad de la evidencia según el tipo de diseño:

- **Ensayos clínicos aleatorizados (EC):** calidad alta
- **Estudios observacionales:** calidad baja
- **Otras fuentes:** calidad muy baja

La siguiente tabla (Tabla 8.2) muestra la clasificación de GRADE, que combina calidad metodológica, balance entre beneficios y riesgos, y las implicancias de cada recomendación:

Tabla 8.2.: Clasificación GRADE de estudios epidemiológicos.

Grado	Recomendación	Calidad	Beneficio vs. Riesgo y cargas	Calidad metodológica que apoya la evidencia	Implicancias
1.A	Fuerte	Alta	Superan ampliamente los riesgos y cargas (o viceversa)	EC sin importantes limitaciones o evidencia abrumadora de estudios observacionales.	Puede aplicarse a la mayoría de los pacientes, en la mayoría de las circunstancias sin reservas.
1.B	Fuerte	Moderada	Superan ampliamente los riesgos y cargas (o viceversa)	EC con importantes limitaciones (resultados inconsistentes, defectos metodológicos indirectos o imprecisos) o pruebas excepcionalmente fuertes a partir de estudios observacionales.	Puede aplicarse a la mayoría de los pacientes, en la mayoría de las circunstancias sin reservas.
1.C	Fuerte	Baja o muy baja	Superan ampliamente los riesgos y cargas (o viceversa)	Estudios observacionales o series de casos.	Puede cambiar cuando se disponga de mayor evidencia de calidad.
2.A	Débil	Alta	Estrechamente equilibrados	EC sin importantes limitaciones o evidencia abrumadora de estudios observacionales.	La mejor acción puede variar dependiendo de las circunstancias de los pacientes o de los valores de la sociedad.
2.B	Débil	Moderada	Estrechamente equilibrados	EC con importantes limitaciones (resultados inconsistentes, defectos metodológicos indirectos o imprecisos) o pruebas excepcionalmente fuertes a partir de estudios observacionales.	La mejor acción puede variar dependiendo de las circunstancias de los pacientes o de los valores de la sociedad.
2.C	Débil	Baja o muy baja	Incertidumbre en las estimaciones de beneficios, riesgos y carga	Estudios observacionales o series de casos.	Otras alternativas pueden ser igual de razonables.

8.2. Planificación y análisis de los estudios epidemiológicos

Una vez definidos los objetivos y el diseño, es fundamental planificar detalladamente el estudio: seleccionar la población, estimar el tamaño muestral, establecer criterios de inclusión y exclusión, definir las variables, las fuentes de datos y los métodos de recolección, procesamiento y análisis.

Mientras que la definición del problema y la elección del diseño ya han sido abordadas en instancias previas, en este curso nos centraremos en la **fase analítica**, haciendo especial énfasis en el tratamiento de los datos

derivados de distintos tipos de estudios.

Dado que una gran proporción de la investigación en salud es observacional, comenzaremos por los análisis correspondientes a estudios transversales, de casos y controles, y de cohortes. En todos los casos, la comunicación de los resultados debe ser clara y estructurada, para facilitar la comprensión del proceso, desde la planificación hasta las conclusiones.

La formulación de recomendaciones en torno a la comunicación de la investigación puede contribuir a mejorar su calidad.

8.2.1. Declaración STROBE

La **Declaración STROBE** (*Strengthening the Reporting of Observational Studies in Epidemiology*) (Elm et al. 2008) es una guía internacional destinada a estandarizar y mejorar la calidad del reporte de estudios observacionales, incluyendo estudios transversales, de casos y controles, y de cohortes.

Se trata de una iniciativa colaborativa que reúne a epidemiólogos, metodólogos, estadísticos, investigadores y editores de revistas científicas, comprometidos con la realización y difusión de investigaciones observacionales rigurosas.

STROBE incluye una lista de **22 ítems**, organizados en secciones del manuscrito (título, resumen, introducción, métodos, resultados, discusión y otra información relevante), que especifican lo que debe informarse para asegurar la **transparencia** y **reproducibilidad** de los hallazgos. Esta guía también es una herramienta útil para investigadores, ya que indica claramente los elementos esenciales que deben estar presentes en un artículo científico basado en estudios observacionales. El respaldo de STROBE en revistas biomédicas es creciente, siendo requisito editorial en gran parte de ellas.

Puede descargarse desde el sitio oficial de STROBE:

Guía completa y *checklist*

Versión en español

La declaración STROBE ha sido adaptada para contextos específicos mediante extensiones temáticas:

STROME-ID

STROME-ID (*Strengthening the Reporting of Molecular Epidemiology for Infectious Diseases*), tiene como objetivo establecer recomendaciones que respalden un adecuado informe científico de estudios en epidemiología molecular de enfermedades infecciosas. Incentiva un reporte adecuado que considere amenazas específicas a la validez, como errores de clasificación, contaminación cruzada, o sesgos derivados de los métodos de laboratorio y análisis genético.

Descargar STROME-ID

STROBE-VET

STROBE-VET (*STROBE - Veterinary Extension*), orienta el reporte de estudios observacionales en medicina veterinaria, salud pública, zoonosis y seguridad alimentaria. Facilita la evaluación crítica y reproducibilidad en investigaciones con poblaciones animales.

Descargar STROBE-VET

RECORD

RECORD (*Reporting of studies Conducted using Observational Routinely-collected health Data*) fue creada para estudios que utilizan datos rutinarios, como registros electrónicos de salud, bases de datos administrativas o registros poblacionales.

Descargar RECORD

8.2.2. Declaración CONSORT

En cuanto a estudios experimentales, la **Declaración CONSORT** (*Consolidated Standards of Reporting Trials*) (Schulz et al. 2010) es una guía internacional destinada a mejorar la calidad del reporte de los ensayos clínicos aleatorizados (ECA). Fue desarrollada en 1996 y revisada por primera vez en 2001, con actualizaciones posteriores (la última versión es de 2010).

Muchas revistas biomédicas la han adoptado como requisito editorial, lo que ha contribuido significativamente a la mejora en la calidad, transparencia y reproducibilidad de los informes sobre ECA.

CONSORT establece un conjunto mínimo de 25 ítems organizados en secciones del artículo científico (título, resumen, introducción, métodos, resultados, discusión y otros), complementados por un diagrama de flujo que muestra el número de participantes en cada fase del estudio (asignación, intervención, seguimiento y análisis). Esta herramienta ofrece un formato estándar para que los autores preparen informes completos y transparentes de los resultados de los ensayos, facilitando su evaluación e interpretación crítica.

Puede descargarse desde el sitio oficial de CONSORT:

https://www.consort-spirit.org/

> ⚠ En este curso nos centraremos en los aspectos metodológicos y en el análisis de datos según el diseño del estudio. Para ello, utilizaremos el **lenguaje R**, un entorno especializado en análisis estadístico que permitirá explorar diferentes enfoques adaptados a cada tipo de diseño epidemiológico. Partiremos de cada diseño y discutiremos posibles caminos de análisis, reconociendo que dichos caminos no siempre son únicos y que ciertos elementos del análisis pueden ser comunes a varios diseños.

(Hernández-Ávila 2011)

8.3. Referencias

9. Estudios de corte transversal

9.1. Introducción

Los estudios transversales se caracterizan por realizar mediciones en un **único momento** en el tiempo, sin llevar a cabo un seguimiento de los participantes. En este diseño, el investigador mide de forma simultánea la **exposición** y el **evento** en los sujetos seleccionados mediante criterios de inclusión y exclusión predefinidos.

Se trata de un diseño **observacional de base individual**, que puede tener un propósito **descriptivo**, **analítico**, o ambos. También se los conoce como estudios de prevalencia o encuestas transversales. Los estudios transversales entonces pueden dividirse en:

- **Descriptivos:** se enfocan en determinar la **frecuencia** y **distribución** de eventos de salud y enfermedad en un momento determinado, midiendo una o más variables o condiciones.
- **Analíticos:** se orientan a explorar **asociaciones** y generar hipótesis de investigación. Algunos autores ubican este diseño en el límite entre los estudios descriptivos y los analíticos.

En los estudios transversales, la medida de frecuencia más utilizada es la **prevalencia**. Para evaluar asociaciones se pueden calcular dos medidas principales:

- **Razón de prevalencia** (*prevalence ratio*)**:** que aproxima el riesgo relativo observado en estudios de cohortes.
- **Odds ratio de prevalencia** (*prevalence odds ratio*): que se utiliza como aproximación al *odds ratio* calculado en estudios de casos y controles.

A continuación se resumen las ventajas y desventajas de este diseño:

Ventajas	Desventajas
Eficientes para estudiar la prevalencia de enfermedades en la población	Problemas para definir y medir exposición
Se pueden estudiar varias exposiciones	Sesgos de selección
Son poco costosos y se pueden realizar en poco tiempo	Sesgos por casos prevalentes
Se puede estimar la prevalencia del evento	La relación causa efecto no siempre es verificable
	Sobrerrepresentación de enfermos con tiempos prolongados de sobrevida o con manifestaciones con mejor curso clínico
	Se puede presentar causalidad débil

> **Nota**: Los aspectos metodológicos esenciales para el reporte de estudios transversales pueden consultarse en la **Declaración STROBE**, sección *Métodos*.

9.2. Análisis de estudios transversales

Al planificar un estudio transversal, es fundamental definir con precisión la **población de estudio**. La selección de los sujetos depende directamente de la pregunta de investigación y los objetivos planteados. Una identificación adecuada de la población garantiza la validez externa de los resultados y su potencial generalización.

En la práctica, estos estudios rara vez incluyen a todos los miembros de la población objetivo, sino que se trabaja con una **muestra**. Solo una muestra representativa, que refleje las características de la población base, permite desarrollar el componente analítico y exploratorio del diseño transversal, de modo que las conclusiones sean relevantes y aplicables.

Una vez obtenidos los datos, éstos pueden representarse gráficamente, resumirse mediante estadísticas descriptivas (como medidas de tendencia central y dispersión) e incluso utilizarse para modelar relaciones entre variables.

- Si el objetivo es **describir los resultados** en la muestra, las herramientas descriptivas pueden ser suficientes.
- Si se busca **extrapolar a la población general,** es necesario aplicar métodos de **inferencia estadística**, fundamentados en la teoría de la probabilidad.

Rodríguez y Mendivelso (2018)

Hernández-Ávila (2011)

9.3. Referencias

10. Inferencia estadística

10.1. Introducción

La **estadística inferencial** es la rama de la estadística que permite formular conclusiones sobre una población a partir del análisis de una muestra. Se apoya en el cálculo de probabilidades, que proporciona el marco teórico para modelar fenómenos aleatorios y generalizar los resultados muestrales a toda la población. Dado que las inferencias basadas en muestras están sujetas a incertidumbre, es fundamental expresarlas siempre en términos **probabilísticos**.

Las dos actividades principales en este proceso son:

- **Estimación de parámetros**: consiste en calcular, a partir de los datos muestrales, valores que aproximen parámetros desconocidos de la población.

  - *Ejemplo: Estimar la prevalencia de una enfermedad X en la población.*

- **Pruebas de hipótesis**: implica evaluar con base estadística afirmaciones acerca de uno o más parámetros.

  Ejemplos:

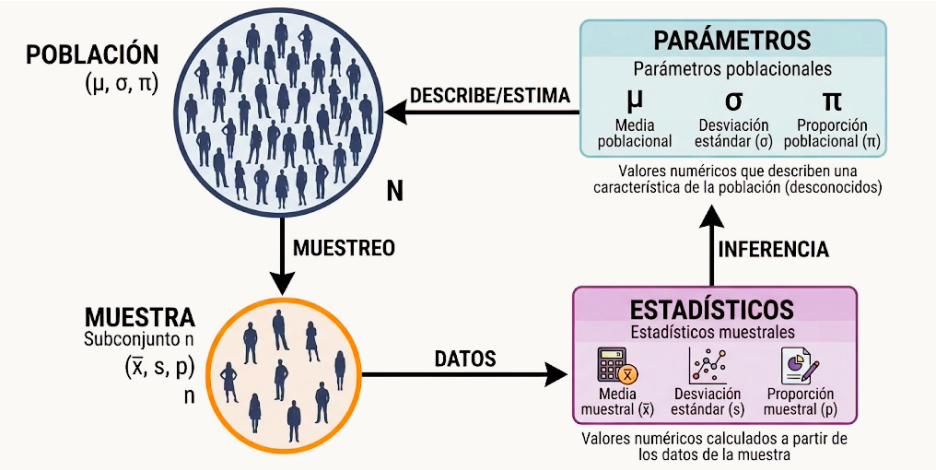
  - *¿La prevalencia de la enfermedad X en Argentina es menor que en Uruguay?*
  - *¿La prevalencia de la enfermedad X en Argentina en 2010 fue menor que en el año 2000?*

Al trabajar con muestras, entra en juego el concepto de **teoría del muestreo**, que, si bien no abordaremos en profundidad aquí, es clave para comprender cómo se relacionan los valores observados en la muestra con los de la población.

Esta teoría estudia la relación entre la distribución de una variable en la población y el comportamiento de dicha variable en muestras aleatorias extraídas de ella. A las medidas obtenidas a partir de la muestra se las denomina **estadísticos muestrales** o simplemente *estadísticos*, mientras que sus contrapartes en la población se denominan **parámetros**.

Por ejemplo, supongamos que queremos conocer el valor medio de colesterol total de la población de Mar del Plata y tomamos una muestra de tamaño n.

- La media poblacional del colesterol total se representa con la letra griega μ y corresponde al **parámetro**.
- La media muestral, que se obtiene a partir de los datos de la muestra, se representa como $\bar{x}$ y es un **estimador** o estadístico muestral.



La **distribución muestral de un estadístico** es la distribución de todos los valores posibles que ese estadístico puede tomar al calcularse en muestras aleatorias del mismo tamaño extraídas de una misma población. Este concepto es central en la inferencia estadística, ya que permite cuantificar la incertidumbre asociada a las estimaciones.

Para comprender mejor qué es una distribución muestral y cómo se construye, pueden ver la siguiente videoclase:

Distribuciones Muestrales

Como construir una distribución muestral puede resultar muy laborioso cuando la población es grande, y directamente imposible si es infinita, se suelen utilizar aproximaciones basadas en la toma de un gran número de muestras aleatorias del mismo tamaño.

10.2. Intervalos de Confianza (IC)

Una forma eficaz de abordar la inferencia estadística es a través de los **intervalos de confianza (IC)**, ya que, aunque son procedimientos inferenciales, están estrechamente vinculados con la estadística descriptiva.

Supongamos que queremos estimar la **media de colesterol** de la población de Mar del Plata. Sería inviable medir el colesterol de cada habitante, por lo que optamos por tomar una muestra de, por ejemplo, 100, 200 o 300 individuos (más adelante veremos cómo determinar el tamaño adecuado de la muestra). Debemos recordar que diferentes muestras producirán en general medias diferentes. Existe, por tanto, un grado de incertidumbre asociado. Si hiciéramos una estimación puntual, obtendríamos un solo valor, pero sin información sobre su variabilidad. No sabríamos qué tan cerca o lejos está nuestra estimación ($\bar{x}$) de la verdadera media poblacional (μ).

El **intervalo de confianza** proporciona un rango de valores dentro del cual se espera que se encuentre el valor verdadero del parámetro poblacional, con un cierto nivel de confianza. A diferencia de la estimación puntual que proporciona un único valor numérico, el intervalo consta de dos valores entre los cuales se supone está contenido el parámetro estimado. Entonces, el intervalo de confianza puede expresarse como:

$$IC = estimador\ puntual \pm (coeficiente\ de\ confiabilidad) * (error\ estandar)$$

donde:

- **Estimador puntual**:

  - Para la media poblacional (μ), se toma la media muestral($\bar{x}$).
  - Para una proporción de la población (p), se toma la proporción muestral ($\hat{p}$).

- **Coeficiente de confiabilidad:** Se relaciona con el nivel de confianza deseado (por ejemplo, 90 %, 95 % o 99 %), y se expresa como $1 - \alpha$, es decir la probabilidad de que el parámetro se encuentre dentro del IC. Recordemos que el nivel de significancia (α) es la probabilidad de que el parámetro no se halle dentro del IC y es un valor generalmente pequeño (por ejemplo, 0.1, 0.05 o 0.01) expresado como probabilidad o porcentaje (por ejemplo, 10 %, 5 % o 1 %).

- **Error estándar (SE):** Representa la variabilidad de la distribución muestral. Por ejemplo, para la media el error estándar se calcula como la raíz cuadrada de la varianza de la distribución muestral:

$$SE = \frac{\sigma}{\sqrt{n}}$$

Donde σ es la desviación estándar poblacional y n el tamaño de la muestra. Si se estima un IC para una proporción, el error estándar es:

$$SE = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

El proceso se fundamenta en el Teorema del Límite Central (TCL), que establece que, para muestras suficientemente grandes, la distribución de $\bar{x}$ es aproximadamente normal, con media μ y varianza σ^2/n. Así, la variable tipificada:

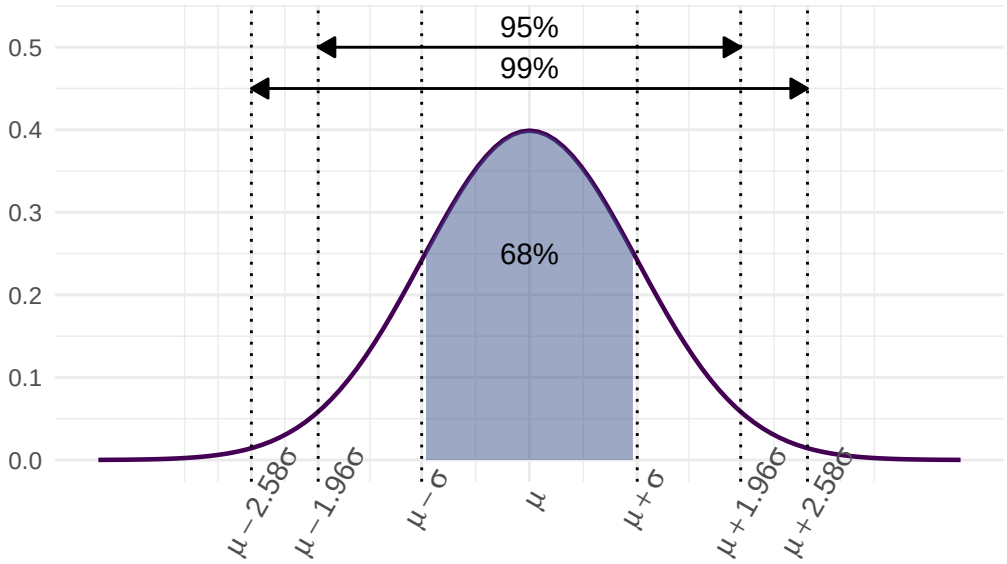
$$Z = \frac{\bar{x} - \mu}{\sigma}$$

sigue una distribución normal estándar (media 0 y desviación estándar 1), lo que permite calcular probabilidades y construir el IC.

En cualquier distribución normal:

- Entre $\mu \pm \sigma$ se encuentra el 68 % de los datos.
- Entre $\mu \pm 2\sigma$ se encuentra el 95 %.
- Entre $\mu \pm 3\sigma$ se encuentra el 99 %.

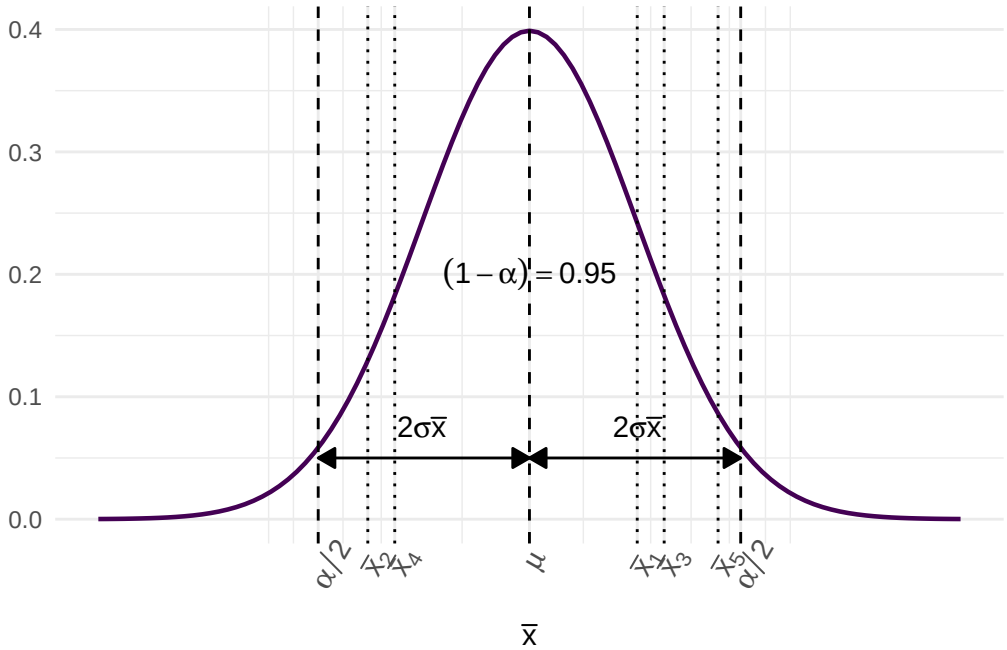
El siguiente gráfico ilustra lo explicado anteriormente:



Sabemos que, independientemente de la localización de los valores, aproximadamente el **95 %** de los valores posibles de $\bar{x}$ en la distribución muestral estarán a menos de dos desviaciones estándar de la media μ. Es decir, el intervalo $\mu \pm 2\sigma$ contendrá el **95 %** de los valores posibles de $\bar{x}$.

Supongamos que formamos intervalos a partir de todos los posibles valores de $\bar{x}$, calculados a partir de todas las muestras posibles de tamaño n tomadas de la población de interés. Esto generará una gran cantidad de intervalos de la forma $\mu \pm 2\sigma$, todos con la misma amplitud, centrados en torno a una μ desconocida.

Aproximadamente el **95 %** de estos intervalos tendrán sus centros dentro del intervalo $\mu \pm 2\sigma$. Cada uno de estos intervalos, que se encuentran dentro de $\mu \pm 2\sigma$, puede contener el valor verdadero de μ.



Finalmente, y basándonos en las propiedades de la distribución Normal, se puede deducir la expresión del IC:

$$P(Z_{\alpha/2} < Z_{1-\alpha/2)} = 1 - \alpha$$

Reemplazando Z:

$$P(Z_{\alpha/2} < \frac{\bar{x} - \mu}{\frac{\sigma}{\sqrt{n}}} < Z_{(1-\alpha/2)}) = 1 - \alpha$$

Reordenando, la expresión del IC para la media queda:

$$\bar{x} - Z_{\alpha/2}\sqrt{\frac{\sigma^2}{n}} < \mu < \bar{x} + Z_{\alpha/2}\frac{\sigma^2}{n}$$

10.2.1. ¿Cómo se interpreta un IC?

Si hubiésemos tomado múltiples muestras del mismo tamaño de la población, en al menos $100 * (1 - \alpha)\,\%$ de las ocasiones el intervalo calculado contendría el parámetro poblacional real. Es decir, un IC al 95 % implica que, a largo plazo, el 95 % de los intervalos obtenidos a partir de muestras repetidas incluirán el valor verdadero del parámetro.

El producto del coeficiente de confiabilidad y el error estándar se denomina **precisión de la estimación** y es el componente responsable de la amplitud del IC. Recordemos que la fórmula general para construir un intervalo de confianza era:

$$IC = estimador\,puntual \pm (coeficiente\,de\,confiabilidad) * (error\,estandar)$$

Para el caso de la media:

- **Aumento de la confiabilidad:** Si se incrementa el nivel de confianza, el coeficiente (por ejemplo, pasando de 1.96 a un valor mayor) aumenta, lo que a su vez incrementa la amplitud del IC.
- **Reducción del error estándar:** Si se fija la confiabilidad (por ejemplo, al 95 %), para disminuir la amplitud del IC es necesario reducir el error estándar. Dado que el error estándar de la media es:

$$SE = \frac{\sigma}{\sqrt{n}}$$

y considerando que σ es constante, la única forma de disminuir el error estándar es aumentando el tamaño muestral (n).

Surge entonces la pregunta: **¿qué tan grande debe ser** n?
La respuesta dependerá de σ, del nivel de significación (α) y de la amplitud deseada para el IC. La relación es:

$$Amplitud = Z\frac{\sigma}{\sqrt{n}} \Longrightarrow n = \frac{Z^2\sigma^2}{Amplitud^2} \quad (Z = 1.96\ si\ \alpha = 0.05)$$

(En la práctica, σ generalmente no se conoce, así que se usa su estimación muestral).

La expresión del error estándar varía según el parámetro a estimar. Hemos visto el caso de la media; si lo que se desea es calcular un IC para una proporción, recordemos que, para muestras grandes, la distribución de las proporciones de la muestra es aproximadamente normal de acuerdo con el TCL. En este caso:

- La media de la distribución es la proporción real p
- La varianza es $p(1-p)/n$, lo que nos lleva a que el error estándar es:

$$SE = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

y el IC para la proporción se expresa como:

$$\hat{p} - Z_{1-\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}} < p < \hat{p} + Z_{1-\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

Dado que un intervalo de confianza implica una declaración probabilística, su cálculo se fundamenta en las distribuciones muestrales de los estimadores y en el correspondiente error estándar. Aunque las fórmulas pueden parecer complejas, los paquetes estadísticos (como R) realizan estos cálculos automáticamente. Lo fundamental es comprender en qué depende la amplitud del IC (nivel de confianza, error estándar y tamaño muestral) y cómo cada uno de estos componentes influye en la precisión de la estimación.

Para profundizar y visualizar simulaciones sobre estos conceptos, pueden explorar recursos interactivos como:

Viendo la teoría: Una introducción visual a probabilidad y estadística

Además, existe un ejemplo práctico de cálculo de IC para la media utilizando la distribución t de Student en el siguiente enlace:

Intervalos de confianza con R

10.3. Test de hipótesis

Aunque los estudios de corte transversal no se diseñan originalmente con grupos de comparación, en aquellos de carácter más analítico es frecuente establecer comparaciones. Por ejemplo, pueden surgir preguntas como:

¿La prevalencia de la enfermedad es mayor en mujeres, en determinados grupos etarios o en una provincia específica?

Con esto en mente, revisaremos las herramientas que permiten comparar grupos mediante test o contrastes de hipótesis. El propósito de estos test es ofrecer al investigador una herramienta para tomar decisiones sobre la población a partir de la información obtenida en una muestra.

Antes de adentrarnos en la parte estadística, es importante distinguir entre dos tipos de hipótesis:

- **Hipótesis de investigación:** Es la conjetura o suposición que motiva la investigación.
- **Hipótesis estadística:** Es aquella que puede ser evaluada mediante técnicas estadísticas apropiadas.

En este texto nos centraremos en aclarar aspectos relativos a las hipótesis estadísticas, asumiendo que las hipótesis de investigación ya han sido discutidas previamente por los investigadores. Describiremos brevemente el razonamiento subyacente a estos test.

Los contrastes de hipótesis parten de una **hipótesis nula**, la cual afirma que los dos grupos comparados son iguales o, en otras palabras, que las diferencias observadas se deben únicamente al azar. Como ya se ha mencionado, la variabilidad intrínseca de cualquier muestra impide que la diferencia entre grupos sea exactamente cero.

El método estadístico nos permite cuantificar la diferencia entre grupos asumiendo que, si repitiésemos el experimento infinitas veces obtenemos todas las posibles muestras del tamaño indicado a partir de nuestras poblaciones (*distribución muestral*), las diferencias entre grupos "iguales" se distribuirían conforme a una curva teórica. Basándonos en las propiedades de esta distribución, podemos determinar un valor límite que comprende, por ejemplo, el 95 % o el 99 % de las diferencias esperadas. Si la diferencia observada entre las muestras supera este valor límite, se considera excesiva para ser atribuible al azar y, por tanto, se rechaza la hipótesis nula. Por el contrario, si la diferencia cae dentro del área del 95 %, se concluye que la diferencia encontrada podría atribuirse al azar, y no habría evidencia que nos permita rechazar la hipótesis nula. En estos casos decimos que los grupos "no son diferentes" pero no "son iguales", ya que la variabilidad inherente impide probar una igualdad exacta.

Los contrastes de hipótesis se realizan generalmente bajo las siguientes condiciones:

- Se asume a priori que la ley de distribución de la población es conocida.
- Se extrae una muestra aleatoria de dicha población.

El conjunto de estas técnicas de inferencia se denomina **técnicas paramétricas**. Sin embargo, existen otros métodos, denominados **técnicas no paramétricas** o contrastes de distribuciones libres, que no requieren estimar parámetros ni suponer una ley de probabilidad específica para la población. Algunos de estos métodos serán desarrollados más adelante en este módulo.

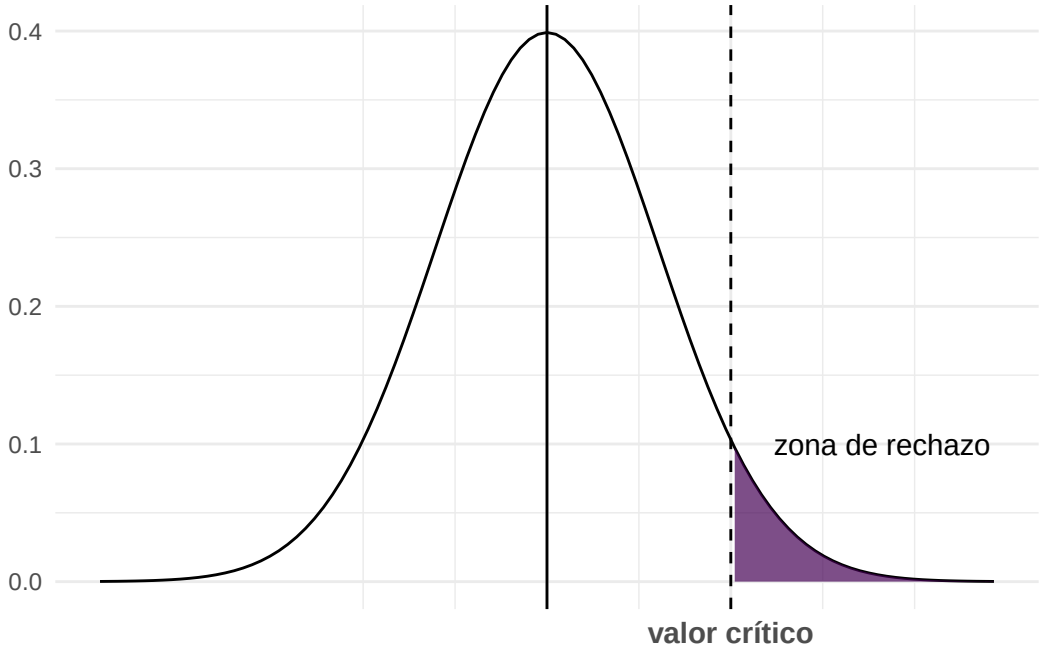
Es importante señalar que los contrastes de hipótesis se utilizan no solo en estudios transversales, sino con mayor frecuencia en diseños en los que *a priori* existen grupos de comparación, como en estudios de casos y controles, cohortes, ensayos clínicos, entre otros. Desarrollaremos aquí la teoría que los sustenta y los utilizaremos a lo largo de todo el curso.

10.3.1. Estructura del test de hipótesis

Los componentes básicos de cualquier test de contraste de hipótesis son:

- **Hipótesis nula** (H_0): Afirma que no existe diferencia entre los grupos que se comparan, es decir, que las variaciones observadas se deben únicamente al azar.
- **Hipótesis alternativa** (H_1): Es la conjetura o suposición que plantea el investigador, estableciendo que sí existe una diferencia entre los grupos. Generalmente es complementaria de la H_0.
- **Estadístico de prueba**: Es el valor calculado a partir de los datos muestrales que se utiliza para tomar la decisión sobre la H_0. Cada tipo de problema tiene un estadístico adecuado, cuya magnitud, al compararse con su distribución muestral (por ejemplo, la distribución normal estándar en el caso del estadístico Z), permite determinar si las diferencias observadas son atribuibles al azar.

- **Valor crítico o Región crítica**: La región crítica se establece en función del nivel de significación (α) y consiste en el conjunto de valores extremos del estadístico de prueba que, de ser alcanzados, llevarían a rechazar la H_0. Todos los posibles valores del estadístico se ubican en el eje horizontal de la gráfica de su distribución, y la región crítica delimita aquellos valores que son muy poco probables bajo la hipótesis nula.



La regla de decisión es la siguiente:

- Si el valor del estadístico de prueba calculado a partir de la muestra cae en la región crítica, se rechaza la H_0 y se concluye que las diferencias observadas son **estadísticamente significativas**.
- Si el valor no cae en la región crítica, no se rechaza la H_0; esto indica que las diferencias entre lo observado y lo esperado pueden explicarse por el azar. Es decir, **no son estadísticamente significativas**.
- **Nivel de significación** (α): Es la probabilidad de cometer un error tipo I, es decir, rechazar H_0 cuando es verdadera. Este valor, elegido por el investigador (comúnmente 5 % o 1 %), determina el límite entre la región de no rechazo y la región crítica.
- **Valor *p***: Es una medida de qué tan probable son los resultados de la muestra, considerando que H_0 sea verdadera.
    - Un valor p muy pequeño indica que es muy poco probable obtener los resultados observados para el estadístico muestral si H_0 fuese cierta, por lo que debemos rechazarla.
    - Esto significa que si el valor $p \leq \alpha$, es posible rechazar H_0; mientras que si $p > \alpha$, no es posible rechazar la hipótesis nula.

  Por ejemplo, si en un test de contraste de dos proporciones se obtiene un estadístico $Z = 3,034$ y un valor $p = 0.0024$, esto significa que la probabilidad de obtener un valor de Z de 3.034 o mayor, suponiendo que H_0 es cierta, es del 0.24 %. Dado que este valor es mucho menor que un nivel de significación del 5 % (o incluso del 1 %), se rechaza H_0 y se concluye que la diferencia observada es estadísticamente significativa. En otras palabras, si bajo un supuesto dado, la probabilidad de un suceso observado particular es excepcionalmente pequeña, concluimos que el supuesto probablemente es incorrecto (*regla del suceso infrecuente*).

10.3.2. Tipos de contrastes

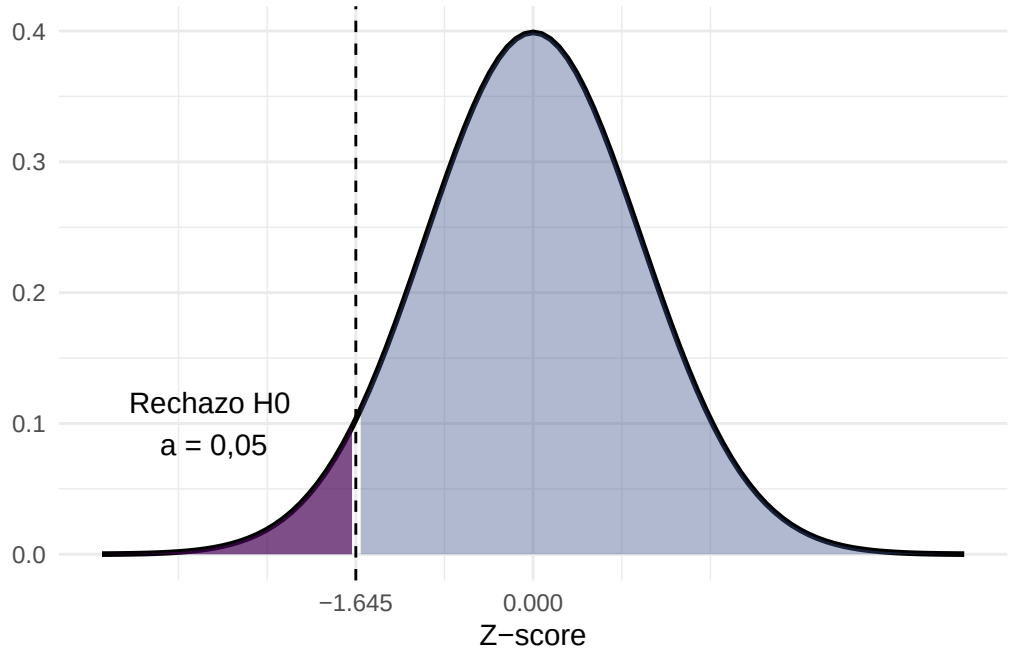
Los contrastes de hipótesis se clasifican según la forma de la hipótesis alternativa (H_1). Esta clasificación determina si la prueba es **unilateral** (de cola izquierda o derecha) o **bilateral** (de dos colas). Siendo las colas de la distribución las regiones extremas limitadas por los valores críticos.

Test de cola izquierda

La hipótesis alternativa plantea que la media del primer grupo es significativamente **menor** que la del segundo:

$$H_1 : \mu_1 < \mu_2$$

La región crítica se encuentra en el extremo izquierdo de la distribución. Todo el área crítica tiene un tamaño α con un valor crítico de $-1,645$.

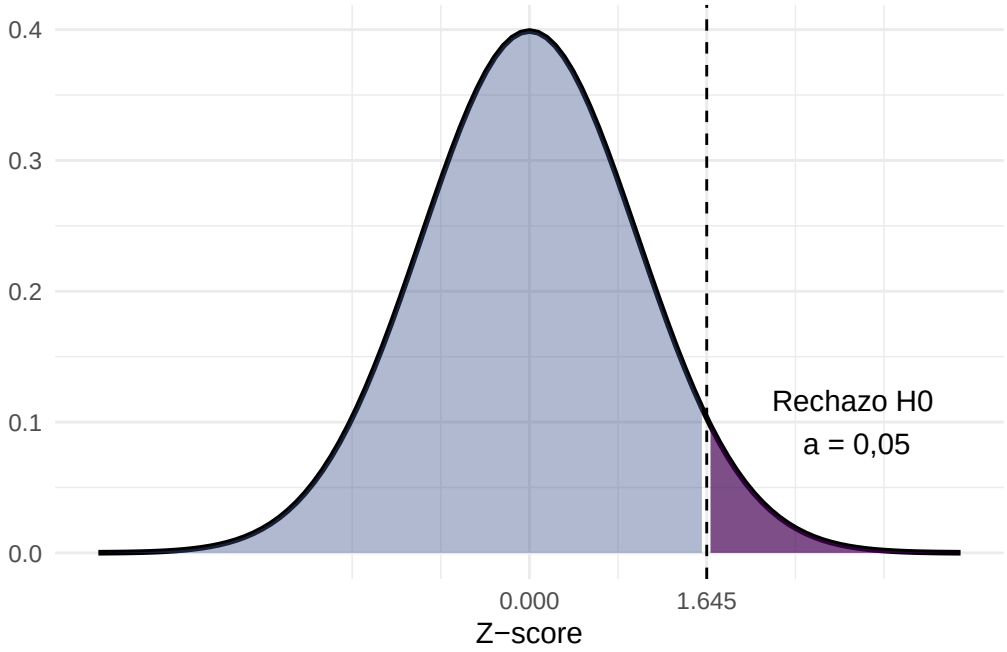


Test de cola derecha

La hipótesis alternativa establece que la media del primer grupo es significativamente **mayor** que la del segundo:

$$H_1 : \mu_1 > \mu_2$$

La región crítica se concentra en el extremo derecho de la distribución y toda el área crítica tiene un tamaño α con un valor crítico de $1,645$.

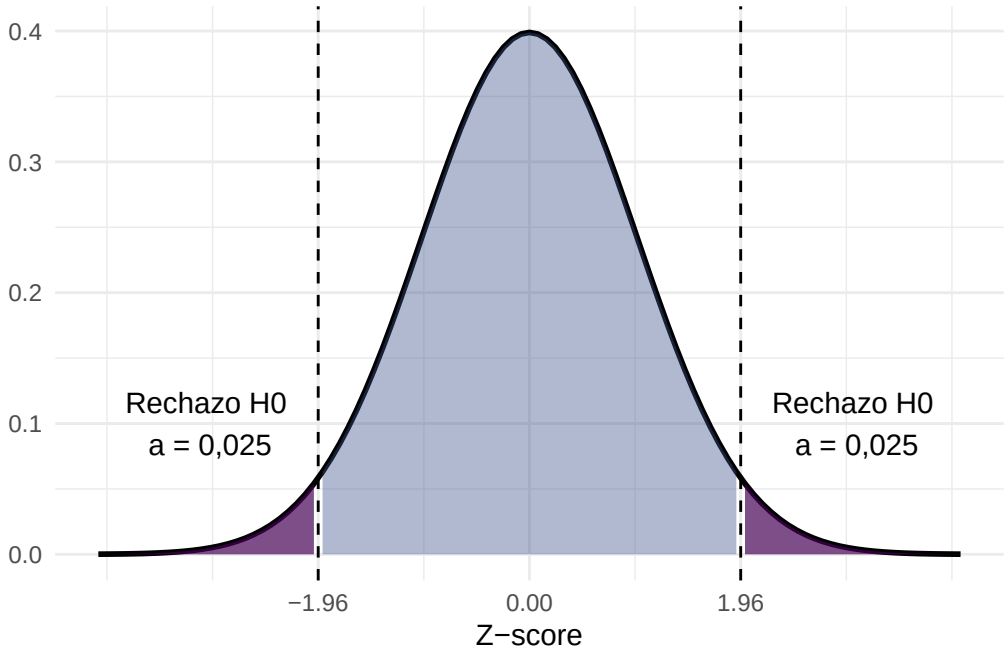


Pruebas bilaterales

La hipótesis alternativa afirma que existen diferencias entre los grupos, **sin especificar la dirección**:

$$H_1 : \mu_1 \neq \mu_2$$

La región crítica se divide entre ambos extremos de la distribución, con valores críticos de $\pm 1,96$. El nivel de significación total (α) se reparte en partes iguales entre las dos colas ($\alpha/2$ en cada una), lo que implica un 2,5 % de probabilidad en cada cola si H_0 es verdadera.



10.3.3. Errores

Al utilizar el razonamiento de los contrastes de hipótesis, existen dos tipos principales de errores que podemos cometer:

- **Error tipo I** (α): Ocurre cuando el investigador rechaza la hipótesis nula (H_0) siendo esta verdadera en la población y se concluye erróneamente que existe una diferencia cuando en realidad no la hay. Se suele eligir un valor pequeño de α (0.01, 0.05 y 0.10) para hacer que la probabilidad de rechazo de H_0 sea pequeña.
- **Error tipo II (**β): Ocurre cuando el investigador no rechaza la H_0 siendo esta falsa en la población, es decir, se falla en detectar una diferencia real. Generalmente β es mayor que α, pero su valor real se desconoce en la práctica.

Es importante notar que, una vez que se realiza el procedimiento de prueba, no es posible saber con certeza si se ha cometido alguno de estos errores, ya que se desconoce el verdadero estado de la realidad. Sin embargo, al fijar un α pequeño, se busca asegurar que, en caso de rechazar la H_0, la probabilidad de haber cometido un error Tipo I sea baja.

En resumen, al interpretar los resultados de un test de hipótesis:

- **Si se rechaza la** H_0: se asume que la probabilidad de haber cometido un error Tipo I es baja (debido al valor pequeño de α).
- **Si no se rechaza la** H_0: se desconoce el riesgo real de un error Tipo II, pero es importante tener en cuenta que, en muchas situaciones, este riesgo es mayor que el de un error Tipo I.

La tabla que se presenta a continuación resume las posibles situaciones a las que nos enfrentamos con los test de hipótesis:

	No rechazar H0	Rechazar H0
H0 es cierta	Correcto (1-α)	Error tipo I (α)
H0 es falsa	Error tipo II (β)	Correcto (1-β)

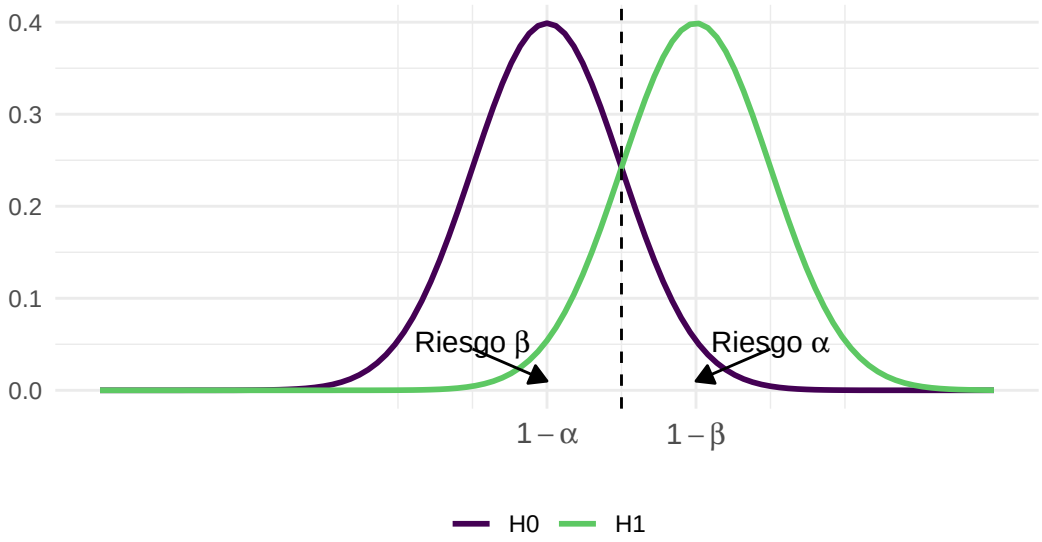
Para quienes están familiarizados con el ámbito del diagnóstico, existe una clara analogía entre los falsos positivos y falsos negativos en las pruebas diagnósticas y, respectivamente, el error Tipo I y el error Tipo II en los contrastes de hipótesis.

Hemos discutido el significado de α (error Tipo I); ahora veamos qué implica β. Recordemos que el error Tipo II es análogo a los falsos negativos de las pruebas diagnósticas: es la probabilidad de no detectar una diferencia cuando, en realidad, ésta existe. En otras palabras, β es la probabilidad de no rechazar la hipótesis nula (H_0) siendo esta falsa.

A diferencia de α que se fija en un único valor y es determinado por el investigador, β varía según el valor real del parámetro en estudio. Por ejemplo, si consideramos la hipótesis nula $H_0 : \mu_1 - \mu_2 = 0$, habrá un valor de β para cada posible diferencia entre μ_1 y μ_2 cuando el valor real no sea cero. La probabilidad de detectar correctamente una diferencia real - es decir, de obtener un resultado estadísticamente significativo cuando la diferencia existe- se denomina **potencia estadística** y se expresa como $1 - \beta$.

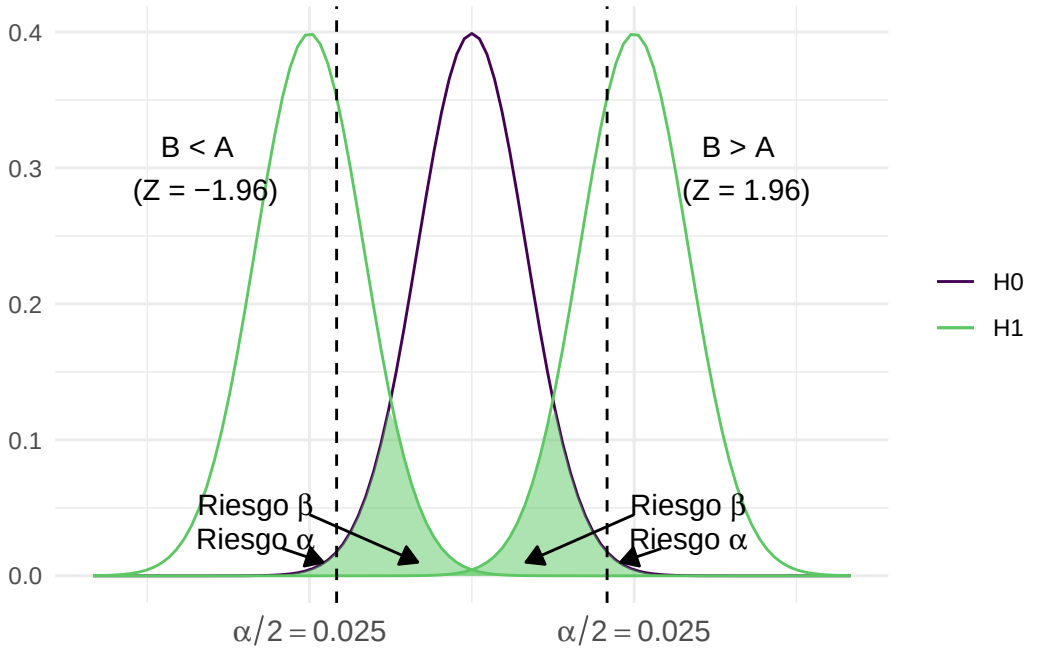
Los contrastes de hipótesis no son exclusivos de los estudios transversales; por el contrario, su uso es más común en estudios analíticos que involucran grupos de comparación, tales como estudios de casos y controles, cohortes o ensayos clínicos. Por ejemplo, en un ensayo clínico que evalúa dos tratamientos, la potencia estadística refleja la capacidad del estudio para identificar un efecto real del tratamiento.

Es deseable que la potencia del estudio sea lo mayor posible, ya que esto incrementa la probabilidad de detectar diferencias verdaderas. Sin embargo, no es posible minimizar ambos errores simultáneamente ya que al disminuir α (es decir, al ser más exigentes para rechazar la H_0), β tiende a aumentar, y viceversa. En los contrastes, la hipótesis privilegiada es H_0 que solo será rechazada cuando la evidencia de su falsedad supere el umbral del $1-\alpha$. Esto significa que, a menos que la evidencia en contra de H_0 sea muy significativa, se opta por no rechazarla. Lo ideal a la hora de definir un test es encontrar un compromiso entre β y α.



Test de una sola cola:
B > A
(Z = 1.645)

Mientras que para tests bilaterales:



Finalmente, podemos decir que la potencia estadística ofrece un segundo mecanismo de seguridad en un contraste de hipótesis. Es como contar con una protección adicional en la toma de decisiones: si solo dispusiéramos del nivel de significación (α), tendríamos menos garantías. Al incorporar la potencia ($1 - \beta$) agregamos un segundo control. Por ello, en un contraste no basta con tener un valor *p* pequeño; también se necesita una potencia alta, que en la práctica suele fijarse en un 80 % (es decir, $1 - \beta = 0.8$).
Por otro lado, cuando existe una diferencia real —o un efecto real de una terapia, o una verdadera diferencia entre dos fármacos—, la **magnitud** de ese efecto influye en la facilidad para detectarlo. Los efectos grandes son más fáciles de identificar que los pequeños. Para estimar la potencia de una prueba, debemos especificar el **efecto mínimo** que valga la pena identificar.

La potencia de un test estadístico depende de tres factores que actúan de manera interrelacionada:

- El **riesgo de error** que se tolerará al rechazar la hipótesis de ausencia de efecto o diferencia (α).
- La **dimensión de la diferencia** que se desea identificar en relación con la variabilidad en las poblaciones.
- El **tamaño de la muestra**.

Del mismo modo que en un problema de estimación se necesita una idea de la magnitud a estimar y del error aceptable para definir el tamaño de la muestra, en un contraste de hipótesis se requiere conocer el **tamaño del efecto** que se quiere detectar. Así, el tamaño muestral se determina en función del nivel de confianza y de la potencia de la prueba, además de otros aspectos relacionados con el diseño y la prueba estadística elegida.

En epidemiología, una de las situaciones más frecuentes al diseñar un estudio es el cálculo del tamaño muestral para un **nivel de confianza del 95 %** y una **potencia del 80 %**, que, como se mencionó, es un nivel de potencia alto y que además permite manejar un α relativamente bajo. El lenguaje **R** cuenta con diversas funciones para **calcular y visualizar** la relación entre tamaño muestral, potencia, tamaño del efecto y nivel de confianza, facilitando el diseño de estudios con un adecuado balance entre la probabilidad de detectar diferencias reales y la de controlar errores estadísticos.

Por ejemplo, la función `pwr.t.test()` del paquete `pwr` (Champely 2020), calcula la potencia para pruebas t de Student de medias (para una muestra, dos muestras y muestras pareadas), basado en el tamaño de la muestra, el nivel de confianza y el tamaño de efecto:

```
pwr.t.test(n, d, sig.level = 0.05, power, type, alternative)
```

Donde:

- `n`: Número de observaciones para cada grupo.
- `d`: tamaño de efecto (d de Cohen) - diferencia (estandarizada) entre grupos.

  > *Nota:* La d de Cohen representa las desviaciones estándar que separan dos o más grupos. Por ejemplo: $d_{Cohen} = 0.5$ representa que la diferencia entre grupo experimental y muestral es de media desviación estándar. Cohen sugirió (provisoriamente) que 0.2 es un tamaño de efecto pequeño, 0.5 es mediano y 0.8 es grande,

- `sig.level`: nivel de significación (probabilidad del error de tipo I).
- `power`: potencia del test (1 menos la probabilidad del error tipo II).
- `type`: tipo de test (`"two.sample"`, `"one.sample"`, `"paired"`).
- `alternative`: palabra que especifica la hipotesis alternativa, debe ser `"two.sided"` (predeterminado), `"greater"` or `"less"`.

La función se ejecuta incorporando todos los argumentos obligatorios (`d`,`n`, `power` y `sig.level`) menos el que se quiere calcular. En ese caso se iguala a `NULL` o se omite.

Supongamos que queremos conocer el tamaño de la muestra para detectar diferencias en la media de la hemoglobina glicosilada entre dos grupos de pacientes con tratamientos de control de la diabetes distintos. Aceptamos un nivel de efecto convencional de una pequeña desviación ($d_{Cohen} = 0.2$), una potencia del 80 % y una significación habitual de 0,05.

Cargamos el paquete requerido:

```
library(pwr)
```

Calculamos el tamaño de muestra:

```
pwr.t.test(
  d = 0.2,
  power = 0.8,
  sig.level = 0.05,
  type = "two.sample",
  alternative = "two.sided"
)
```

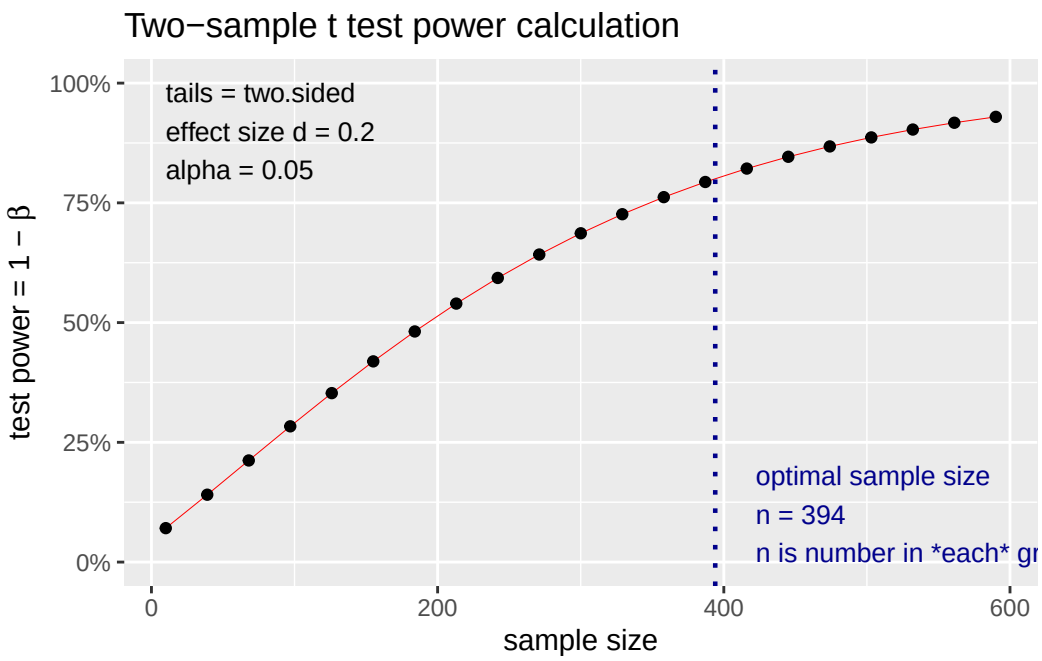
```
     Two-sample t test power calculation

              n = 393.4057
              d = 0.2
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

También podemos graficar la salida:

```
pwr.t.test(
  d = 0.2,
  power = 0.8,
  sig.level = 0.05,
  type = "two.sample",
  alternative = "two.sided"
) |>
  plot()
```



Observamos que, si por ejemplo tomásemos una muestra de 300 individuos por grupo, la potencia del estudio con ese tamaño de efecto y nivel de significación del 0,05 sería aproximadamente de 68 %.

10.3.4. ¿Qué test se debe aplicar en cada caso?

Hemos discutido la mecánica general de los test de hipótesis. Ahora, nos centraremos en orientarnos sobre qué test aplicar en cada situación. Aunque existe un desarrollo teórico detrás de cada caso, aquí nos quedaremos con ciertas reglas prácticas.

Para sistematizar los contrastes de hipótesis, es útil responder dos preguntas fundamentales:

- **¿Qué tipo de variable dependiente tengo?**
  La variable resultado puede ser cuantitativa, cualitativa, de tiempo hasta un evento, etc.
- **¿Qué se está comparando?**
  Esto se traduce en evaluar el tipo de experimento: ¿se comparan dos grupos o más? ¿Son muestras independientes o relacionadas (por ejemplo, medidas en el mismo grupo antes y después de una intervención)?

Variable resultado cuantitativa

Cuando la variable de interés es cuantitativa, la comparación de grupos generalmente se traduce en comparar las medias de dichos grupos. En este contexto, se deben responder una tercera pregunta y es si dicha variable se *distribuye normalmente*, porque si no fuera así, debemos recurrir a los contrastes *no paramétricos*.

> En la **siguiente sección** detallaremos las pruebas de normalidad y heterogeneidad de varianzas y su aplicación en R.

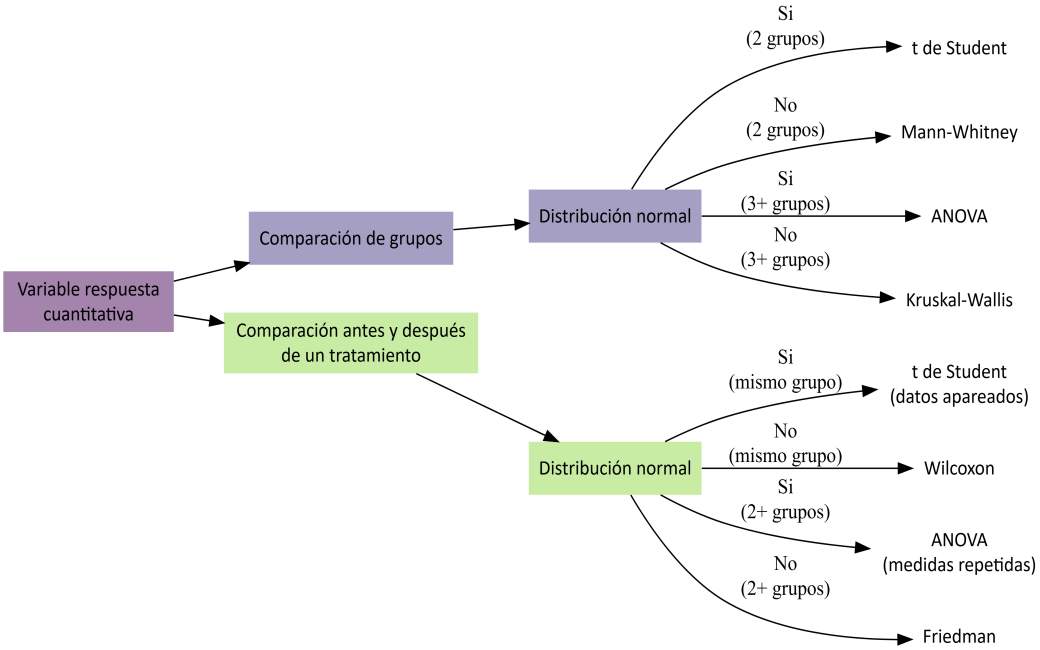
Por ejemplo, para comparar dos grupos se plantearía:

- $H_0 : \mu_1 = \mu_2$ (o su equivalente $\mu_1 - \mu_2 = 0$)

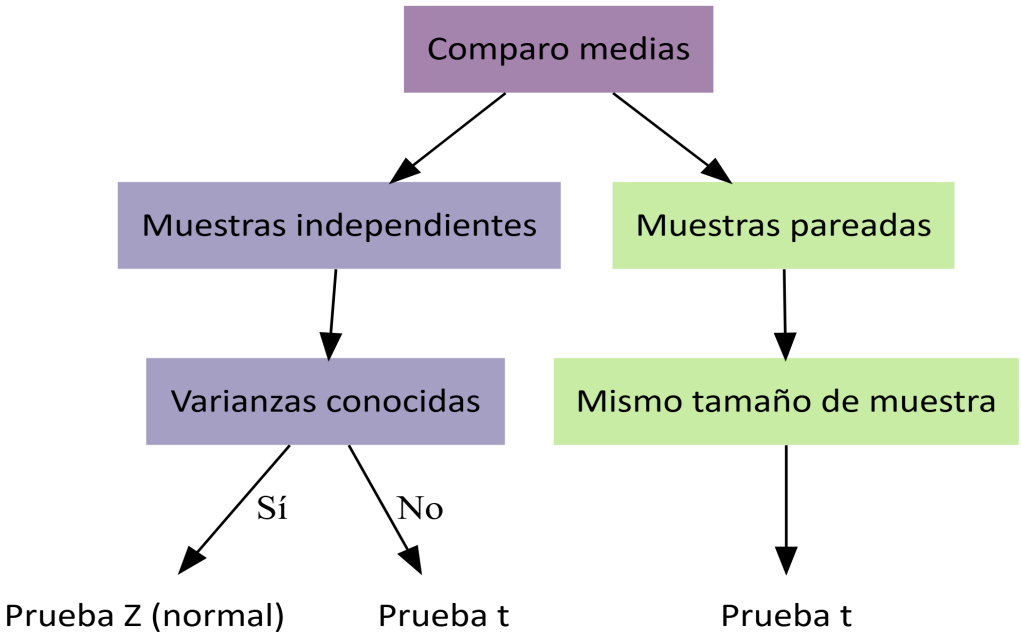
- $H_1 : \mu_1 \neq \mu_2$ (contraste bilateral) o bien $\mu_1 < \mu_2$ o $\mu_1 > \mu_2$ (contrastes unilaterales).

Se calcula el estadístico de prueba (por ejemplo, t de Student o Z, según el tamaño de la muestra y si se conoce la varianza poblacional), se determina la región de rechazo y, finalmente, si $p < \alpha$ rechazo H_0 y si $p > \alpha$ no rechazo H_0.

A continuación presentamos un esquema que sirve de guía para potenciales situaciones:



Hay algo más que los estadísticos nos "obligan" a considerar en el caso que compare medias independientes y se haya verificado la normalidad de la distribución, y se trata de considerar si las varianzas de las poblaciones que comparo son iguales o diferentes. El algoritmo de resolución es el siguiente:



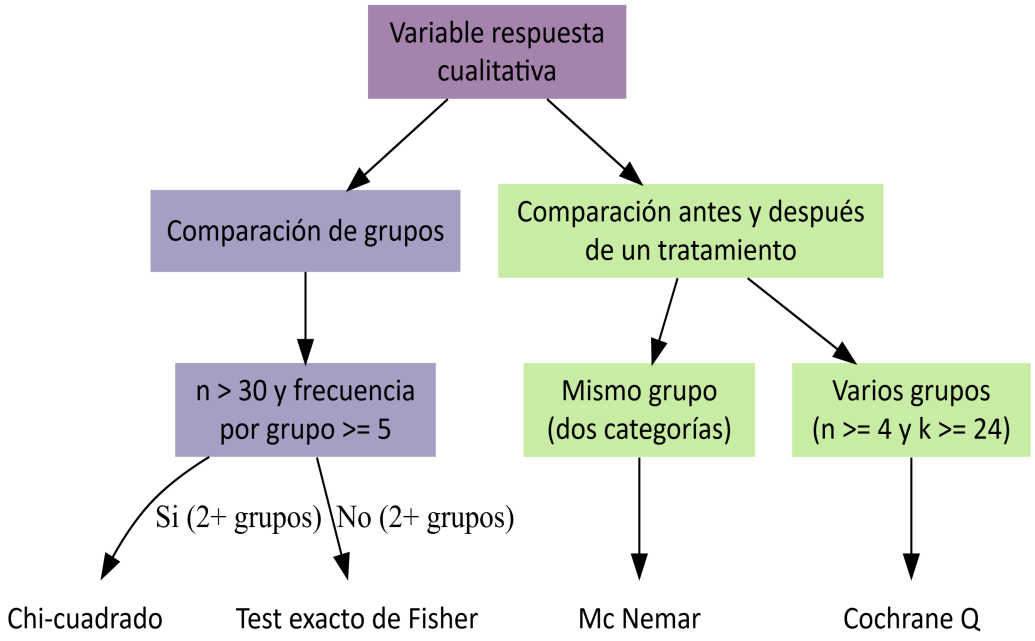
Cuando trabajamos con muestras independientes, a menudo nos encontramos en el escenario de *varianzas desconocidas*. Esto nos obliga a realizar, de manera previa, un test de homogeneidad de varianzas para

determinar si se puede asumir que son iguales o si, por el contrario, difieren. La razón es que, aunque se utiliza el estadístico t de Student para comparar las medias, su cálculo y la forma en que se distribuye varían en función de si las varianzas de los grupos son iguales o no.

Variable resultado cualitativa

Cuando la variable dependiente es cualitativa (categórica), como por ejemplo Enfermo (Sí/No) o Expuesto (Sí/No), la comparación de grupos no se basa en medias, sino en **proporciones**. Para ello, se utilizan pruebas estadísticas específicas según el número de grupos y el diseño del estudio.

A continuación, presentamos una guía análoga a la anterior:



10.4. Ejemplo práctico en R

En el archivo "`bebe1.txt`" se encuentran datos de una muestra aleatoria de los pesos al nacer de 186 bebés cuyas madres no consumieron cocaína durante el embarazo. En el archivo "`bebe2.txt`" se encuentran datos de una muestra aleatoria de 190 bebés cuyas madres consumieron cocaína durante el embarazo. Nos interesa conocer el peso promedio al nacer de los bebés de la primer base de datos y, de acuerdo a estos resultados, saber si el consumo de cocaína afecta el peso que tiene un bebé al nacer.

Podemos resolver la primer parte manualmente:

$$IC = \bar{x} \pm Z_{1-\alpha/2} \frac{\sigma}{\sqrt{n}}$$

donde:

- media ($\bar{x}$): 3103 gramos
- desvío estándar (σ): 696

- número de observaciones (n): 186

$$IC = 3103 \pm 2,57\frac{696}{\sqrt{186}} = 3103 \pm 131,16$$

$$IC = 2971,85 - 3234,15$$

El peso medio al nacer de los bebés oscila entre 2971,85 y 3234,15 gramos ($\alpha = 0,99$).

Para resolver las preguntas en R, comenzaremos cargando los paquetes necesarios:

```
library(tidyverse)
library(dlookr)
```

Cargamos los datos:

```
datos1 <- read_csv2("datos/bebe1.txt")

datos2 <- read_csv2("datos/bebe2.txt")
```

La función `t.test()` del paquete `base` utiliza la distribución t *de Student*:

```
t.test(datos1, conf.level = 0.99)
```

```

    One Sample t-test

data:  datos1
t = 60.804, df = 185, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
99 percent confidence interval:
 2970.178 3235.822
sample estimates:
mean of x
     3103
```

Los IC producidos son similares a los calculados manualmente.

Para comparar los datos de los bebés de madres que consumieron cocaína, debemos realizar un test de comparación de medias para dos poblaciones independientes. Para ello, comenzamos por chequear normalidad:

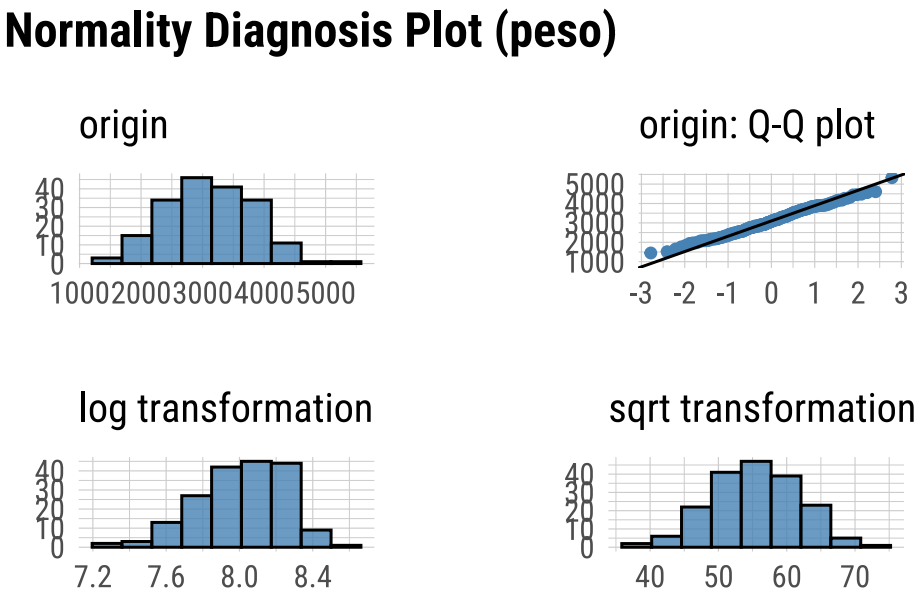
```
shapiro.test(datos1$peso)
```

```

    Shapiro-Wilk normality test

data:  datos1$peso
W = 0.99293, p-value = 0.5092
```

```
datos1 |>
  plot_normality()
```

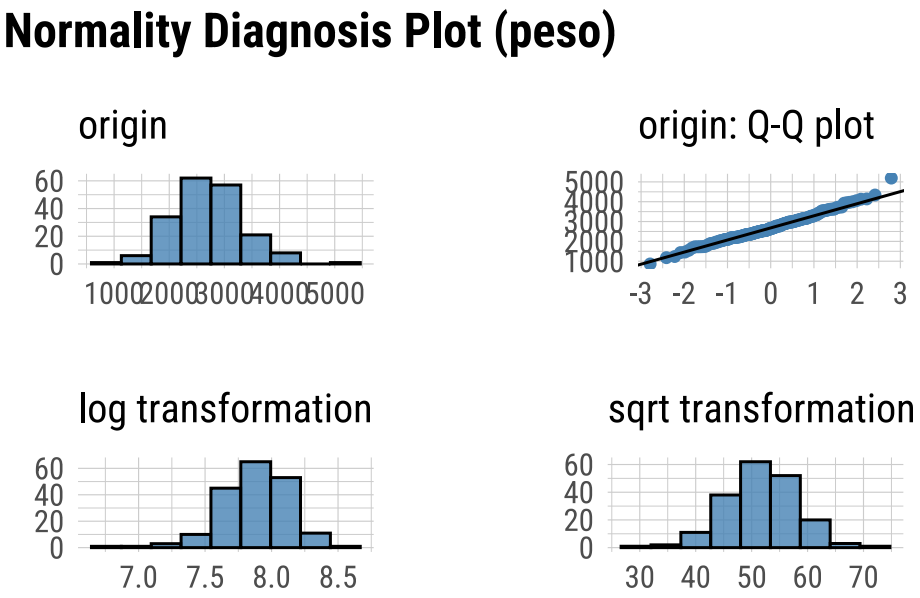



```
shapiro.test(datos2$peso)
```

```
    Shapiro-Wilk normality test

data:  datos2$peso
W = 0.98951, p-value = 0.1774
```

```
datos2 |>
  plot_normality()
```



Se cumple con el supuesto de normalidad, ahora procedemos a comparar varianzas usando la función `var.test()`:

```
var.test(datos1$peso, datos2$peso)
```

```
    F test to compare two variances

data:  datos1$peso and datos2$peso
F = 1.1644, num df = 185, denom df = 189, p-value = 0.2986
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.8736082 1.5526583
sample estimates:
ratio of variances
          1.164392
```

Como las varianzas son iguales, aplicamos un t test para comparar medias de dos poblaciones independientes con varianzas iguales planteando un contraste bilateral:

$$H_0 : \mu_1 = \mu_2 H_1 : \mu_1 \neq \mu_2$$

```
t.test(
       x = datos1$peso,
       y = datos2$peso,
       alternative = "two.sided",
       var.equal = TRUE
)
```

```
    Two Sample t-test

data:  datos1$peso and datos2$peso
t = 5.8252, df = 374, p-value = 1.231e-08
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 266.9644 539.0356
sample estimates:
mean of x mean of y
     3103      2700
```

O también podríamos plantear un contraste unilateral, si consideramos que nuestra hipótesis de trabajo es que el peso de los bebés de madres que no consumen cocaína es mayor que aquellos de madres que sí consumen.

$$H_0 : \mu_1 = \mu_2 H_1 : \mu_1 > \mu_2 \qquad \mu_1 - \mu_2 > 0$$

```r
t.test(
       x = datos1$peso,
       y = datos2$peso,
       alternative = "greater",
       var.equal = TRUE
)
```

```
    Two Sample t-test

data:  datos1$peso and datos2$peso
t = 5.8252, df = 374, p-value = 6.155e-09
alternative hypothesis: true difference in means is greater than 0
95 percent confidence interval:
 288.9222      Inf
sample estimates:
mean of x mean of y
     3103      2700
```

Concluimos que los pesos de los bebés hijos de madres que no consumen cocaína es significativamente mayor respecto de los hijos de madres que sí la consumen.

10.4.1. Inferencia sobre una población

En algunas situaciones, además de obtener intervalos de confianza para estimar parámetros como la media o la proporción poblacional, también puede ser de interés realizar contrastes de hipótesis sobre un valor específico de referencia. Este tipo de análisis se denomina **inferencia sobre una población** y permite evaluar si un parámetro poblacional es significativamente diferente de un valor dado.

Un caso común en epidemiología y medicina es el estudio de la eficacia de un tratamiento. Supongamos que queremos evaluar si un nuevo fármaco reduce la presión arterial en pacientes con un determinado

síndrome. Se sabe que, en ausencia de tratamiento, la presión arterial media en estos pacientes es de **165 mmHg**. Para investigar el efecto del fármaco, se administra a un grupo de **15 pacientes** y se mide su presión arterial después del tratamiento.

Queremos evaluar si la presión arterial media en pacientes tratados con el nuevo fármaco es **significativamente menor** que la presión arterial histórica de 165 mmHg:

- $H_0 : \mu = 165\, mmHg$
- $H_1 : \mu < 165\, mmHg$

Este es un **contraste unilateral a la izquierda**, ya que buscamos evidencia de que el tratamiento reduce la presión arterial.

Cargamos los datos:

```
enfermos <- read_csv2("datos/enfermos.txt")
```

Calculamos la media y el desvío estándar:

```
enfermos |>
  summarise(media = mean(V1), desvio = sd(V1))
```

```
# A tibble: 1 x 2
  media desvio
  <dbl>  <dbl>
1  152.   22.0
```

Evaluamos normalidad:

```
shapiro.test(enfermos$V1)
```

```

    Shapiro-Wilk normality test

data:  enfermos$V1
W = 0.94967, p-value = 0.5192
```

Este grupo de enfermos tratados con el nuevo fármaco tienen una media de aproximadamente 151.9619928, con un desvío de 21.9788237. Además, los datos cumplen con el supuesto de normalidad.

Realizamos la prueba de hipótesis para evaluar si este valor es significativamente menor a 165:

```
t.test(x = enfermos$V1, alternative = "less", mu = 165, conf.level = 0.95)
```

```

    One Sample t-test

data:  enfermos$V1
t = -2.2975, df = 14, p-value = 0.01876
```

```
alternative hypothesis: true mean is less than 165
95 percent confidence interval:
     -Inf 161.9573
sample estimates:
mean of x
  151.962
```

Dado que $p < 0.05$, rechazamos la hipótesis nula y concluimos que, con un nivel de confianza del 95 %, el nuevo fármaco **reduce significativamente la presión arterial** en estos pacientes.

Rodríguez y Mendivelso (2018)

Hernández-Ávila (2011)

Álvarez Cáceres (2007)

Triola (2018)

Rodríguez et al. (2003)

Norman, GR y Streiner, DL (1996)

Ríus Díaz et al. (2012)

Daniel (2002)

Glantz, S (2006)

Ryu (2025)

Manitz et al. (2021)

EPIDAT 4.2 - Consellería de Sanidade - Servizo Galego de Saúde (s. f.)

Referencias

11. Análisis de normalidad y homocedasticidad

11.1. Introducción

Este material es una continuación del capítulo sobre **Análisis exploratorio de datos** de la **Unidad 1**. Recordemos que uno de los objetivos del EDA es conocer cómo se distribuyen los valores de las variables de interés.

Las características fundamentales a la hora de decidir si utilizaremos métodos paramétricos o no paramétricos para la inferencia estadística, es que los datos se ajusten a una distribución normal y conocer si tienen una dispersión homogénea o heterogénea. Ahora que conocemos el funcionamiento interno de los test de hipótesis, podemos usar **pruebas de bondad de ajuste** para evaluar los supuestos de normalidad y homocedasticidad.

11.2. Normalidad

Determinar que una distribución es aproximadamente normal nos permite decidirnos por test de comparaciones paramétricos.

Existen tres enfoques que debemos analizar simultáneamente:

- Métodos gráficos
- Métodos analíticos
- Pruebas de bondad de ajuste

11.2.1. Métodos gráficos

El gráfico por excelencia para evaluar normalidad es el Q-Q Plot que consiste en comparar los cuantiles de la distribución observada con los cuantiles teóricos de una distribución normal con la misma media y desviación estándar que los datos.

Cuanto más se aproximen los datos a una normal, más alineados están los puntos entorno a la recta.

En el lenguaje R hay varios paquetes que tienen funciones para construirlos:

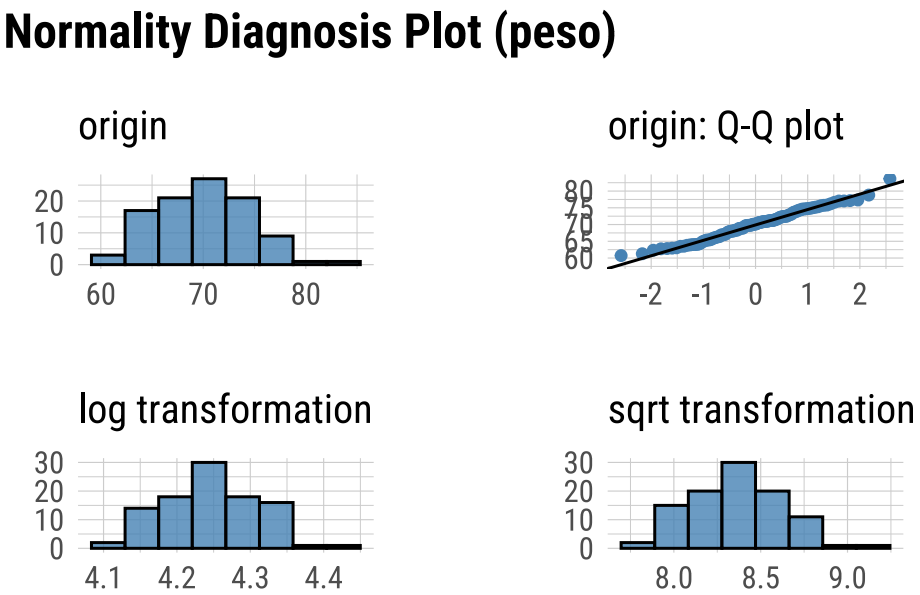
```
library(dlookr)
library(ggpubr)
library(moments)
library(nortest)
library(car)
library(tidyverse)
```

Cargamos datos de ejemplo:

```
datos <- read_csv2("datos/datos_normalidad.csv")
```

Evaluación gráfica de normalidad usando la función `plot_normality()` del paquete `dlookr`:

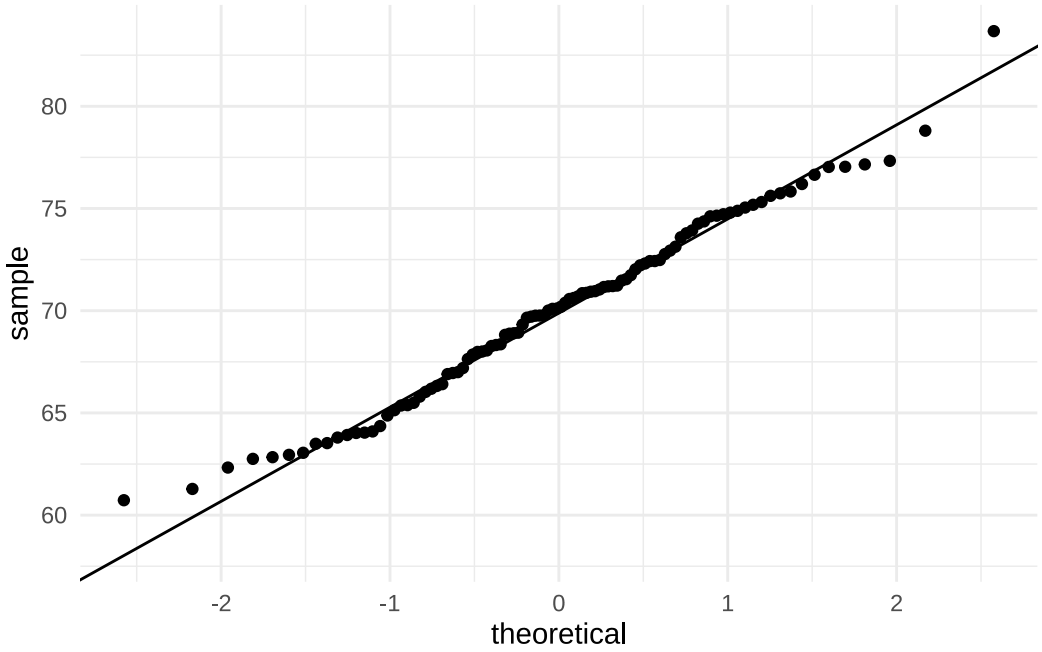
```
datos |>
  plot_normality(peso)
```



A simple vista observamos que los puntos de la variable `peso` *se ajustan bastante bien a la recta*.

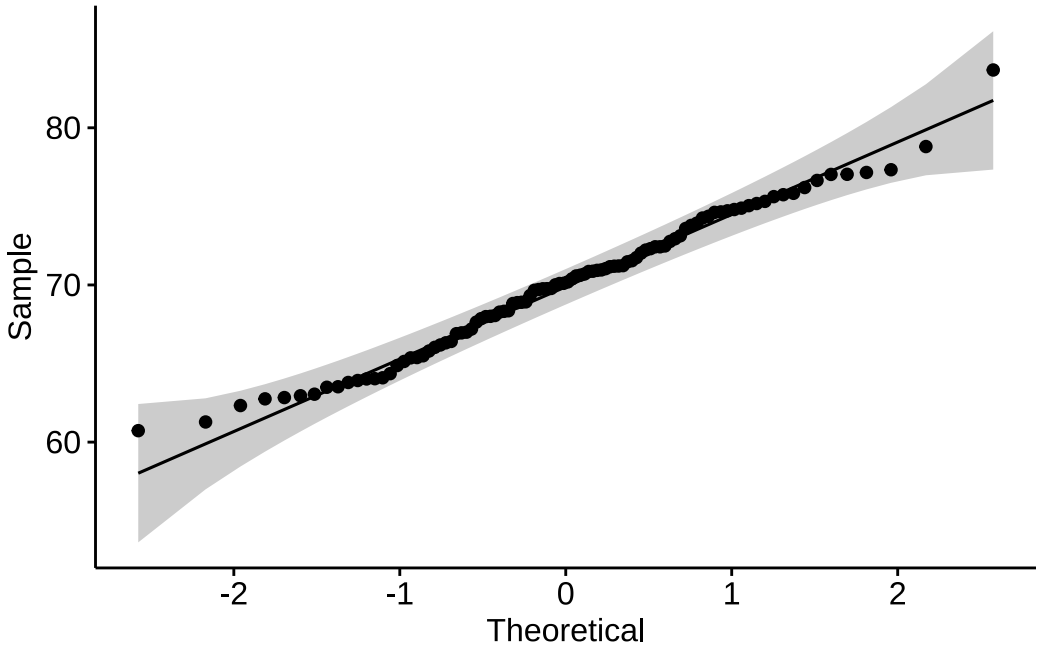
También podemos generar el qqplot usando las funciones `geom_qq_line()` y `geom_qq()` de `ggplot2`:

```
datos |>
  ggplot(mapping = aes(sample = peso)) +
  geom_qq_line() +
  geom_qq() +
  theme_minimal()
```



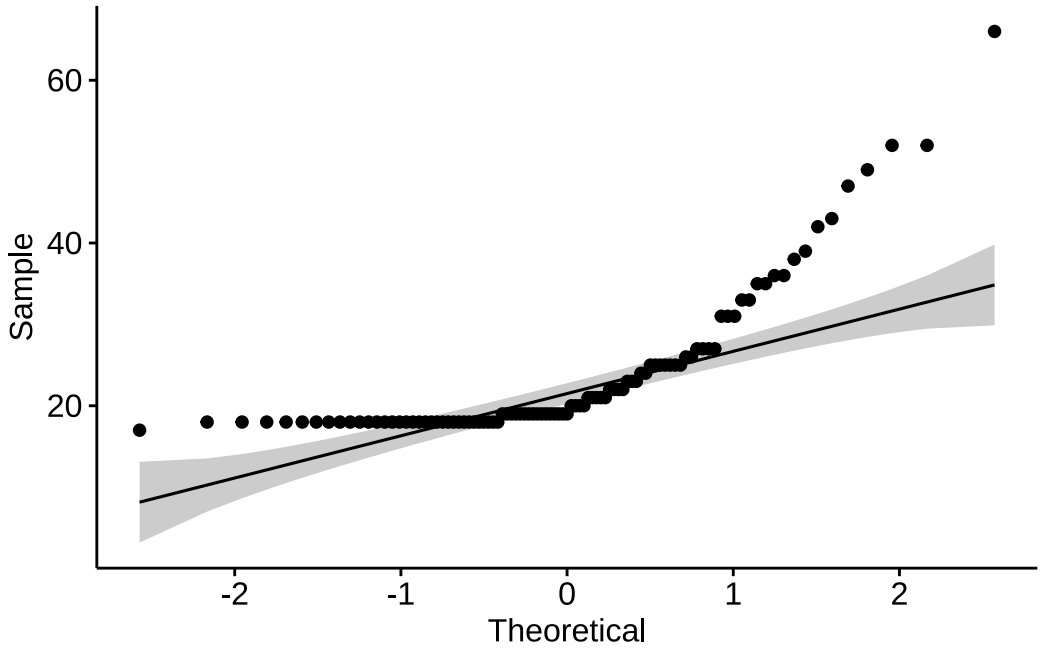
La función `ggqqplot()` del paquete `ggpubr` (Kassambara 2026a) nos permite agregar intervalos de confianza (zona gris alrededor de la recta) que nos orienta mejor sobre "donde caen" los puntos de la variable analizada:

```
datos$peso |>
  ggqqplot()
```



Un ejemplo donde la variable parece no cumplir con el supuesto de normalidad en estos datos de prueba es `edad`:

```
datos$edad |>
  ggqqplot()
```



11.2.2. Métodos analíticos

Medidas de forma

Existen dos medidas de forma útiles que podemos calcular mediante funciones de R.

- La curtosis (kurtosis)
- La asimetría (skewness)

La curtosis mide el grado de agudeza o achatamiento de una distribución con relación a la distribución normal.

- < 0 Distribución platicúrtica (apuntamiento negativo): baja concentración de valores
- > 0 Distribución leptocúrtica (apuntamiento positivo): gran concentración de valores
- $= 0$ Distribución mesocúrtica (apuntamiento normal): concentración como en la distribución normal.

El paquete `moments` (Komsta y Novomestky 2022) posee algunas funciones interesantes para analizar medidas de forma, como el estimador de Pearson para curtosis:

```
datos |>
  summarise(
    kurtosis_edad = kurtosis(edad, na.rm = T),
    kurtosis_peso = kurtosis(peso, na.rm = T)
  )
```

```
# A tibble: 1 x 2
  kurtosis_edad kurtosis_peso
          <dbl>         <dbl>
1          8.05          2.72
```

En los dos casos estamos frente a una *distribución leptocúrtica* pero de magnitudes bien diferentes. Muy alta en el caso de la variable `edad` (8,0) y mucho menor para la variable `peso` (2,7).

El índice de asimetría es un indicador que permite establecer el *grado de asimetría* que presenta una distribución. Los valores menores que 0 indican distribución asimétrica negativa; los mayores a 0: distribución asimétrica positiva y cuando sea 0, o muy próximo a 0, distribución simétrica:

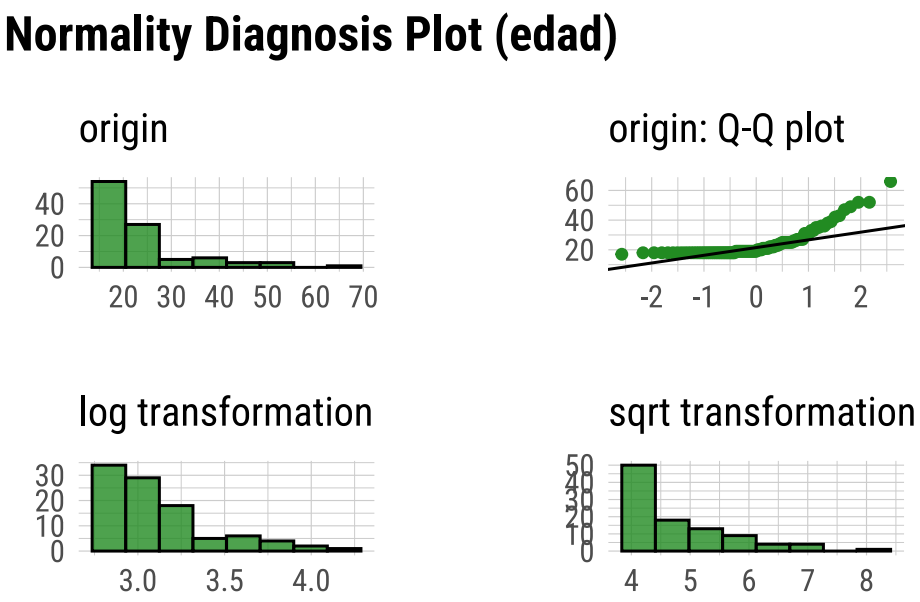
```
datos |>
  summarise(
    asimetria_edad = skewness(edad, na.rm = T),
    asimetria_peso = skewness(peso, na.rm = T)
  )
```

```
# A tibble: 1 x 2
  asimetria_edad asimetria_peso
           <dbl>          <dbl>
1           2.19          0.122
```

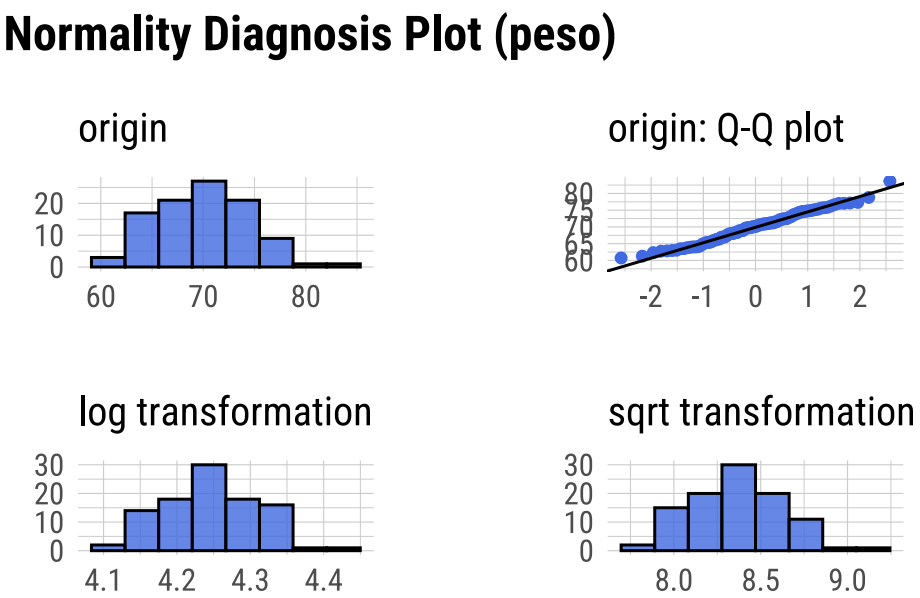
Los valores obtenidos con la función `skewness()` del paquete `moments` nos informan que la distribución de la edad tienen una asimetría positiva (2,2) y que los valores de peso se distribuyen bastante simétricos (0,1).

Estas características de las distribuciones también se pueden ver mediante histogramas o gráficos de densidad:

```
datos |>
  plot_normality(edad, col = "forestgreen")
```



```
datos |>
  plot_normality(peso, col = "royalblue")
```



Los histogramas que se acerquen a la clásica "campana de Gauss" tendrán curtosis y asimetrías alrededor del valor cero.

11.2.3. Pruebas de bondad de ajuste

Una prueba de bondad de ajuste permite testear la hipótesis de que una variable aleatoria sigue cierta distribución de probabilidad y se utiliza en situaciones donde se requiere comparar una distribución observada con una teórica o hipotética.

El mecanismo es idéntico a cualquier test de hipótesis salvo que aquí esperamos no descartar la hipótesis nula de igualdad, por lo que obtener valores *p* de probabilidad mayores a 0,05 es signo de que la distribución de la variable analizada se ajusta.

A continuación, presentaremos los test de hipótesis más utilizados para analizar normalidad.

Test de Shapiro-Wilk

Lleva el nombre de sus autores (Samuel Shapiro y Martin Wilk) y es usado preferentemente para muestras de hasta 50 observaciones.

La función se encuentra desarrollada en el paquete `stats` y se llama `shapiro.test()`:

```
shapiro.test(datos$edad)
```

```
    Shapiro-Wilk normality test

data:  datos$edad
W = 0.69517, p-value = 4.912e-13
```

```
shapiro.test(datos$peso)
```

```
    Shapiro-Wilk normality test

data:  datos$peso
W = 0.98615, p-value = 0.383
```

Interpretación: Siendo la hipótesis nula que la población está distribuida normalmente, si el p-valor es menor a α (nivel de significancia, convencionalmente un 0,05) entonces la hipótesis nula es rechazada (se concluye que los datos no provienen de una distribución normal). Si el p-valor es mayor a α, se concluye que no se puede rechazar dicha hipótesis.

En función de esta interpretación (que es común a todos los test de hipótesis de normalidad), podemos decir que la distribución de la variable **edad** *no se ajusta a la normal* y no podemos rechazar que la distribución de la variable **peso** se ajuste.

Test de Kolmogorov-Smirnov

El test de **Kolmogorov-Smirnov** permite estudiar si una muestra procede de una población con una determinada distribución que no está limitado únicamente a la distribución normal.

El test asume que se conoce la media y varianza poblacional, lo que en la mayoría de los casos no es posible. Para resolver este problema, se realizó una modificación conocida como test Lilliefors.

Test de Lilliefors

El test de **Lilliefors** asume que la media y varianza son desconocidas y está especialmente desarrollado para contrastar la normalidad.

Es la alternativa al test de Shapiro-Wilk cuando el número de observaciones es mayor de 50.

La función `lillie.test()` del paquete `nortest` (Gross y Ligges 2015) permite aplicarlo:

```
lillie.test(datos$edad)
```

```
    Lilliefors (Kolmogorov-Smirnov) normality test

data:  datos$edad
D = 0.24892, p-value < 2.2e-16
```

```
lillie.test(datos$peso)
```

```
    Lilliefors (Kolmogorov-Smirnov) normality test

data:  datos$peso
D = 0.049534, p-value = 0.7905
```

Los resultados son coincidentes con los obtenidos anteriormente.

Test de D'agostino

Esta prueba se basa en las transformaciones de la curtosis y la asimetría de la muestra, y solo tiene poder frente a las alternativas de que la distribución sea sesgada.

El paquete `moments` la tiene implementada en `agostino.test()`:

```
agostino.test(datos$edad)
```

```
    D'Agostino skewness test

data:  datos$edad
skew = 2.1947, z = 6.3465, p-value = 2.203e-10
alternative hypothesis: data have a skewness
```

```
agostino.test(datos$peso)
```

```
    D'Agostino skewness test

data:  datos$peso
skew = 0.12160, z = 0.52683, p-value = 0.5983
alternative hypothesis: data have a skewness
```

Los resultados coinciden con la observación de asimetría que efectuamos con los métodos analíticos, confirmando que la variable `edad` *no se ajusta* a una curva simétrica y la variable `peso` si lo hace.

Cuando estos test se emplean con la finalidad de verificar las condiciones de métodos paramétricos es importante tener en cuenta que, al tratarse de valores probabilidad, cuanto mayor sea el tamaño de la muestra más poder estadístico tienen y más fácil es encontrar evidencias en contra de la hipótesis nula de normalidad.

Por otra parte, cuanto mayor sea el tamaño de la muestra, menos sensibles son los métodos paramétricos a la falta de normalidad. Por esta razón, es importante no basar las conclusiones únicamente en los resultados de los test, sino también considerar los otros métodos (gráfico y analítico) y no olvidar el tamaño de la muestra.

11.3. Homocedasticidad

La homogeneidad de varianzas es un supuesto que considera constante la varianza en los distintos grupos que queremos comparar.

Esta homogeneidad es condición necesaria antes de aplicar algunos test de hipótesis de comparaciones o bien para aplicar correcciones mediante los argumentos de las funciones de R.

Existen diferentes test de bondad de ajuste que permiten evaluar la distribución de la varianza. Todos ellos consideran como H_0 que la varianza es igual entre los grupos y como H_1 que no lo es.

La diferencia entre ellos es el estadístico de centralidad que utilizan:

- Media de la varianza: son los más potentes pero se aplican en distribuciones que se aproximan a la normal.
- Mediana de la varianza: son menos potentes pero consiguen mejores resultados en distribuciones asimétricas.

11.3.1. F-test

Este test es un contraste de la razón de varianzas, mediante el estadístico F que sigue una distribución F-Snedecor.

Se utiliza cuando las distribuciones se aproximan a la "normal" y en R base se la encuentra en la función `var.test()` que permite utilizar la sintaxis de fórmula:

```
variable_cuantitativa ~ variable_categórica_grupos
```

Por ejemplo:

```
var.test(formula = peso ~ sexo, data = datos)
```

```
    F test to compare two variances

data:  peso by sexo
F = 0.67914, num df = 50, denom df = 48, p-value = 0.1781
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.384712 1.194943
sample estimates:
ratio of variances
         0.6791414
```

Comparamos las varianzas de la variable peso entre el grupo de mujeres y hombres. El valor p del test indica que no podemos descartar la igualdad de varianzas entre los grupos (H_0) o lo que es lo mismo el test no encuentra diferencias significativas entre las varianzas de los dos grupos.

11.3.2. Test de Bartlett

Este test se puede utilizar como alternativa al F-test, sobre todo porque nos permite aplicarlo cuando tenemos más de 2 grupos de comparación. Al igual que el anterior es sensible a las desviaciones de la normalidad.

La función en R `base` es `bartlett.test()` y también se pueden usar argumentos tipo fórmula:

```
bartlett.test(formula = peso ~ sexo, data = datos)
```

```
    Bartlett test of homogeneity of variances

data:  peso by sexo
Bartlett's K-squared = 1.8082, df = 1, p-value = 0.1787
```

El resultado es coincidente con el mostrado por `var.test()`. No se encuentran diferencias significativas entres las varianzas de los pesos en los dos grupos (Mujer - Varon)

11.3.3. Test de Levene

El test de Levene sirve para comparar la varianza de 2 o más grupos pero además permite elegir distintos estadísticos de tendencia central. Por lo tanto, la podemos adaptar a distribuciones alejadas de la normalidad seleccionando por ejemplo la mediana.

La función `leveneTest()` se encuentra disponible en el paquete `car`. La vemos aplicada sobre `peso` para los diferentes grupos de `sexo` y utilizando la **media** como estadístico de centralidad, dado que la distribución de `peso` se aproxima a la normal.

```
leveneTest(y = peso ~ sexo, data = datos, center = "mean")
```

```
Levene's Test for Homogeneity of Variance (center = "mean")
      Df F value Pr(>F)
group  1       2 0.1605
      98
```

La conclusión es la misma que la encontrada anteriormente.

Ahora vamos aplicarla sobre la variable `edad`, de la que habíamos descartado "normalidad". Lo hacemos usando el argumento `center` con `"mean"` (media) y con `"median"` (mediana).

```
leveneTest(y = edad ~ sexo, data = datos, center = "mean")
```

```
Levene's Test for Homogeneity of Variance (center = "mean")
      Df F value Pr(>F)
group  1  4.6784  0.033 *
      97
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
leveneTest(y = edad ~ sexo, data = datos, center = "median")
```

```
Levene's Test for Homogeneity of Variance (center = "median")
      Df F value Pr(>F)
group  1  2.4413 0.1214
      97
```

Los resultados son diferentes. Mientras con el centrado en la media nos da un *p* valor significativo menor a 0,05 con el centrado en la mediana no nos permite descartar homocedasticidad.

Observamos aquí las distorsiones sobre la media y las formas paramétricas que devienen de distribuciones asimétricas y alejadas de la curva normal. El código correcto para este caso (variable `edad`) es usar el centrado en la mediana (`center = "median"`).

11.4. Ejemplo práctico en R

La contaminación por plásticos ha alcanzado incluso las zonas más remotas del océano, fragmentándose en partículas microscópicas conocidas como microplásticos (< 5 mm). Debido a su capacidad para filtrar grandes volúmenes de agua, los bivalvos como el mejillón (*Mytilus edulis*) actúan como bioindicadores de la salud ambiental y, al mismo tiempo, como una vía directa de transferencia de contaminantes hacia los seres humanos.

En este ejercicio, analizaremos datos basados en la investigación pionera de Van Cauwenberghe y Janssen (2014). Evaluaremos la carga promedio de microplásticos en muestras destinadas al consumo y estimaremos la magnitud de la proporción de bivalvos que superan los límites de seguridad.

Los valores de concentración de microplásticos (medidos en partículas/g) hallados en estos bivalvos estudiados se encuentran almacenadas en el archivo **bivalvos.txt** y se nos pide realizar los siguientes items:

a. Lea el archivo y explore su contenido
b. Describa estadísticamente las mediciones
c. ¿Estos datos parecen tomados de una población de distribución normal?, ¿por qué?
d. ¿Cuáles son los intervalos de confianza de 90 y 95 % para la concentración media de microplasticos por el método frecuentista tradicional (fórmulas)?
e. Construya la variable **umbral_excedido** tomando el valor de corte hipotético en 0,5 partículas/g (si concentración es menor o igual a 0,5 el umbral_excedido valdrá “No” y si es mayor valdrá “Si”)
f. ¿Cuál es el intervalo de confianza del 95 % de la proporcion de bivalvos con riesgo potencial para el consumo humano?

Veamos como llevamos adelante estos pasos mediante el lenguaje R:

11.4.1. Lectura y exploración inicial

```
# Activamos paquetes necesarios y conocidos
library(tidyverse)
library(dlookr)

# Leemos la tabla de datos
bivalvos <- read_csv2("datos/bivalvos.txt")
```

Visualizamos la estructura de la tabla de datos

```
glimpse(bivalvos)
```

```
Rows: 50
Columns: 2
$ id            <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1~
$ concentracion <dbl> 0.37, 0.42, 0.68, 0.46, 0.47, 0.71, 0.52, 0.26, 0.35, 0.~
```

Tenemos una tabla con 50 observaciones (mediciones en bivalvos) y la variable de interés se llama concentracion (nos informan que está expresada como partículas/g de tejido húmedo)

11.4.2. Describa estadísticamente las mediciones

Con la función `describe()` de **dlookr** podemos obtener los estadísticos de esta variable cuantitativa continua.

```
bivalvos |>
  describe(concentracion)
```

```
# A tibble: 1 x 26
  described_variables     n    na  mean    sd se_mean   IQR skewness kurtosis
  <chr>               <int> <int> <dbl> <dbl>   <dbl> <dbl>    <dbl>    <dbl>
1 concentracion          50     0 0.456 0.138  0.0196 0.188    0.197   -0.299
# i 17 more variables: p00 <dbl>, p01 <dbl>, p05 <dbl>, p10 <dbl>, p20 <dbl>,
#   p25 <dbl>, p30 <dbl>, p40 <dbl>, p50 <dbl>, p60 <dbl>, p70 <dbl>,
#   p75 <dbl>, p80 <dbl>, p90 <dbl>, p95 <dbl>, p99 <dbl>, p100 <dbl>
```

La media (0,4556 partículas/g) y la mediana (0,44 partículas/g) son cercanas. El desvío estandar es de 0,138 partículas/g.

11.4.3. Test de distribución normal

Aplicaremos el test de normalidad de Shapiro-Wilk (forma analítica)

```
shapiro.test(bivalvos$concentracion)
```

```

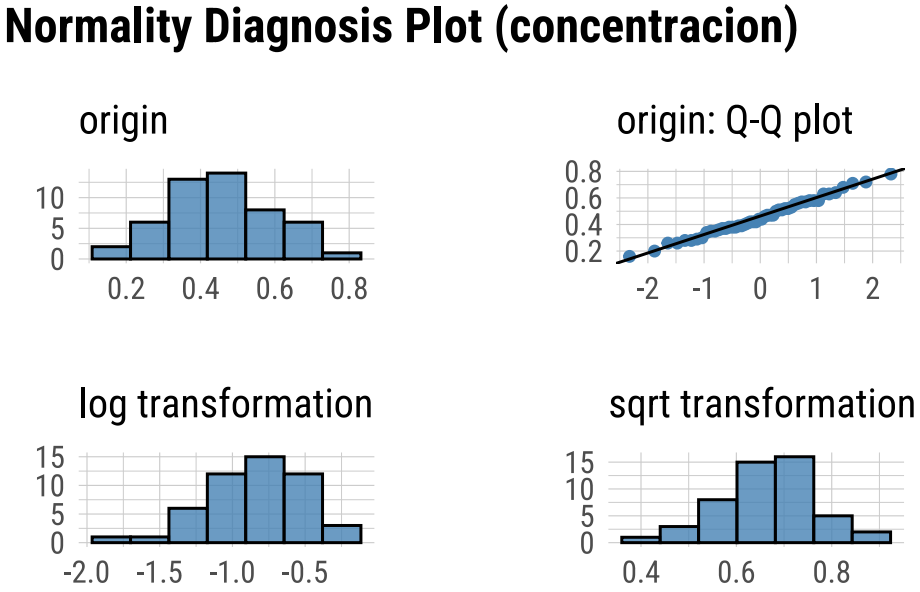
    Shapiro-Wilk normality test

data:  bivalvos$concentracion
W = 0.989, p-value = 0.92
```

Un p-valor mayor a 0,05 habla de un ajuste a la curva normal (es su hipotesis nula de igualdad).

Complementamos visualizandolo graficamente con un Q-QPlot.

```
bivalvos |>
  plot_normality(concentracion)
```



Los puntos se desarrollan “alrededor” a la recta de normalidad.

Con estos dos elementos podemos confirmar que la distribución se aproxima a la “Normal” y por lo tanto es confiable hacer uso de métodos paramétricos.

Si bien con ayuda del Teorema del Límite Central (TCL) sabemos que si el tamaño de la muestra es suficientemente grande, la distribución de la media muestral (o del estadístico) tiende a una distribución normal, independientemente de la forma que tengan los datos originales. En este caso, la definición de “grande” depende de varios factores, pero generalmente se considera que 30 es un buen umbral.

En la práctica, cuando trabajamos con variables continuas:

- Muestras pequeñas ($n < 30$): Si podemos asumir que la población original es aproximadamente normal, utilizamos la distribución t de Student con $n - 1$ grados de libertad. Esta distribución es más ‘ancha’ en las colas, lo que penaliza nuestra incertidumbre por tener pocos datos.

- Muestras grandes ($n \geq 30$): Por el TLC, la distribución del estadístico tiende a la normalidad (z). Sin embargo, dado que la distribución t converge a la z a medida que el tamaño de muestra aumenta, es una práctica estándar (y simplificadora) utilizar la distribución t en toda la escala.Por eso, funciones en R como `t.test()` se usan universalmente: para un n pequeño aplican la corrección necesaria, y para un n grande los resultados son virtualmente idénticos a los de una distribución z.

Es una buena oportunidad para mencionar que el número 30 es una convención histórica (una regla empírica). Actualmente, a veces con $n = 30$ y datos muy sesgados, el TLC aún no ha “actuado” lo suficiente y hay que recurrir a métodos no paramétricos donde obtenemos IC más robustos y honestos que confiar ciegamente en la distribución t“.

11.4.4. Calculo de IC paramétrica

Estamos frente al calculo de IC de una muestra de n = 50 (mayor a 30 observaciones) y que cumple con el supuesto de normalidad, por lo tanto deciamos que podemos aplicar un método paramétrico basado en la distribución t de Student.

Podríamos hacer las operaciones y cálculos individuales utilizando las funciones que R tiene sobre esta distribución (familia TDist: `qt()`, etc), pero la idea de este ejercicio es mostrarles la forma operativa más sencilla de llevarlo a cabo.

Función t.test()

La función `t.test()`, incluída en el paquete stats de R base, sirve apara ejecutar test de hipótesis de medias de una y dos poblaciones (independientes o pareadas) pero además nos calcula automáticamente el intervalo de confianza requerido, basado en la distribución t de Student.

Para este ejemplo debemos utilizar como argumentos:

- la variable de interés que contiene los valores de la muestra
- el nivel de confianza con el que necesitamos el IC

```
t.test(x = bivalvos$concentracion, conf.level = 0.95)
```

```
    One Sample t-test

data:  bivalvos$concentracion
t = 23.296, df = 49, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 0.4162995 0.4949005
sample estimates:
mean of x
   0.4556
```

En la lista de resultados devuelta debemos observar los valores que figuran debajo de “95 percent confidence interval:”. En este caso, el resultado es de una media de 0,4556 IC95 %: 0,4163-0,4949 partículas/g

Si cambiamos el argumento conf.level a 0.90 obtendremos el IC 90 %:

```
t.test(x = bivalvos$concentracion, conf.level = 0.90)
```

```
    One Sample t-test

data:  bivalvos$concentracion
t = 23.296, df = 49, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
90 percent confidence interval:
 0.4228123 0.4883877
sample estimates:
mean of x
   0.4556
```

Este va de 0,4228 a 0,4884 partículas/g, lógicamente más estrecho que el anterior porque redujimos el intervalo de confianza.

La redacción correcta sería *"con un nivel de confianza del 90 %, se estima que el verdadero valor de la media poblacional de la concentración de microplásticos en bivalvos se encuentra entre 0,4228 y 0,4884 partículas/g."*

Técnicamente las funciones estadísticas de R base devuelven generalmente resultados en formato lista y reciben su entrada con formato vector o bien sintaxis fórmula. Ese formato no cumple con ser ordenado como el dataframe. Dado que la filosofía de trabajo del tidyverse (y por lo tanto todos sus paquetes que pertenecen al ecosistema) tienen como premisa recibir y devolver un dataframe, lo que permite usar tambien las características tuberías, Kassambara (2026b) propuso el paquete **rstatix**.

La idea central del paquete es ofrecer funciones que hacen lo mismo que las de R base pero en lugar de devolver una lista de valores como resultado lo haga mediante un dataframe.

Veamos su uso:

```
library(rstatix)

bivalvos |>
  t_test(concentracion ~ 1, conf.level = 0.95, detailed = T) |>
  select(estimate, conf.low, conf.high)
```

```
# A tibble: 1 x 3
  estimate conf.low conf.high
     <dbl>    <dbl>     <dbl>
1    0.456    0.416     0.495
```

11.4.5. Construcción de umbral_excedido

Debemos proceder ahora a construir la variable categórica **umbral_excedido** con valores Si y No en función del valor de corte hipotético de 0,5.

Podemos hacerlo simplemente con el condicional `if_else()`.

```
bivalvos <- bivalvos |>
  mutate(umbral_excedido = if_else(concentracion > 0.5, "Si", "No"))

bivalvos |>
  count(umbral_excedido)
```

```
# A tibble: 2 x 2
  umbral_excedido     n
  <chr>           <int>
1 No                 31
2 Si                 19
```

Observamos que 19 de los 50 bivalvos de nuestra muestra representa un riesgo sanitario para los consumidores.

```
bivalvos |>
  count(umbral_excedido) |>
  mutate(porcentaje = n / sum(n) * 100)
```

```
# A tibble: 2 x 3
  umbral_excedido     n porcentaje
  <chr>           <int>      <dbl>
1 No                 31         62
2 Si                 19         38
```

Eso representa el 38 % de las observaciones que tenemos.

11.4.6. IC 95 % de la proporción

En este punto queremos construir el intervalo de confianza del 95 % que acompañe a la estimación puntual anterior.

Tenemos dos posibilidades que están implementadas en R base: la función `prop.test()` o `binom.test()`.

`prop.test()` utiliza la aproximación de la distribución Normal (puntuación z) a la distribución Binomial. Podríamos decir que es el análogo categórico del `t.test()`. Los elementos necesarios son: **x** número de éxitos y **n** que es tamaño de la muestra.

Se usa con muestras "grandes" ($n > 30$). Pero si en cambio el tamaño de la muestra es pequeño, la proporción es muy cercana a 0 o a 1, o quieres una precisión absoluta y no quieres depender de aproximaciones matemáticas ejecutamos `binom.test()`.

La función aplica la prueba exacta de Clopper-Pearson, donde en **x** definimos cuántos bivalvos superaron el umbral y en **n** el tamaño total de la muestra.

Ejecutamos las dos posibilidades:

```
prop.test(x = 19, n = 50)
```

```
    1-sample proportions test with continuity correction

data:  19 out of 50, null probability 0.5
X-squared = 2.42, df = 1, p-value = 0.1198
alternative hypothesis: true p is not equal to 0.5
95 percent confidence interval:
 0.2499804 0.5283672
sample estimates:
   p
0.38
```

```
binom.test(x = 19, n = 50)
```

```
    Exact binomial test

data:  19 and 50
number of successes = 19, number of trials = 50, p-value = 0.1189
alternative hypothesis: true probability of success is not equal to 0.5
95 percent confidence interval:
 0.2465011 0.5282508
sample estimates:
probability of success
                  0.38
```

Los resultados son muy similares, dado que cumplimos con los supuestos necesarios para usar la aproximación del test de proporciones.

Entonces podemos decir con un nivel de confianza del 95 %, que estimamos que la proporción poblacional de bivalvos que exceden el límite de seguridad sanitaria se encuentra entre 0,25 y 0,53.

Las dos funciones estan representadas en **rstatix**, con `prop_test()` y `binom_test()` respectivamente.

Apredimos con este ejercicio que el lenguaje R siempre ofrece múltiples caminos, pero nuestro criterio es el que decide cuál usar y reportar.

Wickham et al. (2019)

11.5. Referencias

12. Selección de la muestra

12.1. Introducción

Toda investigación epidemiológica requiere, en principio, la selección de una muestra representativa de la población objeto de estudio. Esto implica:

- Determinar el tamaño de la muestra.
- Definir cómo se seleccionarán los sujetos (es decir, las unidades de análisis).
- Calcular la incertidumbre que introduce este proceso de muestreo en nuestras estimaciones.

El tamaño de la muestra se establece en la fase de diseño del estudio, habitualmente cuando se redacta el protocolo de investigación. Es importante considerar que este tamaño es, en cierto sentido, orientador, ya que su cálculo depende de factores que en ocasiones pueden ser subjetivos o estar limitados por la información previa disponible. Además, las exigencias varían según el objetivo del estudio. Por ejemplo:

- En un estudio transversal, donde se estima una prevalencia, se puede aceptar una precisión menor.
- En un estudio experimental, donde se compara la eficacia de un fármaco frente a un placebo, el tamaño de la muestra debe ser suficiente para detectar una "diferencia significativa" entre los grupos.

Cuando el objetivo de la investigación incluye la estimación de un intervalo de confianza (IC) para alguna variable, se busca que dicho IC tenga alta confiabilidad y sea lo más estrecho posible.

En el caso de los estudios de corte transversal, que se orientan a estimar una proporción, se formulan preguntas como:
— ¿Cuál es la prevalencia, proporción o porcentaje del fenómeno en estudio?

Al tomar una muestra para estimar lo que sucede en la realidad, debemos aceptar que la medición obtenida reflejará la situación poblacional, pero siempre con cierto grado de variabilidad (o error). La **precisión** es ese grado de variabilidad aceptable y se expresa a través de los intervalos de confianza, que definen los límites inferior y superior dentro de los cuales se espera encontrar el verdadero valor del parámetro.

12.2. Cálculo del tamaño muestral

Como se mencionó anteriormente, se requiere conocer algunos datos para el cálculo del tamaño de muestra requerido, entre los que destacan:

- **La proporción esperada del evento:** Aunque resulte paradójico, ya que es el parámetro que se desea estimar, el investigador suele contar con información previa que permite delimitar sus posibles valores. Si no fuera el caso, se utiliza el peor escenario, es decir, una proporción del 50 %, ya que este valor maximiza el producto $p(1-p)$ y, por ende, exige el mayor tamaño muestral. De esta forma, muchas veces se recurre a esto para tener al menos una idea del "límite superior" que estaríamos manejando.

- **El nivel de confianza deseado:** Habitualmente se emplea un 95 % (lo que equivale a un α de 0.05). Este valor indica la probabilidad de que el intervalo calculado contenga el parámetro poblacional verdadero. A mayor nivel de confianza (es decir, menor α), mayor es el coeficiente de confiabilidad y, por ende, mayor el tamaño muestral requerido.
- **La precisión (δ) deseada:** Se refiere a la amplitud aceptable del intervalo de confianza. Cuanta más precisión se exija (es decir, un intervalo más estrecho), mayor será el tamaño de la muestra necesario.

La fórmula para el cálculo es la siguiente:

$$N = \frac{p.q.(Z\alpha)^2}{\delta^2}$$

Donde: - N es el tamaño muestra requerido. - p es la proporción esperada de sujetos portadores del evento. - $q = 1–p$ es la proporción complementaria (sujetos que no tienen la variable en estudio). - δ es la precisión o magnitud de error aceptable. - $Z\alpha$ es el valor crítico de la distribución normal asociado al nivel de significancia. Se obtiene de tablas de distribución normal de probabilidades y habitualmente se utiliza un valor α de 0.05, al que le corresponde un valor Z de 1.96

Cuando se conoce el tamaño de la población, es necesario aplicar una corrección por población finita:

$$N = \frac{n^1}{1 + \frac{n^1}{Poblacin}}$$

donde n^1 es el N calculado con la ecuación anterior.

Es fundamental entender que el tamaño de la muestra dependerá de p de α y de la precisión deseada, más que de la complejidad de la fórmula en sí. Existen numerosos softwares que calculan tamaños de muestra. Algunos están especializados en esta tarea (por ejemplo, **EpiDat**, **EpiInfo**, **OpenEp**i), mientras que otros lo incluyen como una opción dentro de un paquete más amplio de análisis. En el lenguaje R, funciones como `power.prop.test()` (para comparación de proporciones) y `power.t.test()` (para pruebas de medias basadas en la distribución t de Student) están incluidas en el paquete `stats`. Otros paquetes útiles incluyen `samplingbook` (Manitz et al. 2021), `samplesize`, `pwr` (Champely 2020) y `epiR` (Stevenson y Sergeant 2026).

A continuación, se presenta un ejemplo práctico utilizando la función `sample.size.prop()` del paquete `samplingbook`. Supongamos que queremos estimar la prevalencia de diabetes en una ciudad de 600,000 habitantes, donde estudios previos sugieren una prevalencia cercana al 10.2 %. El objetivo es determinar el tamaño de la muestra en un muestreo aleatorio simple.

Comenzamos cargando el paquete requerido:

```
library(samplingbook)
```

Calculamos el tamaño muestral en base a los siguientes argumentos:

- `e`: número positivo que especifica la precisión que es la mitad del ancho del intervalo de confianza
- `p`: proporción esperada de eventos con dominio entre los valores 0 y 1.
- `N`: número entero positivo para el tamaño de la población. El valor predeterminado es `Inf`, lo que significa que los cálculos se realizan sin *corrección de población finita*.
- `level`: nivel de confianza para los intervalos de confianza. El valor predeterminado es 95 %.

```
sample.size.prop(e = 0.02, P = 0.102, N = 600000, level = 0.95)
```

```
sample.size.prop object: Sample size for proportion estimate
With finite population correction: N=6e+05, precision e=0.02 and expected proportion P=0.102

Sample size needed: 879
```

El resultado, tras aplicar la corrección por población finita, indica que se requieren 879 sujetos en la muestra. Por simplicidad, es habitual considerar infinita a la población objetivo, ya que hacerlo así minimiza los riesgos estadísticos:

```
sample.size.prop(e = 0.02, P = 0.102, N = Inf, level = 0.95)
```

```
sample.size.prop object: Sample size for proportion estimate
Without finite population correction: N=Inf, precision e=0.02 and expected proportion P=0.102

Sample size needed: 880
```

Si se considera una población infinita, el tamaño muestral calculado es de 880 personas, apenas una unidad más, lo que demuestra que el ajuste por población finita en este caso es mínimo. En cambio, si no se conoce o no se confía en el valor de prevalencia y se opta por el peor escenario ($p = 0.5$):

```
sample.size.prop(e = 0.02, P = 0.5, N = 600000, level = 0.95)
```

```
sample.size.prop object: Sample size for proportion estimate
With finite population correction: N=6e+05, precision e=0.02 and expected proportion P=0.5

Sample size needed: 2392
```

En este caso, el tamaño muestral aumenta considerablemente hasta 2392 unidades, lo que refleja un enfoque más conservador.

Además del cálculo del tamaño muestral, es esencial planificar la forma en que se seleccionarán los individuos. En estudios de corte transversal se debe realizar un muestreo probabilístico, ya que la técnica de muestreo influirá en los factores de expansión necesarios para generalizar los resultados a la población. Aunque el proceso de muestreo no se detalla en este curso, es vital considerarlo para garantizar la validez externa de la investigación.

12.3. Conceptos generales sobre muestreo

Un muestreo se considera **probabilístico** cuando se cumplen dos condiciones fundamentales:

- Es posible conocer de antemano la probabilidad de selección que tiene cada elemento de la población.
- Esta probabilidad es mayor que cero para todos los elementos.

Solo con un muestreo probabilístico se puede medir el grado de precisión de las estimaciones realizadas, ya que se conocen las probabilidades de inclusión. Algunos de los métodos más usados son: Muestreo aleatorio simple, muestreo sistemático en fases, Muestreo aleatorio estratificado, muestreo por conglomerados, etc.

12.3.1. Muestreo aleatorio simple (MAS)

En el MAS, cada elemento de la población tiene la misma probabilidad de ser seleccionado (es decir, es un método **equiprobabilístico**). La probabilidad de inclusión se expresa como:

$$P = \frac{n}{N}$$

donde:

- n es el tamaño de la muestra,
- N es el tamaño de la población.

Este método requiere contar con un listado completo de la población y se utiliza principalmente en poblaciones cerradas (por ejemplo, escuelas, cárceles, hospitales o muestras de laboratorio), donde los elementos pueden identificarse y numerarse fácilmente.

12.3.2. Muestreo sistemático (MS)

El muestreo sistemático es un procedimiento alternativo al MAS, adecuado para la selección de muestras equiprobabilísticas de poblaciones organizadas según algún orden conocido. Sus ventajas incluyen no requerir necesariamente un listado completo de todas las unidades, lo que resulta útil, por ejemplo, para seleccionar a los usuarios que llegan a un servicio hospitalario durante un período determinado.

El procedimiento consiste en:

1. Asignar a cada elemento de la población un número del 1 a N.
2. Calcular el intervalo de selección:

$$k = \frac{N}{n}$$

3. Seleccionar aleatoriamente un número r entre 1 y k.
4. Elegir los elementos que ocupan las posiciones: $r, r+k, r+2k, ...$, es decir, la secuencia $r, r+jk$ con $j = 0, 1, 2, ...$

La probabilidad de ser seleccionado es igual que en el MAS

$$P = \frac{n}{N}$$

12.3.3. Muestreo aleatorio estratificado (MAE)

Este método se emplea cuando la población puede dividirse en subgrupos o **estratos** que difieren en las características de interés (por ejemplo, edad, sexo o nivel socioeconómico). La idea es asegurar que cada estrato esté representado en la muestra. El proceso habitual consiste en:

- Elaborar listas separadas para cada estrato.
- Seleccionar una submuestra de cada uno de ellos.

La probabilidad de selección en cada estrato es:

$$P\,estrato = \frac{n\,estrato}{N\,estrato}$$

Si se aplica un muestreo proporcional, el tamaño de la muestra en cada estrato se calcula como:

$$n\,estrato = n\frac{N\,estrato}{N}$$

El objetivo es obtener una muestra cuya variabilidad interna se asemeje a la de la población, logrando que los estratos sean internamente homogéneos y heterogéneos entre sí.

12.3.4. Muestreo por conglomerados monoetápico

El muestreo por conglomerados es ideal cuando la población es difícil de delimitar o está muy dispersa. Se divide la población en **conglomerados** (o grupos) que actúan como unidades de primera etapa (UPE) y no deben solaparse.

La población se divide en grupos (por ejemplo, áreas geográficas, instituciones o servicios). Se selecciona una muestra de n conglomerados y se incluyen **todos** las unidades de análisis de los conglomerados seleccionados. El tamaño del conglomerado se refiere al número de unidades elementales que contiene, pudiendo ser de igual o distinto tamaño. Los conglomerados deberían ser lo más heterogéneos dentro de ellos y lo más homogéneos entre ellos.

12.3.5. Muestreo por conglomerados bietápico

Se utiliza cuando existe gran variabilidad en el tamaño de los conglomerados o la población es muy grande. En este caso, después de dividir la población en conglomerados y seleccionar algunos de ellos como UPE, se realiza una subselección (muestreo secundario) de los elementos dentro de los conglomerados elegidos.

Existen más posibilidades, que no abordaremos aquí, pero sí nos detendremos en discutir qué implicaciones tiene el muestreo en el proceso de estimación.

	Población	Muestra
Tamaño total	966	120
Número de conglomerados	21	10

12.3.6. Implicaciones en la Estimación y Ponderación

En métodos equiprobabilísticos (como el MAS o el MS), cada elemento tiene la misma probabilidad de selección. Sin embargo, en diseños muestrales más complejos (como el muestreo estratificado o por conglomerados), las probabilidades de selección pueden variar entre individuos.

En estos casos, la probabilidad total de selección de un individuo es el producto de las probabilidades de selección en cada etapa del muestreo. Una vez conocida esta probabilidad, se calcula el peso (o **factor de expansión**) como su inverso. Este peso se interpreta como el número de individuos en la población que representa cada individuo de la muestra.

> El factor de expansión se interpreta como *la cantidad de personas en la población, que representa una persona en la muestra*.

El factor de expansión se utiliza para extrapolar los resultados obtenidos en la muestra a la población total. Por ejemplo, para estimar un total poblacional para una variable, se pondera el valor medido en cada individuo por su factor de expansión y se suman dichos valores.

12.4. Ejemplo práctico

Supongamos que queremos estimar la prevalencia de sobrepeso/obesidad en estudiantes de enseñanza media de una localidad. Se tiene la siguiente información:

- **Población total de alumnos:** 966
- **Número de escuelas (conglomerados):** 21
- **Tamaño de la muestra global (ya calculado):** 120 alumnos

Si se realizara un **muestreo aleatorio simple (MAS)**, se necesitaría disponer del listado completo de los 966 alumnos y, mediante un sorteo, seleccionar 120 individuos. En ese caso, la probabilidad de selección para cada alumno sería:

$$P = \frac{n}{N} = \frac{120}{966} = 0,124$$

Supongamos ahora que decide realizar un muestreo por conglomerados biétapico. Cada escuela se considera un conglomerado; se seleccionan 10 de las 21 escuelas por MAS y, dentro de cada escuela seleccionada, se elige una submuestra de estudiantes también mediante MAS. Se decide que el número de estudiantes elegidos en cada conglomerado sea igual para todos.

Los datos globales quedan de la siguiente forma:

La decisión de cuántos conglomerados forman parte de la muestra, es una decisión del investigador: a veces puede ser necesario fijar un número, a veces puede ser más conveniente fijar el tamaño de la muestra en cada conglomerado. Por otra parte, puede que decidamos que el número de sujetos seleccionados en cada conglomerado sea igual para todos; que sea una fracción fija o algo a definir.

Conglomerado	Tamaño	Muestra
5.1	48	12
1.3	28	12
2.3	64	12
2.2	43	12
3.1	37	12
5.3	86	12
1.4	37	12
3.3	56	12
2.4	48	12
1.5	33	12
Total	480	120

Dentro de los conglomerados seleccionados, se obtiene la siguiente distribución (en este ejemplo, la suma de los tamaños de los conglomerados seleccionados es 480 y en cada uno se eligen 12 alumnos):

12.4.1. Cálculo de las Probabilidades de Selección y Ponderaciones

Para que un individuo sea seleccionado en este diseño, debe ocurrir primero que su conglomerado sea escogido y, posteriormente, que él sea seleccionado dentro de ese conglomerado. Si asumimos que:

> *"Dos sucesos son independientes si la probabilidad de que ocurran ambos simultáneamente es igual al producto de las probabilidades de que ocurra cada uno de ellos"*.

La probabilidad total de selección es el producto de:

1. La probabilidad de que el conglomerado sea seleccionado:

$$P_{conglomerado} = \frac{10}{21}$$

2. La probabilidad de que un individuo sea seleccionado dentro de su conglomerado:

$$P_{dentro} = \frac{N° \, alumnos \, en \, la \, muestra \, del \, conglomerado}{Tamao \, del \, conglomerado}$$

Para los individuos del conglomerado 5.1; la probabilidad de selección será:

$$P_{5.1} = \frac{10}{21} * \frac{12}{48} \approx 0,1190 \quad (11.9\,\%)$$

La ponderación o **factor de expansión** se obtiene como el inverso de la probabilidad de selección:

$$w = \frac{1}{P}$$

Es decir:

Conglomerado	Probabilidad	Ponderación
5.1	11.90	8.40
1.3	20.41	4.90
2.3	8.93	11.20
2.2	13.29	7.53
3.1	15.44	6.48
5.3	6.65	15.05
1.4	15.44	6.48
3.3	10.20	9.80
2.4	11.90	8.40
1.5	17.32	5.78

$$w_{5.1} = \frac{1}{1.190} \approx 8.4$$

Esto significa que cada alumno seleccionado del conglomerado 5.1 representa aproximadamente a 8 alumnos de la población.

Si se realizan los cálculos para cada uno de los conglomerados, se obtiene la siguiente tabla:

Cada individuo seleccionado en el estudio tiene una probabilidad de inclusión que depende tanto de la probabilidad de que su escuela (conglomerado) sea seleccionada como de la probabilidad de que él sea elegido dentro de su escuela. La ponderación resultante (el inverso de esta probabilidad) se utiliza para "expandir" el resultado del estudio, de manera que cada alumno en la muestra represente a un número determinado de alumnos en la población total.

Por ejemplo, en el conglomerado 5.1, cada alumno seleccionado representa aproximadamente a 8 alumnos de la población. De este modo, al calcular la prevalencia de sobrepeso/obesidad en la muestra, se aplican estos factores de expansión para obtener estimaciones que sean representativas de los 966 alumnos de la localidad.

Este proceso es fundamental en estudios basados en muestreos complejos, ya que permite corregir posibles diferencias en las probabilidades de selección y garantizar que las estimaciones sean válidas y precisas para toda la población.

Hernández-Ávila (2011)

Daniel (2002)

Rodríguez y Mendivelso (2018)

Ríus Díaz et al. (2012)

Triola (2018)

Norman, GR y Streiner, DL (1996)

Rodríguez et al. (2003)

EPIDAT 4.2 - Consellería de Sanidade - Servizo Galego de Saúde (s. f.)

Elm et al. (2008)

Schulz et al. (2010)

Álvarez Cáceres (2007)

Glantz, S (2006)

Field et al. (2014)

12.5. Referencias

13. Funciones condicionales en R

13.1. Introducción

Continuamos incorporando funciones útiles para el manejo de datos, enfocándonos en la creación de nuevas variables y en la transformación de variables cuantitativas a variables categóricas, ya sean nominales u ordinales.

Algunas de estas operaciones serán necesarias para alcanzar los objetivos del trabajo práctico grupal de esta unidad, y seguramente también se utilizarán en las unidades siguientes.

Como vimos dentro del universo `tidyverse`, para crear nuevas variables a partir de cálculos utilizamos la función-verbo `mutate()`:

```
datos <- datos |>
  mutate(variable_nueva = funcion(var1))
```

Frecuentemente necesitaremos agrupar, resumir o **discretizar variables continuas**, dividiendo su rango en categorías que pueden ser **dicotómicas** o **politómicas**, según los objetivos del análisis.

13.2. Condicional simple: `if_else()`

Para obtener salidas dicotómicas, podemos usar la función condicional `if_else()` del paquete `dplyr`. Esta función es una versión simplificada y segura del clásico condicional `if` que existe en la mayoría de los lenguajes de programación, así como en hojas de cálculo como *Microsoft Excel®* y *Google Sheets®*, entre otras.

Supongamos que tenemos un dataframe llamado `datos`, con una variable cuantitativa denominada `var1`:

```
# Cargar tidyverse
library(tidyverse)

# Datos ejemplo
set.seed(123)

datos <- tibble(
  var1 = sample(1:90, size = 50, replace = TRUE)
)
```

Queremos crear una variable cualitativa dentro de `datos`, llamada `variable_nueva`, cuyos valores estén dados por el cumplimiento de alguna condición de `var1`. Por ejemplo, si los valores de `var1` son mayores a 10, entonces `variable_nueva`, tomará el valor `"mayor a 10"`, en caso contrario, tomará el valor `"menor o igual a 10"`:

```
# Crear variable_nueva
datos <- datos |>
  mutate(
    variable_nueva = if_else(
      condition = var1 > 10,
      true = "mayor a 10",
      false = "menor o igual a 10"
    )
  )

# Explorar variable_nueva
tabyl(datos, variable_nueva)
```

```
     variable_nueva  n percent
         mayor a 10 44    0.88
 menor o igual a 10  6    0.12
```

La función `if_else()` tiene tres argumentos obligatorios:

- `condition`: la condición a evaluar.
- `true`: valor que tomará la nueva variable si se cumple la condición.
- `false`: valor que tomará la nueva variable si no se cumple la condición.

Este proceso se conoce como **dicotomización** de una variable, ya que el resultado posible consta de solo dos categorías.

Los valores de salida pueden ser de distintos tipos (carácter, numérico o lógico). Sin embargo, cuando discretizamos una variable cuantitativa, habitualmente generamos una variable cualitativa de tipo **ordinal**, con categorías expresadas como texto (tipo `character`).

Ahora bien, al ser ordinal estas categorías de la `variable_nueva` deben "ordenarse" en la forma de los valores de la variable, pero el lenguaje R no sabe con que estamos trabajando y respeta siempre el ordenamiento alfanumérico. Por lo tanto, en este ejemplo las categorías se van a estar ordenando al reves del orden numérico natural (de menor a mayor).

La categoría `"mayor a 10"` se ordena alfabéticamente antes de `"menor o igual a 10"`, porque luego del empate de las letras `m`, le siguen la `a` en el primer caso y la `e` en el segundo.

Para ordenar estas categorías debemos transformar la variable de `character` a `factor`. Esto se puede hacer en un solo paso dentro del `mutate()`:

```
datos <- datos |>
  mutate(
    variable_nueva = if_else(
      condition = var1 > 10,
      true = "mayor a 10",
      false = "menor o igual a 10"
    ),

    variable_nueva = factor(
      variable_nueva,
```

```
      levels = c("menor o igual a 10", "mayor a 10")
    )
  )

# Explorar variable_nueva
tabyl(datos, variable_nueva)
```

```
     variable_nueva  n percent
 menor o igual a 10  6    0.12
         mayor a 10 44    0.88
```

Otra forma más artesanal, igualmente válido, es "forzar" el ordenamiento con las categorías así:

```
datos <- datos |>
  mutate(
    variable_nueva = if_else(
      condition = var1 > 10,
      true = "2.mayor a 10",
      false = "1.menor o igual a 10"
    )
  )

# Explorar variable_nueva
tabyl(datos, variable_nueva)
```

```
       variable_nueva  n percent
 1.menor o igual a 10  6    0.12
         2.mayor a 10 44    0.88
```

Aquí agregamos números iniciales a las etiquetas de las categorías para darle el orden que deseamos, sin necesidad de convertir a factor.

13.2.1. Función `cut_interval()`

El ecosistema `tidyverse` ofrece la función `cut_interval()` para la creación de intervalos regulares.

Es una adaptación de la función `cut()` de R base para *tidy data* y sus argumentos son similares. Volviendo al ejemplo anterior:

```
datos <- datos |>
  mutate(
    grupo_var = cut_interval(
      x = var1,
      length = 10,
      right = TRUE,
      ordered_result = TRUE
    )
  )
```

```
# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var n percent
    [0,10] 6    0.12
   (10,20] 4    0.08
   (20,30] 7    0.14
   (30,40] 6    0.12
   (40,50] 6    0.12
   (50,60] 5    0.10
   (60,70] 4    0.08
   (70,80] 7    0.14
   (80,90] 5    0.10
```

Los argumentos función son:

- `x`: conjunto de datos numéricos de entrada (obligatorio).
- `length`: longitud de cada intervalo regular (obligatorio).
- `n`: cantidad de intervalos a crear (obligatorio, omitir si se definió `length`).
- `right`: indica si los intervalos son cerrados a la derecha o viceversa (valor por defecto `TRUE`, intervalos cerrados a la derecha).
- `labels`: etiquetas de los intervalos automáticas o numéricas (por defecto intervalos matemáticos).
- `ordered_result`: determina si el resultado es un factor ordenado. Por defecto vale `FALSE` (la salida es tipo carácter).

No es necesario definir los argumentos opcionales siempre y cuando los valores por defecto sirvan para nuestra tarea.

13.3. Función `case_when()`

Cuando las condiciones no son simples, es decir, el resultado no es dicotómico y además los intervalos son irregulares, utilizamos la función `case_when()` que es una vectorización de la función `if_else()`.

Supongamos que queremos agrupar la variable cuantitativa de números enteros (`var1`) en tres grupos irregulares:

```
datos <- datos |>
  mutate(
    grupo_var = case_when(
      var1 %in% c(0:24) ~ "Grupo 1",
      var1 %in% c(25:64) ~ "Grupo 2",
      var1 >= 65 ~ "Grupo 3"
    )
  )
```

```
# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var  n percent
   Grupo 1 12    0.24
   Grupo 2 23    0.46
   Grupo 3 15    0.30
```

Existe una condición por cada grupo creado, como si fuese un `if_else()` donde el valor declarado siempre es el verdadero. Se utilizan operadores de comparación como mayor (>), menor (<) y/o igual (==) y conectores lógicos como & (AND), | (NOT) e `%in%`. En cada línea va una virgulilla similar a la usada en la sintaxis formula (~) y luego la etiqueta que tomarán las observaciones que cumplan con esa condición en la nueva variable (`grupo_var`).

Esta evaluación es secuencial y su funcionamiento provoca que el usuario del lenguaje tenga el control de lo que esta sucediendo, por lo que cualquier mala definición de las condiciones puede provocar resultados incorrectos.

Si incorporamos el argumento `.default` podemos indicar que valor toma si no se cumple ninguna de las condiciones anteriores.

Por ejemplo, podríamos tener algún valor perdido (`NA`) en `var1` y queremos que la variable `grupo_var` etiquete esos valores perdidos como `"Sin dato"`.

```
datos <- datos |>
  # Generar valores NA
  mutate(var1 = na_if(var1, 50)) |>

  mutate(
    grupo_var = case_when(
      var1 %in% c(0:24) ~ "Grupo 1",
      var1 %in% c(25:64) ~ "Grupo 2",
      var1 >= 65 ~ "Grupo 3",
      .default = "Sin dato"
    )
  )

# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var  n percent
   Grupo 1 12    0.24
   Grupo 2 22    0.44
   Grupo 3 15    0.30
  Sin dato  1    0.02
```

Otra forma de definir los valores que no cumplen con ninguna condición es usando `TRUE ~ valor`:

```
datos <- datos |>
  # Generar valores NA
  mutate(var1 = na_if(var1, 50)) |>

  mutate(
    grupo_var = case_when(
      var1 %in% c(0:24) ~ "Grupo 1",
      var1 %in% c(25:64) ~ "Grupo 2",
      var1 >= 65 ~ "Grupo 3",
      TRUE ~ "Sin dato"
    )
  )

# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var  n percent
   Grupo 1 12    0.24
   Grupo 2 22    0.44
   Grupo 3 15    0.30
  Sin dato  1    0.02
```

Las salidas son de tipo ***carácter*** (`chr`) y debemos manejar el ordenamiento de las etiquetas como vimos anteriormente, por medio de factores o comenzando con caracteres ordenados alfabéticamente.

Para simplificar el trabajo de estos intervalos de clase irregulares y no provocar errores en la confección de las condiciones, `tidyverse` tiene a la función `between()`.

13.3.1. Intervalos: función `between()`

La función `between()` básicamente opera como un atajo para condiciones de intervalos. Define dentro de los argumentos los límites inferior y superior de un intervalo y se utiliza dentro de una función de condición tipo `if_else()` o `case_when()`.

Aplicado sobre el ejemplo anterior se vería así:

```
datos <- datos |>
  mutate(
    grupo_var = case_when(
      between(var1, 0, 24) ~ "Grupo 1",
      between(var1, 25, 64) ~ "Grupo 2",
      between(var1, 65, Inf) ~ "Grupo 3",
      .default = "Sin dato"
    )
  )

# Explora grupo_var
tabyl(datos, grupo_var)
```

```
 grupo_var  n percent
   Grupo 1 12    0.24
   Grupo 2 22    0.44
   Grupo 3 15    0.30
  Sin dato  1    0.02
```

Los valores declarados como límites quedan incluidos siempre dentro del intervalo (son cerrados ambos). También podemos utilizar valores reservados como `Inf` o `-Inf` cuando desconocemos con que valor máximo o mínimo nos vamos a encontrar en la variable cuantitativa original.

13.3.2. Intervalos: función `age_categories()`

El paquete `epikit` (Spina et al. 2025), contiene herramientas útiles para trabajar con datos en salud públicas. La función `age_categories()`, transforma una variable numérica discreta en grupos de edad. Volviendo al ejemplo anterior, supongamos que `var1` contiene edades de pacientes:

```
# Cargar paquete
library(epikit)

datos <- datos |>
  mutate(
    edad_cat = age_categories(
      x = var1,
      lower = 1,
      upper = 80,
      by = 10,
      separator = " a ",
      above.char = " y más"
    )
  )

# Explora edad_cat
tabyl(datos, edad_cat)
```

```
 edad_cat n percent valid_percent
   1 a 10 6    0.12    0.12244898
  11 a 20 4    0.08    0.08163265
  21 a 30 7    0.14    0.14285714
  31 a 40 6    0.12    0.12244898
  41 a 50 5    0.10    0.10204082
  51 a 60 5    0.10    0.10204082
  61 a 70 4    0.08    0.08163265
  71 a 79 7    0.14    0.14285714
 80 y más 5    0.10    0.10204082
     <NA> 1    0.02            NA
```

Los argumentos principales de la función son:

- `x`: vector numérico (obligatorio).

- `lower`: límite inferior de edades (por defecto `0`).
- `upper`: límite superior de edades.
- `by`: intervalo de años para los grupos.
- `breakers`: alternativo a `by`, para definir manualmente los grupos.
- `separator`: carácter que separa el rango de edades (por defecto `"-"`).
- `ceiling`: define si incluir el valor más alto en los grupos (por defecto `FALSE`).
- `above.char`: el valor que queremos que aparezca para las edades por fuera del grupo de edad más alto (por defecto +). Solo funciona si `ceiling = FALSE`.

13.4. Ejemplo práctico en R

Tomemos un caso clásico como la variable edad medida en años, variable que generalmente tenemos en toda tabla de datos vinculada a personas. Trabajaremos con la base de datos "`edad.txt`" que contiene 106 observaciones de pacientes:

```
# Cargar dataset
datos <- read_csv2("datos/edad.txt")

# Explorar datos
glimpse(datos)
```

```
Rows: 106
Columns: 3
$ id_mujer         <dbl> 115971, 1689710, 114710, 3058900, 3223547, 3234027, 3~
$ fecha_nacimiento <chr> "30/05/1967", "23/04/1991", "06/11/1962", "09/03/1964~
$ fecha_test       <chr> "17/01/2022", "01/02/2022", "16/03/2022", "23/02/2022~
```

A continuación, vamos a transformar las variables cuyo nombre contenga "`fecha_`" a formato fecha para poder calcular edades:

```
datos <- datos |>
  # Transformar a formato fecha
  mutate(across(.cols = starts_with("fecha_"), .fns = ~ dmy(.x))) |>
  # Calcular edad al momento del testeo
  mutate(
    edad = round(
      as.duration(fecha_test - fecha_nacimiento) / dyears(1)
    )
  )
```

Una primera posibilidad es dicotomizar la edad usando el valor de la mediana:

```
datos |>
  summarise(mediana = median(edad))
```

```
# A tibble: 1 x 1
  mediana
    <dbl>
1      56
```

Aplicando el valor de la mediana dentro de un `if_else()` podríamos hacer:

```
datos <- datos |>
  mutate(
    grupo_edad1 = if_else(
      condition = edad > 56,
      true = "mayor a la mediana",
      false = "menor o igual a la mediana"
    )
  )

# Explorar grupo_edad1
datos |>
  count(grupo_edad1)
```

```
# A tibble: 2 x 2
  grupo_edad1                    n
  <chr>                      <int>
1 mayor a la mediana            52
2 menor o igual a la mediana    54
```

Observamos en el conteo que `grupo_edad1` se construyó adecuadamente pero el orden de los niveles no es correcto si queremos que siga el ordenamiento natural de edad (de menor a mayor).

Una de las formas que vimos es convertir a factor:

```
datos <- datos |>
  mutate(
    grupo_edad1 = if_else(
      condition = edad > 56,
      true = "mayor a la mediana",
      false = "menor o igual a la mediana"
    ),
    grupo_edad1 = factor(
      grupo_edad1,
      levels = c("menor o igual a la mediana", "mayor a la mediana")
    )
  )

# Explorar grupo_edad1
datos |>
  count(grupo_edad1)
```

```
# A tibble: 2 x 2
  grupo_edad1                    n
  <fct>                      <int>
1 menor o igual a la mediana    54
2 mayor a la mediana            52
```

Vemos que en el conteo el formato de la variable ya no es `chr` sino `fct` y el orden de las etiquetas siguen la forma *"menor a mayor"*.

Otra forma es:

```
datos <- datos |>
  mutate(
    grupo_edad1 = if_else(
      condition = edad > 56,
      true = "2.mayor a la mediana",
      false = "1.menor o igual a la mediana"
    )
  )

# Explorar grupo_edad1
datos |>
  count(grupo_edad1)
```

```
# A tibble: 2 x 2
  grupo_edad1                      n
  <chr>                        <int>
1 1.menor o igual a la mediana    54
2 2.mayor a la mediana            52
```

Si en cambio necesitamos que los grupos sean mas de dos y que estos intervalos de clase sean regulares, podemos usar `cut_interval()`:

```
datos <- datos |>
  mutate(grupo_edad2 = cut_interval(x = edad, length = 10))

# Explorar grupo_edad2
datos |>
  count(grupo_edad2)
```

```
# A tibble: 8 x 2
  grupo_edad2     n
  <fct>       <int>
1 [0,10]          3
2 (10,20]         3
3 (20,30]         2
4 (30,40]         3
5 (40,50]        13
6 (50,60]        52
7 (60,70]        27
8 (70,80]         3
```

La salida muestra ocho grupos etarios con etiquetas ordenadas con notación matemática, donde un corchete indica que el límite del intervalo es cerrado, es decir contiene el valor y un paréntesis es abierto y no lo hace.Así es que el primer grupo va de 0 a 10 años y el segundo de 11 a 20.

Estos sucede así porque en forma predeterminada el argumento `right = TRUE`. Veamos que pasa si lo cambiamos a `FALSE`:

```
datos <- datos |>
  mutate(grupo_edad2 = cut_interval(x = edad, length = 10, right = F))

# Explorar grupo_edad2
datos |>
  count(grupo_edad2)
```

```
# A tibble: 8 x 2
  grupo_edad2     n
  <fct>       <int>
1 [0,10)          3
2 [10,20)         3
3 [20,30)         2
4 [30,40)         3
5 [40,50)        10
6 [50,60)        48
7 [60,70)        32
8 [70,80]         5
```

En esta salida el primer grupo va de 0 a 9 y el segundo de 10 a 19.

Hasta ahora la variable `grupo_edad2` es de tipo carácter, pero si deseamos que la salida sea factor podemos incorporar el argumento `ordered_result = TRUE`:

```
datos <- datos |>
  mutate(grupo_edad2 = cut_interval(x = edad, length = 10, ordered_result = T))

# Explorar grupo_edad2
datos |>
  count(grupo_edad2)
```

```
# A tibble: 8 x 2
  grupo_edad2     n
  <ord>       <int>
1 [0,10]          3
2 (10,20]         3
3 (20,30]         2
4 (30,40]         3
5 (40,50]        13
6 (50,60]        52
7 (60,70]        27
8 (70,80]         3
```

Construimos así una variable factor ordenada `<ord>`.

Por último, con el argumento `labels= FALSE` hacemos que las etiquetas de los ocho grupos sean numéricas:

```
datos <- datos |>
  mutate(grupo_edad2 = cut_interval(x = edad, length = 10, labels = F))

# Explorar grupo_edad2
datos |>
  count(grupo_edad2)
```

```
# A tibble: 8 x 2
  grupo_edad2     n
        <int> <int>
1           1     3
2           2     3
3           3     2
4           4     3
5           5    13
6           6    52
7           7    27
8           8     3
```

Otro ejemplo, podría ser aplicando `case_when()` donde discretizamos la edad en cuatro grupos irregulares, forzando sus etiquetas para lograr el orden adecuado:

```
datos <- datos |>
  mutate(
    grupo_edad3 = case_when(
      edad < 13 ~ "1.Niño",
      edad > 12 & edad < 26 ~ "2.Adolescente",
      edad > 25 & edad < 65 ~ "3.Adulto_joven",
      edad > 64 ~ "4.Adulto_mayor"
    )
  )

# Explorar grupo_edad3
datos |>
  count(grupo_edad3)
```

```
# A tibble: 4 x 2
  grupo_edad3        n
  <chr>          <int>
1 1.Niño             3
2 2.Adolescente      5
3 3.Adulto_joven    86
4 4.Adulto_mayor    12
```

Si no hubiésemos etiquetado con los números por delante el orden alfabético hacía que Niño fuese a parar al final del conteo.

De la misma forma pero más sencillo y controlado es:

```
datos <- datos |>
  mutate(
    grupo_edad3 = case_when(
      between(edad, 0, 12) ~ "1.Niño",
      between(edad, 13, 25) ~ "2.Adolescente",
      between(edad, 26, 64) ~ "3.Adulto_joven",
      between(edad, 65, Inf) ~ "4.Adulto_mayor"
    )
  )

# Explorar grupo_edad3
datos |>
  count(grupo_edad3)
```

```
# A tibble: 4 x 2
  grupo_edad3        n
  <chr>          <int>
1 1.Niño             3
2 2.Adolescente      5
3 3.Adulto_joven    86
4 4.Adulto_mayor    12
```

> ⚠ Estas funciones condicionales que tratamos en este documento no se limitan a la tarea de construir agrupamientos de variables cuantitativas sino que sirven para cualquier situación donde a partir de una o más condiciones se produzcan una o más valores como respuesta.

Finalmente veamos como quedarían los grupos de edad con la función `age_categories()`:

```
datos <- datos |>
  mutate(
    grupo_edad4 = age_categories(
      x = edad,
      breakers = c(0, 12, 25, 65),
      above.char = " y más"
    )
  )

# Explorar grupo_edad4
datos |>
  count(grupo_edad4)
```

```
# A tibble: 4 x 2
  grupo_edad4     n
  <fct>       <int>
```

```
1 0-11             3
2 12-24            4
3 25-64           87
4 65 y más        12
```

En este caso se genera una variable de tipo factor con los grupos de edad ordenados de mayor a menor.

Referencias

14. Intervalos de confianza en muestras complejas

14.1. Introducción

En los estudios epidemiológicos de corte transversal, es común el uso de encuestas poblacionales para estimar la frecuencia de determinadas condiciones de salud o la presencia de factores de riesgo en la población.

Sin embargo, la implementación de estas encuestas enfrenta restricciones prácticas que hacen inviable o poco conveniente el muestreo aleatorio simple (MAS). En su lugar, se recurre a otras alternativas de muestreo, como la estratificación, la selección en etapas, la formación de conglomerados o el empleo de probabilidades de selección desiguales.

Los diseños muestrales que incorporan combinaciones de estas estrategias se denominan **muestreos complejos**, por contraste con el MAS, en el que las unidades muestrales se seleccionan independientemente unas de otras y todas tienen igual probabilidad de selección y distribución.

El análisis de los datos obtenidos mediante diseños muestrales complejos presenta desafíos adicionales debido a la posible correlación entre observaciones dentro de un mismo conglomerado. Esta correlación impide asumir independencia de las observaciones, condición necesaria para sostener el supuesto de normalidad. Si no se considera el efecto del diseño muestral al realizar estimaciones e intervalos de confianza (IC), los resultados pueden ser erróneos. Aspectos clave como el **efecto del diseño**, los **dominios de estimación** y los **factores de expansión** deben ser incorporados en el análisis para obtener inferencias válidas.

14.1.1. Algunas definiciones necesarias

Estratos

Subpoblaciones “naturales” que, *a priori*, son homogéneos en su interior pero heterogéneos entre sí. El diseño de la encuesta se hace de modo que se garantiza cubrir adecuadamente todos los estratos de interés. Algunas variables de estratos habituales son: sexo, edad (como grupo etario), nivel de educación, urbano/rural, etc.

Conglomerados

Unidades definidas dentro de cada estrato (si es muestreo polietápico), cuyo tamaño es conocido. Lo ideal es que la población dentro de un conglomerado sea lo más heterogénea posible, pero debe haber homogeneidad entre los conglomerados. Cada grupo debe ser una representación a pequeña escala de la población total. Los grupos deben ser mutuamente excluyentes y colectivamente exhaustivos.

Probabilidad de inclusión

Es la probabilidad que cada unidad tiene de estar incluida en la muestra. En los muestreos complejos polietápicos se obtiene, en general, multiplicando las probabilidades asociadas al estrato, a cada **Unidad Primaria de Muestreo** (UPM o PSU en inglés) dentro del estrato, y a la unidad de segunda etapa dentro cada UPM.

Dominios de estimación

Subconjuntos de la población objetivo cuyos elementos pueden ser identificados en el marco muestral sin ambigüedad, y en los que se permite desagregar los resultados de la encuesta. Es aconsejable respetar estos dominios de estimación y no realizar inferencia de parámetros de interés para otros dominios no previstos que conlleva estimaciones inválidas.

Factor de expansión

Es el valor asociado a cada unidad elegible y que responde a la muestra, que se construye a partir de la inversa de la probabilidad de inclusión de cada unidad o peso muestral inicial. Puede incluir distintos tipos de ajustes, para disminuir en lo posible los errores de cobertura y de no respuesta que afectan a la encuesta, y ser tratados por un proceso de calibración que lleva en general a ganar eficiencia y precisión en las estimaciones. Los factores de expansión finales son los que se emplean tanto para generar todas las estimaciones de una encuesta, como en los cálculos del error muestral al determinar la precisión alcanzada.

Efecto de diseño (DEFF por sus siglas en inglés)

Mide la pérdida en precisión al utilizar un diseño muestral complejo en lugar de un diseño aleatorio simple, por ejemplo, un efecto de diseño de 1,5 indica que la varianza del diseño complejo es 1,5 veces más grande que la varianza de un diseño aleatorio simple, en otras palabras se dio un aumento en la varianza de un 50 %.

14.2. Paquetes survey y srvyr

El paquete más conocido y utilizado del lenguaje R para trabajar con datos provenientes de muestreos complejos es `survey` (Lumley 2010) . Desarrollado por *Thomas Lumley* que a su vez es autor del libro *Complex Surveys - A Guide to Analysis Using R (2010)*.

Las funciones que lo integran utilizan en sus argumentos la sintaxis fórmula preferentemente y no son compatibles con el universo "ordenado" de `tidyverse`. Con el objetivo de hacer compatible estas funciones crearon un paquete "*wrapper*" (envoltorio) llamado `srvyr` (Freedman Ellis y Schneider 2026), que incorpora sintaxis *tidy*, haciendo uso de tuberías y funciones tales como `group_by()`, `summarise()`, entre otras.

El primer paso para trabajar con las funciones de `srvyr` es crear un objeto con la información relacionada con el diseño muestral. Esta tarea se realiza con la función `as_survey_design()` que tiene estos argumentos principales:

```
datos |>
  as_survey_design(
    ids = ...,
    strata = ...,
    variables = ...,
    fpc = ...,
    nest = F,
    weights = ...
  )
```

donde:

- `ids`: Variables que especifican identificadores de conglomerados desde el nivel más grande hasta el nivel más pequeño (dejar el argumento vacío, `NULL`, 1 o 0 indica que no hay conglomerados).
- `strata`: Variables que identifican a los estratos. Si no hay estratos, se ignora esta especificación.

Usualmente, se declara a partir de una variable. Por ejemplo, si la variable `estrato` define los estratos, sería,

```
datos |>
  as_survey_design(..., strata = estrato, ...)
```

- `variables`: Variables que se incluirán en el diseño. El valor predeterminado es todas las variables de datos.
- `fpc`: Variables con el factor de corrección por población finita.
- `nest`: Si es `TRUE`, re-etiqueta los conglomerados considerando anidamiento dentro de los estratos. Necesario activar cuando las etiquetas de las categorías de los conglomerados en los distintos estratos se llaman igual.
- **`weights`**: Variables de ponderación de cada observación (inverso a la probabilidad).

Los argumentos mínimos que debemos definir dependerá de la estructura muestral de la base de datos que estemos analizando y de las variables que tengamos a disposición. Habitualmente se tiene referencia de estratos, conglomerados y ponderaciones. También podemos encontrarnos con situaciones donde están definidos los tamaños poblacionales (variable `fpc`).

14.3. Ejemplo práctico en R

Para ejemplificar vamos a tomar a la **Encuesta Mundial de Salud Escolar** (EMSE) en su tercera edición que en la Argentina se realizó en 2018. Fue llevada a cabo por el Ministerio de Salud y Desarrollo Social de la Nación, contó con la colaboración de los Ministerios de Educación Nacional y Provinciales, la OPS/OMS Argentina, OPS/OMS Washington y el CDC.

El diseño de muestreo tuvo dos etapas (selección de escuelas y luego de divisiones al azar) para producir una muestra representativa de alumnos de 1º a 5º año de educación media a nivel nacional (8º EGB a 3º polimodal en el caso de la provincia de Buenos Aires) y provincial. Se relevaron 523 escuelas en todo el país y se encuestaron 57.095 alumnos de los cuales se analizaron 56.981 cuestionarios correspondientes a las edades de 13 a 17 años, con una tasa de respuesta global de 63 %. Se puede acceder a sus datos abiertos en EMSE 2018

Activamos los paquetes necesarios:

```
library(srvyr)
library(tidyverse)
```

Leemos el archivo de datos "EMSE.txt" con:

```
datos <- read_csv2("datos/EMSE.txt")

# dimensiones de la tabla de datos
glimpse(datos)
```

```
Rows: 56,981
Columns: 14
$ record     <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
$ q2         <dbl> 2, 2, 1, 1, 2, 1, 2, 2, 1, 2, 1, 2, 2, 2, 2, 2, 1, 1, 2, 1,~
$ texto_q2   <chr> "Femenino", "Femenino", "Masculino", "Masculino", "Femenino~
$ q3         <dbl> 2, 2, 4, 2, 2, 2, 4, 4, 4, 2, 2, 4, 2, 4, 4, 4, 4, 2, 2, 2,~
$ texto_q3   <chr> "9no grado nivel Primario/Polimodal o 2do año nivel Secunda~
$ q4         <dbl> NA, NA, 1.67, 1.73, NA, 1.72, NA, NA, NA, NA, 1.64, NA, NA,~
$ q5         <dbl> NA, NA, 56, 70, NA, 48, NA, NA, NA, NA, 45, NA, NA, 40, 52,~
$ q6         <dbl> 2, 1, 1, 1, 1, 1, 3, 1, 4, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ texto_q6   <chr> "Rara vez", "Nunca", "Nunca", "Nunca", "Nunca", "Nunca", "A~
$ qn24       <dbl> 1, 2, NA, 2, 2, 2, 2, 2, 2, 1, 2, 1, 2, NA, 1, 2, 2, 2, 2, ~
$ texto_qn24 <chr> "Si", "No", "Dato perdido", "No", "No", "No", "No", "No", "~
$ psu        <dbl> 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19, 19,~
$ stratum    <dbl> 201801010, 201801010, 201801010, 201801010, 201801010, 2018~
$ weight     <dbl> 457.09, 457.09, 261.44, 550.90, 457.09, 550.90, 274.43, 274~
```

El dataframe importado consta de *14 variables y 56.981 observaciones*.

Calculamos el total de la muestra expandida:

```
datos |>
  summarise(total_expandido = sum(weight))
```

```
# A tibble: 1 x 1
  total_expandido
            <dbl>
1         2637546
```

Las variables relevantes para el diseño muestral que el documento de usuario define son:

- `psu`: Unidades primarias de muestreo (conglomerados)
- `stratum`: Estratos del muestreo
- `weight`: Ponderación

Además, nuestras variables de interés son:

- `record`: Nro. de registro
- `q2`: Sexo
- `texto_q2`: Sexo categorizado
- `q3`: Grado / año en el que se encuentra el estudiante
- `texto_q3`: Grado / año en el que se encuentra el estudiante categorizado
- `q4`: Estatura sin zapatos (medido en metros)
- `q5`: Peso sin zapatos (medido en kilogramos)
- `q6`: Pregunta sobre hambre
- `texto_q6`: Pregunta sobre hambre categorizada
- `qn24`: Pregunta sobre idea suicida
- `texto_qn24`: Pregunta sobre idea suicida categorizada

Para facilitar el análisis, vamos a renombrar nuestras variables de interés usando la función `rename()` de `tidyverse`:

```
datos <- datos |>
  rename(
    sexo = texto_q2,
    grado = texto_q3,
    estatura_m = q4,
    peso_kg = q5,
    hambre = texto_q6,
    idea_suicida = texto_qn24
  )
```

A partir de las variables cuantitativas `estatura_m` y `peso_kg` vamos a construir una nueva variable llamada `imc` (Índice de Masa Corporal) :

```
datos <- datos |>
  # IMC: peso sobre talla al cuadrado
  mutate(imc = peso_kg / (estatura_m)^2)
```

Ahora podemos comenzar a definir el objeto de diseño muestral:

```
d <- datos |>
  as_survey_design(
    ids = psu, # conglomerados
    variables = c(
      record,
      psu,
      stratum,
      weight,
      q2,
      sexo,
      q3,
```

```
    grado,
    peso_kg,
    estatura_m,
    imc,
    q6,
    hambre,
    qn24,
    idea_suicida
  ), # variables
  strata = stratum, # estratos
  weights = weight, # ponderación
  nest = T
) # anidación
```

Una vez que tenemos creado el objeto ***"survey.design"***, que en nuestro ejemplo se denomina `d`, podemos avanzar en el calculo de las estimaciones.

Si bien podemos manipular datos dentro de este formato de diseño utilizando algunas de las funciones de `tidyverse` como `select()`, `mutate()`, `filter()` y `rename()`, lo conveniente es modificar previamente la estructura de la tabla de datos, ya sea para crear una nueva variable, como hicimos recién con IMC o cambiar una existente y luego generar el objeto con el diseño muestral para poder hacer uso de esos cambios en las estimaciones.

14.3.1. Estimaciones

> Todos los códigos para las estimaciones comenzaran a partir del objeto de diseño creado inicialmente (llamado `d` en nuestro ejemplo) y evitando utilizar el nombre de la tabla de datos leída que solo tiene los datos de la muestra sin especificar el diseño de muestreo.

En primer lugar tomaremos la variable `imc`, variable cuantitativa continua que construimos.

La función `survey_mean()` computa estimaciones de medias en diseños complejos usando la sintaxis de `tidyverse`.

Los siguientes son los argumentos comunes de la función:

- `x`: Variable o expresión o vacía.
- `na.rm`: Si es `TRUE`, omite los valores `NA` de la variable
- `vartype`: Obtiene la variabilidad como uno o más estimadores: error estándar (`"se"`, predeterminado), intervalo de confianza (`"ci"`), varianza (`"var"`) o coeficiente de variación (`"cv"`).
- `level`: Nivel de confianza (solo se usa si el anterior es `"ci"`)
- `deff`: Si es `TRUE`, calcula el efecto de diseño para la estimación

Aplicamos la función con todas las posibles salidas de varianza sobre la variable de `imc` dentro de `summarise()` de `tidyverse`:

```
d |>
  summarise(
    imc = survey_mean(
      x = imc,
      na.rm = T,
      vartype = c("se", "ci", "var", "cv"),
      level = 0.95
    )
  )
```

```
# A tibble: 1 x 6
    imc imc_se imc_low imc_upp imc_var  imc_cv
  <dbl>  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
1  22.1 0.0869    22.0    22.3 0.00755 0.00392
```

El resultado muestra una media de IMC de 22,1, un error estándar de 0,09, un intervalo de confianza al 95 % de (22,0-22,3), una varianza de 0,007 y un coeficiente de variación de 0,4 %.

Como nos informan que hubo no respuesta en la encuesta y las faltantes de información en altura y/o peso provoca observaciones perdidas en la variable IMC, vamos a ver cuantos de estos valores perdidos tenemos en la variable.

Si deseamos calcular totales, es decir a cuantos estudiantes representa estos valores perdidos podemos usar la función `survey_total()`:

```
d |>
  filter(is.na(imc)) %>% # filtramos los NA en IMC

  summarise(imc_na = survey_total(vartype = "ci"))
```

```
# A tibble: 1 x 3
   imc_na imc_na_low imc_na_upp
    <dbl>      <dbl>      <dbl>
1 976104.    878120.   1074088.
```

Hay aproximadamente 976.104 IC 95 % (87.820-1.074.088) estudiantes sin valor en la variable `imc` del total de la población de 2.637.546 (un 37 %).

Si en lugar de la media quisiéramos la mediana podemos cambiar la función y usar `survey_median()`:

```
d |>
  summarise(
    imc = survey_median(
      x = imc,
      na.rm = T,
      vartype = c("se", "ci", "var", "cv"),
      level = 0.95
    )
  )
```

```
# A tibble: 1 x 6
    imc imc_se imc_low imc_upp imc_var  imc_cv
  <dbl>  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
1  21.5 0.0772    21.4    21.7 0.00596 0.00359
```

Podemos estratificar estas estimaciones haciendo uso del `group_by()`, por ejemplo con la variable `sexo` del estudiante:

```
d |>
  group_by(sexo) |>
  summarise(media_imc = survey_mean(imc, vartype = "ci", na.rm = T))
```

```
# A tibble: 3 x 4
  sexo         media_imc media_imc_low media_imc_upp
  <chr>            <dbl>         <dbl>         <dbl>
1 Dato perdido       0             0             0
2 Femenino          22.0          21.8          22.2
3 Masculino         22.3          22.1          22.5
```

Como hay datos perdidos en la variable sexo nos aparece una categoría más en los grupos de los resultados. Al estar vacía en las estimaciones (evidentemente son "no respuestas" completas) podemos deshacernos de esa línea con un `filter()`:

```
d |>
  filter(sexo != "Dato perdido") |> # o filter(!is.na(q2))

  group_by(sexo) |>
  summarise(media_imc = survey_mean(imc, vartype = "ci", na.rm = T))
```

```
# A tibble: 2 x 4
  sexo      media_imc media_imc_low media_imc_upp
  <chr>         <dbl>         <dbl>         <dbl>
1 Femenino       22.0          21.8          22.2
2 Masculino      22.3          22.1          22.5
```

Esta tabla muestra las estimaciones de `imc` según sexo con sus intervalos de confianza.

Otra variable de la encuesta es la respuesta a la pregunta *"Durante los últimos 30 días ¿con qué frecuencia te quedaste con hambre porque no había suficiente comida en tu hogar?"*. Las categorías válidas fueron: Nunca, Rara vez, Algunas veces, Casi siempre, Siempre y Dato perdido si no hubo respuesta.

Para abordar estas variables cualitativas donde queremos obtener proporciones el paquete tiene la función `survey_prop()` que se utiliza declarando la variable de interés en un `group_by()` previo:

```
d |>
  group_by(hambre) %>% # variable para la estimación
  summarise(prop_hambre = survey_prop(vartype = "ci") * 100) # usamos 100 para %
```

```
# A tibble: 6 x 4
  hambre        prop_hambre prop_hambre_low prop_hambre_upp
  <chr>               <dbl>           <dbl>           <dbl>
1 Algunas veces       9.34            8.49           10.3
2 Casi siempre        1.36            1.18            1.56
3 Dato perdido        0.804           0.678           0.952
4 Nunca              67.8            66.6            69.0
5 Rara vez           20.1            19.2            20.9
6 Siempre             0.627           0.487           0.807
```

El resultado muestra las categorías desordenadas en relación a lo que significa cada una respecto a la definición de la pregunta, es decir a la ordinalidad de la variable, aunque si está ordenada respecto a la forma en que lo hace el lenguaje R (alfabético).

Para darle el orden adecuado, deberíamos convertir la variable `hambre` a *factor* y declarar correctamente sus niveles. Conviene hacer esto previo a la creación del objeto de diseño pero el paquete también lo permite una vez creado:

```
d <- d |>
  mutate(
    hambre = factor(
      hambre,
      c(
        "Nunca",
        "Rara vez",
        "Algunas veces",
        "Casi siempre",
        "Siempre",
        "Dato perdido"
      )
    )
  )
```

Repetimos el código anterior:

```
d |>
  group_by(hambre) |>
  summarise(prop_hambre = survey_prop(vartype = "ci") * 100)
```

```
# A tibble: 6 x 4
  hambre        prop_hambre prop_hambre_low prop_hambre_upp
  <fct>               <dbl>           <dbl>           <dbl>
1 Nunca              67.8            66.6            69.0
2 Rara vez           20.1            19.2            20.9
3 Algunas veces       9.34            8.49           10.3
4 Casi siempre        1.36            1.18            1.56
5 Siempre             0.627           0.487           0.807
6 Dato perdido        0.804           0.678           0.952
```

El ordenamiento de las categorías ahora es el correcto producto de la transformación a factor.

Para estratificar estas proporciones lo único que debemos hacer es incorporar la variable de agrupamiento dentro del `group_by()`:

```
d |>
  group_by(sexo, hambre) |>
  summarise(prop_hambre = survey_prop(vartype = "ci") * 100)
```

```
# A tibble: 18 x 5
# Groups:   sexo [3]
   sexo          hambre        prop_hambre prop_hambre_low prop_hambre_upp
   <chr>         <fct>               <dbl>           <dbl>           <dbl>
 1 Dato perdido Nunca               54.3            43.9            64.3
 2 Dato perdido Rara vez            23.5            17.3            31.2
 3 Dato perdido Algunas veces       14.2             8.18           23.5
 4 Dato perdido Casi siempre         1.07            0.459           2.49
 5 Dato perdido Siempre              1.18            0.589           2.36
 6 Dato perdido Dato perdido         5.74            3.45            9.41
 7 Femenino     Nunca               68.1            66.7            69.4
 8 Femenino     Rara vez            19.7            18.7            20.8
 9 Femenino     Algunas veces        9.78            8.96           10.7
10 Femenino     Casi siempre         1.18            0.933           1.49
11 Femenino     Siempre              0.549           0.389           0.775
12 Femenino     Dato perdido         0.673           0.530           0.854
13 Masculino    Nunca               67.8            66.2            69.4
14 Masculino    Rara vez            20.3            19.2            21.5
15 Masculino    Algunas veces        8.75            7.69            9.94
16 Masculino    Casi siempre         1.56            1.20            2.01
17 Masculino    Siempre              0.700           0.518           0.946
18 Masculino    Dato perdido         0.842           0.624           1.13
```

Este esquema de agrupamientos o estratificaciones se anidan en el orden en que se declaran dentro del `group_by()` siendo la última variable la de estimación. Por ende, los porcentajes también salen anidados, tomando como denominador para el calculo del 100 % al total de cada categoría de `sexo` en el ejemplo anterior.

También podemos hacer que para el calculo de los porcentajes se tome como denominador el total general de la siguiente forma:

```
d |>
  group_by(interact(sexo, hambre)) |>
  summarise(prop_hambre = survey_prop(vartype = "ci") * 100)
```

```
# A tibble: 18 x 5
   sexo          hambre        prop_hambre prop_hambre_low prop_hambre_upp
   <chr>         <fct>               <dbl>           <dbl>           <dbl>
 1 Dato perdido Nunca              0.539           0.429           0.677
 2 Dato perdido Rara vez           0.234           0.160           0.342
 3 Dato perdido Algunas veces      0.141           0.0755          0.263
```

```
 4 Dato perdido Casi siempre       0.0107         0.00464          0.0245
 5 Dato perdido Siempre            0.0117         0.00594          0.0232
 6 Dato perdido Dato perdido       0.0570         0.0328           0.0990
 7 Femenino     Nunca             35.0           33.7             36.4
 8 Femenino     Rara vez          10.2            9.58            10.8
 9 Femenino     Algunas veces      5.03           4.63             5.47
10 Femenino     Casi siempre       0.608          0.481            0.768
11 Femenino     Siempre            0.283          0.200            0.400
12 Femenino     Dato perdido       0.346          0.270            0.443
13 Masculino    Nunca             32.2           31.1             33.4
14 Masculino    Rara vez           9.66           9.11            10.2
15 Masculino    Algunas veces      4.16           3.63             4.76
16 Masculino    Casi siempre       0.741          0.569            0.965
17 Masculino    Siempre            0.333          0.246            0.450
18 Masculino    Dato perdido       0.400          0.295            0.543
```

Un argumento incluido dentro de las posibilidades de `survey_prop()` es el método para su calculo (`prop_method`). En este documento no vamos a profundizar en los métodos que la función ofrece, solo mencionaremos las posibilidades permitidas según la ayuda del paquete:

- `"likelihood"`: utiliza la distribución de chi-cuadrado escalada (Rao-Scott) para la log-likelihood de una distribución binomial.
- `"asin"`: usa la transformación estabilizadora de la varianza para la distribución binomial, la raíz cuadrada del arcoseno, y luego transforma el intervalo a la escala de probabilidad.
- `"beta"`: usa la función beta incompleta como en `binom.test`, con una tamaño de muestra efectivo basado en la varianza estimada de la proporción. (Korn y Graubard, 1998).
- `"mean"`: utiliza un intervalo tipo Wald en la escala de probabilidad, igual que `confint()`.
- `"logit"`: ajusta un modelo de regresión logística y calcula un intervalo tipo Wald en la escala logarítmica de probabilidades, que luego se transforma a la escala de probabilidad. Es el método predeterminado de la función.

Otra función útil es `survey_count()` que estima el conteo poblacional de las observaciones según categoría:

```
d |>
  survey_count(hambre, vartype = "ci")
```

```
# A tibble: 6 x 4
  hambre               n    n_low    n_upp
  <fct>            <dbl>    <dbl>    <dbl>
1 Nunca         1788701. 1683670. 1893733.
2 Rara vez       529025.  498681.  559370.
3 Algunas veces  246227.  213940.  278513.
4 Casi siempre    35857.   30276.   41438.
5 Siempre         16542.   12251.   20832.
6 Dato perdido    21194.   16920.   25468.
```

Aquí lo usamos acompañando con el intervalo de confianza del 95 % (predeterminado).

Ahora tomaremos otra variable de la EMSE2018 llamada `idea_suicida` que refiere a la pregunta: *"Durante los últimos 12 meses, ¿alguna vez consideraste seriamente la posibilidad de intentar suicidarte?"*. Sus posibles respuestas son `"Si"`, `"No"` y el habitual `"Dato perdido"`. Estimaremos su proporción para toda la población pero también para una subpoblación en particular, por ejemplo para *"3er año/12vo grado nivel Polimodal o 5to año nivel Secundario"*.

Como es habitual que necesitemos obtener estimaciones en subgrupos o subpoblaciones determinados por categorías definidas de una variable, debemos tener cuidado al interpretar los elementos del muestro para que esas subpoblaciones hayan sido consideradas y por lo tanto estén incluídas como **dominio de estimación**.

Estos subgrupos requeridos pueden no coincidir con los estratos de la muestra compleja generando un inconveniente para las estimaciones, dado que las ponderaciones muestrales serían correctas pero la probabilidad de muestreo seguramente no, lo que produce estimaciones puntuales correctas con errores estándar incorrectos.

Las funciones del paquete `srvyr` maneja estos detalles sin ningún esfuerzo especial por parte del analista siempre y cuando utilice la muestra completa para definir el objeto de diseño de la encuesta pero no asegura que los resultados tengan la calidad necesaria.

Mostremos el código entonces:

```
d |>
  group_by(idea_suicida) |>

  summarise(prop_suicidio = survey_prop(vartype = c("ci", "cv")) * 100)
```

```
# A tibble: 3 x 5
  idea_suicida prop_suicidio prop_suicidio_low prop_suicidio_upp
  <chr>                <dbl>             <dbl>             <dbl>
1 Dato perdido          2.16              1.90              2.45
2 No                   76.8              76.1              77.6
3 Si                   21.0              20.3              21.7
# i 1 more variable: prop_suicidio_cv <dbl>
```

Seleccionamos una subpoblación correspondiente a los estudiantes de "*3er año/12vo grado nivel Polimodal o 5to año nivel Secundario*" (código 5 de q3):

```
d |>
  filter(q3 == 5) |>

  group_by(idea_suicida) |>

  summarise(prop_suicidio = survey_prop(vartype = c("ci", "cv")) * 100)
```

```
# A tibble: 3 x 5
  idea_suicida prop_suicidio prop_suicidio_low prop_suicidio_upp
  <chr>                <dbl>             <dbl>             <dbl>
1 Dato perdido          1.71              1.27              2.30
```

```
2 No                    79.3              77.6              81.0
3 Si                    18.9              17.5              20.4
# i 1 more variable: prop_suicidio_cv <dbl>
```

Un 21,0 % (95 % IC: 20,3-21,7 %) del total de estudiantes dicen haber considerado un intento de suicidio en el último año, mientras que un 18,9 % (95 % IC: 17,5-20,4 %) dice lo mismo en el grupo de "*3er año/12vo grado nivel Polimodal o 5to año nivel Secundario*".

Agregamos a los resultados algo importante, que va a garantizar la calidad de la estimaciones y evitar estar informando valores poco confiables para los cuales el muestreo quizás no esté preparado. Este es el **coeficiente de variación** (CV). Comparativamente vamos a encontrar valores más elevados de CV, en la medida que se agreguen filtros o agrupamientos anidados que reduzcan la muestra de la estimación.

Que pasa si filtramos y nos quedamos además solo con las mujeres que respondieron que se quedaron con hambre "algunas veces" en los últimos 30 días:

```
d |>
  filter(q3 == 5, sexo == "Femenino", hambre == "Algunas veces") |>

  group_by(idea_suicida) |>
  summarise(prop_suicidio = survey_prop(vartype = c("ci", "cv")) * 100)
```

```
# A tibble: 3 x 5
  idea_suicida prop_suicidio prop_suicidio_low prop_suicidio_upp
  <chr>                <dbl>             <dbl>             <dbl>
1 Dato perdido          3.15             0.941              10.0
2 No                   63.5             53.7                72.3
3 Si                   33.4             25.6                42.2
# i 1 more variable: prop_suicidio_cv <dbl>
```

Los CV de las estimaciones siguen creciendo, por ejemplo en el caso de la respuesta **Si** a un 12,6 % en comparación de la población completa que era de 1,6 %. Este aumento tiene que ver sobre todo con el tamaño muestral que nos queda luego de tantos filtros:

```
# Total muestra
d |>
  survey_count(idea_suicida)
```

```
# A tibble: 3 x 3
  idea_suicida        n   n_se
  <chr>           <dbl>  <dbl>
1 Dato perdido   56885.  4176.
2 No           2026726. 56584.
3 Si            553935. 21644.
```

```
# Subgrupo seleccionado
d |>
  filter(q3 == 5, sexo == "Femenino", hambre == "Algunas veces") |>
  survey_count(idea_suicida)
```

```
# A tibble: 3 x 3
  idea_suicida      n  n_se
  <chr>         <dbl> <dbl>
1 Dato perdido   600.  356.
2 No           12084. 1695.
3 Si            6348.  808.
```

Solo 6.347 estudiantes expandidos que respondieron que **Si** sobre los 553.934 totales que lo hicieron. Y cuantos respecto a la muestra sin expandir?

Podemos verlo así:

```
# Total muestra
d |>
  group_by(idea_suicida) |>

  summarise(n = unweighted(n()))
```

```
# A tibble: 3 x 2
  idea_suicida     n
  <chr>        <int>
1 Dato perdido  1353
2 No           43666
3 Si           11962
```

```
# Subgrupo seleccionado
d |>
  filter(q3 == 5, sexo == "Femenino", hambre == "Algunas veces") |>

  group_by(idea_suicida) |>

  summarise(n = unweighted(n()))
```

```
# A tibble: 3 x 2
  idea_suicida     n
  <chr>        <int>
1 Dato perdido     9
2 No             287
3 Si             194
```

194 estudiantes de la muestra sin expandir respondieron que **Si** sobre 11.962 sin aplicar los filtros.

14.4. Errores estándar según efecto del diseño

En este punto vamos a comparar estimaciones en función de construir objetos de diseños muestrales diferentes sobre la misma base de datos.

Diseño complejo y pesos (igual al ejemplo anterior):

```
d_complejo <- datos |>
  as_survey_design(
    ids = psu, # conglomerados
    variables = c(
      record,
      psu,
      stratum,
      weight,
      q2,
      sexo,
      q3,
      grado,
      peso_kg,
      estatura_m,
      imc,
      q6,
      hambre,
      qn24,
      idea_suicida
    ), # variables
    strata = stratum, # estratos
    weights = weight, # ponderación
    nest = T
  ) # anidación
```

Diseño sin pesos ni estratos:

```
d_simple <- datos |>
  as_survey_design(
    ids = psu, # conglomerados
    variables = c(
      record,
      psu,
      stratum,
      weight,
      q2,
      sexo,
      q3,
      grado,
      peso_kg,
      estatura_m,
      imc,
      q6,
      hambre,
      qn24,
      idea_suicida
    ), # variables
    strata = NULL, # estratos
    weights = NULL, # ponderación
```

```
    nest = T
  ) # anidación
```

Diseño con pesos y sin estratos:

```
d_ponde <- datos |>
  as_survey_design(
    ids = psu, # conglomerados
    variables = c(
      record,
      psu,
      stratum,
      weight,
      q2,
      sexo,
      q3,
      grado,
      peso_kg,
      estatura_m,
      imc,
      q6,
      hambre,
      qn24,
      idea_suicida
    ), # variables
    strata = NULL, # estratos
    weights = weight, # ponderación
    nest = T
  ) # anidación
```

Tenemos tres diseños distintos: un diseño complejo basado en conglomerados, estratos y pesos, un diseño sólo con pesos y un diseño simple sin estratos ni ponderación. Sabemos que el primero es el diseño "real" con el que se llevó a cabo la recolección de los datos.

Ahora volvamos a la variable `imc` para estimar su media en cada diseño:

```
d_complejo |>
  summarise(
    media_IMC = survey_mean(x = imc, na.rm = T, vartype = c("ci", "cv"))
  )
```

```
# A tibble: 1 x 4
  media_IMC media_IMC_low media_IMC_upp media_IMC_cv
      <dbl>         <dbl>         <dbl>        <dbl>
1  22.13994      21.96900      22.31089  0.003923691
```

```
d_simple |>
  summarise(
    media_IMC = survey_mean(x = imc, na.rm = T, vartype = c("ci", "cv"))
  )
```

```
# A tibble: 1 x 4
  media_IMC media_IMC_low media_IMC_upp media_IMC_cv
      <dbl>         <dbl>         <dbl>        <dbl>
1  22.23472      22.16037      22.30906  0.001669130
```

```
d_ponde |>
  summarise(
    media_IMC = survey_mean(x = imc, na.rm = T, vartype = c("ci", "cv"))
  )
```

```
# A tibble: 1 x 4
  media_IMC media_IMC_low media_IMC_upp media_IMC_cv
      <dbl>         <dbl>         <dbl>        <dbl>
1  22.13994      21.98614      22.29375  0.003467962
```

La estimación puntual (`imc` = 22,14) es la misma para los análisis que usan el diseño complejo y los que usan sólo pesos. A su vez es diferente si las estimaciones se llevan a cabo sin considerar el factor de expansión (muestreo simple - `imc` = 22,23). Los intervalos de confianza estimados son diferentes para todos los casos porque la varianza se vincula con la estructura del muestreo (estratos y conglomerados). Si no se utilizan pesos en las estimaciones, los estimadores no serán representativos de la población muestreada.

Si solo se utilizan pesos sin considerar el diseño complejo, en general, se infravalorará la dispersión de los estimadores, llevando a intervalos de confianza excesivamente estrechos y a niveles de significación reales mayores que los nominales (como si se tratase de un MAS). Basta comparar el resultado correcto IMC 22,14 (95 % IC: 21,97-22,31) frente al producido con el diseño simple IMC 22,23 (95 % IC: 22,16-22,30).

Como vemos, el lenguaje R permite que construyamos el objeto de diseño de la muestra compleja con la forma que indiquemos pero esto no siempre es correcto. Debemos conocer previamente la forma en que se diseño el muestreo y como se expresa en las variables de la tabla de datos para hacerlo adecuadamente. **Es fundamental leer detenidamente los documentos de utilización de los datos de toda encuesta donde se hayan recolectado datos mediante muestreos complejos**.

14.4.1. Criterios de calidad en las estimaciones

Todas las estimaciones elaboradas a partir de datos obtenidos por encuestas poblacionales con muestreos complejos están sujetas al error muestral, lo que hace necesario evaluar su validez estadística mediante diversos indicadores de precisión y confiabilidad.

Veamos algunos de estos indicadores de calidad:

Coeficiente de variación

Es el principal indicador de calidad de una estimación. Esta medida configura un acercamiento al error de muestreo que permite verificar si la inferencia es válida. Se caracteriza por ser proporcional a la amplitud del intervalo de confianza, que provee una versión estandarizada y relativa de la precisión alrededor de la estimación puntual.

Un umbral aproximado de CV mayor a 20 %-30 % puede asumirse como un valor de referencia útil a nivel regional para señalar una cifra de poco confiable, *Gutiérrez y otros (2020)*. Para calcular el coeficiente de variación de las estimaciones en estas estimaciones basta con incluirlo dentro del argumento `vartype`.

Tamaño de muestra

Este criterio se encuentra generalmente relacionado al anterior y es relevante a la hora de decidir la calidad de la estimación. La cobertura de los intervalos de confianza y la distribución de los estimadores dependen de que, tanto el tamaño de la subpoblación como su tamaño de muestra asociado, no sean pequeños. Con este espíritu, *Gutiérrez y otros (2020)* proponen que todas las estimaciones basadas en un tamaño de muestra expandida menor a 100 unidades deberían ser marcadas como no confiables.

Conteo de casos no ponderado

Cuando la incidencia de un fenómeno es muy baja y el diseño de la encuesta no lo tuvo en cuenta, entonces es posible que las estimaciones asociadas a tamaños, totales y proporciones sobre este fenómeno no sean confiables. Por ejemplo, *National Research Council (2015)* plantea que si el número de casos no ponderados es menor a 50 unidades entonces la estimación no sería publicable.

EPIDAT 4.2 - Consellería de Sanidade - Servizo Galego de Saúde (s. f.)

Bell et al. (2012)

Sakshaug y West (2014)

Aycaguer (2000)

Gutiérrez et al. (2020)

National Research Council; Division of Behavioral and Social Sciences and Education (2015)

Referencias

Álvarez Cáceres, Rafael. 2007. *Estadística aplicada a las ciencias de la salud*. Ediciones Díaz de Santos.

Aycaguer, Luis Carlos Silva. 2000. *Diseño razonado de muestras y captación de datos para la investigación sanitaria*. Ediciones Díaz de Santos.

Bache, Stefan Milton, y Hadley Wickham. 2026. *magrittr: A Forward-Pipe Operator for R*. https://doi.org/10.32614/CRAN.package.magrittr.

Bell, Bethany A., Anthony J. Onwuegbuzie, John M. Ferron, Qun G. Jiao, Susan T. Hibbard, y Jeffrey D. Kromrey. 2012. «Use of Design Effects and Sample Weights in Complex Health Survey Data: A Review of Published Articles Using Data From 3 Commonly Used Adolescent Health Surveys». *American Journal of Public Health* 102 (7): 1399-405. https://doi.org/10.2105/AJPH.2011.300398.

Champely, Stephane. 2020. *pwr: Basic Functions for Power Analysis*. https://doi.org/10.32614/CRAN.package.pwr.

Daniel, Wayne W. 2002. *Bioestadística: Base para el análisis de las ciencias de la salud*. 4.ª ed. Limusa Wiley.

Elm, Erik von, Douglas G. Altman, Matthias Egger, Stuart J. Pocock, Peter C. Gøtzsche, y Jan P. Vandenbroucke. 2008. «The Strengthening the Reporting of Observational Studies in Epidemiology (STROBE) Statement: Guidelines for Reporting Observational Studies». *Journal of Clinical Epidemiology* 61 (4): 344-49. https://doi.org/10.1016/j.jclinepi.2007.11.008.

EPIDAT 4.2 - Consellería de Sanidade - Servizo Galego de Saúde. s. f. Accedido 11 de junio de 2024. https://www.sergas.es/Saude-publica/EPIDAT-4-2.

Field, Andy, Jeremy Miles, y Zoë Field. 2014. *Discovering statistics using R*. Repr. Sage.

Firke, Sam. 2024. *janitor: Simple Tools for Examining and Cleaning Dirty Data*. https://doi.org/10.32614/CRAN.package.janitor.

Freedman Ellis, Greg, y Ben Schneider. 2026. *srvyr: 'dplyr'-Like Syntax for Summary Statistics of Survey Data*. https://doi.org/10.32614/CRAN.package.srvyr.

Glantz, S. 2006. *Bioestadística*. 6.ª ed. McGraw Hill.

Gross, Juergen, y Uwe Ligges. 2015. *nortest: Tests for Normality*. https://doi.org/10.32614/CRAN.package.nortest.

Gutiérrez, Andrés, Xavier Mancero, Alvaro Fuentes, Felipe López, y Felipe Molina. 2020. *Criterios de calidad en la estimación de indicadores a partir de encuestas de hogares: una aplicación a la migración internacional*.

Hernández-Ávila, Mauricio. 2011. *Epidemiología: diseño y análisis de estudios*. Editorial Médica Panamericana.

Horikoshi, Masaaki, y Yuan Tang. 2018. *ggfortify: Data Visualization Tools for Statistical Analysis Results*. https://CRAN.R-project.org/package=ggfortify.

Kassambara, Alboukadel. 2026a. *ggpubr: 'ggplot2' Based Publication Ready Plots*. https://CRAN.R-project.org/package=ggpubr.

Kassambara, Alboukadel. 2026b. *rstatix: Pipe-Friendly Framework for Basic Statistical Tests*. https://doi.org/10.32614/CRAN.package.rstatix.

Kassambara, Alboukadel, Marcin Kosinski, y Przemyslaw Biecek. 2026. *survminer: Drawing Survival Curves using 'ggplot2'*. https://doi.org/10.32614/CRAN.package.survminer.

Komsta, Lukasz, y Frederick Novomestky. 2022. *moments: Moments, Cumulants, Skewness, Kurtosis and Related Tests*. https://doi.org/10.32614/CRAN.package.moments.

Lüdecke, Daniel, Indrajeet Patil, Mattan S. Ben-Shachar, Brenton M. Wiernik, Philip Waggoner, y Dominique Makowski. 2021. «see: An R Package for Visualizing Statistical Models». *Journal of Open Source Software* 6 (64): 3393. https://doi.org/10.21105/joss.03393.

Lumley, Thomas. 2010. *Complex Surveys: A Guide to Analysis Using R: A Guide to Analysis Using R*. John Wiley; Sons.

Manitz, Juliane, Mark Hempelmann, Goeran Kauermann, et al. 2021. *samplingbook: Survey Sampling Procedures*. https://doi.org/10.32614/CRAN.package.samplingbook.

Meyer, Fanny, y Victor Perrier. 2025. *esquisse: Explore and Visualize Your Data Interactively*. https://doi.org/10.32614/CRAN.package.esquisse.

Müller, Kirill, y Hadley Wickham. 2026. *tibble: Simple Data Frames*. https://doi.org/10.32614/CRAN.package.

tibble.

National Research Council; Division of Behavioral and Social Sciences and Education. 2015. *Realizing the Potential of the American Community Survey: Challenges, Tradeoffs, and Opportunities*. National Academies Press. https://doi.org/10.17226/21653.

Norman, GR, y Streiner, DL. 1996. *Bioestadística*. Mosby/Doyma Libros.

Pedersen, Thomas Lin. 2025. *patchwork: The Composer of Plots*. https://doi.org/10.32614/CRAN.package.patchwork.

Pedersen, Thomas Lin, y Fabio Crameri. 2023. *scico: Colour Palettes Based on the Scientific Colour-Maps*. https://doi.org/10.32614/CRAN.package.scico.

Ríus Díaz, Francisca, Francisco Javier Barón Lopez, Elisa Sánchez Font, y Luis Parras Guijosa. 2012. *Bioestadística: Métodos y Aplicaciones*. http://virtual.uptc.edu.co/ova/estadistica/docs/libros/ftp.bioestadistica.uma.es/libro/.

Rodríguez, Milena, y Fredy Mendivelso. 2018. «Diseño de investigación de corte transversal». *Revista Médica Sanitas* 21 (3): 141-47. https://revistas.unisanitas.edu.co/index.php/rms/article/view/368.

Rodríguez, VM, Verónica A Martínez Ordaz, José Roiz Hernández, Francisco Huazano García, y Armando Nieves Rentería. 2003. «Muestreo y tamaño de la muestra». *Una guía práctica. Coahuila, México: El Cid Editor*.

Ryu, Choonghyun. 2025. *dlookr: Tools for Data Diagnosis, Exploration, Transformation*. https://CRAN.R-project.org/package=dlookr.

Sakshaug, Joseph W., y Brady T. West. 2014. «Important Considerations When Analyzing Health Survey Data Collected Using a Complex Sample Design». *American Journal of Public Health* 104 (1): 15-16. https://doi.org/10.2105/AJPH.2013.301515.

Schloerke, Barret, Di Cook, Joseph Larmarange, et al. 2025. *GGally: Extension to 'ggplot2'*. https://doi.org/10.32614/CRAN.package.GGally.

Schulz, K. F, D. G Altman, y D. Moher. 2010. «CONSORT 2010 Statement: Updated Guidelines for Reporting Parallel Group Randomised Trials». *BMJ* 340 (mar23 1): c332-32. https://doi.org/10.1136/bmj.c332.

Spina, Alexander, Zhian N. Kamvar, y Dirk Schumacher. 2025. *epikit: Miscellaneous Helper Tools for Epidemiologists*. https://doi.org/10.32614/CRAN.package.epikit.

Stevenson, Mark, y Evan Sergeant. 2026. *epiR: Tools for the Analysis of Epidemiological Data*. https://doi.org/10.32614/CRAN.package.epiR.

Tennekes, Martijn, y Marco J. H. Puts. 2023. «cols4all: a color palette analysis tool». *EuroVis (Short Papers)*. https://doi.org/10.2312/evs.20231040.

Triola, Mario F. 2018. *ESTADÍSTICA Decimosegunda Edicion*. Pearson Educación de México, SA de CV.

Van Cauwenberghe, Lisbeth, y Colin R. Janssen. 2014. «Microplastics in Bivalves Cultured for Human Consumption». *Environmental Pollution* 193 (octubre): 65-70. https://doi.org/10.1016/j.envpol.2014.06.010.

Waring, Elin, Michael Quinn, Amelia McNamara, Eduardo Arino de la Rubia, Hao Zhu, y Shannon Ellis. 2026. *skimr: Compact and Flexible Summaries of Data*. https://doi.org/10.32614/CRAN.package.skimr.

Wickham, Hadley. 2011. «The Split-Apply-Combine Strategy for Data Analysis». *Journal of Statistical Software* 40 (1): 1-29. https://www.jstatsoft.org/v40/i01/.

Wickham, Hadley. 2016. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org.

Wickham, Hadley, Mara Averick, Jennifer Bryan, et al. 2019. «Welcome to the tidyverse». *Journal of Open Source Software* 4 (43): 1686. https://doi.org/10.21105/joss.01686.

Wickham, Hadley, y Jennifer Bryan. 2026. *readxl: Read Excel Files*. https://doi.org/10.32614/CRAN.package.readxl.

Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, y Davis Vaughan. 2026. *dplyr: A Grammar of Data Manipulation*. https://doi.org/10.32614/CRAN.package.dplyr.

Wickham, Hadley, Jim Hester, y Jennifer Bryan. 2026. *readr: Read Rectangular Text Data*. https://doi.org/10.32614/CRAN.package.readr.